

Coursera Course Transcripts

Introduction To Data Analytics Course

Generated on June 25, 2026

Total lectures: 164

Modules / Weeks: 1

Introduction to Data Analytics

Meet Your Instructor - Prof. Seetha Parameswaran Video• . Duration: 1 minute

[Notes]

Greetings, I am Sita Parmesharan, an Assistant Professor with Work Integrated Learning Programs of Pillai Institute of Technology and Science, Pillani. I bring over a decade of combined experience from both industry and academia to my role. My academic journey began at Cochin University of Science and Technology, Kerala, where I earned my B.Tech. degree in Information Technology. Passionate about deepening my expertise, I went on to complete my M.Tech. in Computer Science and Engineering, specializing in image processing from the same institution. My professional career has been a blend of industry and academia. I spent two years in IT industry, where I gained valuable experience in software development, testing, and eventually taking on the role of project manager. This hands-on industry experience allows me to provide my learners with real-world perspectives on application of theoretical concepts. At BITS Pillani, I have focused my teaching efforts on data science domain. My courses cover a wide range of cutting-edge topics, reflecting my diverse research interests. These include data analytics, machine learning, and the processing and analysis of audio, image, and video data. I strive to bridge theoretical concepts with practical applications, preparing my learners for the challenges of rapidly evolving tech industry. I am passionate about education and research in the field of data science, and look forward to continuing to contribute this exciting and dynamic field. Happy learning!

Meet Your Instructor - Prof. Aneesh Chivukula Video• . Duration: 57 seconds 57 sec

[Notes]

Hello everyone, I am Anish Srivallabh Chivukula, working currently as Assistant Professor in the Department of Computer Science and Information Systems at the BITS Pilani Hyderabad campus. I have 16 years of teaching, research and industry experience. I have worked in engineering, R&D and consulting roles in the industry. My research interests are in computational algorithms, machine learning, deep learning, data mining, adversarial learning, game theory and robust optimization. I have a PhD in data analytics and machine learning from the University of Technology Sydney, Australia. I hold a Master of Science by Research in Computer Science and Artificial Intelligence from the International Institute of Information Technology, Hyderabad, India. All the best with your studies in this course. Thank you.

Course Introductory Video Video• . Duration: 3 minutes

[Notes]

Welcome, learners, to our course, Introduction to Data Analytics. This curriculum is designed to equip you with essential skills for the dynamic field of data analytics. Our journey begins with an

introduction to data analytics, exploring its key components and applications across industries. You will then dive into Python fundamentals, learning to write efficient code for data manipulation and analysis. Then we will explore fundamental data structures, NumPy library, which provides you powerful tools for numerical computations. The exploratory data analysis module will teach you to uncover insights using Python pandas and create compelling visualizations using the libraries Matplotlib and Seaborn. Finally, you will master data cleaning and preparation techniques, ensuring your datasets are primed for the analysis. Throughout this program, you will gain hands-on experience, preparing you for the challenges and opportunities in the field of data analytics. Thank you for choosing our program. We are excited to guide you on this journey of discovery and innovation in data analytics. Hello, everyone. I am Professor Anish Chukwala. In the last five modules of this course, we will study feature engineering techniques such as feature selection, dimensionality reduction and feature extraction to create the training dataset for building the analytics models. Then we will study several popular analytics techniques such as for regression, classification and clustering. In regression techniques, we will study both linear and nonlinear regression techniques, including the fitting criteria for regression and their optimization using gradient descent algorithm. In classification, we will study binary and multilabel classification, especially with reference to decision trees and their extensions into random forests. Such rule-based techniques will be contrasted with parameter-based models such as logistic regression and discriminant analysis. Appropriate performance metrics to evaluate the goodness of fit of such models will also be introduced. This will be followed by discussion of clustering techniques, including partitional and hierarchical clustering methods to build partitions of the data in a nested fashion. The quality of clustering will be evaluated with cluster validation techniques, including external and internal indices to measure quality. The final module of the course will discuss the privacy, security and fairness aspects of data analytics models, especially with respect to privacy-preserving data mining, algorithmic bias and fairness, and adversarial data mining. Cost-sensitive classification and learning, adversarial and robust data mining will be provided as overarching frameworks to analyze the privacy, security and fairness of the data analytics models. All the best with your learnings in the course. Hope you enjoy the course. Thank you.

Definition of Data Analytics Video• . Duration: 6 minutes

[Notes]

Welcome to our course, Introduction to Data Analytics. Today, we will be exploring the fundamental concepts of data analytics. Let's start with our learning objectives. By the end of this topic, you will be able to define data analytics and its core processes. Explore types of data and its key analytics. Understand the tools and techniques that are required for data analytics. These objectives will guide our discussion and help you grasp the essentials of data analytics. So, what is data analytics? At its core, data analytics is the process of examining data sets to draw conclusions about the information they contain. This definition encapsulates the essence of what we do in this field. The objective of data analytics is to transform raw data into actionable intelligence. This means taking large volumes of data and distilling them into insights that can guide decision-making in various fields. Data analytics comprises four key elements. Data collection. This involves gathering relevant information from various sources. Data processing. Once collected, the data needs to be cleaned and organized. Data analysis. This is where we apply various techniques to uncover patterns and

insights. Fourth one is interpretation of results. We translate our findings into actionable recommendations. Each of these elements is crucial for effective data analytics. The ultimate goal of data analytics is twofold. To extract meaningful insights from raw data. To support decision-making processes. By achieving these goals, data analytics help organizations make informed decisions based on concrete evidence rather than intuition or guesswork. In data analytics, we work with three main types of data. Structured data is highly organized data, typically found in spreadsheets or databases. Unstructured data includes text, videos, and images. It's more complex to analyze, but often contains rich information. Semi-structured data falls between structured and unstructured, like JSON or XML files. Understanding these data types is crucial because each requires different approaches for effective analysis. Data analysts use a variety of tools and technologies. Spreadsheet software like Excel or Google Sheets for basic data manipulation. Statistical software such as R or SAS for more advanced statistical computations. Programming languages like Python and SQL for powerful data processing. Business intelligence tools like Tableau or Power BI for creating visualization and dashboards. Each of these tools has its own strengths. A skilled data analyst often uses a combination of them depending on the task at hand. The data analytics process typically follows these steps. Definition of the question. Clearly articulate what you want to learn from the data. Second, collect relevant data. Gather the data from the appropriate sources. Third, clean and process the data. Ensure that the data is accurate and usable. Analyze the data. Apply various analytical techniques to uncover insights. Fifth one, interpret and present results. Translate the findings into actionable recommendations. This process is often iterative and you may have to revisit the earlier steps to gain new insights. Data analytics employs several key techniques. Statistical analysis using statistical methods to describe and interpret data. Data mining discovering pattern in large data sets. Predictive modeling using historic data to forecast future outcomes. Machine learning algorithms applying artificial intelligence to learn from and make predictions based on the data. These techniques form the backbone of modern data analytics and are constantly evolving. To excel in data analytics, one needs to develop a diverse skill set. Statistical knowledge. Programming and data manipulation skills. Data visualization abilities. Critical thinking and problem solving capabilities. Domain expertise in the field you are analyzing. Strong communication skills. And these skills allow data analysts to extract meaningful insights from the data and effectively communicate them to the decision makers. Data analysts serve as a crucial bridge between raw data and the decision makers in the organization. The primary responsibility is to translate complex findings into actionable insights. This involves not only analyzing the data, but also presenting it in a way that non-technical stakeholders can understand and use to inform their decision. To recap what we have discussed so far. Data analytics is a process of examining data to draw insights. It involves various techniques, tools, and skills and is crucial for informed decision making in various fields. The process transforms raw data into valuable insights that can drive business strategy and innovation. Thank you for the attention. In the next session, we will dive deeper into specific data analytics techniques and their applications. Until then, keep exploring the fascinating world of data.

The Importance of Data Analytics Video• . Duration: 5 minutes

[Notes]

Hello, everyone. Welcome to our second topic, where we will be discussing the importance of data analytics. Let's start with the learning outcomes. By the end of this video, you will be able to

understand the role of data analytics in modern decision making, recognize its impact on business competitiveness and innovation. These objectives will guide our discussion on why data analytics is crucial in today's world. Data analytics matters because it transforms raw data into actionable insights. This transformation is key to driving informed decision making across various industries. By analyzing data, organizations can uncover patterns, trends and correlations that might otherwise remain hidden, leading to more effective strategies and operations. One of the primary benefits of data analytics is enabling data-driven decision making. This represents a shift from relying on intuition to making choices based on concrete evidence. By using data to inform decisions, organizations can reduce uncertainty and mitigate risks in their decision making processes. Data analytics provides a significant competitive advantage. It allows businesses to identify market trends and opportunities that may not be immediately apparent. Moreover, it helps in optimizing operations and strategies based on real-time data, enabling organizations to stay ahead of their competitors and adapt quickly to market changes. Another crucial benefit of data analytics is improved efficiency and cost reduction. By analyzing operational data, organizations streamline their processes, identify areas of waste or inefficiency. This leads to optimized resource allocation and significant cost saving across various business functions. Data analytics play a significant role in enhancing customer experience. By analyzing customer data, businesses can gain deep insights into customer behavior and preferences. This understanding allows for personalization of products and services, leading to improved customer satisfaction and loyalty. In the realm of risk management and fraud detection, data analytics is indispensable. Advanced analytical techniques can identify patterns indicative of fraud or potential risks. This enables organizations to implement proactive risk mitigation strategies, protecting their assets and reputation. Data analytics significantly impacts product development and innovation. By analyzing market data and consumer feedback, companies can guide their research and development efforts more effectively. It helps in predicting market needs and trends, allowing businesses to innovate and develop products that meet future demand. In financial planning and forecasting, data analytics enables more accurate budget projections. By analyzing historical financial data and market trends, organizations can identify financial opportunities and risks more effectively. This leads to more robust financial strategies and better resource allocation. The impact of data analytics extends to healthcare as well. It is improving patient outcomes through analysis of vast amounts of medical data. Data analytics enhance disease prediction and prevention, leading to more effective treatments and public health strategies. In marketing and sales, data analytics enable highly targeted marketing campaigns. By analyzing consumer data, businesses can tailor their marketing efforts to specific audience segments. It also aids in sales and forecasting and pipeline optimizations, leading to more efficient sales processes and increased revenue. Data analytics is revolutionizing supply chain management. It helps in optimizing inventory levels, ensuring that businesses have the right amount of stock at the right time. Moreover, it aids in predicting and mitigating supply chain disruptions, leading to more resilient and efficient supply chains. To summarize, data analytics is crucial for informed, evidence-based decision-making across various sectors. It drives business performance, fosters innovation, and provides a competitive advantage. From improving customer experience to enhancing operational efficiency, the importance of data analytics in today's world cannot be overstated. Thank you for your attention. In our next video, we will discuss the key components of data analytics. Until then, consider how data analytics might be applied in your field of interest or workplace.

Key Components Video• . Duration: 5 minutes

[Notes]

Hello everyone. Welcome to the topic, Key Components of Data Analytics. By the end of this video, you will be able to Understand the fundamental elements of data analytics Explore each component's role in the analytics process Recognize how these components work together These objectives will guide our discussion on the essential building blocks of data analytics. Data analytics comprises of five main components Data Collection, Data Processing, Data Analysis, Data Visualization, Interpretation and Decision Making. Each of these five components play a crucial role in turning raw data into actionable insights. Let us explore each of these in more detail. The first step in data analytics is data collection. This involves gathering raw data from various sources. Methods of data collection include surveys, sensors, web scraping and databases. The significance of this step cannot be overstated. The quality and relevance of the data collected will directly impact the insights we can derive. It is crucial to ensure that the data collected is appropriate for the analysis we want to perform. Once the data is collected, it needs to be processed. Data processing involves preparing raw data for analysis. This step includes three main activities. Data cleaning, which involves handling missing data, removing duplicates, and addressing errors and inconsistencies in the data. Data integration, where we combine data from multiple sources to create a unified data set. Data transformation, which involves converting data into a suitable format for analysis. The goal of data processing is to create a clean, consistent, and analysis-ready data set. Data analysis is at the heart of the analytics process. It involves applying various techniques to derive insights from the data to support decision-making. The steps involved in data analysis include exploratory data analysis. This is initial investigation of the data to discover patterns, spot anomalies, and check assumptions. Statistical analysis. This involves both descriptive statistics to summarize the data and inferential statistics to make predictions. Predictive modeling and pattern recognition, where we use machine learning techniques to identify patterns and make predictions based on the data. Data visualization is a crucial component that involves graphical representation of the data and insights. The main purpose of data visualization is to make complex data more accessible, understandable, and usable. This is particularly important when communicating findings to non-technical stakeholders. Types of data visualization include charts, graphs, dashboards, and infographics. Effective data visualization can reveal trends, patterns, and outliers that may not be apparent from looking at the raw data. The final component is interpretation and decision-making. This involves translating analytical results into actionable insights. It is important to consider the context and limitation of the analysis when interpreting results. Not all statistically significant findings are practically significant or actionable. Communicating findings to the stakeholders is a crucial part of this step. The goal is to present insights in a way that enables informed decision-making. It is important to note that the data analytics process is not linear but iterative. Each step builds on the previous one and feedback from each step is used to improve and refine the previous process. This continuous improvement enhances the overall accuracy and effectiveness of the data analytics process. For example, the insight from data analysis might inform future data collection efforts or challenges in data visualization might lead to improvements in data processing. This iterative nature allows for ongoing refinement and optimization of data analytics techniques and outcomes. To recap, the data analytics process includes five key components ■■■ data collection, processing, analysis, visualization, and interpretation. Each component plays a crucial role in transforming raw data into actionable insights. Remember that these components are not isolated steps but part of the interconnected process. The feedback loops between components allows for continuous

refinement of the analytic process, enhancing the effectiveness over time. Thank you for your attention.

Descriptive Analytics Video• . Duration: 6 minutes

[Notes]

Welcome everyone, welcome to the topic of descriptive analytics. By the end of this video, you will be able to define descriptive analytics, understand its role in data analytics landscape, identify key techniques and applications of descriptive analytics, recognize the strengths and limitations of descriptive analytics. What is descriptive analytics? It is the process of summarizing historical data to gain insights. The focus is on what has happened in the past. The key question that descriptive analytics answer is what occurred. This type of analytics helps us make sense of historical data and provides a basis for further analysis. Descriptive analytics is the first stage of the progression of the data analytics. It forms the foundation of the more advanced analytics types. The analytics spectrum typically progress from descriptive to diagnostic, then to predictive and finally to prescriptive analytics. Each stage builds upon the insights gained from the previous stage, with descriptive analytics providing the essential starting point. The key characteristics of descriptive analytics include retrospective analysis. It looks at the past data. Data aggregation and summarization. It condenses large amounts of data into manageable summaries. Pattern recognition in historical data. It helps identifying trends and patterns that have occurred over time. These characteristics make descriptive analytics an essential tool for understanding what has happened in your business or the area of study. Descriptive analytics employs several key techniques. 1. Data aggregation â combining data from multiple sources. 2. Data mining â discovering patterns in large data sets. 3. Summary statistics â calculating measures that describe the data. 4. Data visualization â creating graphical representations of the data. These techniques work together to provide a comprehensive view of historical data. The common descriptive statistics include measures of central tendency, such as mean, median, and mode. Measures of dispersion, like range, variance, and standard deviation. Percentiles and quartiles. These statistics provide a numerical summary of the data, helping to describe its main features. Data visualization is a crucial part of descriptive analytics. Common visualization types include bar charts and histograms for showing distributions. Line graphs and scatter plots showing patterns and relationships. Pie charts and area charts for showing composition. Heat maps and tree maps for showing hierarchy or complex data. These visualizations help make the data more accessible and easier to understand. Descriptive analytics has numerous applications, including business reporting and dashboards, sales and revenue analysis, customer segmentation, inventory management, and social media metrics. These applications help businesses and organizations understand their current state and the recent history. Let us look at an example of descriptive analytics in retail sales analysis. This might include the total sales by product category, monthly sales trends, identifying the top selling products, summarizing customer demographics. This type of analysis provides a clear picture of a retail business, its performance, and customer base. Descriptive analytics has several strengths. It provides a clear summary of the past events. It is easy to understand and communicate. It forms the basis for future analysis, and it helps identifying patterns and trends. These strengths make descriptive analytics an essential starting point for any data analysis project. However, descriptive analytics also has its limitations. It does not explain why events occurred. It has limited predictive

capabilities. It may oversimplify complex situations. There is a potential for misinterpretation without context. Understanding these limitations is crucial for using descriptive analytics effectively and knowing when to move on to more advanced analytics techniques. To recap, descriptive analytics summarizes historical data to understand what happened. It uses techniques like data aggregation, summary, and visualization. While it provides clear summaries that are easy to understand, it does not explain causality or predict future trends. It is a crucial foundation for more advanced analytics, but it is important to be aware of its limitations. Thank you for your attention.

Diagnostic Analytics Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to the video on Diagnostic Analytics. By the end of this video, you will be able to define Diagnostic Analytics, understand its role in data analytics landscape, identify key techniques used in Diagnostic Analytics, recognize applications and limitations of Diagnostic Analytics. These objectives will guide our exploration of Diagnostic Analytics, which build upon the foundation laid by Descriptive Analytics. What is Diagnostic Analytics? It is a process of examining data to understand why something happened. The focus is on identifying cause and effect relationship. The key question the Diagnostic Analytics answers is why did it happen? This type of analytics helps us dive deeper into our data to uncover reasons behind trends and events. Diagnostic Analytics is the second stage in the progression analytics. It builds upon Descriptive Analytics, which we discussed in the previous video. The analytics spectrum typically progresses from Descriptive to Diagnostic, then Predictive to finally Prescriptive Analytics. Diagnostic Analytics takes the what happened from Descriptive Analytics and helps us understand the why. The key characteristics of Diagnostic Analytics include root cause analysis, digging deep into finding the underlying causes of the events, pattern recognition, identifying recurring patterns in the data, correlation and causation investigation, examining relationship between variables, finally data drill down and discovery, exploring data at increasingly granular levels. These characteristics allow Diagnostic Analytics to provide insights into the reasons behind the data patterns and the trends. Diagnostic Analytics employs several key techniques. Data discovery, exploring data to find meaningful patterns. Data mining, which uses algorithms to extract patterns from large data sets. Correlations and regression analysis, which examine relationship between variables. Principal component analysis to reduce data dimensionality while retaining variation. And finally, time series analysis to analyze time-ordered data points. These techniques work together to help us understand the why behind our data. The Diagnostic Analytics process typically follows the following steps. 1. Identify the problem or anomaly 2. Collect relevant data 3. Analyze patterns and relationships 4. Form hypothesis 5. Test the hypothesis 6. Draw conclusions This systematic approach helps ensure that our diagnostic analysis is thorough and reliable. Diagnostic Analytics has numerous applications, including business performance analysis, customer churn analysis, manufacturing defect analysis, healthcare diagnosis support, marketing campaign effectiveness evaluation. These applications help businesses and organizations understand the reasons behind their successes and challenges. Let us look at an example of Diagnostic Analytics in e-commerce sales drop analysis. If we identify a sales drop, which would be a descriptive analytics insight, we might investigate the potential causes such as website performance issues, competitors' actions, seasonal trends, marketing campaign changes. These type of analysis helps us understand why sales have dropped

and informs our strategy to address the issue. Data visualization plays a crucial role in Diagnostic Analytics. Common visualization types include scatter plots for correlation analysis, heat maps for identifying patterns, decision trees for root cause analysis, fishbone diagrams for cause and effect relationships. These visualizations help make complex relationships in the data more understandable and reveal insights that might be missed in raw data. While powerful Diagnostic Analytics does face some challenges, distinguishing correlation from causation can be difficult. Dealing with complex multi-factor problems require careful analysis. There is a risk of confirmation bias in analysis. Ensuring data quality and relevance is crucial for accurate results. Being aware of these challenges helps us apply Diagnostic Analytics more effectively and interpret results more accurately. To recap, Diagnostic Analytics examines why events or trends occurred. It uses techniques like root cause analysis and pattern recognition to build on descriptive analytics and provide deeper insights. Key applications include performance analysis and problem solving. However, it is important to be aware of the challenges such as distinguishing correlation from causation and avoiding bias in our analysis. Thank you for your attention. Have a good day.

Predictive Analytics Video• . Duration: 6 minutes

[Notes]

Welcome to the topic, Predictive Analytics. By the end of this video, you will be able to define predictive analytics, understand its role in data analytics landscape, identify key techniques and models used in predictive analytics, recognize applications of predictive analytics. These objectives will guide our exploration of predictive analytics, which takes us from understanding past events to forecasting future ones. What is predictive analytics? It is the use of historical data, statistical algorithms and machine learning techniques to identify the likelihood of future outcomes. The focus is on forecasting future trends and behavior. The key question that predictive analytics answers is, what is likely to happen? This type of analytics helps organizations prepare for the future scenarios based on the patterns in their data. Predictive analytics is the third stage in the progression of analytics. It builds upon descriptive, diagnostic, which we have discussed in the previous videos. The analytics spectrum typically progresses from descriptive to diagnostic, then to predictive and finally to prescriptive analytics. The predictive analytics takes the insight from what happened, why did it happen to forecast what may happen next. The key characteristics of predictive analytics include forward-looking analysis. It focuses on future events. Probability-based forecasting, which deals with likelihoods, not certainties. Pattern-based prediction, which uses historical patterns to predict future ones. Continuous model refinement, which models regularly updated with new data. These characteristics allow predictive analytics to provide insights into potential future scenarios. Predictive analytics employs several key techniques. Regression analysis for understanding relationship between variables. Time series forecasting for predicting future values based on the historical time-ordered data. Classification algorithms for categorizing new data points. Clustering for grouping similar data points. Neural networks and deep learning for complex pattern recognition. These techniques form the backbone of predictive modeling. The predictive modeling processes typically follows these steps. 1. Define the objectives. What do we want to predict? 2. Collect and prepare the data. 3. Explore and visualize the data. 4. Select and train the models. 5. Evaluate and refine the models. 6. Deploy and monitor models. This systematic approach helps ensure that our predictive models are robust and reliable. There are several types of predictive models.

Classification models like logistic regression and decision trees, which predicts categories. Regression models like linear and polynomial regression, which predict the numeric values. Time series models like ARIMA and exponential smoothing, which forecast future values based on time-ordered data. The choice of the model depends on the type of prediction that we are trying to make and the nature of our data. Predictive analytics has numerous applications, including sales forecasting, customer churn prediction, risk assessment in finance and insurance, predictive maintenance in manufacturing, weather forecasting, healthcare outcome prediction. These applications help organizations prepare for the future events and make data-driven decisions. Let's look at one example of predictive analytics in action. Customer churn prediction. The process involves collecting historical data, identifying factors correlated with churn, building and training a predictive model, scoring current customers for churn risk, implementing targeted retention strategies. These approaches allow businesses to proactively address potential customer losses. Predictive analytics has specific data requirements. Large volumes of historical data are needed for accurate predictions. The data must be of high quality and clean. Relevant features or variables must be included. For classification problems, data sets should be balanced. Meeting these requirements is crucial for developing effective predictive models. Evaluating predictive models is a critical step. We use accuracy metrics like root mean squared error and mean average error for regression models. Classification metrics like precision and recall and F1 score for classification models. Cross-validation techniques to ensure model robustness. A-V testing in real-world deployments to verify model performance. These evaluation methods help us understand how well our models are likely to perform in practice. To recap, predictive analytics employs various modeling techniques from regression to machine learning. It builds on descriptive and diagnostic analytics to provide forward-looking insights. By forecasting potential future scenarios, predictive analytics enables organizations to make proactive data-driven decisions. Thank you for your attention. Have a good day.

Prescriptive Analytics Video• . Duration: 6 minutes

[Notes]

Hello everyone, welcome to the video on prescriptive analytics. By the end of this video, you will be able to define prescriptive analytics, understand its role in data analytics landscape, identify key techniques and methods used in prescriptive analytics, recognize applications and challenges of prescriptive analytics. These objectives will guide our exploration of prescriptive analytics, the most advanced stage in the analytics spectrum. What is prescriptive analytics? It is the use of advanced analytics techniques to recommend the best course of action. The focus is on optimizing decision-making and outcomes. The key question that prescriptive analytics answers is, what should we do? This type of analytics goes beyond predicting future outcomes to actually suggesting optimal actions. Prescriptive analytics is the fourth and most advanced stage in the progression of analytics. It builds upon descriptive, diagnostic and predictive analytics, which we have discussed in the previous videos. The analytics spectrum progresses from descriptive to diagnostic, then to predict and finally to prescriptive analytics. Prescriptive analytics takes the insights from the previous stages to recommend the best actions. The key characteristics of prescriptive analytics include action-oriented recommendations. It does not just provide insights, but suggests specific actions. Considering of multiple possible scenarios, it accounts for various potential futures. It

provides optimization, where it constantly refines recommendations as new data becomes available. Integration of business rules and constraints, which takes into account the real-world limitations and requirements. These characteristics allow prescriptive analytics to provide practical and actionable insights. Prescriptive analytics comprises key components. Predictive models to forecast potential outcomes. Optimization algorithms to find the best solutions. Business rules and constraints to ensure recommendations that are feasible. Scenario analysis to consider different possible futures. Decision support system to present recommendations effectively. These components work together to generate optimal recommendations. Prescriptive analytics employs several advanced techniques. Linear and non-linear programming for optimization problems. Monte Carlo simulations for modeling uncertainty. Heuristics and meta-heuristics for complex optimization problems. Machine learning for decision optimization. And game theory for strategic decision making. These techniques allow the prescriptive analytics to tackle complex real-world problems. The prescriptive analytics process typically follows these steps. Define the problem and objectives. Gather and prepare relevant data. Develop predictive models. Define constraints and business rules. Generate and evaluate possible scenarios. Optimize and recommend actions. Implement and monitor results. This systematic approach ensures that predictive analytics recommendations are well-grounded and actionable. The prescriptive analytics has numerous applications, including supply chain optimization, resource allocation in healthcare, portfolio optimization in finance, dynamic pricing in retail and commerce, route optimization in transportation and logistics, energy management, and smart grids. These applications demonstrate how prescriptive analytics can drive efficiency and effectiveness across various industries. Let's look at one example of prescriptive analytics in manufacturing production optimization. The process involves defining objectives, such as maximizing profit or minimizing costs, incorporating constraints like machine capacity and raw material availability, constraint variables such as production quantities and inventory levels, using optimization algorithms to determine the best production plan. This approach allows manufacturers to optimize their production processes based on multiple factors. Prescriptive analytics has specific data requirements. High-quality historical and real-time data, well-defined business rules and constraints, accurate predictive models, clearly defined optimization objectives. Meeting these requirements is crucial for developing effective prescriptive analytics solutions. The quality and comprehensiveness of the data directly impact the quality of the recommendations. While powerful prescriptive analytics face several challenges, complexity of real-world problems, dealing with uncertainty and dynamic environments, balancing multiple, sometimes conflicting objectives, ensuring model accuracy and reliability, explaining complex recommendations to stakeholders. Addressing these challenges is key to successful implementation of prescriptive analytics. To summarize, prescriptive analytics recommends the best course of action based on advanced analytics techniques. It builds on the other types of analytics to provide actionable insights. While it offers powerful capabilities for optimizing decision-making, it also faces challenges related to complexity, uncertainty, and need to explain complex recommendations. Thank you for your attention. This concludes our overview of the four types of analytics, descriptive, diagnostic, predictive, and prescriptive. Have a good day.

Industry Applications Video• . Duration: 6 minutes

[Notes]

Welcome, everyone. We'll be discussing industry applications of data analytics. By the end of this video, you will be able to identify key industry applications of data analytics, understand how different sectors leverage data analytics. These objectives will guide our exploration of how data analytics is transforming various industries and sectors. In healthcare, data analytics is making significant impacts. It is used for predictive diagnostics, helping to identify potential health issues before they become severe. Personalized treatment plans, tailoring medical care to individual patients. Hospital resource management by optimizing the use of staff, equipment, and facilities. Epidemic outbreak prediction and helping public health officials prepare for and respond to health crisis. Drug discovery and development by accelerating the process of bringing new treatments to market and applications. These applications are improving patient outcomes and healthcare efficiency. In finance and banking, data analytics is crucial for fraud detection and prevention, identifying unusual patterns that may indicate fraudulent activity. Risk management and assessment by helping institutions make informed decisions. Algorithmic trading using complex models to make rapid trading decisions. Customer segmentation and personalization by tailoring services to individual needs. Anti-money laundering compliance, helping institutions meet regulatory requirements. These applications are enhancing security, efficiency, and customer service in the financial sector. Retail and e-commerce heavily rely on data analytics. They use data analytics for demand forecasting, where predicting what products will be popular. Price optimization by setting prices to maximize profits while remaining competitive. Customer behavior analysis to understand how and why customers make purchases. Supply chain optimization by ensuring products are available when and where they are needed. And finally, personalized recommendation by suggesting products based on individual customer preferences. These applications are helping retailers improve their operations and customer experiences. In manufacturing, data analytics is driving efficiency through predictive maintenance by anticipating when equipment will need repair. Quality control and defect detection by identifying issues before they becoming problems. Inventory optimization by ensuring the right amount of stock is on hand. Production planning and scheduling by maximizing output and minimizing downtime. Energy consumption optimization by reducing cost and environmental impact. These applications are helping manufacturers improve quality, reduce cost, and increase productivity. Transportation and logistics benefit from data analytics in various ways. These include route optimization, where finding the most efficient paths for deliveries becomes important. Fleet management to keep track of vehicles and their performance. Predictive maintenance for vehicles such that reducing breakdowns and downtimes is critical. Demand forecasting for public transportation, matching service to passenger needs. Traffic pattern analysis and condition prediction to help reduce traffic jams. These applications are making transportation more efficient and reliable. In marketing and advertising, data analytics is revolutionizing how companies reach customers. Customer segmentation looks at dividing the market into distinct groups. Campaign performance analysis measures the effectiveness of marketing efforts. Attribution modeling tries to understand which marketing touchpoints lead to conversions. Personalized content delivery tailor messages to individual preferences. Social media sentiment analysis gorges the public opinion about the brands and the products. These help marketers reach the right people with the right message at the right time. Let's look at a specific case study. Netflix's use of data analytics. Netflix used data for content recommendation system, suggesting shows and movies to viewers. Viewership prediction for a new show, which helps decide what content to produce. Personalized thumbnail selection, showing different images to different users. Network optimization for streaming quality, ensuring a smooth viewing experience. Data-driven content

creation decision using viewer data to inform new productions. This case study shows how data analytics can be central to an organization's entire business model. Looking to the future, we see several trends in the industry applications of data analytics. Some of them include increased adoption of AI and machine learning, making analytics more powerful and autonomous. Greater focus on real-time and streaming analytics allowing for instant insights and actions. Integration of unstructured data sources like text, images, and video. More attention to ethical consideration and responsible AI, ensuring analytics is used fairly and safely. These trends suggest that data analytics will become even more central to business operations across industries. To recap, we have seen how data analytics is transforming various industries from healthcare, finance, retail, and manufacturing. It is being used to improve efficiency, enhance customer experience, and drive innovation. Looking ahead, we can expect to see even greater integration of AI and machine learning, more real-time data analytics, and a growing focus on ethical considerations. Thank you for your attention. Have a good day.

Challenges in Data Analytics Video• . Duration: 6 minutes

[Notes]

Welcome, everyone. We will be discussing the challenges that are faced in data analytics. By the end of this video, you will be able to identify key challenges faced in data analytics, understand the implication of these challenges on data-driven decision making, recognize potential solutions and best practices to address these challenges. These objectives will guide our exploration of the hurdles that organizations face when implementing data analytics. So, what are the challenges in implementing data analytics? Data quality and integration issues, privacy and security concerns, data volume and complexity, skill gap and talent acquisition, integration of data sources, interpreting and communicating the results, ethical considerations. Each of these challenges can significantly impact the success of the data analytics initiative. Let us explore each one in more detail. Data quality and integrity is a critical challenge. The issues incomplete, inaccurate or inconsistent data. This can lead to unreliable analysis and flawed decisions, which can have serious consequences for the organization. Solutions to this challenge include implementing data cleaning techniques, validation processes, and establishing robust data governance policies. Ensuring data quality is fundamental to the success of any data analytics project. Data privacy and security is another major concern. Issues include data breaches, unauthorized access and complaints with regulations like GDPR and CCPA. The impact can be severe, including legal consequences and loss of customer trust. To address this, organizations need to implement strong security measures, such as encryption, access control, and anonymization techniques. Regular audits are also crucial for ensuring ongoing compliance and security. The sheer volume and complexity of the data pose significant challenge. Handling big data and unstructured data formats can be overwhelming. This can lead to high storage costs, increased processing time, and difficulty in extracting meaningful insights. Solutions include investing in scalable infrastructure, using distributed computing techniques, and leveraging advanced analytics tools designed to handle big data. There is a significant skills gap and talent shortage in the field of data analytics. Many organizations struggle to find qualified data scientists and data analysts. This can lead to delayed projects and suboptimal use of data analytics capabilities. To address these, the organizations can implement training programs, partner with educational institutions, or consider outsourcing some analytics functions.

Building a strong data-driven culture within the organization is also crucial. Integrated data from various sources is often challenging. Issues include silhouetted data, incompatible formats, and legacy systems. This can result in an incomplete view of the business operations and missed insights. Some of the solutions include implementing data integration platforms, developing APIs to connect different systems, and creating centralized data warehouses. Efficient data integration is key to getting a holistic view of the business. Interpreting and communicating the results to another is a significant challenge. The complexity of the advanced analytics can be difficult for non-technical stakeholders to understand. This can lead to misinterpretation of the results and lack of buy-in for data-driven decisions. To address this, organizations can use data visualization tools, focus on storytelling with data, and ensure regular communication with the stakeholders. Making data insight accessible and understandable is crucial for their effective views in decision making. Finally, the ethical considerations in data analytics are becoming increasingly important. Issues include bias in the data and algorithms, a lack of transparency in the decision-making process. This can lead to unfair outcomes and a loss of public trust. To address these concerns, organizations should establish ethical guidelines, ensure diverse teams are involved in analytics process, and conduct regular bias audits. Responsible use of data analytics is crucial for maintaining trust and ensuring fair outcomes. To recap, we have discussed several key challenges in data analytics, including data quality, privacy, volume, skill gaps, integration, result interpretation, and ethical considerations. These challenges can significantly impact the reliability, efficiency, and effectiveness of data analytics. Addressing these challenges requires a combination of technological tools and organizational strategies and a commitment to continuous learning. By understanding and proactively addressing these challenges, organizations can maximize the value they derive from their data analytics efforts. Thank you for your attention. Have a good day.

Structured Data Video• . Duration: 5 minutes

[Notes]

Welcome everyone. We will be discussing structured data. By the end of this video, you will be able to define structured data and its characteristics, understand the advantages and limitations of structured data, identify common sources and formats of structured data. These objectives, which will guide our exploration of structured data, which forms the backbone of many data analytics projects. So what is structured data? It is data that is organized in a predefined form. The key characteristics of structured data are that it follows a consistent order and it is easily searchable. Common examples include spreadsheet data, relational databases and data in CSV files. This type of data is the most straightforward to work with. Let's delve deeper into the characteristics of structured data. It has a predefined data model, meaning the structure is determined before the data is filled in. It follows a consistent format, making it easy to understand and process. It is easily searchable and sortable, which is crucial for efficient data analysis. It is typically stored in rows and columns like in spreadsheet. It is machine readable, allowing for easy automated processing. Structured data comes from various sources in an organization. For example, customer relationship management systems store data in a structured format. Enterprise resource planning systems contain structured data about business process. Point of sale systems capture transactional data in a structured way. Web forms and surveys collect data in predefined fields. IoT sensors with predefined data formats provide structured data about their physical environments. Structured data

is typically stored in these formats. Relational database like MySQL or PostgreSQL, which organize data into tables with rows and columns. Spreadsheets such as Excel or Google Sheets, which are familiar to many users, also use rows and columns. Comma-separated files or CSV files, which are simple text files where data fields are separated by commas. These formats are widely used and supported by many data analysis tools. Structured data offers several advantages. It is easy to search and analyze due to its consistent format. It allows for efficient storage and retrieval of information. It supports complex queries, enabling detailed data exploration. It facilitates data consistency, reducing errors and inconsistencies. It is compatible with many analysis tools, making it versatile for various applications. However, structured data also has limitations. It can be inflexible in handling varied data types, which can be a problem with complex or diverse data. It may have difficulty capturing complex relationships between data points. There is a potential for data redundancy, which can waste storage space. It may not capture all relevant information, especially unstructured or semi-structured data. Setting up and maintaining structured data systems can be time-consuming. Structured data plays a crucial role in data analytics. It is fundamental in descriptive analytics, providing clear summaries of historical data. It is important in predictive modeling, where consistent data formats are crucial. It is widely used in data visualization, allowing for clear and accurate representation. It forms the foundation for many machine learning algorithms, which often require well-organized input data. Looking at the future, we see several trends in the use of structured data. Increased adoption of AI and machine learning, which often rely on structured data. Greater focus on real-time and streaming analytics, requiring efficient structured data processing. Integration with unstructured data sources, combining the strengths of both data types. Growing emphasis on ethical considerations and responsible AI, ensuring structured data is used appropriately. To summarize, structured data is organized in a predefined format, typically in rows and in columns. Its key characteristics include consistency, searchability, and machine readability. While it offers advantages, it also has limitations such as inflexibility and potential redundancy. Structured data is crucial in many aspects of data analytics, from descriptive analytics to machine learning. As we move forward, we will see structured data continue to play a vital role, even as it's increasingly integrated with other data types and used in more advanced analytical techniques. Thank you for your attention. Have a good day.

Unstructured Data Video• . Duration: 6 minutes

[Notes]

Welcome everyone. We will be discussing unstructured data. By the end of this video, you will be able to define unstructured data and its characteristics. Understand the challenges and opportunities of unstructured data. Identify common sources and types of unstructured data. These objectives will guide our exploration of unstructured data, becoming increasingly important in the field of data analytics. So what is unstructured data? It is a data that does not have a predefined model or organization. The key characteristics of unstructured data are that it lacks a specific format and it is not easily searchable using traditional methods. Common examples include text documents, social media posts, images, and videos. The video that you are currently watching now is an example of unstructured data. Let's delve deeper into the characteristics of unstructured data. It has no predefined data model, meaning its structure is not organized in a predefined manner. Unstructured data has variable format and content, making it diverse but challenging to process

uniformly. It is difficult to search and analyze using traditional methods designed for structured data. It is often rich in content and context, providing valuable insights that might be missed in structured data. Unstructured data is frequently human-generated, reflecting natural communication and behavior patterns. Unstructured data comes from various sources. For example, social media platforms, where users generated vast amount of text, image, and video content. Simple email communications, which contain free-form text and attachments. Customer reviews and feedback, providing valuable but unstructured insights. Surveillance cameras and satellite imagery, generating large volumes of visual data. Scientific research data, such as genomics, which can be complex and unstructured. Unstructured data comes in several types. Text, including documents, emails, and social media posts. Images, such as photographs, diagrams, and medical images. Audio, including voice recordings, music, and podcasts. Video, like CCTV footage and user-generated content. Social media data, including likes, shares, and comments. Each of these types requires different approaches for analysis and interpretation. While working with unstructured data presents several challenges. It's difficult to organize and categorize due to its lack of structure. Unstructured data requires advanced analytics techniques, often beyond traditional database queries. High storage and processing demands due to its volume and complexity. It raises privacy and security concerns, especially with personal or sensitive information. Ensuring data quality and relevance can be challenging without clear structural guidelines. Despite these challenges, unstructured data offers significant opportunities. It is a rich source of insights and context, often capturing nuances missed in structured data. It enables sentiment analysis and opinion mining, valuable for understanding customer feelings. It supports natural language processing application, advancing our ability to interact with computers. It facilitates images and video analysis, opening up new fields of data visualization and interpretation. It drives innovation in AI and machine learning, pushing the boundaries of what is possible in data analysis. Several tools and technologies are used to analyze unstructured data. Natural language processing libraries for text analysis, image recognition software for processing visual data, text mining tools for extracting insights from large volumes of text, machine learning platforms for pattern recognition and prediction, big data analytics for processing large data sets. These tools enable us to extract meaningful insights from unstructured data. Unstructured data analytics has numerous applications. Some of them include customer insight and behavior analysis, helping businesses understand their customers better. Fraud detection and security, identifying unusual patterns that may indicate fraudulent activity. Healthcare diagnostics and research, analyzing complex medical data for better patient outcomes. Predictive maintenance and manufacturing, sensor data to anticipate equipment failures. Personalized marketing and recommendation, tailoring content to individual preferences. These applications demonstrate the wide-ranging impact of unstructured data analytics. To summarize, unstructured data lacks a predefined data model and includes diverse sources like social media, emails, images, and videos. While it presents challenges in organizing and analysis, it also offers rich opportunities for insights and motivations. Working with unstructured data includes natural language processing, image processing, and text mining. As we move forward in the field of data analytics, the ability to effectively analyze unstructured data will become increasingly important. It offers a wealth of information that can complement and enhance insights from structured data. Thank you for your attention. Have a good day.

Semi-Structured Data Video• . Duration: 7 minutes

Welcome, everyone. We'll be discussing the third type of data, semi-structured data. By the end of this video, you will be able to define semi-structured data and its characteristics, understand the advantages and challenges of semi-structured data, identify common formats and sources of semi-structured data. These objectives will guide our exploration of semi-structured data, which bridges the gap between structured and unstructured data. What is semi-structured data?

Semi-structured data is data that does not conform to rigid structural requirements, but has some organizational properties. The key characteristic is that it contains tags or markers to separate semantic elements. Common examples include JSON and XML documents, as well as emails. Semi-structured data combines elements of both structured and unstructured data, offering a balance of flexibility and organization. Let's delve deeper into the characteristics of semi-structured data. It has a flexible schema allowing for variations in the structure. It is self-describing, meaning the structure is embedded within the data. It can contain irregular or incomplete data accommodating real-world inconsistencies. It often has a hierarchical or graph-based organization. While machine-readable, it is more complex to process than purely structured data. Semi-structured data comes in several formats. XML, or Extensive Markup Language, uses tags to define elements. JavaScript Object Notation, which is popular for web applications and APIs. Resource Description Framework, which is used in semantic web applications. Emails, which combine structured headers with free-form-text bodies. These formats allow for flexible data representation, maintaining some level of structure. Semi-structured data comes from various sources. Web scraping results often capture data in formats like HTML or JSON. Log files contain a mix of structured and unstructured information. Sensor data from IoT devices, often in formats like JSON or XML. NoSQL databases require frequently stored semi-structured information. Configuration files combine structured settings with flexible formats. These sources highlight the prevalence of semi-structured data in modern data ecosystems. Semi-structured data offers several advantages. It provides flexibility in data representation, accommodating diverse information. It enables easier integration of heterogeneous data sources. It supports complex data relationships, useful for representing real-world scenarios. It has both human-readable and machine-processable, facilitating both manual and automated analysis. It is adaptable to changing data requirements, crucial in dynamic environments. However, semi-structured data also presents challenges. It is more complex to parse than structured data, requiring specialized parsing tools. There is a potential for inconsistencies across data instances, complicating analysis. It often requires specialized tools for efficient querying. Data validation can be more difficult due to flexible structure. It may require transformation for analysis with traditional tools designed for structured data. Several tools and technologies are used to analyze semi-structured data. NoSQL databases are designed to handle flexible data structures. XML or JSON parsers and processors are used for working with these common formats. Graph database, which is useful for data with complex relationships. Data integration platforms that can handle diverse data types. Big data technologies are capable of processing large volumes of semi-structured data. These tools enable efficient management and analysis of semi-structured data. Analyzing semi-structured data involves several techniques. Data extraction and parsing to convert semi-structured data into a more analyzable form. Schema inference methods to understand the underlying structure of the data. Specialized query languages for XML and JSON. Graph analysis techniques for hierarchical or network-like data. Machine learning applications that can work directly with semi-structured formats. These allow us to extract insights from semi-structured data effectively. Semi-structured data analytics has numerous applications. Web

analytics and clickstream analysis often dealing with log files and JSON data. Social network analysis frequently involves graph-like data structures. Bioinformatics and genomic data processing is requiring flexibility. Financial data analysis such as processing trading data in various formats. Sensor data analysis in IoT environments often involving JSON or XML data streams. To recap, semi-structured data has a flexible schema and some organizational properties. It offers advantages like flexibility and support for complex relationships, but also present challenges in parsing and consistency. Key applications include web analytics, IoT, and bioinformatics. As we continue to generate and collect diverse types of data, the importance of effectively working with semi-structured data will only grow. It offers a valuable middle ground between rigidity of the structured data and the complexity of the unstructured data. Thank you for your attention. Have a good day.

Quantitative Data Video• . Duration: 6 minutes

[Notes]

Welcome everyone. We will be discussing quantitative data. By the end of this video, you will be able to define quantity data and its characteristics, distinguish between discrete and continuous quantitative data, recognize common sources and applications of quantitative data, identify appropriate analysis and techniques used for quantitative data. These objectives will guide our exploration of quantitative data, which forms the backbone of many statistical analysis and data driven decisions. So what is quantitative data? It is numerical data that can be counted or measured. The key characteristics of quantitative data are that it is objective, precise and can be analyzed using statistical methods. Common examples include your age, height, temperature, sales figures. This type of data allows for precise comparisons and mathematical operations, making it crucial for many types of analysis. Quantitative data can be further divided into two types. Discrete data, which is countable and consists of whole numbers. An example would be the number of customers in a store. Second one is continuous data, which is measurable and can take value within a range. Examples include your height or your weight. Understanding this distinction is important as it affects how we analyze and interpret the data. What are the characteristics of quantitative data? It is numerical in nature. It is always represented by numbers. It is measurable and quantifiable, allowing for precise comparisons. It can be used for mathematical operations like addition or multiplication. It allows for statistical analysis, enabling us to draw meaningful conclusions. It is objective and standardized, reducing the impact of personal bias in analysis. Quantitative data comes from various sources. Some of them include surveys and questionnaires, which often collect numerical responses. Scientific measurements and experiments produce quantitative results. Financial transactions and reports are rich sources of numerical data. Sensor readings from IoT devices generate continuous streams of quantitative data. Government and census data often provide large-scale quantitative data sets. These diverse sources highlight the ubiquity of quantitative data in our world. Quantitative data is crucial in analytics for several reasons. It enables precise measurement and comparison, allowing for accurate assessments. It supports statistical analysis and hypothesis testing, forming the basis of scientific inquiry. It facilitates trend identification and forecasting, crucial for business planning. It allows for quantifiable decision-making, reducing reliance on subjective judgments. It forms the basis for many machine learning models, driving advancements in AI and predictive analytics. There are several key

techniques for analyzing quantitative data. Descriptive statistics like the measures of central tendency and measures of dispersion summarize the data well. Inference statistics, including hypothesis testing and confidence intervals, allow us to draw better conclusions. Correlation and regression analysis help us understand the relationship between the data. Time series analysis is used in real-time data that changes over time. Various data visualization techniques help in presenting quantitative data effectively. Quantitative data analysis has numerous applications. Financial modeling and forecasting, crucial for investment decisions. Quality control in manufacturing ensures products meet specifications. Marketing campaign performance analysis measures the effectiveness of advertising efforts. Scientific research and hypothesis testing advance our understanding of the world. Performance metrics and KPI tracking, which are essential for business management. These applications demonstrate the wide range impact of quantitative data analysis. While powerful, working with quantitative data also presents challenges. Ensuring data quality and accuracy is crucial for reliable results. Dealing with outliers and missing values requires careful considerations. There is a risk of interpreting statistical results, especially without context. Balancing statistical significance with practical significance is important. Communicating quantitative results to non-technical audiences can be challenging. Being aware of these challenges help us use quantitative data more effectively. To recap, quantitative data is numerical data that can be counted or measured. It comes in two types, discrete and continuous. Key characteristics include its numerical nature, measurability, and suitability for statistical analysis. We use various techniques to analyze quantitative data, including descriptive and inferential statistics. Quantitative data is crucial in many fields, from finance to scientific research, enabling precise analysis and data-driven decision. However, it is important to be aware of challenges like ensuring high-quality data and avoiding misinterpretation of results. Thank you for your attention. Have a good day.

Qualitative Data Video• . Duration: 6 minutes

[Notes]

Welcome, everyone. We will be discussing qualitative data in this video. By the end of this video, you will be able to define qualitative data and its characteristics, understand the importance of qualitative data in analytics, and identify some common sources and types of qualitative data, recognize appropriate methods for collecting and analyzing qualitative data. These objectives will guide our exploration of qualitative data, which provides rich contextual information that complements quantitative analysis. So what is qualitative data? It is the non-numerical data that describes qualities or characteristics. The key characteristics of qualitative data are that it is descriptive or subjective, and it is rich in context. Common examples include your opinions, descriptions, observations, and interviews. This type of data allows us to explore the why and the how behind the phenomenon, providing depth to our understanding. Qualitative data comes in several types. 1. Nominal data, which categorizes without numerical value, like the eye color or the ethnicity. 2. The ordinal data, which categorizes and ranks, such as satisfaction ratings or shirt sizes. 3. Textual data, including open-ended responses and interviews. 4. Visual data, like images, videos, and artworks. 5. Audio data, such as recorded interviews or ambient sounds. Each type offers unique insights that require a specific analytic approach. Let's look at the characteristics of qualitative data. It is descriptive and explanatory in nature, providing rich details. It is subjective and interpretive, often reflecting personal experience. It is rich in detail and context, offering nuanced

understanding. It is often unstructured or semi-structured, allowing for flexibility. It allows for exploration of complex phenomena that may be difficult to quantify. Qualitative data comes from various sources. Interviews and focus groups providing in-depth individual group perspectives. State survey questions allowing respondents to express themselves freely. Observation notes and field research capturing real-world behaviors and context. Documents and textual data offering historical or organizational insights. Social media posts and comments reflecting public opinions and trends. These diverse sources allow researchers to gather comprehensive qualitative insights. Qualitative data is crucial in analytics for several reasons. It provides context and depth to quantitative findings. It explores the reason behind trends and behaviors, offering deeper understanding. It can uncover unexpected insights and generate new hypotheses for further research. It captures nuanced information and experiences that might be missed in quantitative data. It is essential for understanding complex social phenomenon that resists simple quantification. There are several key techniques for analyzing qualitative data. Context analysis systematically categorizes and interprets textual data. Thematic analysis identifies patterns and themes across the data. Grounded theory approach develops theories based on the data itself. Discourse analysis examines language use and its social context. Narrative analysis focuses on the stories and experiences shared by participants. These methods allow researchers to extract meaningful insights from qualitative data. Qualitative data analysis has numerous applications. Some of them include market research and consumer insights. Consumer motivations and preferences. User experience research, which targets improving product and service design. Policy analysis and evaluation, which assess the impact and effectiveness of the policies. Healthcare and patient experience studies to improve care delivery and patient satisfaction. Organizational culture assessment to understand workplace dynamics and employee perspectives. These applications demonstrate the wide-ranging impact of qualitative data analysis. While valuable, working with qualitative data also presents challenges. It can be time-intensive to collect and analyze, requiring careful interpretation. There is potential for researcher bias in interpretation and analysis. It can be difficult to generalize findings to larger populations. Ensuring rigor and trustworthiness in the analysis process is crucial. Integrating qualitative insights with quantitative data can be complex. Being aware of these challenges helps researchers use qualitative data more effectively. To recap, qualitative data is non-numerical data that describes qualities or characteristics. It includes types such as nominal, ordinal, text, visual and audio data. Key characteristics include its descriptive nature, subjectivity and richness in the context. Thank you for your attention. Have a good day.

Primary and Secondary Data Sources Video• . Duration: 6 minutes

[Notes]

Welcome everyone. We will be discussing primary and secondary sources of data. By the end of this video, you will be able to define primary and secondary data sources, understand the advantages and limitations of each type, identify common examples of each of the data sources, recognize pentious primary against secondary data in analytics projects. These objectives will guide our exploration of data sources, which is crucial for understanding where our data comes from and how to choose the right data in our analysis. So what are data sources? Data sources are the origins of data used in research or analysis. The importance of data source in analytics cannot be overstated. They determine the quality, reliability and relevance of our data. We primarily recognize

and categorize two types of data sources, primary and secondary. Understanding these categories is essential for effective data collection and analysis. Primary data sources refer to data collected firsthand by the researcher for the specific purposes. This data is tailored to the research question or analytical need at hand. Examples of primary data sources include surveys, interviews, experiments and observations. The key characteristic here is that the data is original and collected specifically for the current research or analysis. Let us dive deeper into the characteristics of primary data. Primary data is original and unique to the research, providing fresh insights. It is collected for a specific purpose, ensuring relevance. Researcher has more control over the data collection process. It is often more time consuming and expensive to collect. Primary data can be either qualitative or quantitative, depending on the research needs. There are several methods for primary data collection. Surveys and questionnaires can reach a large number of respondents. Interviews can be structured, semi-structured or unstructured. Focus groups allow for group discussions and interactions. Observations, either participant or non-participant observations. Experiments and field trials test specific hypotheses under controlled conditions. Each method has its strengths and is chosen based on the research objectives. Now let's look into secondary data sources. These are data that already exist, collected by someone else for another purpose. Secondary data is repurposed for the current research or analysis needs. Examples include government reports, academic publications, and company records. The key here is that data wasn't originally collected for your specific requirement. The characteristics of secondary data include its pre-existing and not original to the current research. It is often more readily available and less expensive than primary data. It may not perfectly fit to the current requirements. It requires careful evaluation. It can provide valuable historical context or trends. Secondary data can be internal, that is within the organization, or external, that is outside the organization. There are various types of secondary data sources. Some of them include government publications and databases, which often provide large-scale data. Academic journals and research papers offering peer-reviewed findings. Industry reports and market research providing sector-specific insights. Social media and web analytics data offering real-time consumer insights. Public records and archives, which can provide historical data. These diverse sources offer a wealth of information for researchers and analysts. Primary data has several advantages. It is specific to research needs, offering high control and current data. However, the limitations include that it is often time-consuming and expensive to collect. There is potential for bias in the data collection process. These factors need to be weighted when deciding whether to use primary data. Secondary data also has its own advantages and limitations. Secondary data is cost-effective and time-saving. It can provide valuable context for the research, but at the same time, the data can be outdated. There is a lack of control over the data quality. There is a potential for misfit with your specific research needs. These factors need to be considered when opting for secondary data. To summarize, primary data is collected first-hand for specific research, while secondary data is pre-existing data collected by others. Primary data offers control and specificity, but it is time-consuming and expensive. Secondary data is cost-effective, but may lack a perfect fit for your needs. The choice between primary and secondary data depends on your research needs, available resources, and constraints. Thank you for your attention. Have a good day.

Internal and External Data Sources Video • Duration: 6 minutes

[Notes]

Welcome everyone. We will be discussing internal and external data sources. By the end of this video, you will be able to define internal and external data sources. Identify common examples of internal and external data sources. Understand the advantages and limitations of each type. Recognize the importance of integrating internal and external data in data analytics. These objectives will guide our exploration of where data comes from within and outside an organization. And it is crucial for comprehensive data analytics. Let us define our key terms. Internal data refers to information generated within an organization. External data, on the other hand, is information from sources outside the organization. The importance of understanding both in data analytics cannot be overstated. Together, they provide comprehensive insights for decision-making, offering both detailed internal views and broader marketing context. Diving deeper into internal data sources, these are data created and stored within an organization. They are often proprietary and highly relevant to specific business needs. Examples include sales records, customer databases, and employee data. This data is unique to your organization and can provide insights that are not available from external sources. There are several common types of internal data sources. Transactional data, where sales and purchase records are maintained. Customer relationship management. Enterprise resource planning systems. Human resource information systems. Internal surveys and feedback. Each of these sources provide valuable data about different aspects of the organization's operations. Internal data comes with several advantages. It is highly relevant to the organization, directly reflecting its operations and performance. There is greater control over data quality as the organization manages the data collection and storage. It is often more detailed and specific, providing granular insights into the organization's process and customers. However, internal data has its own limitations. It may be silhouetted or fragmented across different departments or systems. There is a potential for internal biases as data reflects only the organization's perspective. It provides limited external context, potentially missing broader market trends and competitor information. Now let's look at the external data sources. These are originating from outside the organization. They provide broader context, market trends, benchmarking opportunities. Examples include government data, social media, and market research reports. External data helps organizations understand their position in the broader market landscape. Common types of external sources include government and public sector database, social media and web data, market research reports, economic indicators, and financial markets data, weather and environment data. These diverse sources can provide a wealth of information to complement the internal data. External data offers several advantages. It provides broader context and benchmarking, allowing organizations to compare themselves to industry standards. It offers new perspectives and insights that might not be available internally. It can fill gaps in internal data, providing information on areas where the organization lacks data collection capabilities. However, external data has its limitations. There is less control over the data quality and collection methods. It may require significant processing or integration to be useful with internal data. There are potential costs associated with acquiring external data, especially from specialized providers. Integrating internal and external data can be powerful. Here are some use cases for you. For market analysis, combining internal sales data with external market trends provide a better analysis. In risk management, use internal financial data alongside economic indicators which are external to the organization. For customer segmentation, merge the internal customer data with external demographic information. In supply chain optimization, internal inventory data with external weather or traffic data. These combinations provide a more comprehensive view for decision making. To recap, the internal data is generated within the organization and is highly relevant but potentially

limited. External data comes from outside sources providing broader context but less control. Both types have their own advantages and limitations. The key considerations when working with these data sources include high data quality, relevance, integration challenges, and ethical use. Thank you for your attention. This concludes our module on types of data and data sources. Have a good day.

Recording of Introduction to Data Analytics: Week 1 - Live Session on 26-06-06 Video• . Duration: 1 hour 1 minute 1h 1m

[Notes]

you OK, so good evening all and welcome to the first session of our introduction to data analytics. Yes, so in this session we are going to see it is actually based and designed to equip with the essential skills for the dynamic field of data analytics. So here you are going to see about the introduction to data analytics, exploring the key components and applications of data analytics across the industries. So here you are going to learn the basic Python fundamentals. OK, and explore the fundamentals of data structures using your NumPy libraries and with the powerful which provides you the this NumPy libraries will provide you with the powerful tools for your numerical computations and all. So you are using the Python pandas for creating and compiling the visualizations. And using visualizations and data cleaning and preparation techniques. And also in the first five modules. In the first five modules, we'll be studying the feature engineering techniques like feature selection. OK, and dimensionality reduction, extraction, feature extractions to create the like training data sets for building the analytical models. And you're going to study several popular analytical techniques for regression, classification and clustering and all. And also. And also in regression techniques, you will be studying about the linear regression and nonlinear regression and fitting criteria. And in the classification, you will be studying about the binary classification, multilabel classifications using the decision trees. And in the rule based techniques, you will be with the parameter based model as well as the logistic regressions. OK, and in the clustering techniques, you have the partitional and hierarchical clustering. And the quality of clustering will be validated with the cluster validations techniques, which include the external and internal external and internal indices. And the final module of the course, you will discuss about the privacy, security and the fairness aspects of the data analytics. And we have the privacy preserving data mining techniques, algorithm bias and fairness and adversarial data mining and all. Then we'll be having the cost sensitive classifications and learning adversarial and robust data mining and all you will be learning in this. So at the end of the course, you'll be able to perform end to end data analytics, ranging from or starting from the collection of the data, cleansing the data, analyzing the data, visualizing the data and using the data modeling techniques. Like you have regression classification and clustering. For this, you will be using the Python along with all the essential libraries like you have the NumPy, pandas, matplotlib, seaborn and segit. OK, so all these are using you will be able to have a knowledge, basic knowledge of all this. So. Oh, yeah, you must be aware by this time, you must be aware of the like evaluation process. So you have you will be having graded quizzes after every two sessions. You might you would be having one graded quiz. Then you will have the graded assignments. So how does this Instagram know what is my like? OK, what, what, what suggestion to be given to me? So basically all these decisions, they are made using one concept, which is called as data analytics. OK, now coming to the point, then what is this data analytics is? So data

analytics is nothing but the process of examining the data set to draw conclusions about the information they contain. OK, it is going to try to extract or examine the data set, what we have, what is there and try to draw some conclusions from the information what it has. OK, so the objective of data analytics is to transform the raw data into actionable intelligence so that it can guide the decision making in various fields. So the motto of your data analytics is take the raw data, turn the raw data into some useful information. OK, again, some insight from this and take a smart decision out of it. So this data analytics, it comprises of basically like, for example, if I give if I consider the marks of a student. Right. So you will let us suppose I have students and I have marks of the students. Like first of all, I have a student number eight. He scored, let us suppose 90 marks. I have another student B and he scored 70 marks. I have another student C. Let us suppose they scored 50 marks. OK, so what is the data here? I have data is nothing but the marks that are secured by the students. That is the data what I have. So as it is, this data is called as a raw data. What do you mean by the raw data here? I'm not able to come to any. I'm not understand anything. OK, so they just like figures. They are just 90, 70, 50 are just like raw figures. They are numbers. But it is a real data is nothing but raw data is nothing but the real data. It is happened. OK, it is genuine data, real data. But if you look at the data, it is nothing. You are not inferred. You're not inferring anything out of the thing what you are seeing. So what do you have to extract from it? When you process this data, you get some information. OK, how do you want to process this data? Maybe I want to find the average mark. OK, so that I can get the class average from this marks. OK, so now again, this information, it depends upon the perception of the user. How the user wants to use this raw data. Right. So this is the raw form of data. How do I want to use this data is up to the perception of the user. Now here in this case, I want to see what is the class average. OK, if somebody X wants to view this data, he wants to process this data and to analyze who is the topper of this data. Or somebody X might want to see this data, process this data to check how many have scored above 80 percent of the marks like this. OK, so it is purely depends upon the perception of the user. How do they want to analyze the data? So what is the information I get from this data after processing it? Maybe I find the average of the marks. OK, what is the insight from this? So insight is the class performance is moderate. After calculating the averages from these marks, I come to one insight. I get one knowledge that the performance class performance is moderate. So what is the smart decisions you are taking? You may conduct remedial classes. OK, we may conduct remedial classes for fixing some percentage of students for students who are securing this percentage. But then I'll conduct a remedial class. OK, so like that, a decision can be taken based upon the information I get and the like insight I get from the information. Like another example, if I want to give you, we have an online shopping example. Right. So now in the online shopping, what would be the data? The purchase history of the customers. OK, so the purchase history now in the online shopping. First of all, I have my website. So here, if I check the data, what I have, I have all the list of products that have been ordered, that have been processed, that have been delivered, that have been like under queue, whatsoever. So all the information I will be having that is called as a data. That is called the purchase history of the customers, which is the data. Means data is nothing but a raw fact. OK, data is a raw. Why I'm telling it raw, raw, raw again? Because it is not processed. As it is, if you want to look at the data, you get no information, you get no knowledge from the data. So that is why it is called as a raw fact. Now, OK, so in the online shopping, I get the purchase history of the customers, which is the data. Now, what is the information I get? Maybe I may analyze that most purchased products. OK, so when I read the data, I understand what is the most purchased products by the customers. OK, now what is the insight I have from this? The insight what I have

from this is customers buy mobile phones, most of the mobile phones during festivals. OK, during festivals. So what is the decision I can take? A wise decision that would be OK. Because in the trend, what I observe from the data, I understood that. During the festival season. Customers are buying more mobile phones. So what should I do as a company person? I should see that there is a stock in my shelf that I am satisfying my customer. So I'll increase the stock. I will increase the stock based upon the customer's requirement. OK, so that is what I have. OK, we so we take the raw data. We process the raw data to get some information. And based upon the information, we have some insights and we take a smart decision. OK, now, why does why this data analytics is important then? So without analytics decisions, they are just based on assumptions. OK, so if you're not doing analytics, so you may think we may take we may go wrong in taking some decisions sometimes. So big decisions you may take because there is no solid proof. There is no concrete analysis of the data. Right. You are going to have assumptions. Maybe a customer may take more mobile. Customer may buy this product. They may not be willing to have this. So that is the assumption based decisions would be. OK, so here when you have assumption, when you go by assumption, the risk is you are the chances of you making mistakes may be more when you go by assumptions. The same thing when you do it with analytics. OK, when you apply some analytics, then you have evidence based. OK, so I have taken this decision of improving the stock. OK, I have taken the decision of improving the stock of mobile. based upon the evidence. What is the evidence? For the past five years, every year during the time of Diwali, let us suppose, during the time of Diwali, there is an increase of 40% of sales in the mobile phones. Okay. Maybe, whatever may be the brand and the model and all, but cell phone sales are increased by 40% or 30% by analyzing the past. Now, analyzing the data doesn't mean that you are just inquiring with the one week data, 10 days data or one month data. No. Okay. We will try to study huge amount of data. Maybe, let us suppose a small example of five years of data. Okay. So, in the five years of data, when I analysis, I found a similar pattern that for every Diwali, the sales of mobile phones is been increasing by on an average of 40%. Then I will take a decision. Okay. What is that? That I will, okay, let me increase the sales, increase the stock, so that automatically the sales also will be increased because there is a demand for that particular product. Okay. So, what is the result of this? You will have a better business outcomes of it. Okay. You will have a better business outcomes. Like that, like to say simple example, you have this Netflix recommends you with the movies. Okay. When I watch one type of movie, let us suppose some patriotic movie, if I'm watching. Next time it will give me in the list of movies. It will suggest me, maybe you see one line there. Maybe you like these movies. Okay. It will suggest you some four or five movies. If I watch some family comedy movie, it will try to suggest me that type of movies. It's one, two, three, four in the list. Okay. Like again, in the example to say is your Google maps. Okay. So Google maps also suggest use the fastest routes. Yeah. Right. So this is benefits of using the data analytics and not using the data analytics. Okay. Now, what is data analytics? It comprises of the four key elements. Now, what are the four key elements of your data analytics? That is data collection, data processing, data analysis, and interpretation of data. That is data collection, processing the data, analysis of the data, visualizing the data and making the correct decision out of that data. That is data collection means, it will gather the relevant information from various sources. Okay. And then you process the data means once the collection is done, the collected data need to be cleaned and organized. Okay. Then you have the analysis part where we apply various techniques to uncover the patterns and insights. Then we have the interpretation of the result. Interpretation of the results can be like, you can have various types of visualizing the data or translate findings into actionable

recommendations. So the ultimate goal of your data analytics is to extract the meaningful insights from the raw data. And this will support the decision making process. So these insights what you have, that will help us to take a wise decision making process. By achieving these goals, data analytics helps organization make informed decisions based on concrete evidence rather than intrusions or guesswork. Like for example, if you're talking about the college attendance system, let us suppose. Okay. So in the college attendance system, what is the collection of the data? We collect all the attendance records. Okay. The next step is what? Collection will be having all collection of all records. Okay. Then, then you have the cleaning of the data. Now, and then after collection, you process the data. Okay. Processing is nothing but cleaning the data. Now, what do you mean by cleaning of the data? Cleaning means unwanted data, right? Or here in this example, records of your students. Now, you may say duplicate records. Okay. Duplicates, you may, what do you say? You may ignore them or you may clear them or you may delete them. Okay. So that is called as, you can call it as cleaning of the data. Then we have, you clean it and you process it. Then you analyze it. What is the analysis you do? You calculate the attendance percentage. Okay. See, for suppose, if you want to write for you also, if you want to write the examinations and the end time exams, there will be some criteria that attendance should be minimum of 70%. So if you're not having 70% of attendance, you're not eligible to write the examination. So I take all your biometric attendance and remove all the duplicates. Maybe somebody is giving two times attendance, three times attendance at the same time. Then I'll cleanse the data. Means I'll remove all the duplicates and then I try to process the data that is removing duplicates. And then I try to analyze the data by calculating the attendance percentage of each student. After calculating the attendance percentage of each student, I may go for the visualizing of the data. May take a bar graph of the percentages to check maybe, to check in which month I have full attendance, at what time of the month I have very low attendance, low attendance, or on what time of the month I have the average attendance. So I like that I can, if I visualize it, it will be more easy for me to understand and grab the information. So like that, we will visualize using the bar graph for your like line graph or any histogram or anything, depending upon the data, what we are considering. Then what is the ultimate decision you can take? We can make it out of it. And that is, who are the students who are to be detained? And what is the percentage of the students that are getting detained? Okay. So I can understand that. Right? So the ultimate goal of your data analytics is to transform or to extract the meaningful insight from the raw data and to support the decision making process. So by achieving these goals, our data analytics, it helped the organization to make a informed decision based upon a concrete evidence rather than going on a guesswork. Right. Now let us see. There are three main types of data we have. What is there? We have the structured data, we have unstructured data and semi-structured data. Right? So what is this structured data? Structured data is highly organized data. They're typically found in spreadsheets, typically found in spreadsheets or in the databases. So here we store the data in the form of rows and columns. Okay. We store the data in the form of rows and column. Like you see, for example, row number 101. Name is Ajay. Date is 10th June. Something. Absent or present. Present. Okay. So we have rows and columns. So this is a structured data. Like you have, you can consider it as an Excel sheets or your databases and all. Next type of data, what we have is an unstructured data. So this unstructured data, it doesn't have any fixed format. Okay. It's more complex to analyze, but often it contains a very important or very rich information. Okay. It contains rich information. See, a student's handwritten assignment photo. Basically these unstructured data, they include text, they include audios, they include videos, they include emails, images. So this unstructured data is combination of all this. So for example, if I say a student's

handwritten assignment photo or a video of a classroom lecture or a paragraph of feedback. Okay. So all this, we can consider it as an unstructured data. Then we have the third type of data we have is the semi-structured data. So what is this semi-structured data? So this semi-structured is in between as the name itself suggests, is in between your unstructured and structured data. Okay. Like you have your JSON files, JSON files, XML files, and all. So like for example, if you can see here, JSON structure file you have, name is Ravi, role number is 101, subjects are math, science, English, like this. So these JSON files, you can consider it as a semi-structured data, which is in between your structured and unstructured data. Okay. So what are the tools and technologies that are used in your data analytics? So the spreadsheet software, like you have Excel or your Google Sheets can be used for the basic data manipulations and statistical softwares like your R, your SAS, they are more advanced statistical computations. If you have more advanced statistical computations, then we use R programming or SAS. Then for the SQL, for powerful data processing, we use SQL. Okay. Or we can use the Python programming languages and all. And for business intelligence tools, we use Tableau and Power BI, which are used for creating the visualizations as well as for the powerful dashboards. So data analytic process, it typically follows three steps. What are they? Definition, like what you want to learn from the data. Second is collect the relevant data, rather than gather the data from the appropriate sources. First step is what? You have to understand actually what you want to learn from the data. What type of data you want to collect. When I want to know the attendance of the student, I need not have the marks of the student, right? What type of data I should have? I should have the data of the students. How much attendance is having in maths? How much attendance is having in physics? How much is having in computers? Like in different subjects, I want attendance rather than marks. So first you should understand what is the data you want to collect. Okay. Now, after understanding and after knowing what exactly you want, then you start collecting the data. Now, what is this collecting the data means? Like for suppose, if you are having five subjects, five subjects teachers will be taking the attendance, right? Now, as an admin, I have to pull the data from all these five subject teachers and then do the cleansing, right? Means that is called as gather the data from the appropriate sources. Now, what is the source I want? I have different faculty for different subjects. Okay. What is appropriate date? I don't want the marks. I want the attendance. That is the appropriate data what I want. Then third is what? You have first thing, you have understood what you want to collect. Next is you have identified what is the source from where you want to collect the data. And third is you have to cleanse the data. So cleansing the data means we are ensuring the data is accurate and it is usable the date. It is in the usable format. Okay. So you are collecting the exact data and it is in a usable format, right? Next is we have to analyze the data. So analyzing the data after collecting all the attendance from various subject teachers. Now, after removing all that maybe duplicate records or maybe the records which is not having values like that or removing all the cleansing the data. Now I go for analyzing the data. So to analyze the data, we apply various analytical techniques. Okay. Now why you are applying analytical techniques? Now I want to see what is the percentage, what is the percentage of each student in each subject and similarly in the overall all subjects. Is he meeting the criteria or not? Is he eligible? That is after analyzing, the fifth one is you're going to take the, interpret the results and take a wise decision, right? So that is, I will check after finding all, uncovering all the patterns and all, then I will. take a wise decision, okay, that is interpret the results and translate the findings into actionable recommendations. So, what is the recommendations? You may say, okay, this student, the students who are, this is the list of students who are not having attendance above 60 percent or who are not meeting the criteria of attending for the next level. So,

like that we will have. So, these are the steps that are involved in data analytical process, okay. Now, what are the techniques that are used? So, to excel in the data analytics, one needs to develop diverse skill sets, like you should have the statistical knowledge, you have the programming and data manipulation skills, data visualization abilities, like using maybe using your Python or using your Tableau and all Power BI Tableau and all, then critical thinking and problem solving capabilities. What are the key components of data analytics? So, data analytics, it comprises of five main components. Data analytics comprises of five main components. They are data collection is the first one. Okay, so the first component is data collection. So, this involves gathering of the raw data from various sources. So, methods of collections, you can use the survey method, or from the sensors, or from the web scrapping and databases, maybe anything, okay. So, these some of the collection sources, it is crucial to ensure that the data collected is appropriate for the analysis we want to perform. So, the data what you are collecting should be appropriate to the analysis for which we are trying to collect the data. Then the next step is processing. So, once the data is collected, it needs to be processed, means you are preparing the data, the raw data for the analysis. So, this step includes three main activities, what is there, that is data cleaning, data integration and data transformation. Now, what is data cleaning here, handling the missing data, removing the duplicates and addressing the errors and inconsistencies in the data. See, for example, if the student's name appeared twice, or marks are missing, we correct it, okay. So, if the same student names is appearing twice, we check on that. If the marks of any particular students is missing, we check on that, okay, that is data cleaning. Then next, you have data integration. So, this all in under data collection process only, that is processing of the data. Now, data integration is, we combine the data from multiple sources to create a uniform data set. Like for example, attendance data from one system, plus marks data from another system, you combine it to one table, you can name it as a data, student data set, okay. Maybe I have a student name and student roll number from, or name from my admin record, okay. I have marks from my teacher's record, attendance from my coordinator record. So, I have to pull everything and collect it to one place. Then you have the data transformation. Now, what do you mean by data transformation? So, you convert the data into suitable format for analysis, like which, like text category into numerical category, okay. So, the format which is easy for analysis, to that format you are going to transform the data. So, your data processing is three, what is the data cleaning, data integration and data transformation. So, the overall goal of your data processing is to create a clean, consistent and analysis ready data set. Next step is data analysis. So, data analyst is the heart of the analytical process. So, here you are applying various techniques to derive insights from the data for helping analyst to take a, or manager to take a wise decision. So, this step involved in data analysis include your explorative data analysis. So, the initial investigation of data to discover the patterns, spot the anomalies and check the assumptions. So, this is very crucial step, okay. So, in this step you are going to check whether what you are going to check for the patterns, okay. You are going to check does the data has any anomalies, okay, or any assumptions are given in the data. So, you are going to spot all this in the data analysis step. Then you have one part which is called a statistical analysis. So, in this statistical analysis both the descriptive statistics to which you are used to summarize the data and inferential statistics is used to make the predictions. So, the predictive modeling and the pattern recognition where we use machine learning techniques to identify the patterns and to make a predictions based on the data. The next step after processing we have the data visualization. What is this data visualization? So, the data visualization is crucial component. It involves a graphical representation of the data and insights. The main purpose of your visualization is to make it, make a

complex data more accessible, understandable and see generally when you, whenever we create any report it, the reports would be some 20 pages, 30 pages, all statistics, all figures, all day, all number of columns, number of, see if I submit that report as this to a manager it may be very difficult for him to understand what actually the data is going to tell me. At the same time if I put that all in a format of a graph, maybe a line graph or a box graph, okay, it is, it will be very easy to understand, okay, during this month this is the hype, during this month the sales are falling down, okay, during this month the sales are very moderate. So, like that manager can take a wise, can easily understand the data what we are trying to present. Now, effective data visualization can reveal, yeah, it can reveal the trend, it can reveal the patterns and outliers that may not be apparent. Next, you have the data interpretation finally. So, this step involves drawing meaningful insights and conclusions from the visualized data. So, carefully examine the charts, the graphs and the plots. After examining them, we'll try to identify the trends, patterns and anomalies and outliers. Note the differences between the group, categories or time periods, then compare the visual findings with your original research questions and hypothesis or the business objectives and then we determine whether the data supports, contradicts or whether it will support our assumptions or it will contradict our assumptions. For example, if I have sales increased in the second quarter across all regions, but in west region it grew 40 percent more than the average. So, the question would be what does this trend mean? What do you mean by this trend? Why might this change have occurred or what external or internal factors could explain the observations? So, your thought processing would be that after seeing the results. See, the next is the visualization. So, here if you can see the graph, a line chart showing monthly revenue over a year. So, over a year what is the monthly revenue? So, by looking at this graph, you can see there is one line starting from this low range, it is reached to the peak at this level, then again there is a fall, downfall, then again the graph is slowly increasing. So, what do you understand by this? So, we infer that revenue is increasing steadily from January to June, slowly it was increasing the revenue from January to June, then followed by a sharp drop. So, there was sudden drop, sharp drop from June to July. So, this may correlate with a discontinued product or the seasonal change in the demand. So, further what is the conclusion? We say that further investigation is needed over here. So, for example, let me give you very simple example with this four steps what we have. Imagine a college canteen, it wants to decide whether food items to keep in the menu or not. There is a college canteen which is serving some snacks, items and all. So, it wants to decide, the manager, canteen manager or management wants to decide whether to have some ABC products in the menu or not. Then they start collecting the data, what is that, like how many students are buying the food items every day, which item most of the students are buying every day. Now, how do you collect, which item they are liking the most. So, maybe from the primary source or maybe by distributing the feedback forms or I can collect the data from the sales report I have, but from the billing section. So, I will collect the data. After collecting the data, we'll process the data, means the data is clean, the data is organized, means you are removing the duplicate entries. For example, you are collecting the data from the sales as well as from the, like what do you say, from the students food item list also. So, duplicates I will try to remove. Okay, and correct, I'll check with the spelling mistakes, maybe samosa is like mistaken with the sandwiches, like that I'll try to identify the anomalies and try to remove all that and arrange the data in a tabular form. Okay, like for suppose I'm every day I'm selling 120 students, I analyze the 120 students are buying samosas, 30 students are buying sandwiches, 10 students are buying salads. Okay, so like this, I'll form a table of this. Okay, what is the item, what is the sales, daily sales. The next step is what we'll analyze the data. So, we look for the patterns. So, samosas are selling the most. This is a

very simple example with a simple count. Okay, so by 120 sales, we understand that samosas are sold the most, then sandwich sales are moderate with 30. Then what is the least sale that is with the salads, salads are sold very less. Okay, now what is the next step we have, we have the interpretation of the analysis, what we have done, and we have to take the best decision so that we make profit in the business, we have the growth in the business. So, we use the insights to make a decision. Okay, now what is the insights, because the, okay, right. So, so, so, because samosas are being sold more, right, we keep the samosas. Okay, so we try to improve or we try to promote the sandwiches. Okay, and we remove or we try to replace the salads by some other menu, because salads are very least sold. Okay, so we try to replace the salads. So, this is the steps what we have. Okay, so that is the, what do you say processing steps what we have, then what are the types of analytics we have. Okay. So, we have the first type of analytics is the descriptive analytics. So, this descriptive analytics is the process of analyzing historical data. understand what happens or what happened in the past using the summaries like you may use the charts, you may use the graphs, you may use the statistics. Okay, so it answers the question what exactly happened? Okay, so what is the question answered in the descriptive analytics? What happened? Okay, so this type of analytics help us to make sense of historical data and provides a basis for the further analysis. Okay, so means here with the descriptive analytics, you are going to answer the question what happened in the past? Okay, historical data. So what are the key characteristics of descriptive analytics? You have the retrospective means it looks at the past data. Retrospective means it looks at the past data. Then you have the data aggregation and summarization. So here it will condense the large amount of data into some manageable summaries. Then you have the pattern recognition in historical data means you are trying to identify the trends and the patterns that have occurred over the time. So what are the key techniques you have? So in the descriptive analytics, it employs several key techniques like you have like data aggregation first one means data aggregation is you are combining data from multiple sources. Data mining is you are discovering pattern in large data sets. See data mining the word mining itself is you are you are dealing with as I told you to analyze the data and to understand the pattern. You're not just using a small chunk of data, right? You will be using large bulk of data, right? Means maybe the data of five years history or 10 years or 15 years like that. That is why the word mining. Okay, that is why we say data mining. Okay, means we are trying to discover the patterns in large data chunks or data sets. Then you have a summary statistics means calculating measures that describe the data. Then you have data visualization. So in this we are creating some graphical representations of the data which will you have the visualization. Then you have the statistic measures of central tendency like you have mean median and mode. Okay, and also the measure of dispersion sorry measure of dispersion like you have range, you can have variance, you calculate the standard deviation, standard percentile and quartiles and all. So all these statistics, they, they provide a numerical summary of the data. And this will help us to describe its main features. Okay, the summary of statistical summary what we are going to have, they will give us the or describe the main features of the data. Okay, then what are the applications of it? So descriptive analytics has numerous applications, you can have the business reporting and dashboards, revenue analysis, customer segmentation, you have inventory management also used in social media metrics. Okay, let us see an example of your retail sales example, wherein this descriptive analytics techniques is used. So first step you have with the data aggregation. So here we'll combine the data from all the stores or branches and to see the total monthly sales. Let us suppose we are talking about retail sales, right? So total monthly sales, I want to check monthly sales from various stores, I have various branches I'm having, let us

take the month, then next step is we have data mining. So we try to find the patterns like what are the sales in the weekends. Okay, so we identify that weekends are having more sales, or people buy more jackets in December. Okay, like that, we try to find some patterns, some some similar, what do you say, same, like, same action being repeated. Okay, so that like pattern, then you have the summary statistics. Summary statistics means we find the average that is mean. So the average number of items that are sold per day, then we calculate the, we calculate the mean, then you calculate the median. Okay, median is the middle value of daily sales. Good when data is in the data is skewed. This, this will be advantageous when you have a skewed data means going down left towards left or towards right, then we'll calculate the mode. So the most commonly to identify the most commonly sold product like that. So you have the summary statistics, then you have data visualization. So here we will create the bar charts to show which product categories are sold the most. Okay, then you have measures of dispersion. So in this you find range you find standard deviation. Okay, so ranges difference between your highest and the lowest daily sales deviation, it will tell you how much daily sales are fluctuating. Okay, then you have the percentiles and quartiles. So this percentiles and quartiles, this will help to divide the customers or the products into groups like 25% are best selling, which product are not selling at all. So like that you can identify the sales. And then what are the limitations you have limitations means in the descriptive analytics also has a limitation means it does not help why the event has occurred, why there is a change in the sales, why there is a like sales are falling in the month of January, why the sales are very high in them during the festive season, so it is not able to answer the why point. Okay, so it has limited predictive capabilities, it may oversimplify the complex situations. So there is a potential for misinterpretation without the context. So that are the limitations, then you have the diagnostic analytics. So in the diagnostic analytics, we answer the question why it has happened. So in the descriptive, it was what about the past. Now, this diagnostic analysis, it will tell you why it has happened. Okay, means it will show a drop in shoe sale in the February. So diagnostic analytics will dig deeper into and reveal the reason why the sales of jackets or sales of shoes are dropping in the month of February, maybe there was a supply issue, or there was no promotions, promotions were not done proper, or new competitor has entered into the market during that period. Okay, so because of that the sales of that particular product were a drop. So while descriptive tells you what happened, this diagnostic analysis will tell you why it has happened. Then we have the predictive analysis. So this predictive analysis, it is based on it will understand the past events to forecasting the future ones. Okay, so it focuses on the forecasting future trends and behavior. This takes insight from what happened, why it happened to forecast what may happen next. Okay, so in the descriptive, what happened in the diagnostic, it is why it has happened now in the predictive, it will tell you what may happen next. Okay, so it like for example, on the based past data predictive analytics, it may forecast that we have seen 40% jacket sales are dropped in February, right. So jacket sales will rise, it may predict the jacket sales may rise in December, because sales have increased every winter over the past five years or over the past 10 years. So it may raise again in the month of December. So it uses the historical trends again, okay, it's not giving you any assumptions, it will study the previous data, historical trends, historical patterns, models, okay, and then it will predict the future outcomes, right. So it is basically focusing on two concepts that is the probability based as well as the pattern based. Probability based is it is forecasting which deals with likelihood, not the certainties. Okay, means there is a 70% chance that the jacket sales will go up in December. Pattern based is a prediction which uses historical patterns to predict the future outcomes. Now, here you may say that every December for the past five years, jacket sales have rose. So this December also, it may happen

again. So that is a prediction. So here you have the continuous model refinement. So model regularly updated with the new data. So new sales data outcome each week. So the system will update the prediction. So we say accurate focus on. Then you have the prescriptive analytics, next analytics, you have the prescriptive analytics. So in this, it uses the advanced analytics techniques to recommend the best course of action, like prescriptive analytics answer the question, what should we do? Okay. So in the retail sales, if jacket sales are expected to rise in December, the pressure by using the predictive analytics, the prescriptive analytics recommends increase the jacket stock by 30% or offer a 10% discount in the second week or focus ads in the colder regions. Okay. Now, by using the predictive, you are predicting that in the month of December, you may have the high raise in the jacket sale. Now to have the raise in the jacket sale, what should I do? What is the prescription as the doctor gives you a prescription, right? So here also, and this analyst will give the prescription. What should be the prescriptive analytics? So prescriptive analytics will recommend us, okay, increase the stock of jacket to meet the demand or offer a discount before the sales of December. Okay. Or focus on the cold regions where the sales are very, very low. Right. So it suggests the best course of action using the optimization simulation or AI helping business make decisions. So you have what happened first one, then why it has happened? What may happen? What should we do? We do. Okay. Now that is the examples we have. So challenges faced in the data analytics, you have like quality, data quality and integration issues. You have privacy issues, security concerns, data volume and complexity. So skill gap and talent acquisition, integration of data sources, interpreting and communicating the results as well as the ethical consideration. These are all would be the challenges for your data analytics. Same thing like data types and data sources, data types we have seen structured data, unstructured data and semi-structured data. Data sources here you have quantitative data, qualitative data and primary data and secondary data source. So in the quantitative data, you have the numeric data that can be measured or counted. Okay. Quantity. Okay. It is fixed means which can be countable. Quantity means which can be counted. See like there are 50 students in a class. So you can count 1, 2, 3, 4, 5, so on to 50. Right. Similarly, sales revenue is 5 lakhs. You can count finance. Then you have qualitative data. Qualitative data is a descriptive data that describes the quality or a character. You cannot count it. Okay. You cannot count it. So like customer feedback, the product is excellent. Can you count this excellent? Not possible, right? Yes, I am satisfied. Can you count the measure? Can do you have any measure for the satisfaction? No, it is not measurable. That is qualitative data. Next is the primary data source. Now primary data source means the data that is collected in the first hand. Right. So like first hand means if I am directly asking you a question and taking the answer, that is a first hand data. Survey responses, no middleman. Okay. Or interviews, observations, experimental results. So all these types of collection of data, it is called as a first hand data. Then the next type of data source you have is a secondary data source. Secondary data source is nothing but you are collecting the data by somebody else and you are using the data analysis. You are using it data for the data. Like as I told you, we use five years back data, 10 years back data, that 5, 15, 10 years are all collected by XYZ, somebody else. Now you are using that data. Okay. That is called a secondary data. Like example is you have government reports, you have research articles, your company financial statements. So all these are called as secondary data sources. Internal data source, another type of data is a data generated within the organization. Like data that is generated within the organization. You have employee reports, you have sales data of that organization, you have inventory logs. So all this is within the organization. Now external data source means data collected from outside the organization. Like you have market research reports, social media trends, competitor analysis. So

all these are called as your like, what do you say, external data sources. So to end with, data is everywhere. Data analytics help us to convert that data into a meaningful decisions. So in upcoming classes, we will learn how with use of Python, NumPy, Pandas and visualization tool will help us to perform this analytical analytics practically, okay, right, yeah. So I'm going to check the chat box, and then you check your chat box, who's this? Hello? Who's speaking? Yes, ma'am, this is me. Can you please check your chat and send your message? TA, right? Yes, ma'am. You are the TA for this course, okay? Yes, ma'am. Yeah, yeah. Neil, I'll talk to you. Just give me one minute. In this session, anybody is having any doubt? No? Okay.

Python Interpreter Video• . Duration: 6 minutes

[Notes]

Hello everyone. Welcome to our first video on the fundamentals of Python programming language. By the end of this video, you will have a solid understanding of Python and its key features. You will grasp the purpose and function of the Python interpreter and be able to use the Python interactive shell for basic operations. Additionally, you will recognize the features and benefits of Jupyter Notebook. These objectives will guide our exploration of Python and its interpreter. Python is a high-level interpreted programming language that was created by Gurdjieff van Rossum in 1991. It has gained immense popularity due to its simplicity and readability, making it an excellent choice for beginners and experienced programmers alike. Python's versatility is evident in its wide use across various fields, including data analytics, machine learning, web development, and more. Python boasts several key features that contribute to its popularity. Its ease of learning and use makes it accessible to newcomers. The language is supported by a large and active community, ensuring continuous development and support. Python offers an extensive library of modules and packages, enhancing its functionality. It is cross-platform compatible, supports multiple programming paradigms, and employs dynamic typing and automatic memory management, making it both flexible and powerful. In the realm of data analytics, Python shines with its powerful libraries. NumPy excels in computing numerical data, while Pandas is the go-to for data manipulation and analysis. For data visualization, Matplotlib and Seaborn offer robust capabilities. Skykit Learn provides comprehensive tools for machine learning tasks. Furthermore, Jupyter Notebook serves as an ideal environment for data exploration and analysis, integrating code, visualization, and documentation seamlessly. The Python interpreter is a crucial component that reads and executes the Python code. It acts as a bridge between your code and the computer's hardware, enabling immediate execution of the Python commands. When you write Python code, you save it in files with a py extension. To run these files, you generally use the command `python3 filename.py` in your terminal or command prompt. The Python Interactive Shell is a powerful tool for learning and experimentation. It allows you to enter Python commands and see the results immediately. This immediate feedback is invaluable for testing small code snippets and understanding Python concepts in real time. Starting the Python interpreter is pretty straightforward. You simply open the command line or the terminal and type `python` or `python3` as shown here. Once initiated, you will see the prompt. The prompt indicates that the interpreter is ready to accept and execute Python expressions. When you are done in Python interpreter, exiting is quite simple. You can either type `exit` or `quit` at the prompt. This command will close the interpreter and return you to your regular command prompt or the terminal. The Python interpreter includes a built-in help system. If you need

information about a specific function or an object, you can use the help function. By typing help object or function in brackets, you will receive detailed information about that particular function or object, making it easier to understand and use Python's features. Jupyter Notebook is a web-based interactive development environment that has revolutionized the way we work with Python, especially in data analysis. It combines live code execution with the ability to include visualization and narrative text. This combination makes it particularly suited for data analysis, scientific computing, and machine learning tasks where documentation and visual representation of data are crucial. Jupyter Notebook offers a range of features that enhance the coding experience. It allows the code execution in isolated cells, enabling you to run parts of the code independently. You can use the Markdown for rich text editing, adding explanations and documents to your code. Jupyter Notebook seamlessly integrates plots and visualization, making it easy to see your data analysis results immediately. Moreover, it supports multiple programming languages through kernels, extending its versatility beyond Python. To start using Jupyter Notebook, you launch it from your command line by typing Jupyter Notebook. This opens the Jupyter interface in your web browser. From there, you can create a new notebook or open an existing one. In a notebook, you write and execute codes in cells, seeing the output immediately below each cell. You can also add Markdown cells for documentation, creating a comprehensive interactive document that combines codes and explanations and results. To summarize, the Python interpreter is a powerful tool that executes Python code interactively. The interactive shell provides immediate feedback, making it excellent for learning and testing code snippets. Jupyter Notebook takes this interactivity further by providing a web-based environment that combines code execution, documentation, and visualization. These tools collectively make Python a powerful and accessible language for data analytics and various other applications. Thank you for your attention. Do try out the Python interpreter and the Jupyter Notebook. Have a good day.

Jupyter Python Video• . Duration: 5 minutes

[Notes]

Welcome, everyone. In this video, we will discuss more about Jupyter Notebooks. By the end of this video, you will have gained several important skills. You will be able to access Jupyter Notebook in Coursera Labs and navigate its interface with ease. You will learn how to create, run, and manage Jupyter Notebook cells effectively. We will also cover the basic features of Jupyter Notebook, ensuring you have a solid foundation. Lastly, you will understand how to write both single-line and multi-line comments in Python, an essential skill for code documentation. To access Jupyter Notebook in Coursera Labs, you will first need to log in into your Coursera account. Once logged in, navigate to the appropriate course module. There, you will find a Launch Lab button. Clicking on this button will open the Jupyter Notebook environment ready for you to start coding. The Jupyter Notebook interface is straightforward, yet powerful. The main area in the Notebook editor is where you write your code and see its output. In the top right corner, you will find the kernel and the notification area. This shows the status of your Notebook and provides important information about your current session. Creating a new Notebook is simple. Click on the New button in the file browser, then select Python 3 from the drop-down menu. This action will open a new Notebook in a separate tab ready for you to start coding. Understanding the structure of Jupyter Notebook is crucial. The basic units of a Notebook are cells. These can be of different types. Code cells for

writing Python code, markdown cells for formatted text, and raw nbconvert cells for special purposes. The active cell is always highlighted with a border indicating where you are currently working. Code cells are where you will write your Python code. You can type the code directly onto these cells. Once you have typed the code snippet to run a cell, you can either press Shift-Enter or click the Run button. After execution, the output of the code will appear below the cell, providing immediate feedback on your code's performance. Markdown cells allow you to include formatted text in your Notebook. To use a markdown cell, first change the cell type to markdown. Then use markdown syntax or HTML syntax for formatting your text. When you run the cell, the markdown will be rendered displaying your formatted text. Proper Notebook management is essential. To save your work, use Control-S or go to File, Save, and Checkpoint. If you want to rename your Notebook, simply click on the current Notebook name. This allows you to save your work locally or share it with others. The kernel is the computational engine running your code. You can restart the kernel by going to Kernel, Restart. If you want to restart and clear all outputs, choose Kernel, Restart and Clear Output. For a fresh start and full execution of the Notebook, select Kernel, Restart and Run All. These options give you control over the Notebook's execution state. Comments are crucial for code documentation. In Python, you can create a single line comment by using the hash symbol. Anything following the symbol on the same line will be treated as a comment and not executed as a code. This is useful for brief explanation or notes within your code. For longer comments spanning multiple lines, Python allows the use of triple codes. You can use either three single codes or three double codes to enclose your multi-line comment. This is useful for class documentation or for temporarily commenting out large blocks of code. When writing comments, it is important to follow best practices. Use comments to explain why certain code exists rather than what the code does. Always keep your comments up to date with code changes to avoid confusion. Use clear and concise language in your comments. Avoid writing obvious comments that don't add value. Your code should be self-explanatory wherever possible. To recap, Jupyter Notebooks are powerful tools that combine code, output, and documentation. Code cells are used for Python code, while markdown cells allow for formatted text, and effective notebook management includes regular saving and organization. In Python, comments can be either single line using hash or multi-line with triple codes. Good commenting practices significantly improve code readability and maintainability. Thank you for your attention during this introduction of Jupyter Notebook. I encourage you to practice using Jupyter Notebook and experiment with writing and commenting your work.

Input and Print Video• . Duration: 5 minutes

[Notes]

Welcome all. We will be discussing input and print functions in Python. By the end of this video, you will be able to use the input function to receive user input in Python, allowing your programs to interact with the users. You will also learn to utilize the print function for outputting information. We will explore various techniques for formatting output using the print function. Lastly, you will understand the importance of type conversion when dealing with user input, ensuring your programs handle the data correctly. The input function is a powerful tool that allows your program to receive user input. Its syntax is simple, input with an optional prompt inside the parenthesis. It's important to note that the input will always return a string, regardless of what is typed by the user. Let's look at an example. If we write name equals input, enter your name, and then print hello name, the program

will ask for the user's name and greet them personally as given by the code. As mentioned, the input is always returning a string. This means that if you need a number, you will have to convert the input. We can use functions like `int` or `float` for this purpose. In the example given, the program will ask for the user's age, convert it into an integer using type conversions, and then tell them how old they will be the next year. The `print` function is used to display output to the console. It is versatile and can print multiple items separated by commas. By default, it adds a new line at the end of each print statement. Let's look at an example. If we write `print hello learner`, it will display that greeting. We can also print variables, like `x equals 5`. This should be followed by `print the value of x equals, comma, and x`. This will display the value of `x` as stored in the variable. Python version 3.6 above introduced fstrings, which makes formatting output easy and readable. To use fstring, you prefix the string with `f` and then include variables in curly braces. Here is an example. If we have `name, Alice`, and `age is 30`, we can write `print fname is age, both a name and age in curly braces`. This will neatly format our output, inserting the values of `name` and `age` into the string according to the user's input. Sometimes you may want to print multiple lines of text. Python makes this easy with triple quotes. By enclosing your text in triple quotes, you can create multiple lines that preserve formatting. The example shown will print each phrase on a new line, creating a neat multi-line output. Now let's see how we can combine input and print to create interactive programs. Here is an example. We can write `name equals input, enter your name to get the user's name`. Then `age equals int input, enter your age to get their age`. Note that we are typecasting the entered input. Finally, we can use `print` using fstring. Inside brackets, `in 10 years, name in curly brackets will be again in curly braces, age plus 10 years old`, to give them a prediction about their future age. This creates a simple but interactive program. When using input and print, it is important to follow some best practices. Always use descriptive prompts for input so users know what to enter. Remember to validate and type convert user inputs to prevent errors. Use fstrings for readable string formatting whenever possible. Consider using `separation` and `end` parameters to print for custom output formatting when needed. These practices will make your code more robust and user-friendly. To recap, the `input` function is used to receive user input, and it always returns a string. The `print` function is to use display output to the console. Remember that type conversion is necessary when you need non-string input from users. We have explored various formatting options available for print, including fstrings and multiline printing. By combining input and print, we can create interactive programs that engage with the users. Thank you for your attention. I encourage you to practice using input and print to create your own interactive programming. Happy coding!

Indentations Video• . Duration: 5 minutes

[Notes]

Welcome all. In this video, we will be discussing indentation in Python. By the end of this video, you will be able to understand the importance of indentation in Python, use proper indentation to define code blocks, recognize and avoid common indentation errors, apply best practices for consistent indentation. Indentation in Python refers to white spaces at the beginning of the line of code. It is used to define the structure and scope of the code blocks, replacing the brackets and keywords used in other programming languages. This unique feature of Python contributes significantly to its clean and readable syntax. Indentation in Python is not just for aesthetics. It is syntactically significant. It defines code blocks such as if statements, loops, and functions. Proper indentation

improves code readability and enforces a consistent code structure. It is crucial to note that incorrect indentation in Python causes errors, unlike in many other languages where indentation is optional. In Python, indentation typically uses four spaces or one tab per indentation level. Consistency within the same block is crucial. You increase the indent for nested blocks and decrease to close a block. This system creates a visual hierarchy that reflects the logical structure of your code. Let's look at an example of indentation in an if statement. In this code, we set x to 10. The next line, if x greater than 5 colon, is followed by two indented lines, which form the if block. The last line is not indented, so it is outside the if block. This indentation clearly shows which lines of code are conditional and which are always executed. Here is another example for indentation in function definition. We define a function called greet with a parameter name. The two print statements are indented, indicating they are part of the function body. The last line calls the function with the argument Alice. This indentation clearly delineates the function's scope. This is another example that demonstrates indentation in a loop with a nested if statement. The for loop iterates three times. All code in the loop is indented. The if statement is inside the loop is further indented, creating a nested block. The last print statement is at the same level as the for loop, indicating it is outside the loop. New Python programmers may cause several common indentation errors. These include mixing tabs and spaces, using inconsistent indentation levels, forget to indent after a colon, and over or under indenting. The example code shows an indentation error caused by not indenting the line after the if statement. To fix indentation errors, use a consistent indentation method throughout your code, either spaces or tabs. Ensure that all lines in a block have the same indentation level. Always check for missing or extra indentation, especially after colons. Proper indentation significantly improves code readability and understandability. It helps in identifying the structure and flow of the program at a glance. This is especially important in collaborating coding environments and during code reviews, where others need to quickly understand your code's logic. For best practices, the Python style guide recommends using four spaces per indentation level. Consistency throughout your code base is key. Many IDEs offer visual indentation guides, which can be very helpful. Always review your code for correct indentation before running it to avoid unexpected behaviors or errors. To summarize, indentation is a fundamental aspect of Python that defines code structure. Consistent indentation is crucial for correct execution of your codes. like mixing tabs and spaces and using inconsistent indentation levels. The best practice is to use four spaces per indentation level. Remember, proper indentation not only ensures your code runs correctly, but also improves its readability and maintainability. Thank you for your attention. I encourage you to practice writing code with proper indentation to reinforce these important principles. Happy coding!

Lesson 1 Demo Video • Duration: 4 minutes

[Notes]

Greetings, Python beginners. Today, we will demonstrate all the code snippets we have covered in our lesson on Python basics. We will run these examples in Jupyter Notebook and examine their outputs together. Let us dive in. In lesson one, we were talking about input and print functions. The first line of code explains the input function. We have given the input function, enter your name. Let's execute and see how this works. It prompts for the user to enter a name. When I press enter, it prints, hello, Alice, the greeting and the name, what we had entered. In the next line of code, we are again asking for an input from the user. Here it is an age, which is going to be entered as a number.

So we are typecasting so that the entered string gets converted to int. Let's execute it and see the results. It did the addition correctly. That means it is getting casted to int and then printing the results. Now we did see some examples of printing, but we'll continue seeing the effect of printing with and without formats. The first line, that is line number three, is without any formatting. In line number four, we will execute and see the effect of formatting or using the F string. F name is age. The name and age value is getting replaced by what was there in the variable. To print multiple lines, we have to use triple quotes. To combine both input and print, let's execute this line and see the results for us. So we have even typecasted or type converted the value of age and then printed after 20 years what will be the age of Manav. The next video talked about indentation. That is the number of spaces that you have to add when we are using if statements or functions or loops. So let's see how the effect of if works. In this one, after colon, note down the space that we have added. And the third print line is outside the if loop because it does not have any space written there. This works. Now what if we do not provide the space? What will happen if the space is missed? It will give us an indentation error. Look for yourself that before the print, it is showing an expected intended block. So this gives an error. It does not get executed. Same goes for function. Inside function, we have given print statements with a space in front, which says that that is the code snippet or code block for the function. On executing, we get to see the results. Now let's slightly modify this. That is, we added for the first statement, for the first print statement, we added the intent. But for the second print statement, we didn't add. So what will be the result? Which statement will get executed first? Let's see. Of course, the first statements get executed first because that is printed before the line greet Alice is invoked. So here, note that this print statement, print welcome to Python, gets printed first. Then the function gets called. That is when hello and the name gets printed. So intents are very important in our Python programming. How about loops? How will it work? Now the same example that we saw in the session. With intents, it works beautifully well where two iterations are executed. Then you will see the middle iteration getting printed. Loop finished also. Loop finished is outside the for loop. Now, if we do not intend the for loop, the first line, the space is removed. It is again giving a wrong intent. So what will be the effect of this? We will get the indentation error as expected. As we wrap up, I encourage you to experiment with these files on your own. Happy coding and enjoy your journey into Python fundamentals.

Python Scalar Types Video• . Duration: 6 minutes

[Notes]

Welcome everyone. We will be discussing scalar types in Python. By the end of this video, you will be able to identify the main scalar types in Python, understand the characteristics of each scalar type, use scalar types in Python programs, perform basic operations with scalar types. What are scalar types? Scalar types in Python are the basic indivisible data types that hold a single value. The main scalar types we will be discussing today are integers, floating point numbers given by float, Boolean values specified by bool, and strings specified by str. These form the foundation data representation in Python. Integers in Python represent whole numbers, both positive and negative, without a decimal point. Example includes negative 5, 0, and 42. Here is a quick code example. If we assign x equals 10, then print its type, we will see it is an integer. Python supports various operations on integers. These include addition, subtraction, multiplication, all of which use the same standard mathematical symbols. But division with single slash represents floating point division. And

for integer division, we need to use double slashes. The modulus operation is represented using percentage symbol. And for exponentiation, you have to use double asterisk symbols. These operations allow you to perform a wide range of mathematical calculations. Floating point numbers or floats represent numbers with decimal point. They can also be expressed in scientific notations. Example includes 3.14, negative 0.001, and 2.5e to the power of negative 4. As a code example, if we assign y equals 3.14 and print its type, we will see that it is a float. Floating operations are similar to integer operations. But it is important to be aware of floating point precision issues. For example, sum of 0.1 and 0.2 does not exactly equal 0.3 due to how computers represent floats. This issue is normally referred to as floating point precision issues. We can work around this by using the round function. Booleans represent truth values in python. There are only two possible values, true and false, with T and F both in caps. They are often used in conditional statements and logical operations. If we create a variable, is python fun equals true and check its type, we will see that it is bool. Python provides several operations for bools. The and operator returns true if both operands are true. The or operator returns true if at least one operand is true. The not operator inverts the bool value. These operations are crucial for creating complex logical conditions in the programs. For example, true and true will always return false. Whereas, applying true or false will return true. Remember, and and or are the operators that we just discussed. Not true will display false. Strings in python are sequences of characters enclosed in single codes or double codes. They can represent text, numbers as text, or even empty text. Examples include hello, python, and 123 all in single or double codes. If we create a variable, say name equals alice, and check its type, we will see that it is a string. Python provides functions to convert between different scalar types. We use int brackets to convert to integer, float brackets for float, bool function for boolean, and str for string. This allows us to change the type of a value when needed for our computations. Let us look at some examples. The first line converts str to int. The second line converts str to float. The next one converts int to bool, and the fourth line converts int to str. To check the type of a value, we have been using the type function. In the earlier examples, we saw how to use type to check if each of the variables belongs to a particular scalar. If you want to check if a value is of specific type, we can use isinstance function. These are useful for ensuring our data is of expected type before performing operations. There are a few common pitfalls to be aware of when working with scalar types. Division of integers results in a float, not an int. Comparing floats for exact equality can become a problem due to precision issues. Lastly, remember to convert input strings to the appropriate type before performing numerical operations. To recap, the main scalar types in Python include int, float, bool, and str. Each type has specific characteristics and operations. We have also discussed type conversion and checking the type of value or a variable. Remember to be aware of potential issues like floating point precisions. Thank you for your attention. Happy coding.

Objects Video• . Duration: 4 minutes

[Notes]

Welcome, learners. We will be discussing objects in Python. By the end of this video, you will be able to understand the concept of objects in Python, recognize that everything is an object in Python, creating and manipulating objects. Objects in Python are instances of a class. They contain both data, which we will call as attributes, and code, which we call as methods. A crucial point to remember is that in Python, everything is an object. This principle underlies the entire language and

its functionality. To understand objects, we need to understand classes. A class is like a blueprint for creating objects. An object then is an instance of a class. For example, in the code, `name` equals `Alice`, `str` is a class, and `name` is an object of the `str` class. We can confirm this by printing the type of the `name`. Objects are created in Python when you assign values to variables or when you use class constructors. In this example, we define a book class with a constructor given by `init`, which takes two parameters, `title` and `author`. We then create an object called `MyBook` and print its title. Object attributes are data associated with an object. We access these attributes using dot notation. In our book class, the `title` and `author` are attributes. We create a book object, then access its title attribute using `MyBook.title`. Note the usage of dot notation to access the object attributes. Object methods are functions associated with an object. They are accessed using dot notation. Here we define a book class with `GetCitation` method. We create an object of the book class and call its `GetCitation` method, which returns a formatted string with the author's name and title. Python comes with many built-in objects. These include lists, dictionaries, strings, and integers. In this example, we create objects of these types. Understanding these built-in objects is crucial for effective Python programming. In Python, even functions and classes themselves are objects. This is a powerful feature of the language. In this example, we have defined a function `create` and then print its type. We see that it is an object of class `function`. Every object in Python has a unique identity. We can get this identity using the `id` function. In this example, we create two lists, `x` and `y`, with the same content. Despite having the same content, these list objects have different identities because they have separate memory. The print statement will return `false`. Objects in Python can be either mutable or immutable. Mutable objects like list and dictionary can be changed after creation. Immutable objects are string and tuple, which cannot be changed after creation. When working with objects, follow the practices so that the code is more readable and maintainable. Use meaningful names for objects and classes. Follow the `Pepe` style guide for naming conventions. Keep classes and objects focused on a single responsibility. Use doc strings to document classes and methods. To recap, everything in Python is an object. Objects have attributes and methods. Objects are instances of classes. Python has many built-in objects and we can create our own custom objects. Understanding objects is key to mastering Python programming. Thank you for your attention. I encourage you to practice creating and manipulating objects to reinforce what we have learned now. Happy coding!

Attributes Video• . Duration: 5 minutes

[Notes]

Welcome all. We will be discussing attributes in Python. By the end of this video, you will be able to understand what attributes are, distinguish between instances and class attributes, create access and modify attributes, use built-in functions to work with attributes. Attributes in Python are variables that belong to an object or class. They store data associated with the object or class and access using dot notation. For example, we can access an attribute of an object by writing `object.attribute`. This simple syntax allows us to interact with the data stored in our objects. There are two main types of attributes in Python. Instance attributes are specific to each instance of a class, meaning each object can have its own unique values for these attributes. Class attributes, on the other hand, are shared by all instances of the class. This distinction is very important for understanding how data is stored and accessed in Python objects. Instance attributes are defined inside the `init` method of the

class and are unique to each object instance. We access the instance attributes using dot notation. In the example given, we define a book class with title and author as instance attributes. When we create an object of the book class and print its title, we can see that each book can have its own unique title. Class attributes are defined outside any method in the class and are shared by all the instances of the classes. We access class attributes using dot notation. In the example given, we define a category class attribute for our book class. When we create two objects of the same class and print its category, we see that this attribute is shared among all book instances. Both print statements will display tragedy. Attributes can be modified after object creation, but we should be cautious when modifying class attributes. In the example, we change the title of a book after it has been created. This flexibility allows us to update our objects as needed, but it is important to do so carefully to maintain data integrity. Python provides several built-in functions for working with attributes. The `get attr` function gets the value of the attribute, `has attr` checks if an attribute exists, and `set attr` sets the value of an attribute. `Delete attr` deletes an attribute from the memory. These functions provide programmatic ways to interact with the attributes. Here is an example of using these built-in functions. We create a car class and a car object, then use `get attr` to get the make attribute from the object. `Has attribute` to check for a model attribute, `set attr` to add a model attribute, and finally `delete attr` to remove it from the memory. These functions give us powerful ways to manipulate object attributes. Property decorators in Python allow for getter, setter, and deleter methods, providing more control over attribute access. This is useful when you need to add logic to the getting or setting of an attribute values, such as validation or computation. Let's see an example for this. We define an instance attribute, price, for the book class. We define a getter method that returns the price. We also define a setter method that checks if the new price is negative before setting it. This allows us to enforce rules on how our attributes are accessed and modified. The attributes are accessed in the same way as instance attributes, that is, using the dot notation. When working with attributes, it is important to follow best practices. Use clear and descriptive attribute names to enhance readability. Initialize all instance attributes in an `init` method to ensure consistency. Use properties for controlled attribute access when needed, and try to avoid adding new attributes outside of the `init` to maintain predictable object structures. To recap, attributes store data in objects and classes. Instance attributes are specific to each object, while class attributes are shared among all the instances. We can use built-in functions and properties to gain more control over attributes. This is key to effective object-oriented programming in Python. Thank you for your attention. Happy coding!

Methods Video• . Duration: 4 minutes

[Notes]

Welcome learner, we will be discussing methods in Python. By the end of this video, you will be able to understand what methods are in Python, distinguish between instance, class and static methods, use and create methods in Python classes. Methods in Python are functions associated with a class. They are defined inside a class and operate on objects or the class itself. We access methods using dot notation, either as `object.method` or `class.method`. Methods are essential for defining the behavior of objects in object-oriented programming. In Python, we have three main types of objects. Instance methods, class methods and static methods. Each type serves a different purpose and it has its own unique characters. Instance methods are the most common type of methods that you

will see. They always have `self` as their first parameter, which represents that it is an instance of the class. These methods can access and modify instance attributes. In the example, we define a `book` class with an instance method, `getInfo`. This method is an instance method and it returns a string containing information about the book, accessing the instance attributes like `title`, `author` and `pages`. Class methods are marked with the `@classmethod` decorator. Their first parameter is always `CLS`, representing the class itself. These methods can access and modify class attributes. They are often used as alternate constructors. Let's see an example for this. We define a `book` class with a class attribute `numBooks` and two class methods `fromString` as an alternate constructor and `getTotalBooks` to return the total number of books. `fromString` splits the text using the text by and return title and author. It behaves like a constructor. `getTotalBooks` behaves like a getter method. Static methods are marked with the `@staticmethod` decorator. Unlike instances and class methods, they do not have access to `self` or `CLS`. They are typically used as utility functions related to the class. In the example, we define a static method `isLong` in the `book` class, which checks if a book is long based on the number of pages. Static method overriding allows us to define a method in a subclass customizing inherited behavior. Let's see an example. We define the `book` class with an `str` method. The `book` class is the superclass. Now we create another class `ebook`, which is a subclass. Now we will overwrite the `str` method. The `ebook`'s `str` method calls the parent's class method using `super` and adds additional information. This demonstrates how we can extend and customize inherited behavior. When working with methods, it is important to follow some best practices. Use clear and descriptive method names to enhance readability. Document methods focused on a single task to maintain simplicity and usability. Use appropriate method type, like instance, class, or static based on your needs. Document your methods with doc strings to explain the purpose and usage. Consider using properties for methods that behave like attributes. To recap, methods are functions associated with classes. We have explored three types of method. Instance method, which operate on object instances. Class methods, which operate on the class itself. And static methods, which are utility functions related to the class. Remember, good method design improves code organization and reusability, which are key principles in object-oriented programming. Thank you for your attention. Happy coding.

Operators Video• . Duration: 6 minutes

[Notes]

Welcome all to our session on operators in Python. By the end of this video, you will be able to understand what operators are in Python, identify different types of operators, use operators with built-in and custom objects, implement operator overloading in custom classes. Operators in Python are symbols that perform operations on operands. Essentially, they are built-in functions that work on data. One of the powerful features of Python is that you can overload these operators, allowing for custom behavior with user-defined classes. This flexibility is part of what makes Python so versatile and user-friendly. Python offers a wide variety of operators to work with. We have arithmetic operators for mathematical operations, comparison operators for testing equality and order, logical operators for Boolean operations. There are also assignment operators for variable assignment, bitwise operators for bit-level operations, membership operators for testing inclusion, and identity operators for object comparison. Each of these serves a specific purpose in Python programming. Let us look at arithmetic operators in more detail. These include addition, subtraction,

multiplication, division, integer division given by double slashes, modulus operation provided by percentage symbol, and exponentiation provided by double asterisk symbol. To illustrate their use, let us consider a book class. In this example, we have defined a custom add method that allows us to add the prices of two books. This demonstrates how we can use operator overloading to give meaning to the addition operator for our custom book classes. Moving on to comparison operators, we have equals with the double equal to symbol, not equals, less than, greater than, less than or equal to, greater than or equal to. In our book class example, we have implemented the LT method to compare books based on their page count. The EQ method is to check if two books are the same. This allows us to use comparison operators directly on our book objects, making our code more intuitive and readable. Logical operators in Python include and, or, and not. These are particularly useful for combining conditions. In our example, we use logical operators to check if a book is both in stock and affordable. This demonstrates how logical operators can be used to create complex conditions that control the flow of our program. Assignment operators are used to assign values to variables. Beyond the basic assignment, we have compound assignments like addition assignment, subtraction assignment, multiplication assignment, and so on. In our book example, we have implemented ink add method to allow us to increase the quantity of a book using the addition assignment operator. This shows how we can make our custom object work seamlessly with Python's assignment operators. Bitwise operators work on bit level of numbers. These include and, or, xor, not, left shift, and right shift. In our example, we have creatively used the and operator to find common categories between books. It will return common categories as Python and AI and demonstrates how bitwise operators can be repurposed for set operations when working with custom objects. The in and not in operators are used to test membership. In our book collection class, we use it to check if a book title is in the collection using the in operator. This makes our code more Pythonic and easier to read. Identity operators are is and is not. They are used to compare object identity. Our example shows the difference between identity comparison and equality comparison. Two books with the same title and page count are equal, but they are not the same object in memory. Operator overloading is a powerful feature in Python that allows us to define how operators work with our custom objects. We implement operator overloading using special methods, also known as magic methods. All the previous examples used magic methods. When working with operators, especially when overloading them, we always try to maintain intuitive behavior. For instance, the addition operator should either combine or add in a way that makes sense to our objects. Try to be consistent with how built-in types behave. And remember, while operator overloading is powerful, it shouldn't be abused. Use it when it genuinely makes your code clearer and more intuitive. To wrap up, we have learned that operators in Python perform operations on operands and come in various types, including arithmetic, comparison, logic, and more. We have seen how operator overloading allows us to define custom behavior for our user-defined classes. Remember, proper use of operators can make your code more intuitive and readable. Thank you for your attention. Happy coding.

Lesson 2 Demo Video• . Duration: 11 minutes

[Notes]

Welcome semantic explorers. We are about to bring to life the code examples from our Python semantics lesson. Using Jupyter Notebook, we will execute these snippets and analyze the results.

Shall we begin? First we will talk about the Python scalar types. We have four scalar types, int, float, string, and bool. Python does not require any kind of declaration, we just have to create the variable name and initialize with the values. The first two lines is for integer data type. When we print the type of the variable, it tells whether which scalar type it belongs to. So the first one x belongs to int because it's an integer number. 3.14 is a floating point number, so automatically it takes it as float class and the last one is given in double quotes. So it is a string belonging to str class. Some of the errors that we get during floating point operations. See the first one it's supposed to give me true, but because of the precision issues it will give false as expected. Whereas in the second one, since we are using the rounding of operation, it gives us true. What about bool data type? Now we have created a variable with the name is Python fun with all the underscores in between? To make it meaningful, we are going to print the type of the bool variable and look at some boolean operations. As expected, it gives the class bool for the type and then the results for the boolean operation and or and not. Now, to check whether it belongs to data type that we are or the scalar type that we are expecting, you can use isinstance. In this example, x is int data type. So both the print statements return true because both of them are belonging to int scalar type. The next video talked about objects, so let's create some object. Now the first example is creating an object of class str. So we create an object name and then assign it with a value, a string value here alice. And then we try to figure out what is the type of that object name. By using the type method, it returns str as expected. Now how to create our own classes? We were talking about the book class. We use the class keyword to create a class of book type. Then define the constructor, this is the constructor. And then we have a method Also inside the constructor we are getting two parameters which will be acting as the properties or the attributes of this book class. For us, it is title and author. Let's execute this. So the class is created. Now we can create objects of this particular class, right? We are going to create two objects, book one and book two on executing that is also working perfectly fine. Now how do we see the results? Like how do I know what is the title of book one? What is the author name of book one? So there we have to access using the dot operator. So book one title will return the name of the title of the book which is represented by the object. Book one Data Science Handbook. Now to access methods we have a method here, get_citation. It has to be through the objects. So we have to go for the method, book1.get_citation. Return f string. So here the values are printed by the print method. Now one more thing that we wanted to see was the object identity. Different objects say book three, but the data of the book, i.e. the title and the author is same as that of book two and see if they are same. So book two and book three have different identities. Let's just print the identity of book. So print statement with book two and print of book three. You can see that it is different. The objects are located at two different memory locations. That is why we are getting it as false. But if we compare the author names and the title names of each of the books, they will return true because they are one and the same. Continuing to the next video, we were talking about attributes. We just saw some examples. Authors and title as attributes of the class. But what about the class attribute? We have created a class attribute called category inside the class. Now accessing the class attribute through an object, it will be the same way. For instance, book object name dot the attribute name. Can we modify the object attributes? Yes, the object attributes can be modified by using the dot notation book title and give a different title name. It works. The error is because we have not created the book asset. So let's create the book object. Instead of changing the name, let's create a book object and then change the title. Now how will the built-in functions like getattr, setattr work? We will create a different class, this time called car to execute or to see the effects of those built in functions. So we create a class car which has only one attribute, now make. Then we create

an object of that particular class which we call as my car. So let's see the effect of this particular line. What does it print? It prints the object ID where it is getting stored. Now to access the attribute right we have to use get attribute and the attribute name. It gives the results of Toyota, which is what we have mentioned. Now if we check whether it has a different attribute like make the same attribute that we have mentioned, it should return True because the class has that particular object. What if we try to create a new attribute? Can we do that? Here, using set method, setattr method, we can do that. We are going to create a new attribute called model and pass on the value that it is supposed to take in and execute the statement. So it creates a new attribute in the class car. Now if we check whether it has the attribute, it will return true because now we have created it, it is available in the class to work with. Let's delete the attribute model, the one that we created just now by using delattr. Now let's try to print back to check whether the attribute is existing or not. So the same attribute model we are going to check using the hasattr method if it exists in the system or not. So on executing we can see that it is false. That means the attribute got deleted successfully. We also have some property decorators. So for the book class, we are going to define two property methods, one getter and one setter. The getter method will just extract the value of the price from the book class and setter method will check. It will check the validation rule whether the price can be negative or not. So we are going to create a book called Data Analytics Basics 29.99 as the price and we will print the results it executes. Now what if we create the book price so as a negative value? So I'll just put it as negative and see the results for ourselves. It returns an error what we had given here, which is called as value error. It says that price cannot be negative. Now we will create another method called get info to return all the values in a specific order. Let's execute this particular line and see how it works. It says that data science book by the author with the number of pages the way we have written in the f string statement. Now how about class methods? Remember we said that the class method should have the key word as at the rate class method. So we have created two class methods from string and get total books. On executing this, it will start creating books. Now let's check whether we can create one more book. See the counter increments because this is a class method. So depending on the total number of books, it just increments the count number of books as it is a global or it belongs to the class not belonging to the object. Now how to do overriding of methods? We need to define two classes, the superclass and the subclass. So here the superclass is book and subclass is ebook. The superclass has str method which the subclass is going to override. Now, to see the result of this, we need to create two books. One by the superclass value we will change the name. The first line talks about creating an object of book class. When we print, it just prints the author name. And when we create the second one it prints only the title. It does not print anything else. The getter method is defined using the name of the attribute itself. The setter method is again defined by the name of the attribute itself. The only difference is we will have a parameters where we give the value. Moving on we will go to Video 4 where we were talking about operators and how to do operator overloading. Now this is how we specify the markdown. If I double click on that particular cell, you can see that it is a markdown. And I've already mentioned this as markdown here. We are using two hash symbols to say that HTML heading 2 tag has to be used and this is using HTML4 tag. And when we run this it shows in the proper way. We are going to overload the operators add, lt, and eq or overload those methods appropriately. Where we are going to when we say add, it is going to add the price of the two books. When we are going to use the less than or equal to simple, it is going to compare the pages for less than and for equal. It will compare the title and the pages together. Now once we create the class, we can create the objects and do the comparison. How about logical operators? For logical operators we are going to specify them in

the if statement. We are going to compare whether the books are available in the stock and what is the price if it is if the price is less than 50 and the book is available in stock, we are going to print available and affordable. After creating a book object, in case of assignment overloading, you have to use I add where we are going to increment the addition operation. And for bitwise operator, we have to use the and. These are all the magic methods that we are going to use so that we can apply the proper overloading. For membership, it is contains and not contains magic methods. We can overload those based on the parameters that we are or based on the attributes that we are specifying and overload the membership operator. To conclude, make sure to run all these files independently. May your coding be clear and your understanding of Python semantics grow deeper.

Conditional Statement Video• . Duration: 6 minutes

[Notes]

Welcome, everyone, to our lesson on conditional statements in Python. By the end of this video, you will be able to understand the concept of conditional statements. Use if, elif, and else statements in Python. Apply conditional statements to solve programming problems. Conditional statements are control structures that allow different code execution based on specific conditions. They are the decision makers in our programs, enabling our code to respond differently to various situations. This flexibility is what makes conditional statements a cornerstone for creating dynamic and responsive programs. The basic structure of a conditional statement in Python is straightforward. We use the if keyword followed by a condition. If that condition is true, the intended code block beneath it executes. We can also include an else statement to specify what should happen if the condition is false. This simple structure allows for powerful decision making in our code. Let's look closer at the if statement. In its simplest form, a conditional statement executes a block of code only when the specified condition is true. In our example, we are checking if the book has more than 100 pages. If it does, we print a message stating it's a substantial book. This demonstrates how we can use conditions to make judgments about our data. The else statement provides an alternative path of execution when the if condition is false. It is like a backup plan for your code. For example, if you are categorizing books as either long or relatively short based on their page count, this shows how we can handle both possible outcomes of our condition. Sometimes we need to check multiple conditions. That is where the else if statement comes in, short for else if. It allows us to chain multiple conditions together. Let's look at an example. We are categorizing books into short, medium, or long based on the page count. This demonstrates how we can create more complex decision trees in our program. We can create even more sophisticated conditions by combining them using logical operators like AND, OR, and NOT. For an example, we are checking if the book is longer than 200 pages and costs less than 30. This shows how we can make decisions based on multiple criteria simultaneously. Python also offers a concise way to write simple if-else statements called the ternary operator. It is a one-line condition statement that can make your code more readable for simple conditions. What is the syntax of this? Let's see. We are determining whether a book is an e-book or a physical book based on its format. Look at the syntax. The if condition is coming after the assignment operator. For more complex decision making, we can use nested conditional statements. This involves placing one condition statement inside another. For example, we might categorize programming books based on their category and page count. In our demonstration, we have created layered decision making processes. If the category is programming

and the book pages is more than 500, both the conditions are true, then it will print Comprehensive Programming Guide. The else parts are getting executed. When working with conditionals, there are some common mistakes that we need to avoid. One frequent error is using single equal symbol rather than double equal symbol. We always have to use double equals for comparison. Another is forgetting to indent properly, which can completely change the logic of our code. It is also important not to overcomplicate our conditions, as this can lead to confusion and bugs. We need to keep some best practices in our mind for effective conditional statement writing. Keep your conditions simple and readable. Use positive conditions whenever possible, as they often are easier to understand and read. And try to avoid deep nesting of conditional statements, as this can make our code hard to follow. These practices will help you write cleaner, more maintainable code. To summarize, we have seen how we can control the program flow based on conditions, using if, elif and else statements. We have explored how to combine conditions with logical operators and how to use nested conditionals for making complex decisions. Remember, conditional statements are incredibly useful for decision making in data analysis tasks. By keeping our conditions simple and readable, we will be able to write effective, dynamic Python programs. Thank you for your attention. Happy coding!

Nested Conditional Statement Video • Duration: 4 minutes

[Notes]

Welcome everyone to our lesson on nested conditional statements. By the end of this video, you will be able to understand the concept of nested conditional statement, implement nested if-elif-else structures in Python, recognize when to use nested conditionals, and apply nested conditionals to solve complex decision-making problems. Nested conditional statements are simply conditional statements placed inside another conditional statement. They allow for more complex decision-making process in our code, enabling us to handle multiple interdependent conditions. Think of them as a way to create decision tree within your program. The basic structure of a nested conditional statement involves an outer condition and an inner condition. If the outer condition is true, we then evaluate the inner condition. This allows us to create more nuanced logic in our programs. We can have code that executes when both conditions are true, when only the outer condition is true or when the outer condition is false. Let's look at a more practical example for this. Here we have a function that categorizes books. If a book is about programming, we further check whether the page count is for a comprehensive guide or for an introductory book. For non-programming books, we simply categorize them as long or short based on the page count. This demonstrates how nested conditionals allows us to make decisions based on multiple criteria. We can take this concept further by multiple levels of nesting. In this example, we are analyzing books with three levels of condition. First we check the category, then the page count and finally the price of programming books. This allows for very specific categorization. But as you can see, the code starts to become a bit more complex. Nested structures can also include else-if statements. In this example, we are classifying books first by the category, fiction or non-fiction and then by the page count. The else-if statement allows us to handle multiple categories elegantly. This shows how we can create quite sophisticated classification system using nested conditionals. While nested conditionals are powerful, they come with some pitfalls. They can quickly become difficult to read and maintain, sometimes leading to what is called arrow code due to increasing indentations. As

complexity increases, so does the potential for errors. If we find ourselves with a deeply nested condition, it might be a sign that we need to refactor the code or consider a different approach. One alternative to deep nesting is using early returns. We check conditions and return early if they are met. This can lead to flatter, more readable code. Instead of nesting conditions, we are handling each case separately and returning as soon as we have an answer. When working with nested conditionals, it is important to follow some best practices. Try to limit the nesting to two or three levels for readability. Use clear and descriptive variable names to make your logic easier to follow. If we find our conditions becoming too complex, consider refactoring, that is, using alternate methods. Use comments to explain complex logic so that it is readable and understandable. Always consider alternate approaches like early returns or using dictionary mapping for complex categorization. To wrap up, nested conditionals are a powerful tool for complex decision-making in Python. They allow us to handle multiple interdependent conditions and are particularly useful in data analysis for multilevel categorization. However, we need to be cautious of overuse. When nesting becomes too deep, it's often a sign to consider alternatives. The key is to strike a balance between logical complexity and code readability. Thank you for your attention. Happy coding!

For and While Loops Video• . Duration: 5 minutes

[Notes]

Hello, everyone, to our lesson on iterations in Python. By the end of this video, you will be able to understand the concept of iterations, mainly for and while loops. Implementing for loops to iterate over sequences using while loops for condition-based iterations and apply the iterations on loops to solve common programming problems. Loops or iterations are used for repetitive execution of a block of code. In Python, we have two main types of loops, for loops and while loops. These are essential tools for data processing and analysis tasks, allowing us to perform operations on large data sets effectively. The for loop is used to iterate over a sequence such as list, tuple, or string. It executes the intended block of code for each item in the sequence. The basic structure is as shown there. We use the keyword for followed by the variable name and the keyword in and then the sequence we want to iterate over. This structure allows us to process each item in a collection one by one. One common use of for loops is with the range function. This allows us to iterate a specific number of times. In our example, we are printing numbers 0 to 4 in the first set of code and the second set of code, we are printing the numbers 1 to 5. Note the way the range values are provided. Range 5 function generates a sequence of numbers from 0 to 4 and our loop iterates over each of these numbers. This is particularly useful when you need to perform an action a specific number of times. Note the second variation where you are giving a starting number and the ending number. Always remember that whenever we are using range, the last number is skipped. Enumerate function adds a counter to an iterable. It is a powerful tool when working with for loops. It adds a counter to an iterable, allowing us to keep track of the index of each item. In our example, we are printing a numbered list of book titles. We start with 1 using the start parameter and instead of using default 0. This is especially useful when you need both the index and value of each item in a sequence. While loops have a different structure, they repeat execution as long as a specific condition is true. The condition is checked before each iteration. If the condition becomes false, the loop stops. This makes while loops particularly useful when we don't know in advance how many times we need to iterate. Let's look at an example of a while loop. Here we are simulating reading a

book. We start with a total of 1000 pages and keep track of how many pages we have read. The loop continues as long as we haven't read all the pages. In each iteration, we read 50 pages and print our progress. This demonstrates how while loops can be used to model real-world processes. Let's talk about infinite loops and the break statement. An infinite loop is a loop that never ends, which can be useful in some situations but should not be used quite often. We need to be very cautious. The break statement allows us to exit a loop immediately. In the example, we have an infinite while loop that repeatedly asks for user input. If the user enters quit, we use break statement to exit the loop. This pattern is common in interactive programs. The continue statement is another useful tool when working with loops. It skips the rest of the current iteration and moves to the next one. In our example, we are iterating over a list of books but we only print information about the books with more than 200 pages. For shorter books, we use continue to skip to the next book. This allows us to selectively process items in our loop. Nested loops are loops inside other loops. They are powerful but should be used judiciously as they can quickly increase the complexity of the code. In the example, we are using nested loops to print combinations of book categories and lengths. The outer loop iterates over categories while the inner loop iterates over the lengths of each category. This demonstrates how we can create more complex iterations using nested loops. To recap, it is important to choose the appropriate loop type for your task. Overloops are typically used when you know the number of iterations in advance. While-loops are used when the number of iterations is not known. Avoid infinite loops unless explicitly needed and use break and continue statements judiciously. Remember, loops are essential for data iteration and processing in Python. Thank you for your attention. Happy coding!

Lesson 3 Demo Video• . Duration: 8 minutes

[Notes]

Hello, flow control enthusiast. It is time to put our knowledge of control statements and iterations into practice. We will run through all the code samples from our lesson using Jupyter Notebook. Let's get started. In lesson three, we were talking about the conditional statements. So we execute the class book where we have title, pages, price, and format. In the if part, as you can see the syntax, it is if and the condition. The condition is given before the colon. We always have to end the if statement with the colon. And once we type in, that is if I type the same thing, automatically it prompts the intent. So most of the IDEs or the integrated development environment provide us the I with the intents. Please work with the same intent. So we create a book class in the first line. In the second line, we are giving you an if statement where we are checking whether the number of pages is more than a hundred or not. And then print accordingly. What if we give without or with less number of pages, will it get printed? Let's just see. For the same example, it doesn't print anything. That means the if statement did not get executed. Now, instead of giving it for guessing, we can specify the else part to make it very clear. The example is shown in the line number four, where it has else part also. So if in the previous case, where the number of pages is 50, it goes to the else part and replies or returns the book title is a relatively short book. If we do with else if, let me just change the book object here so that we get something which is different. As expected, it prints the value medium length because 250 is between 100 and 300. So the else if part or elif part is getting executed. Now what if we change to 500? It goes to the else part and it says long read as the output. Instead of doing this, we can combine also if required, based on the logic that we are trying to execute. For

tertiary operator, as you can see, it is given in the single line, right? The if and else part. It just simple does a simple check and the result based on the result, we will either provide the book type as ebook or otherwise. Now let's just print the book type. For nested conditional statements, we have to put one if inside the if part or the else part as shown here. It checks both the conditions. If it is true, it executes the if part. An alternate of conditional statements, you can use the return. Early return helps work with nested if or it avoids nested if it can work simply with the return statement. We'll move on to the next video, which is nested conditional statement. In nested lifts, that is, if I have to nest or if I have to bring in multiple conditions, one if inside another if, it is possible. It is possible to provide the if statement inside the else also. So both the conditions has to be true or false, depending on that the statements will get executed. Now, if the book category is true, and if the number of pages is less than 300, it goes to the last statement. The number of nesting can be increased as per our logic. In this example, there are three levels of nesting. As we can see, there are three if blocks. And for each of the if block, there is a continuous else part also. So else is also nested thrice. For each of the if, we can provide else if also. But as you can see, this becomes complicated. It becomes difficult to follow what the logic is, right? If the logic states that we can combine the if conditions using logical operators, it is preferred to do that. This is an example where format, we are checking the number of pages and book format so that the if statements or the if else nesting can be reduced. Another alternative is to use early return as we had seen in the earlier case. Moving on, we will go to the video on for and while loop. We'll create a class book, which has just titles and pages as attributes. Then we create an object. It's a list which contains three books so that we can experiment with our for loops. The first example, we are just going to use the for loop in its pure form. We are just saying for book in books. So instead of curly brackets or anything, we are using the membership operator there. For book in books, print the title and the pages. We can work with range also in loops. So we, in this example, in this dummy example, we are just going to take the value of i in the range five. Remember, this will start with zero. Always the for loop starts with zero and ends one number less than five. In the second example, we are modifying it slightly by providing two values in the range. So in the range, what we have done is we have set start and end values. We start at one and end at six. Remember, again, six will not be printed because that is the ending criteria. It works like for i equals one to five, print the value of i. We can also use enumerate with the for loops. For index comma book in enumerate books, where we specify the start criteria for the enumeration or for the index, we can do that. So this will list, remember, this was the output earlier. Just glance at it, it says that there are no numbering, just the book name and the number of pages. Now, when we enumerate, we'll get the number. This time we have set index and the book title. Number of pages is not printed. Moving on to while, to read books or the number of pages from the book, right, we have a logic here where we say that the number of pages read, if it is less than total pages, then you get into the loop where you are just going to add the number of pages by 50 and then print that you have read that many books. And when you have completed, when you have reached the total pages, you are saying you have finished reading the book. The loop is executed 20 times, depending on the number of pages that you are reading. We can have infinite loops also. This is an example where we are getting, we are waiting for the user to enter a data and only when the user enters quit or exit, it will exit. Let's execute this. So we are saying it is entering. So let's say clean code, okay? You entered the value. Now you enter another value. It all gets stored in user underscore input. As long as we are not entering quit, the code gets executed. Now what if, if we enter quit, then we move out of the loop. That is break gets executed and we move out of the loop or it moves to the next line that is written after the while. What is the effect of continue statement? In

this example, in the for loop, we are using continue. We had the book list. As long as the number of pages are less than 200, it will continue. If it is more than 200, it will print the title and saying that it is a lengthy book. Now just for the reference, we'll just go back. We have three books, Titan Basics of 200 pages. We have Data Science and ML Intro with 350 and 280 respectively. Let's see what is the result. All the books are printed. Why? Because we said less than 200. Now if we had to change it to less than equal to, you can see the result. Only two books as expected is printed. Nested loops where we write one loop inside the another loop. It's pretty simple and straightforward. It is just going through the categories and links and printing each of the six values based on the categories and links. It's nesting two for loops. To finish up, please execute these files on your own. Enjoy mastering the art of controlling your code flow.

Regular Functions Video• . Duration: 6 minutes

[Notes]

Welcome learners to our lesson on regular functions in Python. By the end of this video, you will be able to understand the concept of functions in Python. Define and call regular functions, use parameters and return values in functions, apply functions to solve programming problems, recognize best practices for writing effective functions. We have been referring to functions a lot in the previous videos. So what are functions? Functions are reusable blocks of code that perform a single specific task. They are essential for organizing your code and avoiding repetition. Functions can take inputs, which we call as parameters, and can return outputs. Think of them as mini-programs within your main program, each designed to do one job well. Defining a function in Python is straightforward. We use def keyword followed by the function name and parenthesis containing any parameters. Parameters are optional. The function body is intended and contains the code to be executed. If the function needs to send a return or result back, we use the return statement. This structure allows us to create custom operations that we can use throughout our program. Let's look at a simple example. Here we have a function called greet that takes a name parameter and returns a greeting message. We call the function with the argument, Alice, and store the result in the message variable. When we print the message, we get, hello, Alice. Why don't we try executing this function using your name? Another change that you can try is instead of hello, use a different word to greet yourself. Now let's dive deeper into function parameters. Parameters are variables listed in the function definition. While arguments are the actual values we pass to the function when we call it. Let's see an example. We have a calculate price function that takes price and discount as parameters. When we call the function with the arguments 100 and 0.1, it calculates and returns the discounted price, which in this case is \$90. Python allows us to set default values for parameters. This is particularly useful when some parameters can be made optional. In this example, our create book function has a default value of 200 for the pages parameter. This means that we can call the function with or without specifying the number of pages. As in the first case, if we do not provide the value for pages, it will use the default value of 200. Keyword arguments provide flexibility when calling functions. By using parameter names in the function call, we can provide an argument in any order. What does this mean? A simple example will demonstrate this. In our describe book function, we are able to specify the year, author, and title in a different order than they were defined in the function. This can make our code more readable, especially when dealing with functions that have many parameters. The return statement is used to send a result back from

the function. A function can have a single value or multiple values as a tuple. Let's see an example for both. In our books example, get book info function, we are returning the title, author, and the number of pages as a tuple. We can then unpack these values when we call the function, assigning each return value to a separate variable, as shown in the example. Functions can have multiple return statements, allowing them to return different values based on certain conditions. In the analyze book function given below, we return different strings based on books length, along with the page count. This allows our function to make decisions and provide different outputs accordingly. Understanding the scope of a variable is crucial when we are working with functions. Local variables are defined inside the functions and are not accessible outside the function. Global variables, on the other hand, are defined outside the functions and can be accessed throughout the program. It is generally best to rely on local variables within the functions to avoid unintended side effects. Let's see an example. We use a global variable with a function. We define total books as a global variable and then use the global keyword within our add book function to modify it. While this is possible, it is often better to pass values as parameters and return results to maintain cleaner, more predictable code. So what are the best practices? Keep your functions small and focused on the single tasks. Use descriptive names for both functions and parameters. Provide default values for parameters when it makes sense. Use doc strings to document what your functions do. And try to avoid modifying global parameters within the functions when possible. These practices will make your code more readable, maintainable, and less prone to errors. To recap, remember that functions are reusable blocks of code that can take parameters and return values. They are essential for organizing your code and promoting reusability. Use descriptive names and doc strings to make your functions clear and easy to use. By mastering functions, you will be able to write more efficient, more organized, and powerful Python programs. Thank you for your attention. Do try out the examples provided. Happy coding.

Lambda Functions Video• . Duration: 6 minutes

[Notes]

Welcome everyone to our topic, Lambda Functions in Python. By the end of this video, you will be able to understand the concept of Lambda Functions, create and use Lambda Functions in Python, recognize the situations where Lambda Functions are appropriate, apply Lambda Functions in common programming scenarios, and compare Lambda Functions with regular functions that we had seen in the previous video. Lambda Functions are small, anonymous functions in Python. They can have any number of arguments, but are restricted to a single expression. You might also hear them referred as Lambda Expressions or Anonymous Functions. These functions are particularly useful for creating quick throwaway functions without the need of a formal definition. The syntax of Lambda Functions is quite different from the regular functions. There is no def keyword and no explicit return statement. Instead, we use lambda keyword followed by the arguments, a colon, and the expression. The result of this expression is automatically returned. This concise syntax allows us to define simple functions in a single line of code. Let's look at an example. We will compare a regular function with its lambda equivalent. Here we have a function that squares a number, a very simple function. The regular function uses the keyword and a return statement, while the lambda functions achieves the same task in a single line. Note the use of the keyword lambda. Both the functions produce the same output when called with the argument 5, returning 25. Lambda

Functions aren't limited to a single argument. They can take multiple arguments, separated by commas. For example, we have a lambda function that adds two numbers. `sum equals lambda a b colon a plus b` There is another function which multiplies three numbers. This demonstrates how Lambda Functions can handle various operations with multiple inputs, all in a compact form. Note that in both the cases, there is only one expression that is evaluated. Now let's see how Lambda Functions can be useful in a real-world scenario. In this example, we have a list of book objects, each with a title, author, and page count. We use a lambda function as the key in the sorted function to sort the books based on their page numbers. Sorted is a built-in function that we are using. This showcases how Lambda Functions can be used in-line as arguments to other functions, making our code more concise. Lambda Functions work well with Python's built-in functions like map. Let's see an example. We use map with a lambda function to extract the titles of our list of books. The lambda function takes each book object and returns its title. The map function applies this to every book in the list, and we convert the result to a list. This demonstrates how Lambda Functions can be used for quick transformations of data. Another common use of Lambda Function is with the filter function. In this example, we use a lambda function to filter out the list of books, keeping only those with more than 300 pages. The lambda function returns true for books meeting this criteria, and filter uses this to create a new list. Let me print the titles and page counts of these long books. This shows how Lambda Functions can be used for concise data filtering. Now let's compare Lambda Functions with regular functions. Lambda Functions are ideal for single expressions. They are anonymous and are very concise. Regular functions, on the other hand, can contain multiple expressions, have names, and are generally more readable for complicated logic. In our example, we see how a simple discount calculation can be written in both ways. The choice between them often depends on the complexity of the operation and the context in which it is used. Lambda Functions are best suited for simple operations. Functions you will use only once, and so arguments to higher-order functions like map, filter, and sorted. Their concise nature makes them perfect for these scenarios, allowing you to define the function right where it is needed, without cluttering your code with formal functions. However, Lambda Functions have limitations. They are restricted to single expression, which means you cannot use multiple statements like if, for, or while within them. They can also become less readable for complex operations. It is important to consider these limitations when deciding whether to use Lambda Functions or regular functions. When working with Lambda Functions, it is better to follow these practices. Use them for simple, single-line, one-time use functions, where their conciseness adds value. For complex logic, prefer regular functions as they are often more readable and maintainable. Even in short Lambda Functions, use descriptive variables to maintain clarity. Always consider readability when choosing between Lambda and regular functions. To summarize, remember that Lambda Functions are small, anonymous functions in Python. They are practically useful for simple operations and as arguments to higher-order functions. While they often replace simple regular functions, they are best suited for short, simple operations to improve code conciseness. Thank you for your attention. Do try out Lambda Functions. Happy coding!

Lesson 4 Demo Video• . Duration: 4 minutes

[Notes]

Greetings, function fanatics! Today, we will demonstrate the code snippets from our lesson Python Functions. We will use Jupyter Notebook to execute these examples and preview their outputs. Are you ready to begin? Let's go to the next lesson, where we will be talking about functions. The simple function that we had was just the function with greeting, where we greeted the name. It prints the message as expected. Then we had a function which calculates the price based on the discount value that we have provided. It prints as expected. It does the computation and calculates the price. The computation is given inside the function. Note the function definition. We use the def keyword, give the function name, give the parameter names, colon, and then we have to provide the intent. The last statement is returned. We can give default parameter values also, as given in this example where the pages is given 200. We will execute this and see for results. By using a different method, we can see the results of this. We are using getBookInfo function, where we provide the book details and it returns the title, author, and pages. For the object that we created earlier, book1, let's see how many pages it had. We had not given any value, so it should take the default number of pages. It is giving the default number of pages based on the parameter value that we give. When we are calling the function, we can reorder it by providing the keyword arguments. In the earlier case, we had to use positional arguments. Now we are going to give keyword arguments. Even if the position is different, it takes in according to the values that we have provided. Functions can have multiple returns. In the if and else part, we are providing return so that it returns out of the function according to the code or according to the logic that we have written. So it compares the number of pages and returns accordingly. Against regular functions, we had lambda functions. This is an example where we are using a function to square. The syntax is use the keyword lambda, provide the variable name, then colon, and then the expression. Both the functions, both the regular function and the lambda functions return the value, the same value as expected. A lambda function can take multiple arguments, so the syntax will be lambda, the number of arguments separated by commas. Here also we have three arguments separated by commas, and then the expression that we want to execute. In the books example, we have created a list with three books, their name and the number of pages. Lambda function can effectively work on this. Based on the book pages, it is going to sort. So we are going to rely on the built-in function sorted. And we are seeing that the sorting is based on the book pages and execute the code. Another way is use lambda with map or filter function. This particular line executes lambda with map. We say lambda, book, book title, and from the list of books. Books is the list that we have provided. It will map all the book titles and give the results. Let's see the example. So it is extracting all the book titles from the list and just returning that particular list. If I'm going to go for filter, we have to specify a condition on which it has to be filtered. Again, the syntax is filter lambda and what is the function or what is the condition on which it has to be filtered. So based on the pages, it will filter the values and give the results. Both the cases, it is stored as a list. As we conclude, I encourage you to dive into these files yourself. Happy coding and may your functions be efficient and reusable. Thank you for your attention.

Writing Files Video• . Duration: 5 minutes

[Notes]

Welcome everyone to our lesson on writing files in Python. By the end of this video, you will be able to understand the importance of writing to files in Python, open files in write and append modes, write data to files using different methods. Writing to files is a fundamental operation in programming

and data analysis. It allows us to persist data for later use, create logs and reports, export analysis results, and share data with other programs. This skill is essential for data output in analytics, enabling us to save our findings and make them accessible for future reference or further processing. In Python, we have two main modes for writing to files. The write mode, denoted by W, creates a new file or overwrites an existing one. The append mode, denoted by A, adds content to the end of the existing file. It is crucial to choose the write mode to avoid accidentally overwriting important data. The syntax for write mode is with open file, mode is W, then do the write operation. Whereas for the append mode, it is with open file name, mode is A, and then do the append operations. Writing text to a file is straightforward in Python. In our example, we use the with statement to ensure the file is properly closed after the operations are completed. We are writing two book titles to a file named books.txt. We use the write method to the file object to add strings to our file. Notice that we add a new line character, slash n, at the end of each line to ensure each title appears on a separate line in the file. For efficiency, we can write multiple lines to a file at once using write lines method. This approach is more memory efficient for large datasets compared to joining all the strings beforehand. For example, we have a list of book titles and we use a generator expression to add new line character to each title. When we want to add content to an existing file without overwriting its current contents, we use the append mode. This is useful for logs, ongoing lists, or any situation where we want to preserve existing data by adding new information. In our example, we are adding a new book to an existing reading list by appending to the existing list. CSV files are common in data analysis and Python makes it easy to write to them. We use the CSV module to create a writer object, which handles the formatting for us. Let's look at an example. We are writing a list of books, including a header row to a CSV file. The new line parameter is important to ensure consistent line endings across different platforms. JSON is another popular format for storing structured data. Python's JSON module provides functions for writing JSON data to files. In the example, we use JSON.dump to write a dictionary containing book information to a JSON file. The indent parameter makes the output file more readable by adding line breaks or indentations. Sometimes we need to ensure data is returned to the disk immediately, which we can do with the flush method. This is useful for critical data that shouldn't be lost if the program crashes. However, it is generally better to use the with statement, which automatically closes the file and ensures all data is returned. If you are not using with, then we have to remember to close our files manually to free up system resources. When writing files, it is important to always use the with statement to ensure proper file closure. Choose the appropriate mode, W for overwrite and A for append, based on your needs. Specific file formats consider using specialized libraries like Pandas for CSV files. Be cautious when overwriting files to avoid accidental data loss. To recap, remember that writing to files is crucial for data persistence and output in Python. We have explored various methods to write files, including write function and write lines function. It is important to use the appropriate file modes, W for write and A for append. Choose the right technique based on your data structure and format, ensures data integrity and prevents data loss, which is essential in any data analytics workflow. Thank you for your attention. Happy coding.

Lesson 5 Demo Video• . Duration: 3 minutes

[Notes]

Welcome file handlers. We are about to embark on a practical journey through our file handling lesson. We will execute all the code examples in Jupyter Notebook and examine their results together. Let's dive in. Moving on to Lesson 5, the last file where we are going to read files. Now for reading files, remember these particular files should exist in the current working directory. So if you look at the current working directory, I do have books.txt, books.csv, books underscore info.json. All the three files exist. So books.txt is what we are going to use first for reading. We are going to use in the read-only mode. We are not going to write it. File.read and print the content, whatever the content was in that particular file. If we have to read line by line the same example, that's also possible by using the for loop. From the CSV file, if we have to read, we have to rely on CSV module of Python. We give the file name. Again, in the read-only mode, we use a CSV reader. Using a for loop, we extract each row from the reader and execute or do whatever operations we are required to do. Here, the simple operation of printing is what is done. If you are going to read JSON files, we have to rely on JSON module of Python. Give the file name, give the read mode, and use json.load. The JSON file is accessed and printed. So we have a file name, write mode, and provide whatever we have to write using file.write method. Remember, in all the functions, or notice all the functions we are using the with operator. We don't have to explicitly close the file. Writing multiple lines at once is simple. Here, we have a list of book names. We are going to read it. Now, we are going to say append it to the book list. This is a different file name. Now, instead of executing here, let me just copy the read one and see what is the content from that. It is listing the five books that we had. Right, so append mode works perfectly well. Again, to see the results for ourselves, let's go back to the books.txt. As you can see, there are five books against the earlier. Writing to a CSV file, we again rely on the CSV module of Python. We use CSV writer contents of the list to the file, whatever is pointed by the writer object. New line is used to ensure that it goes into the next line. Each line goes into the next line. And writing to a JSON file requires the JSON module, where we have to use JSON.dump method. We also use the intent to specify the number of intents that we have to provide so that it is written in the correct JSON format. To wrap up, please run these files independently. Enjoy exploring the world of file input-output handling in Python. And may your data always flow smoothly. Thank you.

Recording of Introduction to Data Analytics: Week 2 - Live Session on 26-06-13 Video• . Duration: 54 minutes 54 min Resume . Click to resume

[Notes]

Okay, so in today's session, we shall learn about the fundamentals of Python programming. So before we start coding, let us have a small discussion like how many of you have been used or used applications like YouTube, Instagram, Chachapiti or Netflix or Amazon, right? So generally all these applications, they all use a programming language behind the scenes, right? So Python is one of the most popular programming language, which is used in the data science, in the artificial intelligence, in the machine learning, in the web development, okay, in the web development as well as in the automation. So the good thing in the Python is it is one of the easiest programming language. It is the one of the easiest programming language, which is used in the data science, and which is very easy to learn, okay. So by the end of this session, you will be able to write simple Python program. You learn to take simple inputs from the user, make decisions using the if statements, we try to use the loops like for loop by loop and all, and you will try to understand about the functions, basic

functions, lambda functions and all. So to start with, what are the basic features of the Python? So Python is a high level language. It is a interpreted programming language, which is very much known for its simplicity, its readability as well as its versatility. This high level language here means it is very simple or very close to the human language. Like for example, instead of writing 10 lines code, and to print something, what we can do with using Python is, we may require only one line, okay, means because it is easy to learn, easy to read, it is platform independent, and it is also open source most useful. And it is widely used in AI as well as in the data science. So to name some of the companies which use Python is, you have Google, you have Netflix, you have Instagram, you have Spotify. So all these are famous companies which use the Python, okay. So this, for this, we see, we first install Python, and then we use the Jupyter Notebook to write our Python programs, okay. So let me just take you to the... Okay, so this is the Python platform, okay. So here I'll open a new notebook, Jupyter Notebook. So in the file we have, I'll create a new notebook, right? Yeah, so this is your new Jupyter Notebook, okay. So here, if you can see, you can rename the Jupyter, your Python file here, okay. You can rename the file and move it as sample, rename the file, okay. So here, these are called the cells, wherein we give the input, okay. These are called the cells, where we write our code, or like code of lines and all. Here basically, these cells, they are like, they are of three types. If you can see this, I have this in the drop-down list, this code, markdown, and raw. So these are the three types what a cell has, okay. A cell can be a code, a cell can be a markdown, or a cell can be a raw. So what do you mean by code is, so here, if you write anything in this cell, keeping the type of the cell as code, means you can execute them, okay. You can execute the one which is, like for example, I'll write very simple one. See I selected the type as code. Now come to the cell, okay, and write simple line, print, hello world, okay. Now, so this line I have written. Now if I want to execute, either you can just write shift, enter. Now see, you can just write the code and output also, it's visible in the same screen, okay. See how quickly we have got the response, right. So this is called as a code. Now when I, what do you say, when I selected the cell type as a code, okay, on just on with the click on the button, you see, you can use even this button, okay, this is used to run the code. So when I clicked on this, or when I just entered shift plus enter, immediately I got the output, okay. So you can use, even to write, do the operations, like for example, I want to find the sum of two integers, okay, just click, see the output here, okay. So this is the cell type, okay, if it is in the code. Now what is the markdown, another cell type is called as a markdown. Now in the markdown, what happens is, see you can, what is it, like for example, I want to hash, welcome to Python, right, no run this cell, see, can you see the difference? Now what did I write, I just mentioned hash, welcome to Python, okay. Now what happens here is, actually it is not executed, what happens here is, when I type one hash, I will get the text, whatever I have written, that will be printed in the header type of one, okay, that much size, header type one, it will be printed, okay. So for the formatting of the text, we can use the markdown feature, okay, markdown feature, we can use for formatting the text. Now if I, again, I will change the cell type to markdown, hash, welcome to Python, okay, yeah, you can do in BS code also, okay, you can use that also, but for the starting session, because this will be very comfortable, I have used this, later on you can move on to the BS code, no issue, see, this is the second heading type, okay, here what did I do, I to do, you have to first select the cell type as markdown, okay, cell type as markdown and type the text, so I wanted the text to be printed in bold, so I have used a double star, okay, double star to print the text in bold, now again change the cell type to markdown, by default it will be in bold type, so now, can you see the difference, here I got the text in italic, okay, so to print the text in italic, we will use single star, so like this for formatting the for documentation type features, we can use the markdown type of cell,

okay, and then you have raw, now what is raw, it will not do any type of processing here, whatever you enter into the cell that will be just as it is, it will not process it, okay, like this is some text, okay, so try to execute on this code, nothing will be displayed, okay, nothing will be displayed, the text will not be formatted, nothing will be done, okay, you will get, you can do for documentation purpose, okay, writing the, for writing purpose, you can use that cell, okay, so that is what you will understand here, basically we have three types of cells in the Jupyter notebook, one is code, other one is markdown, then you have the raw, code is a user to do all type of processing, okay, like you can, what do you say, you can do any type of coding, you can use the cell of type code, then markdown is used for formatting the text and raw cell is used just to write or enter the text into the cell, okay, now next is basic thing is input output, now how do you give input to a cell, let us see, like for example, I want to read input from the user, enter your name, I want to read your name, okay, then I want to print the name, okay, see, when I just, this cell is in the code type, I just entered name is equal to input, enter your name, then I try to print, see no, no need of terminating, what do you say, same column, nothing, okay, so just I have typed enter your name, then I have said print hello, so this message hello will be printed as it is, let us see what will happen, so enter the name, now once when I enter, see, hello site, that is printed, okay, next term, so it will ask you first to enter the name, once you have entered the name, that name will be, it will be sent to this print statement, print hello name, another type of, let us see with another example, enter input, I want to read some number, enter a number, I'll read a number now this number you can do some arithmetic operation also multiply this number by two and try to print the number. Okay. It's an indication error print should come right. Yeah. Yeah, that was an indentation error. Sorry for that. I didn't check that. Okay, so here now I'll give some number like to see so it has taken two and multiplied with this to certain number into two. Again, if I remember to five five into two is equal to ten. Here I got that. Now here what is happening is by using the input key. I am trying to read a number. Okay from the user now this number actually when I am reading the number it will be in a string. Now. I have to convert that string into integer types. That is why I'm using this data conversion. Okay in convert the number that is coming from the keyboard or from the user into an integer type and that will be stored into this variable called num and that is multiplied with two and then printed. Okay, that is why you got the output as 10. And another example, for suppose. You can see I have assigned a value string value to the variable name and I have just printed the string name. Okay, and here. Another thing what we can do is. Yes. Could you see? Yeah, I'll use your stream. So when I use everything so it is generally a modern way of formatting the output. So here the name will be printed and in this the age will be printed. Let us execute this and see my name is Sai and I am 20 years old. So like that by using the F you can format the output also you can print you can. And. Welcome to. See see the difference in the output here. Can you see so it is printed in different lines? Okay. Now, let us check with the data types. What are the different data types that are supported by the Python? like A is equal to 10 B is equal to 3.14 C is equal to. Then to print the data type. We have one function called type. Okay, so if you use the type keyword. You can get the type of the data type. What we are use that variable is. Or data type that variable is. Okay. See. Data type or also called as class. So here a we have assigned it as 10. Okay. So now that is the data type is of integer. Now B is 3.14, right? It is decimal that is float. Then we have C. We have written it as Python, which is a string data type. Then D we have assigned it as to which is a Boolean data type. Okay, so these are the data type that are supported by the Python and next if we check with the. Yeah. You see the input output. So we take the numeric data. And convert into the integer data. We have seen this. F3. So this is short for formatted string. Okay, just now we have seen the string, right? F stands for formatted formatted

string. So you can insert variable values directly into a string in Python. And then yeah, so if you want to print multiple lines, we have seen we used three columns to print the multiple lines. And we are scalar types or data types basic data types. What we have seen integer float string and Boolean. And then we have the objects. Okay, so in Python everything is an object. We have numbers strings and list. Okay, so all are considered as objects. So like. If I have. Text is equal to. Hello. I have a text. Hello. Okay. Pinter. Text dot upper. Okay, and I have one more statement that text dot replace. H with Y. Okay. Now, let us see here. I have just used one variable called text and assign the value as hello, right? Then I have used the print statement text dot upper. Now, what is this upper doing upper is a method which is converting the value of text into uppercase value of text into uppercase. Then I have used one more way that is replaced. I want to replace the character H in the text with the white. Now here the object is nothing but a thing. Okay, the object is a thing and method is an action which is performed on the thing. For example, if you take a human being so object will be a student if I consider a student, he will be an object. Okay, a student we can consider him as an object. Now what you can what you can what you say methods or operations that can be performed on a student a student can walk a student can speak a student can write a student can read. Okay, walk speak write read. So these are all the methods on that object that is called as a student right now in the string. What example we have taken? Hello. Hello is an object here. Okay. Hello is an object. Now, what is the what are the methods I can perform on this object we can perform upper means I can change the string into uppercase. I can replace the characters of this string. I can change the string into lowercase. Okay, I can consider the substring like that. You have many methods that can be performed on this. What do you say object, right? So if you just run this code. Yes, it first. What did I say text dot upper on this object? I want this method to be applied. What is that method upper? So this text string of hello will be converted into capital letter. Okay, and next is replace replace this character H with Y. So why e l l o w so it is hello. No, okay. So that are the objects and the methods. That can be performed on this objects. Now attribute is the property that belongs to an object. So upper is an attribute specifically a method attribute of a string object, which is hello. Now next is method. We have seen upper and replace and all then next is the operators. So operators they perform action on the variables. So like all the arithmetic operators that they are supported here like you have the addition operator. You have multiplication operator. You have division operator. You have comparison operators. Okay greater than or equal to so these comparison operators output will be true or false only. Okay. And let us see with the operators example. So here I will say a is equal to 10. I'll assign another variable B is equal to 3 so I can say print a less B directly. You can perform the operation and print then another operator print a into B. Then we can say print. in in the print a greater than B logical operators. Okay, then print. equal to B, right? So any guesses what would be the output of this anybody? Very simple. So we shall start with very basic initially. What would be the expected output of this? Okay, that is with only one print, right? 10 plus 3 is 13. So you have all other print statements also 10 into 3. This will be 10 divided by 3. This will be 10 greater than 3. So for the output from this statement will be logical output, right? Whether true or false even from this. Okay. So here you will be getting it as true and here you will be getting it as false because a is not equal to B. Let us execute this. Okay, can you see here? So Saif good that you have given one answer, but you have considered only one print statement that is a plus B, but I have given all five print statements. So when you execute you will get the output of all the five print statements like this. Okay. So the relational operators output will be always either true or false. Okay. Now next let us. Right. Okay. Now next let us see with the conditional statement. So the conditional statement with if okay and else if and else. So what does this conditional statement

do is the it will check whether a given whether the given condition is true or false. For example, here if you take this example, we have a number into input enter a number. We are reading a number. Now I am checking here whether the number is greater than zero. If the number is greater than zero, then we are printing the number is positive else if else if is written like this e l if else if we don't write the complete spelling. it will be an error again. So else if number equal to zero, so this is comparison. Okay, if you write single equal to that is will that will be an assignment. So this is a comparison if number equal to zero print the number is zero else print the number is negative. Okay. So this is now what does the if statement do if statement will check first will check with the given condition. If the condition is true the true statement will be printed else again else if you you can even check the multiple conditions see. First condition is checked if this condition is wrong. I am checking one more condition. Okay. So multiple conditions. I am checking here with if else if statement right. Okay, so condition is checked if it condition is true true statement will be printed else another condition is checked if it is true next statement will be printed else the state false statement will be printed. Okay. Like let us see with a simple example in our code. If I want to read the number is equal to convert the number into integer input. Enter for number. I will read the number here. if num is greater than zero. Then what should I do? We should print the the statement that is correct. If the condition is true and just simply say print the number is positive. Okay One minute is equal to zero. No. Okay Great. Okay. So if number is 0, print positive. Else if number is 0, sorry, if number is greater than 0, print positive. Else if number equal to 0, print 0. Else print negative. Now let us run this code and see. Yes. I just wanted to show you here. See in this python, indent is very very important. Okay. Indent is, now what do you mean by indent? See here, I have started the code. If number is greater than 0, okay, print. I am printing the positive. See the position here. Okay. See the position. That is why I am getting the error also. Print. We change the indent, we see. Else if number is equal to 0, print positive. One more. I forgot the bracket here. Closing bracket was missing. Okay. So here closing bracket was missing. So that is why that was error. See here, the indent is very important. You cannot randomly write from any part of the line. Okay. So that indent is important. Be very careful over there. So I will enter one number. It is a positive number. Again run the code and enter 0. It is 0 printed. Okay. So this is the conditional statement what we have. Next we have nested conditional statement. Okay. So nested conditional statement means, see, I am checking if age is greater than 8, print you are eligible to vote. If age is greater than C. Okay. Nested conditions we are checking. Check if someone is eligible to vote or if they are a senior citizen. So nested conditions you will be checking with nested ifs. Okay. So next you are having this for loops. So in the loops, we have two types of loops. One is the for loop and one is the while loop. So when you want to perform some repeated operations, we use the loops. So here used to repeat action a fixed number of times. Okay. Then we use a for loop. So simple for loop is for i in a range of 1 to 6, print i. Now, let us see here. For i in range of 1, 6, print i. Now here, what do you expect the output to be anybody? What is the output here? For i in range of 1 to 6, print i. Can anyone tell me what is the expected output there? Very good. So, yeah. Newsome Lahiri ready. Good. So here, what happens is when I execute this, you can see only till 5, it will be printed. Okay. So Python will start at 1 and stop at 6. So ranges from 1 to 6, not including the upper limit. Okay. Now, similarly, if you check with the. Yeah. So starts from the first number and stops before the second number. It's exclusive. Okay. It doesn't include the upper bound. So we use the for loop when we know how many times you want to repeat it. Okay. Now, next is the while loop. So use it to repeat actions as long as the condition is true. Okay. So repeat the action as long as the condition is true. You are not aware of the fixed number of repetitions. Then you go with the while loop. So here you have the

simple example of your while loop. Now, for example, first I'll assign a variable `i` is equal to 1. While `i` is less than or equal to 5. What you will do? I want print `i`. Okay. I want to print `i`. Then `i` is equal to increment the `i` value `i` plus 1. Simple statement. Right. Now you see. Okay. So what is happen until this condition is true. Okay. You are entering into the loop and you are performing the operation that is inside the loop inside the loop. What do you have? You have only one operation that is print `i`. Okay. So until this condition is true, you are going on printing the `i` value. Okay. That is print `i`. So first initially `i` will be 1. So you will be checking 1 is less than or equal to 5. Yes. 1 is less than 5. So enter into the loop. Print 1. Then again I am incrementing the `i` value here next. `i` is equal to `i` plus 1. So 1 plus 1. Now `i` will be 2. So again it will go to every time initialization of `i` will be not done. Okay. Only these two statements will be performed and every time it will be checking for the condition. Okay. Loop will be like this. So now when `i` is 2, it will check again. Is 2 less than or equal to 5? Yes. Okay. It is true. So enter into the loop. Print 2. Then again increment `i`. Now `i` will be 3. Go to the while condition. 3 less than 5? Yes. Come. Print 3. Then again increment `i`. `i` will be 3 plus 1. Now 4. Again check the condition. Is 4 less than or equal to 5? True. Okay. So print 4. Then again increment `i`. 4 plus 1 is equal to 5. Again go and check the while condition. Is 4 less than or equal to 5? Yes. Come. Print 3. Then again increment `i`. `i` will be 3 plus 1. Come. Print 3. Then again check the while condition. Is 5 less than or equal to 5? Yes. I am checking less than or equal to 5. So equal to condition is satisfied. So we are printing 5 also. Then now again this incrementing of `i` will be performed. Okay. So 5 plus 1 is 6 now. So again go to the while condition and check. Is 6 less than or equal to 5? No. Now the condition is false. Condition is false. Now I will come out of the loop. Okay. Now I will be coming out of the loop. So in the for loop it is used for the known repetitions. In the while loop it is a conditional based repetition. That is the simple. Okay. So I will make it again. Row. Simple. For loop is used for known repetitions. Okay. For loop is used for known number of repetitions. While loop is used for condition based repetitions. Okay. So that is the basic difference between your while loop and for loop. So this I made the cell as raw cell. Now try to execute this cell. See nothing. I wanted to make a note of it. So I have written something, some note in this cell. So nothing will be executed in this cell. Okay. Another thing is, if you want to remove this output, just right click on the cell and say just say clear cell output. Now I want to clear all the outputs, clear all cells output. Can you see? All the outputs are gone here. Okay. That is one simple note. Okay. And next what do we have? This is the while loop. Now let us see with the functions, regular functions. So a regular function is defined using the keyword that is called as `def`. So you will define a function in another programming languages, we define the functions with the keyword called function with the function keyword, right? So whereas in the Python, we use the `def`, `def` keyword is used to define a function. Okay. And it can have multiple lines. So must have a name used for reusable and complex logic, and it supports multiple lines of code. So this `def` is the keyword that is used to define a function and every function shouldn't must have a name. Okay. And it, what is the use of use of functions is for the reusabilities, okay, we use functions for reusability. And like, for example, if you take your calculator, right, so we are not every time we are not writing any program or any code to add numbers, right, directly we give numbers just a symbol and automatically we will get the output. So instead of writing the addition logic repeatedly, we create a function which does the operation, okay. So operations that can be done repeatedly, those operations can be written under a function for reusability. Now here, so in the Python, regular functions are used by a keyword called `def`. Now in this example, I'm taking a name called `add`, `add` is my function name, it is taking two parameters that is `a` and `b`, two variables will be considered, you know, what is that and what is the returning of a function is some of the two variables `a` plus `b`, okay, now print `add` of three comma five. So I'm

calling the function here inside the print statement itself, we can call a function, we are calling the function here with the parameters where I'm passing the values three comma five. So three and five will be assigned, three is assigned to a and five will be assigned to b. And I'll get the sum of three plus five, which is nothing but eight, okay, now, square of a number. So you can another example is to find the square of a number like def, def square of x and return x into x. So we are printing square of, so actual value will be passed over here and like, for example, def add a comma b is my function colon, okay, return a plus b. So this is my function. Now, if you write here only with this indent only, it will be part of your definition, a part of your add function, right? So I'm done with my function here, only one line statement inside my function that is written. Okay. Now here I want to print, I want to call the add function with the passing the parameters 3 plus 10, close the arrows, Laurent, see 3 plus 10 is 13. Now here in the print statement, I have not written, I have not written any logic. So for a simple example, we are using one statement, two statement, complex code also you can write, okay, another function type is lambda function, which is also called as anonymous function. So this lambda function, if you have a one line functions, okay, one line functions, you can use a lambda function is a short one line function used for simple operations. Okay, see, add equal to lambda a, a comma b colon a plus b, right or print add three comma five output will be a. So square of lambda x, okay, this is the like function name, okay, what type of function in line one line function, right? So in a simple one line, I am writing the function code completely, what is that I want to find this where x into x, that's it. So print, I'm printing the function, I'm calling the function here inside the print statement, which will do to four, right? So four into four, four, okay, this is also what we are doing, we are finding the square, lambda x colon x into x, that's it single line function. Now, here in the print statement, I will call the function square of five, okay, so you got it. So lambda is a short function, whereas normal function, see, for this in if you are writing a normal function, we write it as square of x, then you will see what happened when I have written square of x, and when I pressed enter, it is starting from the first line itself. Why? Because here, by after writing the function name and the parameters, you should use colon. Now see the indent, observe the indent, which is starting from the function name, okay, return x into x square. Now, see, if you want to continue the function more lines, multiple lines, then you will follow the same indent, but I am done with the function. So I am just printing print square of five, see, these two lines of code, okay, I am just writing in one line using the lambda, both are having the same output, right, both gave me the same output, but here the line of code was more than one, here it is just in simple one line, that is the basic difference between your lambda and your function, okay. So to give an overview, today, we have learned what are the basics of your Python. So, this all we shall see in the next session, okay, so today we have seen with the Python basics and Jupyter Notebook, what is the use of your Jupyter Notebook, what is the how do you give the input using the Python, how do you give the input, how do you take the input, how do you give the output, what are the various data types, basic data types that are supported by the Python, what are the conditional statement, what are the loops, okay, how does the loop work, and how do you write the loop statements, and what are the functions, functions like basic functions and lambda functions, one line function and multiple line functions, okay. So, this is the foundation that is required before we go in deep into learning about the NumPy or pandas or the data analytics and machine learning and all. So, in the next session, we will be we will start working on Python collections and data analysis libraries and all, okay. So, yeah. So, that's it. Any? Yeah, user defined functions. Okay, any other queries? Because this is the very first session with the Python, I made it very simple. Okay, so as we go on, we can, yes, next time on, we can use the VS code also. Thank you, Abhijit. So, the concepts are clear, right? Thank you.

Tuple Video• . Duration: 6 minutes

[Notes]

Welcome everyone to our session on tuples in Python. By the end of this video, you will be able to understand what tuples are and their characteristics, create and initialize tuples, access and unpack tuple elements, recognize the immutability of tuples and its implications. A tuple in Python is an ordered immutable sequence of elements. Unlike lists, tuples can contain elements of different data types, making them versatile for grouping related data. They are defined using parentheses and are useful when you want to ensure that a collection of data remains unchanged throughout your program. Tuples are used for grouping related data that shouldn't be modified. There are multiple ways to create tuples in Python. We can create empty tuples, tuples with elements and even tuples with a single element. The example code shows all those. It is important to note that when creating a tuple with a single element, we need to include a trailing comma to distinguish it from a regular parenthesized expression. The code demonstrates these different ways of creating tuples, including the special case for single element tuples. We can access tuple elements using indexing, similar to list. Python uses zero-based indexing, so the first element is always at index zero. We can also use negative indexing to access elements from the end of the tuple, where negative one refers to the last element. The code shows how to access the first and last elements of the tuple containing book information. Tuple unpacking is a powerful feature that allows you to assign the elements of a tuple to individual variables in one go. This is useful when working with functions that return multiple values or when you want to swap variable values without using a temporary variable. The code demonstrates unpacking a book information tuple into separate variables for title, author, year, and pages. Tuples can contain other tuples or even lists, creating nested structures. This allows for more complex data organization. When accessing elements in nested tuple, you will use multiple square brackets to navigate through the levels of nesting. The code example shows a nested tuple and how to access an element within the nested structure. One of the key characteristics of tuple is their immutability. Once a tuple is created, its elements cannot be changed, added, or removed. This immutability is what makes tuples useful for representing data that should not change like constants or fixed collections of values. The code attempts to modify a tuple, which would result in an error if executed. If we try to modify a tuple, Python will raise a type error. While we cannot change the elements of a tuple, we can create new tuples by concatenating existing ones. This is done using the addition operator. The result is a new tuple containing all the elements of both original tuples. The code demonstrates how to concatenate two tuples to create a new combined tuple. Another operation we perform on tuples is repetition. By using multiplication operator, we can create a new tuple that repeats the elements of the original tuple a specified number of times. This can be useful for quickly creating tuples with repeating patterns. This code shows how to create a new tuple by repeating a single element three times. Tuples and lists have some key differences that are important to understand. Tuples are immutable, meaning once created, they cannot be changed, while lists are mutable and they can be modified. Tuple use parentheses for their syntax, whereas lists use square brackets. Tuples are typically faster than lists due to their immutability. Tuples are ideal for certain scenarios in Python programming. They are perfect for storing data that should not change, such as coordinates or RGB values. Because of their immutability, tuples can be used as dictionary keys, which is not possible with lists. Another common use case for tuple is when returning multiple values from a function. Tuples offer several advantages in Python programming.

Their immutability helps protect data that shouldn't be changed, preventing accidental modifications. As mentioned earlier, they can be used as dictionary keys due to its immutable nature. Tuples are also slightly more memory efficient than lists, which can be significant when working with large datasets. Finally, using tuples clearly communicates your intent to keep the data unchangeable, making your code more readable and self-documenting. In summary, tuples are ordered immutable sequences in Python. They can be created using parentheses and can contain elements of different types. Tuples are useful for grouping related, unchangeable data. They support various operations such as indexing, unpacking, and basic operations like concatenation. Remember that while tuple is immutable, you can still perform operations that create new tuples based on existing ones. The choice between tuple and list often depends on whether you need a mutable or an immutable sequence. Thank you for your attention. Happy coding!

Tuple Methods Video• . Duration: 4 minutes

[Notes]

Welcome, everyone. We will continue our discussion on tuples and the methods that can be applied to tuples. By the end of this video, you will be able to understand and use the built-in methods for tuples. Apply tuple methods to solve common programming problems. Recognize the immutability of tuples and its implications. Tuples in Python have fewer methods compared to list due to its immutable nature. However, they still offer several built-in functions that are useful. The primary methods we will be discussing today are count, index, length, max, and min. These methods allow us to extract information from tuples without modifying their contents, aligning with the immutable characteristics of tuples. The count method is a straightforward yet powerful tool in the tuple toolkit. It returns the number of times a specified value occurs in a tuple. This method is useful when you need to analyze the frequency of elements in your data. In our example, we are counting how many times the number 2 appears in the tuple of numbers, which demonstrates the basic usage of count. Next we have the index method, which returns the index of the first occurrence of a specified value in the tuple. This method is helpful when you need to locate a specific element within your tuple. It's important to remember that Python uses zero-based indexing, so the first element is at index 0. Our code example shows how to find the index of green in a tuple of colors, illustrating the basic use of index method. The index method becomes even more versatile when used with start and end parameters. These parameters allow you to specify a range within the tuple to search for a value. This can be useful when working with large tuples when you need to find subsequent occurrences of a value. Our code demonstrates how to find the index of number 30, but only between the indices 3 and 6 of the tuple. The len() method is a simple yet essential function that returns the number of items in the tuple. This method is crucial for understanding the size of our data and is often used in loops and conditions. In the example with book information, we can quickly determine how many pieces of information have been stored in the tuple using the len() method. The max() and min() functions are powerful tools when working with tuples containing comparable elements. They return the item with the highest value and the lowest value in the tuple, respectively. These functions allow you to quickly identify extremes in your data. The code example demonstrates how to find the maximum and the minimum value of a tuple of numbers. While tuples themselves are immutable, we can use the sorted() function to create a new sorted list based on the contents of the tuple. It is important to note that sorted() always returns a list, not tuple. This function is useful when you need

to analyze your data in a specific order without modifying the original tuple. Our code shows how to create a new sorted list from a tuple of scores, including how to sort it in the descending order. Let's recap the key tuple methods we have carved. Count, index, len, max, min and the sorted() function. Remember, due to the immutable nature of tuples, these methods do not modify the original tuple, but instead return new information or objects. These methods are incredibly helpful for data analysis and manipulation, allowing you to extract valuable insights from your tuple data without altering its structure. Thank you for your attention. Happy coding.

Strings Video• . Duration: 6 minutes

[Notes]

Welcome everyone to our lesson on strings in Python. By the end of this video, you will be able to understand what string is and its characteristics. Create and initialize string in Python. Perform basic operation on string. Recognize the immutability of string and its implications. In Python, a string is a sequence of characters. It is an immutable data type, which means once a string is created, it cannot be changed. Strings are represented by enclosing the text in either single codes or double codes. They are used to store and manipulate text data in our programs, making them a data type for various text processing tasks. Creating string in Python is flexible and straightforward. You can use single codes for simple string, double codes if your string contains single codes or triple codes for multi-line strings. This flexibility allows you to work with various types of text data efficiently. The code demonstrates these ways of creating strings, including multi-line string with triple codes. One common operation with string is concatenation, which means combining two or more strings. In Python, we use the addition operator to concatenate strings. This is useful when we need to build a complex string from smaller parts, like combining the first name and the last name to create the complete name. The code shows how to concatenate strings to form a full name. Another interesting operation is string repetition. By using the multiplication operator with a string and an integer, we can create a new string that repeats the original string a specified number of times. This can be handy for creating patterns or padding in your text data. The code demonstrates how to repeat a string multiple times using the multiplication operator. Often you will need to know the length of the string. Python provides the length function for this purpose. It returns the number of characters in the string, including spaces and punctuation. This is crucial for many string operations and validations. The code shows how to use the length function to get the length of a string. Strings in Python are sequences, which means you can access the individual characters using indexing. Remember, Python uses zero-based indexing, so the first character is at zero. You can use negative indexing to access characters from the end of the string, with negative one referring to the last character. The code demonstrates how to access characters in a string using both positive and negative indices. String slicing allows you to extract a portion of a string by specifying a range of indices. You can omit the start or the end index to slice from the beginning or the end of the string. A step can also be specified, allowing you to skip characters or even reverse the string. The code shows various examples of string slicing, including how to reverse a string using slicing. Python provides several methods for changing the case of string. The upper method converts all the characters to uppercase. The lower method converts to lowercase and capitalized method capitalizes the first character while making the rest lowercase. These methods are useful for standardizing text data. The code demonstrates how to use these case conversion methods on a

string. Searching within a string is the common operation. The find method returns the index of the first occurrence of a substring. The count method counts the number of times a substring appears. And the in operator checks if a substring exists in the string, returning a boolean value. The code shows how to use these searching methods on a string. The replace method allows you to substitute one substring with another within the entire string. This is useful for correcting errors, standardizing text, or making wholesome changes to your string data. The code demonstrates how to use the replace method to modify a string. It replaces cool with awesome. It is crucial to remember that string in Python is immutable. This means you cannot change the individual characters in a string directly. Instead, you need to create a new string with the desired changes. This immutability helps prevent accidental modifications and allows Python to optimize string handling. The code shows an attempt to modify a string directly, which raises a type error. And it also shows the correct way to create a new modified string. To summarize, remember that string in Python is immutable sequence of characters. They can be created using single, double, or triple quotes. String supports various operations like concatenation and repetition. And offer numerous methods for manipulation and searching. Always keep in mind that string methods return new strings. They do not modify the original string. These concepts are fundamental to working the text data in Python. Thank you for your attention. Happy coding.

Accessing Strings Video• . Duration: 6 minutes

[Notes]

Hello everyone, let us continue the discussion on strings and how to access string in Python. By the end of this video, you will be able to use indexing to access individual characters of a string, apply slicing techniques to extract substrings, utilize string methods for finding and extracting parts of strings. String indexing is a fundamental concept in Python that allows us to access individual characters in a string. In Python, indexing starts at 0, meaning the first character of a string is at index 0, the second is at index 1, and so on. The code demonstrates how to access different characters in a string using positive indices, including the first and third character of the word Python. Python also supports negative indexing, which allows us to access characters from the end of the string. The index negative 1 refers to the last character, negative 2 to the second to last, and so on. This feature is useful when you need to access elements from the end of the string without knowing its exact length. The code shows how to access the last and last to last characters of the word Python using negative indexing. String slicing is a powerful technique that allows you to extract a portion of a string. The basic syntax of slicing is string, start, end, and step. Start is the index where the slice begins. End is the index where it ends. It is exclusive of the last number. The step is the increment between each character in the slice. If the step is omitted, it defaults to 1. The code demonstrates how to extract substrings from a larger string using slicing. If we omit the start index, Python assumes we want to start from the beginning of the string. If we omit the end index, it assumes we want to go to the end of the string. Omitting both gives us the entire string, which can be useful for creating a copy of the string. The code shows examples of slicing with default values to extract different parts of the string. The step parameter in slicing opens up more possibilities. A step of 2 would give us every second character. A negative step reverses the direction of the slice, allowing us to easily reverse the string. This can be useful for various string manipulation tasks. The code demonstrates how to use step values in slicing, including how to reverse a string. Python

provides a method to search for substrings within a string. The index method returns the index of the first occurrence of a substring. While find does the same, but returns negative 1 if the substring is not found. The key difference is that index method raises a value error if the substring isn't found. While find does not. The code shows how to use both methods and highlights the difference when a substring is not found. The count method is a straightforward but powerful tool that returns the number of times a substring appears in the string. This can be very useful for text analysis tasks, like determining the frequency of certain words or phrases in the body of the text. The code demonstrates how to use the count method to find occurrences of substrings in a larger string. Python provides a convenient way to check if a string starts or ends with a specific substring. The startsWith method returns true if the string starts with the specified substring and endsWith does the same for the end of the string. These methods are often used in file handling to check file extensions or in text processing to verify string formats. The code shows how to use these methods to check the beginning and end of a file name. The split method is a tool for breaking a string into a list of substrings based on a specified delimiter. If no delimiter is provided, it splits on whitespace. This is extremely useful for parsing structured text data, like csv files or log entries. The code demonstrates how to use split to break a comma-separated string into a list of individual items. The join method is the counterpart of split. It allows us to concatenate a list of strings into a single string with a specified delimiter between each element. This method is more efficient than using the concatenation operator. The code shows how to use join to combine a list of words into a single sentence. To wrap up, remember that indexing allows us to access individual characters in a string while slicing lets us extract substrings. Methods like find, index, and split are tools for locating and extracting parts of strings. Always keep in mind that string in Python is immutable, so these operations return new string rather than modifying the original. By mastering these techniques, you will be equipped to handle a wide range of string manipulation tasks in your Python programs. Thank you for your attention. Happy coding!

Lesson 1 Demo Video• . Duration: 5 minutes

[Notes]

Hello everyone, we will do a demo to execute all the code snippets we have explained in immutable data structures. All the Python snippets are executed in Jupyter Notebook. The first video is on tuples. Let's create an empty tuple with just blank parentheses. To add tuples with elements, in the parentheses, we add values. It can have strings or numbers. For a single element, it's the same way, but we have to provide the comma with it. To access the tuple elements, we use the indexing method. This will return the first element and the last element of the tuple that we have created in the line 2. Unpacking the information will just list out all the details in the nested tuples will have one tuple inside the another. We can have lists also inside another tuple. So this tuple will have 4 elements, 1, 2, the tuple of a and b, list of 3 and 4. We can check whether tuples can be mutable or not. The expectation is executing this line of code will raise an error. To concatenate, we overload the addition operation. It will return a different tuple concatenating both the values. Repetition also returns a different tuple. The next video is on tuple methods. Count the number of elements in the tuple using count methods. Access each element using the index method. It will return the first occurrence. We can access between a set of values using start and end parameters. This code will search for 30 between the index 3 and 6. The length method will return the total number of elements

in the tuple, which is 4 here. The max will return the max value, which is 9 and min value, which is 1 in this example. Sorting will return the tuple in the sorted form. By default, it is ascending. If we have to make it descending, then we have to put it as reverse is true. Remember, both the sorted methods will return list. The next video is on strings. So let's create empty strings with single quote, double quote, and triple quote. To concatenate, we use the addition operation for repetition multiplication. Remember, while we concatenated, we used the space so that it comes as proper name. And in this part, since we didn't add space, it gave all the three Python values together without any space in between. Length will return the total number of characters in that string. Indexing starts with 0 and ends with the length of the character. So to access the first character, you have to use 0. Access the last one, we'll use negative 1. Slicing is pretty straightforward. We use the start parameter and the end parameter. It will take out the number of characters in that string. So the expectation is this will return the first six characters. The seventh character is not included. From the seventh character to the end, if you require it, you have to use 7 colon and leave it as it is. Using negative 1 in the step parameter will reverse the string. For case conversion, we use upper, lower, and capitalize. Searching is pretty straightforward. We can use either find, count, or int. Find and count will return the position where it was found the first occurrence again. Int will return true or false based on whether the text is available or not. Replace will replace one substring with another. To check immutability, we can try this out. The expectation is it will give us a type error saying that the string cannot be modified. The next video was on accessing the strings using indexing operations. As we had already discussed, it will return p, t, and n. The negative indexing starts with the end. If we say negative 1, it will return the last character. Negative 3 will return the last third character. Slicing will return the number of characters, the start point that we provide, till the end point. End is not included. If we use a default value, that means it will start with 0 and end with the last character. If we provide just colon, it will return the entire string. Step is an interesting one. It will execute the slicing method and skip the number of characters that we have specified by step. If we say step is 2, every second character starting from 0th position is given. If we provide negative 3, it will skip every third character from the end. Let's execute this. To find substrings, either you can use index or find. Remember, when we try to execute the last line, it will raise an error because it is not available in the original string. Count will return the number of occurrences of a particular string. Here python is repeated twice. Is is repeated twice. To check whether we have a start string and an end string, we will use startsWith and endsWith method. We can split a text using the comma separator in the split method. And to join, we use join words. In this video, we executed the python code snippets related to immutable data structures in Jupyter notebook and previewed the outputs. Do execute the files. Happy coding and thank you for your attention.

Lists Video• . Duration: 4 minutes

[Notes]

Welcome everyone to our lesson on list in Python. By the end of this video, you will be able to understand what list is and its characteristics. Create and initialize list in Python. Access and modify list elements. Perform basic operation on a list. And recognize the mutability of list and its implications. List in Python is ordered, mutable sequence of elements. Unlike tuple, which we discussed earlier, list can be modified after creation. It can contain elements of different data types, making it versatile for storing various kinds of related items. List is defined using square brackets

and is commonly used for storing collections of related items in Python. This flexibility makes list one of the most frequently used data structure in Python programming. There are three main ways of creating a list in Python. You can create an empty list using empty square brackets. Or create a list with elements by enclosing them in square brackets. The code demonstrate the methods including creating an empty list, a list of numbers, a mixed type list and creating a list from a string using the list constructor. Accessing elements in a list is similar to accessing characters in a string. We use indexing. Of course, it starts with zero. Arbitrary indexing allows us to access elements from the end of the list. We can also use slicing to access a range of elements, just like with string. The code shows how to access individual elements and slices a list of fruits. One of the key features of list is its mutability. We can change any element in a list by assigning a new value to its index. This allows us to update our data collections dynamically as our program runs. The code demonstrates how to modify an element in a list of numbers, changing the third element that is index two to 30. Lists can be combined using the addition operator. This creates a new list containing all the elements from the first list, followed by all the elements from the second list. This is useful when you need to merge two collections of data. The code demonstrates how to concatenate two lists of numbers into a single combined list. A list can contain another list as elements, creating nested structures. This allows us to represent more complex data, such as matrices or hierarchical structures. We can access elements in a nested list using multiple indices. The code demonstrates creating a 3x3 matrix as a nested list and accessing a specific element within it using two indices. Python provides several built-in functions for working with lists. We can find the length of a list with `len()` function, the maximum and min values with `max` and `min`, respectively, and sum of the all elements with `sum()` method. These functions are useful when working with numerical data. The code shows how to use these functions on a list of numbers. It is important to understand the differences between a list and a tuple. While both are sequences, list is mutable, typically used for collections of similar items that might need to be modified. Tuple is immutable and used for collections of dissimilar items or when you want to ensure that data does not change. List use square brackets, while tuple use parentheses. List is generally more flexible but slightly less efficient than tuple. To recap, remember that list is mutable ordered sequence in Python. It generally contains elements of same type and support a wide range of operations and methods for modification. Thank you for your attention. Happy coding.

Slicing List Video• . Duration: 6 minutes

[Notes]

Welcome everyone to our lesson on performing slicing operations on a list in Python. By the end of this video, you will be able to understand the concept of list slicing, using slicing syntax to extract portions of a list, apply advanced slicing techniques and modify lists using slicing. List slicing is a powerful technique in Python that allows us to extract a portion of a list, creating a new list contained with specific elements. It uses a concise syntax, list, start, end and step. Start is the index where slice begins and end is where it ends. The step is the increment between each element. This tool is useful for data manipulation techniques, allowing us to work with specific sections of our data efficiently. The basic slicing syntax involves specifying a start index and an end index. The start index is inclusive, meaning the element at this index is included in the slice, while the end index is exclusive, meaning the element at its end is not included. This allows for precise control over which

elements are extracted from the list. The code demonstrates how to extract a portion of a list of numbers, creating a new list with elements from index 2 to index 6. Python's slicing syntax becomes even more powerful when we use default values. If we omit the start index, Python assumes we want to start from the beginning of the list. If we omit the end index, it assumes we want to go to the end of the list. Omitting both gives us a copy of the entire list. The code shows all the three examples. Slicing from the start of a specific index, from a specific index to an end, and creating a copy of the entire list. Negative indexing is a feature in Python that allows us to access elements from the end of the list. The index negative 1 refers to the last element, negative 2 to the second last, and so on. This is useful in slicing when we want to work with the end of the list without knowing the exact length. The code demonstrates how to use negative indexing to get the last 5 elements of a list and all the elements except the last 3. The step parameter in slicing adds another layer of flexibility. It allows us to specify the increment between each element in the slice. A step of 2, for example, would give us every second element. This is useful for extracting patterns or subsampling data. The code shows how to use step to get every second element of a list and how to offset this to start from the second element. One of the most elegant applications of slicing is reversing a list. By using a step of negative 1, we can create a new list with the elements in the reverse order. This is often more readable and efficient than using a loop to reverse a list. The code demonstrates how to reverse a list of numbers using this slicing technique. Slicing can also be applied to a nested list, allowing us to work with multidimensional data structures. We can slice the outer list to select a specific nested list or use list comprehension to apply slicing to each nested list. The code shows two examples, selecting a portion of the outer list and using list comprehension to slice each inner list. While slicing typically creates a new list, we can also use it to modify the original list. By assigning to a slice, we can replace a range of elements with new values. This allows for efficient bulk updates to a list. The code demonstrates how to replace two elements in a list using slice assignment. It modifies the third and the fifth element. Slicing can be combined with the del statement to remove a range of elements from a list. This is more efficient than removing elements one by one, especially for a large list. The code shows how to delete a slice of elements from a list using del. It is important to understand the difference between slicing and indexing. Indexing returns a single element from the list, while slicing returns a new list containing a range of elements. Slicing is more flexible for working with portions of a list, allowing us to extract, modify, or delete multiple elements in a single operation. To summarize, remember that list slicing uses the syntax list-start-end-step to extract portions of a list. Slicing is useful for reversing lists, extracting patterns, and modifying lists efficiently. Please keep in mind that slicing typically creates a new list and does not modify the original unless you are using slice assignment or deletion. Thank you for your attention. Happy coding!

List Methods Video• . Duration: 7 minutes

[Notes]

Welcome everyone to our lesson on various methods or operations that can be performed on a list in Python. By the end of this video, you will be able to understand and use common list methods in Python, modify lists using various built-in methods, apply list methods to solve common programming problems, and recognize the impact of different methods on a list structure. List in Python comes equipped with a lot of built-in methods that allow for efficient manipulation of their

contents. These methods provide powerful tools for modifying, analyzing, and reorganizing list data. It is important to note that most list methods modify the list in place, meaning they change the original list rather than creating a new one. However, some methods return a new list or values, which is crucial to understand when working with list data in the programs. The append method is one of the most commonly used list methods. It adds a single element to the end of the list, effectively increasing the list length by one. This method is particularly useful when you need to add items to a list one at a time, perhaps in a loop or based on some condition. The code examples show how to use append to add cherry to a list of fruits, resulting in a new list with the added element at the end. While append adds a single element, the extend method allows you to add multiple elements to a list at once. It takes an iterable, such as another list, as an argument and adds each of its elements to the end of the list. This is more efficient than using append in a loop when you need to add multiple elements. The code demonstrates how to use extend to add two numbers to an existing list of numbers. The insert method provides more control over where new elements are added to the list. It allows you to insert an element at a specified position in the entire list, shifting all elements after the position one index to the right. This is useful when you need to add an element at a specific location rather than at the end. The example demonstrates inserting green at index one in the list of colors. The remove method is used to remove the first occurrence of a specified element from the list. If the element appears multiple times in the list, only the first instance is removed. This method is helpful when you know the value you want to remove, but not its index. The code shows how to remove the first occurrence of number two from a list of numbers. The pop method serves a dual purpose. It removes an element from the list and returns that element. By default, it removes and returns the last element, but you can also specify an index to remove an element from a particular position. This method is useful when you need to work with a removed element after taking it out from the list. The example shows removing the element at index one from a list of fruits and printing both the removed element and the modified list. When you need to remove all the elements from the list, clear method is the most efficient way to do so. It empties the list, leaving you with an empty list object. This is efficient than reassigning the variable to an empty list, as it modifies the existing list object rather than creating a new one. The code shows how to use clear to empty a list of numbers. The index method is used to find the position of a specific element in the list. It returns the index of first occurrence of element. This method is helpful when you need to know where in the list a particular element is located. The example shows finding the index of blue in a list of colors. When you need to know how many times a particular element appears in a list, the count method is your go-to tool. It returns the number of occurrences of a specific element. This can be useful for various data analysis tasks or for checking the frequency of elements. The code shows how to count the number of occurrences of number 2 in the list of numbers. Sort is another powerful tool that we can use for ordering the elements in a list. By default, it sorts the list in the ascending order. But you can also specify a reverse sort to get in the descending order. This method modifies the list in place, which is more memory efficient than creating a new sorted list. The example demonstrates sorting a list of numbers in both ascending and descending order. When you need to reverse the order of elements in a list, the reverse method provides a simple and efficient solution. It modifies the list in place, reversing the order of all the elements. This can be useful in scenarios such as implementing last-in-first-out data structure. The code shows how to reverse the list of fruits. Copy method creates a shallow copy of the list. This means it creates a new list object, but the elements themselves are references to the same objects as in the original list. This method is useful when you need to work with a copy of a list without modifying the original. The example

demonstrates a copy of list, modifying the copy and showing that the original remains the same. To wrap up, list methods in Python provide powerful ways to manipulate lists efficiently. Most of these methods, such as append, extend, sort, modify the list in place, changing the original list. Others, like pop, index, and count, return values based on the list contents. Thank you for your attention. Happy coding!

Dictionary Video• . Duration: 6 minutes

[Notes]

Welcome everyone to our lesson on Dictionaries in Python. By the end of this video, you will be able to understand what dictionaries are and their characteristics Create and initialize dictionaries in Python Access, modify, delete dictionary elements Utilize common dictionary methods What are dictionaries? Dictionaries in Python are unordered collections of key-value pairs, offering a mutable data structure that allows for efficient data retrieval and manipulation. The keys in a dictionary must be unique and immutable, typically strings, numbers or tuples, while the values can be of any type. This flexibility makes dictionaries ideal for representing complex data relationships. Dictionaries are defined using curly braces or dict constructor. Creating dictionaries in Python is straightforward and can be done in multiple ways. You can use curly braces to define key-value pairs directly, employ the dict constructor for a more function-like syntax, or start with an empty dictionary and add elements later. The code examples demonstrate these three methods, creating a person dictionary with curly braces, a car dictionary using the dict constructor, and of course an empty dictionary for future use. Accessing values in a dictionary is typically done using square bracket notation with a key, but Python also offers the get method for safer access when a key might not exist. The code shows how to access the name value using square brackets and the age value using the get method, as well as demonstrating how the get method can provide a default value if a key is not found. Dictionaries in Python are mutable, allowing you to add new key-value pairs or modify existing values easily. The code illustrates how to add a new city key-value pair to a person dictionary and how to modify the existing age value, demonstrating the flexibility of dictionaries in updating and expanding data structures. Python provides several ways to remove elements from a dictionary, each with its own use case. The del keyword allows you to remove a specific item by a key. The pop method removes an item and returns its value, whereas the pop item removes and returns the last inserted item. The code demonstrates these three methods, ultimately resulting in an empty dictionary. Dictionaries come with built-in methods that allow you to access their components efficiently. The keys method returns a view of all the keys, values return a view of all the values, and items return a view of all key-value pairs as tuples. The code shows how to use these methods on a car dictionary, providing different ways to inspect and iterate over the dictionary contents. Iterating through dictionaries is a common operation, and Python offers multiple ways to do this efficiently. You can iterate over keys directly or use the items method that we had seen to access both keys and values simultaneously. The code demonstrates both the methods of iteration, showing how to print each key-value pair in the car dictionary. Python offers multiple ways of merge dictionaries, with newer versions of Python providing more concise syntax. There are two ways of doing this, using the double asterisk unpacking operator and the newer pipe operator, both of which combine two dictionaries with the second dictionary's value overwriting any duplicate keys from the first. Dictionaries find a wide application in various programming scenarios due to its flexibility and

efficiency. They are excellent for storing configuration settings, caching function results to improve performance, counting occurrences of items in a dataset, representing real-world objects with multiple attributes, and even implementing simple databases. Their key-value structure makes them ideal for these tasks, allowing for intuitive data organization and fast lookups. While both dictionaries and lists are essential data structures in Python, they serve different purposes. Dictionaries excel at storing key-value pairs and offer fast lookups based on keys, making them ideal when you need named access to values. Lists, on the other hand, are ordered collections of integer indices, better suited for sequential data or when the order of elements matters. Choose dictionaries when you have natural keys for your data and need quick access. Choose lists when you need to maintain a specific order or work with simple sequential collections. To summarize, dictionaries in Python are powerful data structures that store key-value pairs, keys being unique and immutable. They offer fast lookups and a flexible data structure, making them ideal for many data processing and analysis tasks. Common methods like GET, KEYS, VALUES, and ITEMS provide various ways to interact with dictionary data. Dictionaries are particularly useful for representing complex data structures and are essential in many programming scenarios. Using dictionaries will significantly enhance your ability to organize, manipulate, and analyze data efficiently in Python. Thank you for your attention. Happy coding!

Set Video• . Duration: 5 minutes

[Notes]

Welcome everyone to our lesson on sets in Python. By the end of this video, you will be able to understand what sets are and their characteristics. Create and initialize sets in Python. Add and remove elements from sets. Sets in Python are unordered collections of unique elements, offering a mutable data structure that excels at storing distinct values. The elements in their set must be immutable, such as numbers, strings, or tuples, ensuring that the contents of a set remains hashable. One of the key features of sets is that they do not allow duplicate elements, automatically eliminating any redundancy in the data. Sets can be defined either by using curly braces or the set constructor. Creating sets in Python can be done in multiple ways, each suited to different scenarios. You can use curly braces to define a set directly with elements, or employ the set constructor to create a set from an iterable list or initialize an empty set using the set constructor. The code example demonstrates these three methods, creating a set of fruits using curly braces, a set of numbers using set constructor with a list, and an empty set using the set constructor. Sets in Python have several distinct characteristics that can make them unique among data structures. They are unordered, meaning the elements have no specific sequence and cannot be indexed. The uniqueness property ensures that each element appears only once in the set, automatically eliminating duplicates. Sets are mutable, i.e. you can add or remove elements after creation. However, it is important to note that sets cannot contain mutable elements like list or dictionary, as these would compromise the set's ability to hash its contents efficiently. Python provides methods to add elements to a set, allowing you to expand your collection of unique items. The add method is used to insert a single element into the set, while update method allows you to add multiple elements at once. The code example shows how to use add to include yellow in a set of colors, and then use update to add both orange and purple to the same set, demonstrating the flexibility of set modification. Removing elements from a set can be done in several ways, each with its own use

case. The remove method eliminates a specific element, but raises a key error if the element is not found. The pop method removes and returns an arbitrary element, which can be useful in some algorithms. The discard method removes a specific element if it is present, without raising an error if it is not found. The code demonstrates these three methods on a set of numbers, showing the different approaches to element removal. Frozen sets are an immutable version of the regular sets. It provides all the benefits of sets with the added guarantee that their contents cannot be changed after creation. They are created using the frozen set constructor, and are useful when you have a hashable set, such as for use as a dictionary key. The code shows the creation of a frozen set and comments on the fact that attempting to add an element would raise an attribute error, highlighting the immutability of frozen sets. Sets find wide application in various programming scenarios due to their unique properties. They excel at removing duplicates from sequences, providing a quick way to obtain unique values. Sets are highly efficient for membership testing, allowing for fast lookups to check if an element exists. They support mathematical set operations like union, intersection, and difference, making them ideal for comparing and combining datasets. Sets are also valuable for finding unique elements in the data and eliminating duplicate records in the database operations, showing their versatility in data processing tasks. To summarize, sets in Python are an unordered collection of unique elements, offering powerful tools for managing distinct data. They are useful for removing duplicates and performing set operations efficiently. Common methods like add and remove provide for manipulating sets. Frozen sets extend the concept to immutable collections, providing hashable set functionality. Thank you for your attention. Happy coding!

Set Operations Video• . Duration: 6 minutes

[Notes]

Welcome everyone to our lesson on set operations in Python. By the end of this video, you will be able to understand and apply various set operations in Python. Use set methods and operators for different set operations. The union operation combines elements from two sets, effectively merging them while removing any duplicates. In Python, this can be achieved using the union method or the pipe operator. Both approaches produce the same result, creating a new set that contains all unique elements from both original sets. The code example shows how to perform a union operation on two sets of numbers, resulting in a set containing all unique elements from both the sets. The next operation is intersection, which returns the elements that are common to both sets, essentially finding the overlap between two collections. This can be performed using the intersection method or the ambassador operator in Python. Both the methods yield the same result, creating a new set with only the elements that exist in both original sets. The code demonstrates how to find the intersection of two sets of numbers, resulting in a set containing only the elements present in both original sets. The difference operation returns elements that are there in the first set and not in the second set, allowing us to identify unique elements in one set relative to another. This operation can be performed using the difference method or the negative operator. Both approaches produce a new set containing elements from the first set that are not present in the second set. The code example illustrates how to find the difference between two sets of numbers, resulting in a set with elements unique to the first set alone. The symmetric difference operation returns elements that are in either set but not in both, essentially finding the non-overlapping elements between two sets. This can be achieved using the symmetric difference method or the caret operator. Both methods create a new

set containing elements that are unique to each set when compared to the other. The code shows how to find the symmetric difference between two sets of numbers, resulting in a new set with elements that are either in the first set or in the second set but not in both. The subset and superset operations allow us to check the relationship between two sets, determining if one set is contained within another or if it contains another set entirely. The `is-subset` and `is-superset` methods, as well as the `less-than-equal-to` or `greater-than-equal-to` can be used for these checks. The code demonstrates how to check if one set is a subset of another or if one set is a superset of another, using both methods and the operators. The concept of disjoint sets refers to sets that have no elements in common. Python provides `is-disjoint` method to check if two sets are disjoint. This method returns `true` if the sets have no common elements and `false` otherwise. The code example shows how to check if two sets of numbers are disjoint, which would return `true` in this case as the sets have no overlapping elements. Another method is update operation, which allows us to modify a set in place based on another set, which can be more efficient than creating new sets for each operation. Python provides methods like `update` and `intersection update` for these in-place modifications. The code demonstrates how to use method `update` to add all elements from one set to another and how to use `intersection update` to keep only the common elements between the two sets. One practical application of set operation is data deduplication, where we remove duplicate entries from a collection of data. Sets are particularly efficient for this task due to its property of only containing unique elements. The code example shows how to use a set to remove duplicate email addresses from a list, resulting in a set of unique email addresses. To wrap up, we have covered various set operations, including union, intersection, difference, and symmetric difference. These operations can be performed using either methods or the corresponding operators in Python, providing flexibility in how you write your code. Set operations are useful for tasks like data deduplication and finding unique elements across collections. They offer efficient solutions for membership testing and removing duplicates while making them invaluable tools in data processing and analysis. By mastering these set operations, you will be well-equipped to handle complex data manipulation tasks in your Python programs. Thank you for your attention. Happy coding!

Lesson 2 Demo Video• . Duration: 10 minutes

[Notes]

Hello everyone, we will do a demo to execute all the code snippets we have explained in the lesson Mutable Data Structures. All the Python snippets are executed in Jupyter Notebook. Let's get started. Create an empty list using square bracket. For creating list with just elements of one type, add the elements in the square brackets. The second line, mixed list, displays a list of multiple elements. It has numbers, or integers, strings, float, and another list also, with 4 and 5 elements. List can also be created using the list constructor. Here each character will go as part of the list, given in single quotes. Place the list elements using the slice operation. Modify the list. Since it is mutable, it can be modified. So list name in square brackets the position and it will change. So we have a list 1, 2, 3, 4, 5. The expectation is the position 2 will have the value of 30. Yes, we are getting it. Position 2 is 0, 1, 2. Concatenate will give us another combined list. It's the third list that gets created. To create metrics, we can nest list. So we have one square bracket, which is mentioned here, which is highlighted in green. And within that, we have different other lists separated by comma. Let's print that metrics as it is, instead of each element. It prints the metrics as it is, what we have provided.

The common list operations include length, max, min and sum. You don't have to write. The next video is on slicing list. Let's execute those statements where what we had discussed in the lesson. To slice, we use the square brackets and provide the start and end. What will this give? All the, the first line, the expectation is it will list out all the values till the index 5. The second will list out all the values from index 5 to the end. The third will give the list as it is. Negative indexing will give us from the end of the given string. Step will skip. So the expectation of first is it will give 0th position, second position, fourth position like that. And when we give start and step, it will start at this position and skip. So the first line will give us all the even numbers and the second line will give us all the odd numbers. Reversing a string can be done using the negative indexing method. To slice a nested string, we have to specify either the position or the row value. We can modify the string by providing the slicing again on the left-hand side of all the assignment operator. The expectation is between 2 and 4, whatever the numbers are, it is getting replaced by 20 and 30. To delete a string, to delete the elements of a list, we use slice operator. This will delete 2 and 3 which are there in position 2 and 3 respectively. The next video is on list methods. AppendCherry will add third element to the given fruits list. Extend will do the same thing. It will extend the given numbers with the second list that is provided. Note that both the lists are getting modified. It is the same list that gets modified. You can use the insert a new value at a position. So here red is at 0, blue is at 1, but now insert 1, green will insert green at the position 1. So the expected result is red, green, and blue. Remove will remove an element. Here it will remove the first occurrence of the number 2. Stop will remove the element. It will get stored in a different list. If the removed element is random, clear will empty the entire list. Index of blue will return the position of the word or the element in the list, which is 1. Sort method will sort the list in either ascending or descending order. For descending, you have to give reverse equals true. To reverse the entire list, you can use the reverse method also in addition to the slice operation. Create a copy of the entire list. So you get two copies and we are modifying copied 0 equals 10 is modifying the copied version of the original list. Let's proceed to the video on dictionaries. Create dictionaries using multiple methods. Use curly braces. This is the key. This is a value. So each pair will have a key value associated with it. You can also create it using the dictionary constructor. Access the dictionary values by providing the key. So this will list out by using the key name. It will give us the value for the name. Instead of that, you can use the get method also. Modifying a dictionary is possible because it is a mutable one. We are modifying the dictionary by providing additional key value pair. The key is city and value is New York. We can change the existing values also. Age was 30 initially, now it has been modified to 31. Delete the dictionary using the pop method. Pop method will list out the key that we are providing. Pop item will list or will remove the last element in the dictionary. So the expectation is the first line will give return New York, second will return 30. Delete person age. This line will delete that particular key value pair. And when we are trying to access it, it will give us error because there is no age provided there. Delete person name will delete that particular key value pair also as shown. And finally, you get an empty dictionary. Keys will return brand, model, year, etc. Values will be Toyota, Corolla and 2020. And items will be each of the pair that you see in the original dictionary. To iterate through the dictionaries, you can use the for method. You may have to add an iterator if required. Nexted dictionaries are pretty straightforward to be made. You just create a dictionary of dictionary. Dictionary can have lists and tuples also inside it. Merging a dictionary is not pretty straightforward as we have. If we are using 3.5 version, you have to use the merge. You have to use the double asterisk symbol. My version is 3.5. So only that gets executed 3.9 version can we can use the pipe symbol. The next video is on sets. Let's create various sets by using either the curly braces, the set

constructor or the empty set using the set constructor. To add elements, set again is mutable. So we can add elements to it. We use either the add method or the update method. To remove elements, let's do one by one. We have a set here, which is having 1, 2, 3, 4, 5. If I say remove three, this will give error if it is not found. Remove will remove some random value and discard two will discard that particular element. It will not return an error if it is not found. Frozen sets can be created using frozen set constructor. Remember this cannot be modified. So that is why we are getting an attribute error. To figure out if there are any unique values in the set, you can just say set duplicate list, where the duplicate list has different duplicate values. Look at the result of the set. It will return only those elements which are unique. The next video is on set operations, where we are trying to do all the set operations that we have learned in our mathematics course. Union can be done using either union or the pipe operator. It will combine both the lists. Repeated elements will be retained only once. So the expected is 1, 2, 3, 4, 5. For intersection, the expectation is only the common elements. So it should return a set of 3 and 4. For the difference operator, whatever is available in the first set will be retained because it is set 1 minus set 2. What will happen if we do the reverse? Let's try it out. Set 2 minus set 1. It will return the elements which are present in set 2, not present in set 1. For symmetric difference, we use the caret operator. Let's see the result. It will return those elements which are not there in the other list. So it will return 1, 2, 5, and 6. Subset. If set 1 is a subset of set 2, it will return true. These are Boolean methods. So set 1 is set 2. Is set 2 subset of superset of set 1 true? So that is also returning true or false value. Disjoint sets. Are they disjoint? Are they different? It will return true if they are different. For updating the existing set, you use the update method or intersection update method. These two will update based on the common elements that you have. The first one returns 1, 2, 4, and 5. The second one will return only 3 because it is intersection. Multiple sets can be combined together. So we say set 1 union set 2 and 3. So this will return 1, 2, 3, 4, 5, and 6, 7. An example where we are duplicating or removing duplicates. User at example.com are duplicated. So when you execute it as a set or when you convert it as a set, only one copy is maintained. In this video, we have executed the Python code snippets related to mutable data structures in Jupyter Notebook and previewed the outputs. Do execute the files. Happy coding and thank you for your attention.

NumPy Arrays Video• . Duration: 7 minutes

[Notes]

Welcome all to the topic NumPy Arrays in Python. By the end of this video, you will be able to understand what NumPy Arrays are and their advantages, create NumPy Arrays using various methods, perform basic operations on NumPy Arrays, and understand the difference between NumPy Array and Python List. NumPy is short for Numerical Python. It is a fundamental library for scientific computing in Python. It provides a robust support for large multidimensional arrays and matrices, which are essential for handling complex numerical data. Additionally, NumPy offers a comprehensive collection of mathematical functions designed to operate efficiently on these arrays, making it an indispensable tool for data scientists and researchers. NumPy Arrays offer several advantages over standard Python List when it comes to numerical operations. They are significantly faster and more memory efficient, which is crucial when dealing with large datasets. NumPy provides a wide range of mathematical operations tailored for array manipulation, allowing for complex calculations to be performed with ease. Furthermore, NumPy's compatibility with other

scientific Python libraries makes it an excellent foundation for more advanced data analysis and machine learning tasks. To start using NumPy in your Python scripts, you need to import it. The standard convention is to import NumPy with the alias `np`, which makes it easy to refer to NumPy functions throughout the code. NumPy offers various methods to create arrays. You can create an array from a Python list using `np.array`, or generate a sequence of numbers with `np.arange`, or create arrays filled with 0s or 1s using `np.zeros` and `np.ones` respectively. These methods provide flexibility in initializing arrays for different purposes. For more specialized array creation, NumPy provides functions like `np.linspace`, which creates an array of evenly spaced numbers over a specified interval. If you need arrays of random numbers, `np.random.rand` is useful, allowing you to generate arrays of any shape filled with random values. The code demonstrates using `np.linspace` to create an array of evenly spaced numbers between 0 and 1, and `np.random.rand` to generate a 3x3 array of random numbers. NumPy arrays come with several useful attributes that provide information about the array's structure and content. The `shape` attribute tells you the dimensions of the array, `ndim` gives you the number of dimensions, `size` returns the total number of elements, and `dtype` indicates the data type of the elements. The code example creates a 2D array and demonstrates how to access its attributes. Accessing elements in NumPy arrays is similar to Python List, but with added capabilities for multi-dimensional arrays. You can access individual elements using square brackets. For multi-dimensional arrays, you can use comma-separated indices to access specific elements efficiently. The code shows how to access individual elements using indexing, array in square brackets too, or slice arrays and access the elements in a 2D array using comma-separated indices. NumPy allows for efficient element-wise operations on arrays. You can easily add or multiply arrays with the addition and multiplication operation. The multiplication is a scalar operation. NumPy also provides a range of functions for array calculations, like `np.sum`, for adding all the elements. These operations are optimized for performance, making NumPy ideal for numeric computations. Broadcasting is a powerful feature in NumPy that allows operations between arrays of different shapes. It automatically applies operations to each element, making it possible to perform calculations between arrays and scalars, or between arrays of compatible shapes. This feature significantly simplifies many numerical computations and reduces the need for explicit loops. The code example shows how adding a scalar to a 2D array automatically applies the operation to each element of the array. NumPy allows you to change the shape of an array without altering its data using the `reshape` method. For example, `array.reshape(3,4)` would transform a 1-dimensional array of 10 elements into a 2-dimensional array with 3 rows and 4 columns. This flexibility is crucial when working with data that needs to be reshaped for specific algorithms or visualizations. NumPy provides functions to combine multiple arrays or split a single array. You can concatenate arrays using `np.concatenate`, which joins arrays along an existing axis. Conversely, you can split an array into multiple sub-arrays using `np.split`. The code shows how to use `np.concatenate` to join two arrays and `np.split` to divide an array into two equal parts. While NumPy arrays and Python lists may seem similar, they have key differences. NumPy arrays are homogeneous, meaning all elements must be of the same type, whereas Python lists can contain elements of different types. NumPy arrays support vectorized operations, allowing for different element-wise calculations without explicit loops. Additionally, NumPy arrays have a fixed size after creation, while Python lists are dynamic and can change size. To summarize, NumPy arrays are fundamental to numerical computing in Python, offering efficiency and a wide range of mathematical operations. They can be created using various methods and support operations like indexing, slicing, and reshaping. The ability to perform vectorized operations and broadcasting makes NumPy arrays powerful tools for efficient

computations. Thank you for your attention. Happy coding!

NumPy Data Types Video• . Duration: 7 minutes

[Notes]

Welcome all to the topic NumPy datatypes in Python. By the end of this video, you will be able to understand the various datatypes available in NumPy, specify datatypes when creating NumPy arrays, convert between different NumPy datatypes, and recognize the importance of datatypes in memory management and performance. NumPy provides a more extensive and specific set of datatypes compared to standard Python. These specialized datatypes allow for more efficient memory usage and faster computations, which is crucial when dealing with large datasets. In NumPy, these datatypes are referred to as D-types, and understanding them is a key to optimizing your numerical computations. NumPy offers a wide range of datatypes to suit various computational needs. For integers, we have `int8`, `int16`, `int32`, and `int64`, with corresponding unsigned versions. Floating point numbers are represented by `float16`, `float32`, and `float64`. Complex numbers use `complex64` and `complex128`. Precision values are stored as `bool` and strings as `str`. This variety allows you to choose the most appropriate D-type for your specific requirements, balancing precision and memory use. When creating NumPy arrays, you can specify the datatype using the D-type parameter. This allows for precision control over how your data is represented in memory. The code explains creating two arrays, one with 32-bit integers and another with 64-bit floating point numbers. By explicitly specifying the datatype, you ensure that your data is stored and processed exactly as intended. To determine the datatype of an existing array, you can use D-type attribute. The code example shows how to check the datatypes of integer and floating point arrays. It is important to note that the default integer type can be either 64-bit or 32-bit, depending on your system. Understanding the datatypes of your arrays is crucial for predicting behavior and optimizing the performance. NumPy provides the `astype` method for converting between different datatypes. This is useful when you need to change the precision of your data or convert between integer and floating point representations. The code example demonstrates converting an integer array to a 64-bit floating point array. It is important to be aware of potential precision loss or overflow when converting between datatypes. When performing operations between arrays of different types, NumPy automatically upcasts to a more precise type. This ensures that precision is maintained in calculations. The code shows adding a 32-integer array to a 64-bit floating point array, resulting in a 64-bit floating point array. This automatic upcasting prevents unexpected loss of precision in your computations. Different datatypes use varying amounts of memory. The `itemSize` attribute allows you to check the memory usage per element in bytes. The code example compares the memory usage of 8-bit and 64-bit integer arrays. Improving memory usage is crucial for optimizing storage and processing large datasets. The choice of datatype can significantly impact computation speed. Smaller datatypes generally lead to faster computation, but may be prone to overflow or underflow. The code compares performance of summing arrays with different integer sizes. It is important to balance speed and numerical precision based on your specific requirements. Note that the last two lines at this % time is a magic command, and it is specific to Jupyter Notebooks, and it may not work in other Python environments. For the other environments, it is suggested that you might use a different timing method. NumPy provides special datatypes that adapt to the system architecture. `np.int` matches the system's word size, whereas `np.float` and `np.complex` are shorthands for 64-bit

floating point and 128-bit complex numbers, respectively. These types can be useful for writing more portable code. Structured arrays allow you to create arrays with compound datatypes, including multiple named fields of different types. This is useful for representing complex data structures, such as database records. The code examples demonstrate creating a structured array to store student information with names and grades. NumPy provides support for handling missing or invalid data through masked arrays and the use of NaN, standing for Not a Number. The `np.isnan` function allows you to check for NaN values in your arrays. This capability is crucial for dealing with real-world datasets that often contain incomplete or missing information. When working with NumPy datatypes, it is important to choose the smallest datatype that you can accurately represent your data. Be aware of potential overflow and underflow issues, especially when working with smaller integer types. Use appropriate types for better interoperability with other libraries and always consider the trade-offs between memory usage, performance, precision when selecting datatypes of your arrays. In conclusion, NumPy offers a variety of precise datatypes that significantly affect memory usage and computation speed. The ability to specify and convert between different datatypes give you fine-grained control over your numerical computations. Understanding these datatypes is crucial for efficient numerical computing, allowing you to optimize your code for both performance and accuracy. Remember to choose appropriate datatypes based on your specific needs and constraints, balancing the factors such as precision, memory usage, and computation speed. Thank you for your attention. Happy coding!

Arithmetic with NumPy Video• . Duration: 5 minutes

[Notes]

Welcome all to the topic Arithmetic with NumPy in Python. By the end of this video, you will be able to perform basic arithmetic operations on NumPy arrays. Understand and utilize NumPy's broadcasting feature. Recognize the efficiency of NumPy operations compared to Python Lists. NumPy allows us to perform basic arithmetic operations on arrays efficiently. These operations are applied element-wise, meaning they are performed on corresponding elements of the arrays. In this example, we create two arrays A and B and demonstrate addition, subtraction, multiplication and division. The code shows that when we add A and B, each element of A is added to the corresponding element of B. Similarly, for subtraction, multiplication and division, the operations are performed element by element. This allows for quick and easy calculations on large datasets without the need for explicit loops. NumPy extends beyond basic arithmetic to more complex mathematical operations, all performed element-wise. Here we demonstrate power, square root and exponential functions. The code shows how we can easily square each element of an array, calculate the square root of each element and compute the exponential of each element. Note that NumPy automatically handles type conversion, ensuring that the results are in the appropriate format. Scaling is a powerful feature in NumPy that allows operations between arrays and scalars. In this example, we see how a scalar value can be applied to each element of an array. The code demonstrates adding 5 to each element, multiplying each element by 2 and dividing each element by 2. This feature eliminates the need for explicit loops, making the code more concise and efficient. It's particularly useful when you need to perform the same operation on all elements of an array. Broadcasting also applies to operations between arrays of different shapes. In this example, we have a 2D array A and a one-dimensional array B. NumPy automatically broadcasts the smaller

array B across the larger array A. Each row of A is added to B, resulting in a new two-dimensional array. This feature is powerful for operations that would otherwise require complex looping structures, making NumPy code both more readable and more efficient. NumPy also supports element-wise comparison operations, which return Boolean arrays. The code demonstrates equality greater than and less than comparisons between two arrays. Each comparison is performed element by element, resulting in a Boolean array of same shape as input arrays. These operations are useful for data analysis and filtering tasks, allowing you to easily identify elements that meet a specific criteria. For Boolean arrays, NumPy provides logical operations. The code shows examples of logicalAND, logicalOR, and logicalNOT operations. These functions operate element-wise on the Boolean arrays, allowing for complex logical operations to be performed efficiently. The logicalAND function returns true only when both the input arrays are true. The logicalOR returns true if either input array is true. The logicalNOT function inverts the Boolean values. These operations are essential for conditional processing of data. NumPy's universal functions, or ufuncs, are functions that operate element-wise on arrays. They provide fast, vectorized wrappers for simple functions. The code demonstrates the use of np.add, np.multiply, and np.power as alternates to the standard arithmetic operators. These ufuncs can often be more explicit and can offer performance benefits, especially when working with large arrays. They also provide additional optional parameters for more complex operations. In conclusion, NumPy provides efficient element-wise arithmetic operations that can significantly speed up numerical computations compared to standard Python lists. The broadcasting feature allows for operations between arrays of different shapes, greatly simplifying many common programming tasks. These capabilities make NumPy an essential tool for scientific computing and data analysis in Python. By leveraging these features, you can write more concise, readable, and efficient code for numerical operations. Thank you for your attention. Happy coding!

Indexing and Slicing Arrays Video• . Duration: 6 minutes

[Notes]

Welcome all to the topic Indexing and Slicing Arrays in Python. By the end of this video, you will be able to understand basic indexing in NumPy arrays, apply slicing techniques to extract sub-arrays, use advanced indexing methods including Boolean and fancy indexing, modify array elements using indexing and slicing. In NumPy, we can access individual elements of a one-dimensional array using square brackets, similar to Python Lists. We can use both positive and negative indices. The code example demonstrates creating a one-dimensional array and accessing its first element with arr[0], which outputs 1, and its last element with negative 1, which outputs 5. The indexing allows us to easily retrieve specific elements from our array. For two-dimensional arrays, we use comma-separated indices to specify both row and column. The code creates a two-dimensional array and shows how to access a specific element using the array name in square brackets, 1, 2, which returns the element in the second row and third column. Remember, indexing starts at 0. We can also access the entire rows by specifying only the row index as shown with array name 0, which returns the entire first row. Slicing in NumPy allows us to extract sub-arrays using the syntax start, stop, and step. The code demonstrates three slicing operations. Array 2 colon 7 extracts elements from index 2 to 6. Array colon colon 2 selects every second element from the entire array. And array colon colon minus 1 reverses the array. Omitting the start or stop values slices from the beginning or

the end of the array, respectively. Slicing can also be applied on 2D arrays, allowing us to select specific rows and columns. The code shows how to slice both dimensions simultaneously with `array[2:4, 1:]`. This selects the first two rows and all columns from the second onwards, resulting in a 2 by 2 sub-array. NumPy allows to modify multiple elements at once using slicing. The code demonstrates assigning the value 10 to a slice of array with `array[2:4] = 10`. This operation modifies the third and fourth element of the array. When assigning a scalar value to a slice, it is broadcast across all elements in that slice. Boolean indexing is a powerful feature that allows us to select elements based on conditions. The code shows creating a Boolean mask using a comparison operation, `array > 2`, then using this mask to select elements. We can also combine the comparison and indexing in one step, as shown. This technique is incredibly useful for filtering data based on specific conditions. Fancy indexing allows us to select multiple elements using an array of indices. The code demonstrates selecting elements at indices 1, 3, and 4 from the original array using `array[indices]`. This method provides a flexible way to extract specific elements in a custom order. Another way is using `np.ix_` function. It is useful for selecting subsets of multi-dimensional arrays. It creates an open mesh from multiple sequences. The code demonstrates how to use `np.ix_` to select specific rows and specific columns from a two-dimensional array. This method is useful when you need to select non-contiguous subsets of your data. We can use multi-dimensional arrays of indices for even more complex indexing. The code creates a one-dimensional array with a range 10, then uses a two-dimensional array of indices to select elements. The result is a 2D array, where each element corresponds to the element in the original array at the specified index. NumPy provides functions to find indices where conditions are met. `np.where` returns the indices where the condition is true, while `np.nonzero` returns indices of non-zero elements. The code demonstrates using these functions to find elements greater than 3 in an array. These functions are useful for locating specific elements in large datasets. Masked arrays allows us to work with data that may contain missing or invalid values. The code shows how to create a masked array using `np.ma.masked_array`, where certain elements are hidden based on the Boolean mask. The feature is invaluable when dealing with real-world data that often contains gaps or errors. In summary, NumPy provides a rich set of indexing and slicing capabilities. Basic indexing allows us to access individual elements, while slicing extracts sub-arrays. Boolean indexing selects elements based on conditions. Fancy indexing uses integer arrays for selection. Thank you for your attention. Happy coding!

NumPy Functions Video • Duration: 6 minutes

[Notes]

Welcome all to the topic NumPy Functions in Python. By the end of this video, you will be able to understand and apply various NumPy mathematical functions, utilize NumPy's statistical functions for data analysis, apply array manipulation and utility functions, recognize the efficiency of NumPy functions compared to Python built-ins. NumPy provides a variety of basic mathematical functions that operate element-wise on arrays. In this example, we use `np.abs()` to calculate the absolute value of each element in the array, `np.sqrt()` to compute the square root, and `np.square()` to square each element. These functions can be combined for more complex operations as shown in the second line, where we take the square root of the absolute values. NumPy also offers exponential and logarithmic functions that operate element-wise on arrays. The code demonstrates the use of

`np.exp()` for exponential calculations and `np.log()` for natural logarithm. Use `np.log10()` for base-10 logarithm. The trigonometric functions in NumPy allow for easy computation of sine, cosine, and tangent values. The code creates an array of angles in radians and applies `np.sine`, `np.cos`, and `np.tan` functions to it. These functions expect input in radians, which is why we use `np.pi()` to represent the pi value. NumPy provides efficient functions for calculating measures of central tendency. The code demonstrates the use of `np.mean()` to calculate the arithmetic mean, `np.median()` for median value, `np.average()` for a weighted average, which in this case without weight specified is equivalent to the mean. These functions are fundamental in descriptive statistics and data analysis, allowing quick summarization of large data. To measure the spread and variability of data, NumPy uses functions for calculating variance, standard deviation, and range. The code shows the use of `np.var()` for variance, `np.std()` for standard deviation, and `np.ptp()` for the range. PTP stands for peak-to-peak. NumPy provides functions for counting and binning data, which are essential for data analysis and visualization. The code demonstrates `np.unique()` to find unique values, `np.bincount()` to count occurrences of non-negative integers, and `np.histogram()` to create a histogram of the data. These functions are useful when preparing data for visualization or when analyzing the distribution of values in the dataset. Aggregation functions in NumPy allow for efficient computation of sums or other aggregations across arrays. The code shows how to use `np.sum()` to calculate the sum of all elements in an array as well as sums along specific axes. The axis parameter determines whether the operation is performed row-wise, axis is one in row, or column-wise where axis is zero. NumPy provides a comprehensive set of linear algebra operations. The code demonstrates the use of `np.dot()` for matrix multiplication and `np.linalg.det()` for determining the determinant of a matrix. Reshaping arrays is a common operation in data preprocessing. The code here displays how to use `np.reshape()` to change the shape of an array without altering its data. `np.transpose()` is used to swap the axes of an array. These functions are used when working with multidimensional data or when preparing data for specific algorithms that expect a certain input shape. NumPy provides functions for combining arrays vertically or horizontally. The code demonstrates `np.vstack()` for vertical stacking and `np.hstack()` for horizontal stacking. These functions are useful when you need to combine multiple arrays when merging different data sets or creating more complex data structures for simpler computations. Splitting arrays into smaller sections is the inverse operation of stacking. The code shows how to use `np.hsplit()` for horizontal splitting and `np.vsplit()` for vertical splitting. These functions are used when you need to separate data into training and testing sets or when working with data that naturally divides into distinct subsets. Another functionality of NumPy is some utility functions provided for creating arrays with specific values or patterns. We have already seen the usage of `np.zeros` to create an array filled with 0s, `np.ones` for an array of 1s. Similar to those, we have `np.eye` for identity metrics, `np.linspace()` to create an array of evenly spaced values. These functions are used for initializing arrays efficiently. How about generating random numbers? The code shows how to use `np.random.rand()` for generating random numbers from a uniform distribution. If we want normal distribution, then we use `np.random.randn()`, `np.random.randint()` for random integers within a specified range. To summarize, NumPy provides a wide range of efficient mathematical and statistical functions that are essential for data analysis and scientific computing. The array manipulation functions allow for reshaping, stacking, and splitting arrays, providing flexibility in data handling. NumPy functions simplify common tasks like array creation and random number generation. Thank you for your attention. Happy coding!

Lesson 3 Demo Video• . Duration: 11 minutes

[Notes]

Hello, everyone. We will do a demo to execute all the Python code snippets related to NumPy library. All the snippets are executed in Jupyter Notebook. Let's get started. We'll start off with a video on NumPy arrays. Remember for NumPy arrays, we have to import the module NumPy, and np is the alias that we commonly provide. Create the NumPy arrays using a Python list or using AR range or zeros or ones. Now, np.zeros, it will create five zeros here, as shown, and np.one seven will create an array of seven values. The dot symbol in the output says that it is a floating point number. We can create using linspace also. Linspace will create an array from zero to one with five values. All of them are again floating point. Random.rand will create a matrix here, three metrics, with random values. All the random values will be between zero and one. To access the array attributes like shape and dim, we just provide the array name and the attribute name. So the first one will give us a shape of, for this particular array, it will give a shape of two, three. This will give us the dimensions, which is two dimensions. There are two rows and two, three columns, so two rows and columns. The size will specify the size of the array, and the type will specify the data type that is being used. So shape is two, three. Number of dimension, it is just rows and columns, so two dimensions. The number of values are six, and dtype is int64, the default for integers. The indexing and slicing operations remain the same that we have mentioned so far. We will just use the square brackets and provide the start index and the end index for slicing. If it is a 2D array, again, we use the comma separator to specify which cell we are referring to. So this will return row one, column two. So row one, column two from this one, row one, column two, which will be six. Remember, it is zero index. Broadcasting operation will help us to do the addition or add 10 to each element, we get an element of 11, 12, 13, 14, 15, 16, respectively. Let's look at the interesting one, shape. Arrange 12 will return 12 values in one dimension. Let me keep this as a separate one so that it is visible for you. So let's see how this gets executed. This will return 12 values, zero to 11. Now what about reshape? This single matrix or this single array is returned as a matrix of size 3, 4. So reshape is a very powerful method that we use, especially in machine learning. Concatenate and splitting, we use the concatenate and split method. It's pretty straightforward. Concatenate will join the two arrays and split the concatenated array into number of components that you have mentioned. So here it is two, so it will return two arrays. The next video was on arithmetic in NumPy. The basic arithmetic operation include addition, subtraction, multiplication, and division. Division is floating point division. Remember each value, it's an element-wise operator. So you will see that each value, corresponding values are added, multiplied or subtracted. We can extend this element-wise operation for other mathematical functions like exponentiation, square root, or power. Broadcast the operation where the value will be applied on every element of the array. So you will get 6, 7, and 8 for addition. Similarly for multiplication and division. Again, this division is floating point division, so you get it in decimal values. If we broadcast two arrays, the smaller will be broadcasted to the larger one. So the expected output is 11, 22, 33, etc. Comparison operations are again straightforward. It will compare two arrays. Let's see how this works. We are comparing whether A is equal to B, whether A is greater than B, or whether A is less than B. On executing, it will give us a matrix where they are equal, not equal, or less than or greater than. Logical operators again, it will return a Boolean list based on the logical operations that we are performing. And will return true only when both the values are true, or will return true if any one is true, and not is the reverse of whatever we have provided. So it will reverse the values of A. Trigonometric functions are pretty straightforward where you are going to apply on an array of angles. Remember this has to be in radians. So pi by 2

and pi will be like 90 and 180 degrees respectively. Linear algebra operations like dot and determinant. For determinant, we have to use linear algebra dot determinant. It will convert universal functions include add, multiply, and power, which will do the scalar operations, whatever we have discussed so far. The next video was on NumPy indexing and slicing. It remains pretty straightforward. So I'm just going to execute each line to see the elements. This is going to return the first element 1, the last element 5. This will return the value in the first row, second column. This will return, since we have mentioned only one A value, it will return the first row. Using start index, end index, and step, you can slice more elements from the array. Slice to 7 will return the value from the second to the seventh. Every second element can be returned by using step 2 and reverse the array using minus 1 in the step. For 2D arrays, remember you have to mention two values. So it's separated by comma. Colon 2 will return the second row. 1 colon will return everything in the first column. This will effectively give you a smaller array. We can modify arrays using slicing. We can change the value from 2 to 4 with 10. 10 gets repeated. So 2 and 3 in the original array gets modified. Here it will return a Boolean index wherever this condition is true. Boolean index will begin an array based on the number of elements that you have in the original array. For fancy indexing, you can use a list of indexes, which you want to retrieve. So this will return the values in the first position, third position, and the fourth position, which will be 20, 40, 50. IP.index does the same thing. You can provide a list of row indices or column indices and provide those as part of the indexing method. It will return those values alone. It is a very powerful tool that we will use for machine learning. Array indexes. You can specify the indices as a 2D matrix also. It will return based on that. WHERE and ARGWHERE will return true or false based on where it is positioned. WHERE and ARGWHERE will return the position of the element that you are providing. And masked arrays will help us to perform operations on the original array based on the mask that we are providing. So it will give us only these values. Only these values will be returned for us. The false values will be kept as it is. The next video is on NumPy functions. The basic arithmetic functions we have seen in the previous video. So let's look at functions like SQUARE, ROOT, SQUARE, and ABSOLUTE. This will return the absolute value. This will return SQUARE, ROOT, and this is SQUARE. Remember, press CTRL, ENTER to execute the lines. Exponential and log can be found using EXP and log methods. Log 10 will give you the log to the base 10. Trigonometric functions, we just explained it. So repeating, it will return the sine, cos, and tan values of the angles provided in gradients. For statistical functions, we have mean, median, and average, which can run on the NumPy arrays. For dispersion, it will be variance, standard, and peak-to-peak or the range. Counting, you can count the unique values. You can have the bins. You can have the histograms based on the number of bins. So the last line is something that we have to look at. The number of bins is specified as 4, and the histogram method will convert this array into a binning of 4. So you can see that the array that is returned is this. Aggregation can be done on each axis. So let's see. Let's split this into two lines and see what it is doing. This particular line will sum all the elements in the array. Pretty straightforward. You get 21. But if I have to sum it column-wise, then we have to provide axis as 0. And if we have to sum row-wise, we have to provide the axis as 1. The first line is column-wise summing. So 3 plus 6, 2 plus 5, and 1 plus 4 in the reverse order. And the second line will give you the column-wise sum. Reshape is another tool that we use for modifying the array. And we can take the transpose of the array using the transpose function. So this is the reshaped array, which gets transposed and becomes a 4 by 3 matrix. We can stack arrays either in the vertical or horizontal. Let's see how the output comes in. When I'm doing a vertical stacking, it will stack the first array, then the second array goes at the bottom. If we are doing a horizontal stacking, it is

similar to the append method of a list. All of them are put together in one array. We can split using either edge split or bead split. We create 9 element matrix in the shape 3 by 3. So we are creating a 3 by 3 matrix. And then we are going to split it into 3 parts. Remember, all the outputs are arrays. So this is your original array. Horizontal split will split into the column-wise form. 0, 3, 6 will be the elements, which again is an array on its own. Vertical split will split row-wise. 0, 1, 2 will be the elements. Again, it is an array. Utility functions, we saw zeros and ones and in space. There is one more that is good enough, which is I. It will return an identity matrix based on the shape that you have mentioned. So this will return a 3 by 3 identity matrix where 1 is given in the diagonal elements. Random number generation is very critical in machine learning. So create random numbers in the uniform distribution. It will create a matrix of 3 by 2. RAND n will create normal distribution numbers. And if you want random integers, you have to use RAND int. The RAND and RAND n will be returning floating point values. In this video, we executed the Python code snippets related to NumPy library in Jupyter notebook and viewed the results. Do execute the files. Happy coding and thank you for your attention.

Recording of Introduction to Data Analytics: Week 3 - Live Session on 26-06-20 Video• . Duration: 1 hour 1 minute 1h 1m

[Notes]

Hi, everybody. Today is a part of the third session, we should see the topic slight. Before I start this session, I just want to have a small question for you. Suppose we are assigned roll numbers to the students, picture your class. Can I change the roll numbers? Can I give the permission to the student to change the roll numbers? No. Once the roll numbers are being assigned, they should not be changed. For example, date of birth of a student, which is stored in the system. Can it be changed every day? No, absolutely. Some information should never be changed. The changes once created are not allowed on some data. Such type of data should not be changed. For example, if I give more examples, if I am calculating the marks of a student, can they be altered? No. If I have the other number of a student, can you change the other number? No. Date of birth should not be changed. Bank account numbers should not be changed. All these as an example, should never be changed, these are no changeable data. Our Python will give us the facility of restricting the user on modifying the data. These type of data they are generally called as immutable data structures. We will start today in our session with immutable data types like you have the tuples, you have strings and others, as an example of our immutable data. Starting with what is a tuple? A tuple is a ordered collection of values that cannot be changed after creation of that tuple. For example, how do you define a tuple here? To define a tuple, generally what we do is equal to within a simple bracket, write the value. That's it. This is how you are going to define a tuple. Now if you want to print a tuple, just give a value like this, value of the tuple is being printed. Now here I'm not using any print statement also. Generally, if you want to print the immediate statement value, we did not give the print statement. It means following lines output if I want to get, I can directly give the variable name. Automatically, Python will give us the output. First thing, how do you define tuple? Tuple this is a variable name of my tuple and just within the simple brackets, we define the values of tuple. This is same as. Both are having the same values, value as 3. If you want to print automatically the previous immediate expression, that should be the last one. We can directly use the variable lab instead of the print statement. What is the tuple? How do you define it? Variable lab equal to open

the brackets and define. Now, why we are using tuple, why not a list? What do you see? See in the list, when you define a list, we have the provision of altering the values in a list. You can change the values in. Hello. List is mutable. Very good. You can change the values because it is mutable, whereas tuple is immutable. You cannot change the values in the tuple. For that reason when I don't want to change any values, I use the tuples, means accidentally also, I don't want the user to do any changes to the data. As we have seen, I have stored the marks of the student or I have entered the date of birth, other number, email addresses. These data should not be changed. Then we use the tuples here. Another example of your tuple with a string. See student equal to this is Ravi, 101, MBA. Now if I want to print the student variable, print of student. I'll get the output like this. This is Ravi, 101, MBA. You cannot alter the values of this variable that is called Ravi. Other one is singleton tuple. Now what do you mean by this singleton tuple? When a tuple with only one element, then we say it is a singleton tuple. Now, see here I have defined it `a=(3)`. This is also with one value, but this is not a singleton tuple. Whereas this is a singleton tuple. Remember, always a singleton tuple, if you can see, this is a tuple type. See here type of `t` means, it is getting type as tuple with a single value. This is called a singleton tuple. Tuple with one element. Always it should have a comma. Next, how do you access the tuple elements? Just like we do with list and arrays, tuple elements also can be accessed with the indices. Like this is `t(0)` means first element of the tuple will be printed. Now here, I can access the element from the last location, last index also. How do you access from the last indexes? With the minus negative. For example, `t(-1)`. Writing `t(-1)` means in the reverse order, you are accessing the tuple elements. For example, if I say `t(-1)`, I will be getting the output as 8. `T(-1) = 8`. `T(0)` is, I'll be getting this 6. With a normal index values, it will be from left to right, with a negative values, you can access from right to left as seen here. You see, as I told you, output is 8 with -1, output is 6 with 0. 0 is the first element, 8 is with the last element. Now, next is the nested access. You can use the tuples for the nested elements also. Means tuple inside a tuple. See for example, here, this is one tuple. Inside this tuple, I'm using one more tuple. This is a nested access. We can use the nested access also with the tuples. Like for example, I have one variable that is book information. Here I have one is Python Basics, one string, other is John, and then is the maybe the date of publication and number of pages that a [inaudible]. Inside this, also, I can use tuple structure. Now here, if I want to print, how do you access this? See this will be Index 0, this will be Index 1, and this will be Index 2. Now, if you use this Index 2, you will be getting this complete output. Now, again, to access this also, you have to use with Index 2 of 0, Index 2 of 1. Where is it? See here, like this. Like this, you're going to access the elements of your tuple. This is a nested access. If I run this, see the last element of the tuple, book information, is last element is this right. This is what? 0, 1, 2. Book info of 2. I'll be getting this last element, so I got this here, 2020, 350. Now if you want to get this second element, this is 0 and 1. Book info of 1, I'll get the second element. Now here, first element of the nested element is. This is at Index 0, 1, 2. `[2][0]`. Book info of `[2][0]`, so I'll be getting 2020, the year of publication, maybe. Number of pages if I want to get. Then book info of `[2][1]`. I'll get the number of pages. Like this, you can access the tuple elements. Just like arrays, in the tuples also you can perform some operations on a tuple or methods because tuple is an immutable data, therefore, this Python provides few methods which can be operated on the tuples. Like you can have concatenate, you can count. Another method is find by index value, then you find the length of the string. To find the maximum element, max function is there. Minimum value, sorted; to get the list in the sorted order. These are some of the methods that can be performed on the tuples. Let us see here. Now here I have a variable `t` with these values, H, E, L, L, O as string of characters. Now count how many elements are there if you want to count. One

method is the count method, `t.count("L")`. Now count of L means, how many times this character L is repeated in this. It will give you that count. Now, next is first occurrence. You can even find the first occurrence of any character by using the index method. Now, `t.index("L")`, I want to find when is the first letter L seen in this string. May I run this? See, first occurrence of L is at Index 2; 0, 1, 2. First letter of L was seen at Index 2. Now how many times letter L is repeated in this? 1, 2, so I got the output as 2. L is repeated two times. By using the count method, you can find how many times any particular character is being repeated, you can check. By using the index, I can find what is the first occurrence of any particular character. You can find the length of this tuple by using the len function or len method. Total how many pieces? 1, 2, 3, 4, 5, 6, so I got this, length is 6. Similarly, you can find the maximum element. You can find the minimum element by using min. You can sort the elements by using the sorted. This sort method will give you the output as a list. You can print in a reverse order also. See, I'm printing the elements in a reverse order, I'm printing the elements in a sorted order by using the sort method and finding the minimum element in this list. Finding the maximum element by using the max function and others. Now, next is the string. Let us go with a string. Again, string is another immutable data type. We can define multiple lines of strings by using this three times quotes symbol. Three times if you use this, you can print in multiple lines. For example, `t="""hello, and hai in the next one and three times. See I will be printing this in multiple lines. How many lines ever you want, you can just use within this three quotes. To print the string in multiple lines, we can use this quotes. Then concatenation of the string. Just like plus is used, for concatenation of the strings, we can use plus symbol. Now, if I want to have a space here, I'm using just plus N for the empty space, I'm using this, then again, concatenating the next string is the last name with plus N, last name. Here what will happen between this first name and the last name, I will be creating some space. Like this. If you don't want any space, just use the plus symbol. Here I wanted to have some space, so I have used these pots. Plus space and plus and the last name. If you don't want any space, just use the plus symbol. I don't want any space between the two strings. You can add two strings or join two strings to say, by using this concatenation symbol, like plus. Now I can have the repeat on of the string. You can use the reputation of the string. Just by CM. 'First into two' means this first name will be repeated two times. First name is Rivali. It is repeated two times because I wanted to repeat it two times. Like that, you can have the repetition of the string words. Then another operation that you can perform on the stings is slicing of the strings. So you can split the string. How do you slice them? Now here, I have let us see with the first value, like step value. Now what do you mean by this step value? I want to slice I want to divide the string. How is 'print T' of zero, nine, two. What does this mean? Start skip. This is what 'start' and 'skip' mean: you start from the index zero till the index nine. Skipping what? Skipping two characters. As alternate character will be printed over here. For example. Cons: This is start, skip. It is T of start column and column. Skip skip count. How much you want to skip. Start here. For example, start it at index zero, that is, H till index nine, one, zero, one, two, three, four, five, six, seven, eight, nine, means last character by skipping two means alternate character will be printed. Our alternate character is printed. By skipping every character, you are having the output. Skip by two, then. Now, this minus one is what you are coming from the reverse. Start at index four. This is what start from index four. This is start end by how much start at index four, okay? And move from the left, move from the left, starting and stopping before the index point. This is the end index at the index one, and this is minus means what we have seen right, minus means you will be coming in the reverse direction. What will be the output of this NAN? It is starting from index four. So zero, one, two, three, four. Index four is N, and you have to stop where you have to stop at index one means at A, N, A, N. The end is`

exclusive. 'Exclusive' means 'end' will not be considered end will be exclusive. Now, similarly here also. Now next is this text is equal to A, B, C, D, F. Now I have text pollen pollen minus one. Now, what is this pollen, pollen here? I'm not specifying any start and not specifying any end. And negative means what? It will tell the Python that you have to move in the backward through the string. There is no start and no end. Move from the backward through the string so that the characters are all collected from the last position to the first position, like this, what will happen? What is expected here? Yes, what is the expected output? It will be in reverse string. Let us see this. You'll be getting a reverse string: F, E, D, C, B, A. You will be getting a reverse string over here. Let's string in the reverse order. Right. So here, what you are doing? I did not specify the start, neither I have specified the end, but I said you have to go from last to the first. I got this string in reverse order. Now, before going further to the functions of the string, Now, I said the tuple as well as the string; they are immutable, right? Now, 'immutable' means you cannot change the values once you have declared or when you have assigned the values to the string or the tuple. So let us see one example for this. I suppose I have name is one two three Now, how do I access this? I can access using the index values. I'll be accessing. Now, with a list, all, if you want to change, I can just replace the characters. But suppose I want to replace it with a small little character. You got that right does not support this assignment because it is a string. This is the beauty of using or having the strings because you cannot change once you have assigned the params. Now let us move to the string functions. So what are the functions you can find a character? You can count the number of repetitions of a character. You can find the character in a string. You can change the case of the string to upper case, lower case. You can concatenate the characters you can find and you can arrange or replace the character. These are all the functions or methods that you can function on a string. Let us see with this. Now, see here. I have variable T Small letter T. String T is banana. In this cell, I have defined it. Now I want to use it here. If you're not changing, you can use the same variable, no issue. T.find of A. T.find of A, this is one function. Then count of N. Count of N, find AN. Now when I say find A, N, the combination of A, N how many times when is the first location first time it is found in the string, this combination, that index will be given us. Also, you can say 'print BA in T. Let us see.' Now, this find 'A' means 'my T is here'. Our T is banana. When is the first A found here at the index zero, one. I got the output as one. Next is what count L. Now, in the spelling 'banana' in the string 'banana', do I have 'L'? No, there is no L, so I got the answer as zero. Next is what find AN. This sequence AN, when is it first found from here? Zero, one. So I got the output as one, first appearance or occurrence. Then next one is 'print BA in T means what? Do you have the sequence BA in this variable or string T? Yes, I found BA. That is why I got the answer or, say, print AA. Can you see it is false? Now, Now, I got the error here. Why? Anybody can anybody guess this? Same statement I got previously. I got the output for all the things, but now it is giving me an error. Anybody can you guess? Yes. Just think, when I have executed these lines, I got all the outputs without any error, but now I made a small change. But I'm getting an error. Can anybody find why is the error here or what is the error? Very good. Fatima? Yes. This Python is K-sensed. Very good. See. What did I do? I had changed it into capital letters. That is why I got it. I got the error, so it is. It is false. There is no string AA in the string T with the banana. I got it as error, that is false. Similarly, you can convert into upper case, you can capitalize it. means first letter will be capital letter. Then you can find the characters; we can found how many times a character is repeated. You can find the first utterance of any string. Same thing again, upper-end catalytic functions are being used here. Now replace method, replace this. I'm first trying to replace the elements function. Print dot replace of this mango, zero with O. Replaced. No. Next is the list. What is the list? List is an

ordered collection of items that can be modified. List is mutable. Now, let us see. Suppose I have a shopping cart. Okay? Now, the items that we add into our card, can I alter the items? Can I alter the items, what I add into the card? Yes, obviously, I can change. I can add new item, or I can delete any item or even at the time of payment, or I can go back and change my card. I can modify the list. So then Tuple is not suitable. And I want to do any changes because tuple is immutable; we go for the list. The structure that help us to do the changes is called as a list. Let us see with the list. Now if you want to create the list, it is a square bracket. Now, see the changes. For the tuple, it is a closed bracket. For the 'what do you say' list, it will be a square bracket. For the list, it will be a square bracket. Within the square brackets, I want to have a string. Apple, banana, cherry, these are the root names. Now first it's got printing the original list fruits. Then I want to access the elements of the list. Same thing by using the index, we will access the elements of the list. Print fruit. Fruits of zero means first element of that list will be printed for us. Now, if you want to modify the elements, that also you can do. I'm changing the first element that is apple to blueberry. That I can do. How you are doing that, so fruits of one is equal to blueberry. Fruit of one is what? Apple. Now, this apple will be altered with blueberry, so I can change. I can check the change also by saying print of fruits. Now, adding an element, you can add new element by using the append method, fruits.append of orange. As the last element in the list, it will be added to us. Now next is inserting at a specific. Now when you use the append method, it will be added as a last element into the list. When you want to add at a specific location, you can just use the insert method. Insert at what position you want to insert and what is the value that you want to insert? What is it? Fruits.insert one, grape. I want to insert at index value one, the fruit as grape. Now, if you want to remove any element, you can remove it by using the remove method. Remove what? Cherry. This element will be removed. Popping an element. Popping means what? You will be getting the last element of the list. Last element will be printed. Now, next is the slicing. Slicing means I will split the list like see, from the index fruits of 1:3. From the index 1:3 always the last will not be considered. It will be excluded. Same here also as we have seen in the tuple and the string, same in the list also. The last end will not be considered. Still that point you will be considered means from 1-1,2. That's it. Three will not be considered. Index 1 and Index 2 will be printed. Now next is the length of the list. Same by using the len method, we can find the length of the list, then looping through the list. By using the for loop, we can loop through the elements of the list. One by one elements, you can just print. Now, checking if an item exists or not in a list. Say if apple in fruits means if this is true, I'm going to print this statement, apple is in the list. Checking for them. If apple in fruits, in this list, then print the apple is there. There is some statement to be printed. Now, next method, what we can do is you can sort the list by using the sort method, fruits.sort. Alphabetically, the list will be sorted for us. Next is reversing the list. Yes, by using the reverse method, you can reverse the elements of the list. Next is copying a list. You can copy the list to a new list like see new_fruits is equal to fruits.copy method. What is the method you have? Copy. This method will copy all the elements, what you have in the list to the new variable. Now, if you want to clear the list, you have the clear list of fruits. The variable will be there, but the elements will not be there. You have the variable, but not the elements. Now let us run and see. What was the first function we have used? We have just created the list and we have printed the list elements. Original list is what? Apple, banana, cherry. Then what did you do? We want to access the elements of the list by using fruits of zero, means first element. What is the first element? Apple. I got the output as apple. Then what is that we are doing? We are modifying the list. First element was apple. Now that is changed to blueberry. Did you get the output? After modifying the list here see, zero, but Index 1 is blueberry. Now next is appending. Appending is

what? It will add the element at the end. After cherry, I have the orange. Indexing at one. At index Number 1, I have blueberry, so I want to insert grapes in that place, so apple and grapes. Now berry has taken second position. That is index Number 2. Now, removing the cherry, if you can just remove the element also. See, you can remove the element. We have used the variance. Removing the cherry element. Now, pop is what? By using P-O-P, pop method, you are retrieving the last element of the list. In this list, what is the last element? Orange is the last element, so we got the last element. Now, popping means what? The element will be removed from the list. Not only displaying, you are removing. That is see, after popping, what is my list having? My list is having only three elements. See, last item, fruits.pop. Now here by using this, I am retrieving the last element from the list. Now what is that? Popped item is last item. Now, see, after popping, what are the fruits we have? What is the fruit list is having? See, after popping, the list is apple, grape, berry. But actually, you are having four elements. You have popped the last element means you have removed the last element from the list. This is only what you have. Now, slicing. You have taken fruits of 1-3. Means excluding the third. Here see, 0, 1, 2. There is no third element. I got grape and berry. List length, how many elements you have? Len of? One to three. I got three here. Now, this looping through the list elements, where is it exactly? See, looping through the list elements for fruits in fruits, print dash and fruit name for every element. That is why I've got all the elements in a different line. We are checking the if condition, checking. See, if apple is in the fruits, that is a list, then I'm printing this statement. Yes apple was there in the list, so I have printed that apple is in the list. Then we have sorted the list elements. Sorting will be according to the alphabetical order. Actually, my list was apple, grape, and berry. Now it has been sorted with apple, berry and grapes. Now reverse list is in the reverse order, you are going to print the list and copied list is this list has been copied to the new list, see, with the copy. It is copied with a new list. Next is we have cleared the list. We have clear. As I told you, when you use the clear list, the variable will be there. Means the list will be there, but without any elements. That is where you got this empty list. These are all the methods that can be performed on your list. Now, next is the dictionary. Dictionary generally, it is like a key value pair. Now let's see some example here. I have student variable with names. Now dictionaries also, we have to use the flare braces, name as this is key, and this is the value. What is the key here? Name. What is the value? John. What is the key here? Age. What is the value? Twenty. Name it is a pair of key value. Multiple values you can give. Name is John, age is 20. This is key, and this is the value. Key value pair is your dictionary. You can assign value to a key by using the dictionaries. This is what? You can append a new key to the existing dictionary by using this student of. See, actually in the student, I have name and age. Now new one that is course, I'm adding it. Course is what? Math. New item will be added. You can add a new item to an existing dictionary and you can modify the value also. This is also mutable. The dictionaries are also mutable. You can change the values. If you print, it will be now initial. I have defined dictionary with only two items. But when I print, I'll have three items because I have appended a new item here. I have a name with John, age is 21 because I have modified the age from 20-21. Age is 21 and course is math. With three elements, I have a dictionary. Similarly, by using the index, you can just print the dictionary items also rather than the index because see, what is the main uses? See, in the list if I want to print, I have to remember. For example, see here in my list I had apple, new berries and cherries. Now if you want to print cherries, now I have to remember the index values of cherry. It is 0, 1, 2, I have to remember and I have to say fruit of two. Then I'll get cherry. If you have more number of elements, it will be difficult for us to remember the index value. For that reason, we will have this name key values. With the keys, I can retrieve the values. Now when I say student of name, what is the name? That is John. I'll get the

answer is John. Now next thing is the sets. Why do we use sets? Now, when I'm storing email IDs of all the students, I don't want any duplication in the email IDs. When I'm storing the roll numbers, I don't want any duplications of the roll numbers. I want the unique roll numbers. Or when I'm storing any other card number, I don't want any duplicate numbers. When you don't want any duplicates, then we use the feature data structure that is called as sets. Sets can be used to remove the duplicate values. What are the operations you can do? On the sets, you can do union, you can do intersection, you can do difference. All these operations also you can perform on sets and again, sets are declared using the flare braces. For example, here I have two sets that is a is equal to 1,2, 3, b is equal to 2, 3, 4. Now, print a/b. This is the symbol union, or symbol. A or b means, for example, I want to retrieve the students who are either registered in Python or Java class. Then I say a or b. That is union. Now here when you say a or b, the only unique you will get. For example, two and three, see here, some students are in Python as well as in Java. I will not have repetition over here. I'll get output as 1, 2, 3, 4 only for this union. Now next operation is the intersection, that is and, Java and Python. The students who are registered in Java and Python. Now I want to retrieve them, then I use this and. And symbol is used for intersection. Means the students, those who are in Python, as well as in the Java class. I get two and three. Now, difference is a-b. Means that are there in a, but not in b. That is a difference in Python, not in Java. That is a difference we can have with the sets. See, in a, but not in b. That is a-b. These are the operations that you can perform in sets. To have just brief idea, now see. For the tuples, it is rounded brackets. Now, duplicates are allowed in the tuples. You cannot modify the data once it is created. While the list it is square braces, duplicates are allowed, and in the list, we can modify the data. Whereas in the dictionary, it is in the flare braces like key and values. Here keys, yes, you can modify the data. Similarly with the sets, again, flare braces duplicates are not allowed in the sets, and yes, you can modify the data. Now I will come to this at the end. Let's see, for example, let us have a quick review of this. Aggregate scorecard, it contains team runs and wickets. Now, what do you think the data type we use for this? Aggregate scorecard, it contains Team Name, Runs and Wickets. What type of data structure is applicable for this? Yes. Any idea? Very good. We use doubles because we are talking about the scorecard, it cannot be altered. Now, a music playlist where songs can be added and removed. Very good. Good. I got two answers, Nari, and Fatima. Thank you. Employee details. Very good. Store all the unique email addresses. Names of students in order. Very good. Thank you. Yes. Names of students in the order they are registered we use list because the order in which they are registered matters. Fine. Very good. No. before going to numpy, I just want to clear the difference between the list and arrays. List also are just like arrays only? What is the main difference between this list and arrays? For example, um, I'll take one list marks 10,20,30. This is one list of students who secured marks in one subject title subject. Now I want to print if I want to add five marks to all this, increase the marks by five. What is expected? I'll just say print mark plus five because I want to add five marks to all. I have used. Now, will it be executed I'll get an error because this Python, it will it doesn't know actually how to add this five marks to every element of the list. It doesn't know how to add five marks to every element. Now, I wanted to add five marks to each element of the list. It demands for a loop. If you go for looping, if you use loop, you can add, because I want to add five to each and every mark of the list. What you can do is on this. For i in range of length of, your have lens for length. What I want, marks plus i is equal to, marks plus i+5. Let me change this. Now, you got it. Every element is added with five and it is increased by five marks. Now, see what we are doing we are using, what do you think? A loop, to add five marks to each element. Just remember this. Now, how I can make this work easy? Is there any way that I can

improvise this? Yes, definitely, we can improvise this. Let us see how. For that, just go with this numpy. What do you mean by a numpy? Numpy is nothing but a numerical Python. Okay? Numpy is a numerical Python. Which is used to do fast calculations, which is generally used to do fast calculations. To use this numpy, first we have to install the numpy. P-I-P, with this you have to run before you want to use this numpy methods. first you have to do, and then you have to import the Numpy libraries. Import numpy as NP. This NP is nothing just as Elias name we are using, Numpy NP. Otherwise, if you don't want to use NP, you have to use this complete spelling numpy. Either you use numpy completely or just NP. NP used as for this number. We are importing the libraries. From numpy. Now here, how do you use? Now I want to create an array. Import A is equal to NP I'm using Elias name np.array of (1,2,3),(4,5,6). This is how I'm going to declare an array in the numpy. Now print A, I'll be printing the array elements (1,2,3), (4,5,6). Now here, np.array of (1,2,3). This is two by two array and this is single dimension array. I'm printing the array. Now, print B D types, what is the data type of this array? These are integers. I got int. Now next this is a force data type means I can change or I can source the array elements data type also np.array. what is array elements (1,2,3). Now, D type is equal to float. actually, these are integers, right? They will be converted into float. This is calls force data type. They will be converted into float. Now if you run this, you can see 1.0, 2.3 point like this. Actually, they were integers. I have force converted them into the float. Now, next is what yea np.zeros(2). You will be getting a two by two array with all zeros, with all zeros. Now np.ones(3) means, one row with three ones means three columns. This row will be of three columns with ones, and this row will be of two by two array with zeros. Now see two by two array with zeros, and this is one array three columns np.range(5). You will get five elements (0,1-4) elements. Now, np.linspace(0,10,5, dtype=int) ow, what this is line space. This method will give a line space, a spacing between the elements of an array. It will generate equal spacing between the start and stop of your elements. Equal spacing will be created. Now, see. Yeah. This is the start. This is the stop. What is the spacing spacing is five. What are the elements you want? They are the integer data type. Integer integer type. I want with equal spacing of five, starting from zero, ending at ten. You got 0, 2, 5, 7,10 five elements 0-10. No cilium, zero to one with five. Start is zero, end is one, 1, 2,3,4,5 I got five elements. Now here, I did not say anything like int or float or something, so it will divide. I want five elements 0-1, equal spacing of five from 0-1. It will calculate automatically what should be the values 0-1 so that I'm generating five with equal spacing. Means equal difference between two consecutive integers. Now next is the split method. np.split (a1,2), a1 is array 1,2,3,4,5,6, so it is array (1,2,3) 0-2 and test elements, an array. Mean value, maximum value of this array elements will be printed. Minimum value will be printed with mean, length of the array elements will be printed. Now see. We can do the arithmetic operations also with the methods, sum of elements with the sum function, mean maximum, minimum. We can even find the square root also. You can even find the square root. Sum of all the elements is 150 means average of these elements is 30.0. Maximum element in this is 50, minimum element is ten, minimum element is ten, square root, each value square root will be found and you will have the bas. Now see. We have just seen what is the list? Now, if you want to add Pi to all the elements, yeah. Add Pi to all the elements of the list, what happened? I could not do it by directly adding, I had to use Lou for Lou then only I could add the elements. Now, same thing if I use arrays. Same thing, if I use arrays, max is equal to, I just said it is a numpy array. It is a nump instead of list, I just said np.array and I used (marks+5). That's it. Run it every element. Did I get the same output? Yes. With the list with the fur loop, I have incremented or added five to each elements of the list. With this, I have added five to each element, simple, one line, that's it because I'm using a numpy array. What is the future?. With the list, you

don't need any libraries. But numpy array needs a library. That is why you use numpy NP AA's name. These list the stores different data types, whereas with the numpy arrays they are usually same type. with the list, you can perform limited mathematical operations, whereas see here. efficiently, you can just perform the operations. Moreover, with the numpy arrays, it is very fast when compared to your normal lists. Also memory storage also with the numpy arrays it requires less when compared to your list. Another beauty is if you're using numpy arrays, they are used in the u they can be used in the data science fields or AI for the analytics and all other than your general Python Send. By this, No, anyone has any doubts? Yes, no.

Series Video• . Duration: 7 minutes

[Notes]

Welcome learners to the topic Pandas Series. By the end of this video, you will be able to understand what a Pandas Series is, create series objects from various data sources, access and manipulate series data, perform basic operations on series data. A Pandas Series is a one-dimensional labeled array that serves as a fundamental data structure in the Pandas library. It is designed to hold data of any type, such as integers, floats, strings, making it incredibly versatile for various data analysis tasks. While the size of a series is immutable, meaning that you cannot change its length after creation, the values within it can be modified, allowing for flexible data manipulation. Before we dive into working with Pandas Series, we need to import the Pandas library into our Python environment. We typically import Pandas using the alias `pd`, which is a convention widely used in data science community. The import statement allows us to access all of Pandas functionality using the `pd` alias. Creating a series from a list is one of the most straightforward ways to get started with Pandas. When we pass a list to `pd.series` function, Pandas automatically assigns an integer index starting from zero to each element in the list. The code creates a series from the list of numbers and prints it, showing both the automatically assigned index and the corresponding values. It is a quick way to convert a simple list into a powerful Pandas data structure. Let's look at how the output is generated. The data is 10, 20, up to 50. Please observe that the Pandas library adds an index starting with zero. Sometimes we want more control over how our data is labeled in the series. Pandas allows us to create a series with custom index labels, which can create our data more meaningful and easier to work with. In this example, we are creating a series with custom string labels as index. This approach allows us for intuitive data representation, especially when working with categorical or time series data where meaningful labels are important. Another flexible way to create a series is by using a dictionary. When we create a series from dictionary, the dictionary keys automatically becomes the series index and the dictionary values becomes the series data. Let's see the code for this. This method is useful when already you have your data organized in a key value format as it preserves the relationship between your labels and the data values. It is an efficient way to create label data directly from existing structured information. Once we have created a series, Pandas provides multiple ways to access its elements. We can access elements by their label, by their position in the series, or even slice the series to get a range of values. The code demonstrates how to access individual elements and slices of a series. Pandas series comes with several useful attributes that provide information about the data structure. The main attributes are `index`, `values`, and `dtypes`, which gives us access to the series labels, its underlying data, and the data type of the series, respectively. These attributes are crucial for

understanding the structure and content of a series. They allow us to inspect our data quickly and make informed decision about how to process or analyze it. Pandas series supports various operations that can be performed element-wise, making data manipulation efficient and intuitive. We can perform arithmetic operation between series or between a series and a scalar value. The code shows how to add series together and how to multiply a series by a scalar. These operations are performed element-wise, allowing for easy and efficient data transformations. Filtering is a crucial operation in data analysis, and Pandas makes it straightforward to filter a series based on conditions. When we apply a condition to a series, it returns a new series containing only the elements that satisfy the given condition. This code demonstrates how to filter a series to include only elements greater than 30. Pandas provides a convenient way to apply functions to every element in the series using the apply method. This method allows us to transform our data using custom functions or lambda expressions. In this code, we are using a lambda function to square each element in the series. The apply method executes this function on each element returning a new series from the transformed value. Dealing with missing data is a common task in data analysis, and Pandas offers robust tools for handling null values in a series. The isNull method checks for null values in the series, while dropNa removes null values from the series. These methods are essential for ensuring the integrity of our data during analysis. They allow us to identify where we have missing data and provide options for how to handle it. One of the powerful features of Pandas series is their ability to handle different types within a single structure. This flexibility is useful when working with real-world data and often comes in mixed formats. In this code, we create a series with mixed data types, including an integer, a string, a float, and a boolean. Pandas automatically determines the most appropriate data type to accommodate all the elements in the series, making it easy to work with diverse data. Let's recap. A series is a one-dimensional labeled array in Pandas, fundamental for data analysis. We have explored creating series from lists, dictionaries, and arrays, offering flexibility in data input. Series support custom indexing, allowing meaningful labels for data points. We have seen how series enable easy data access and manipulation, streamlining our work with information. Importantly, series can handle missing data and mixed data types, crucial for real-world data sets. An essential tool for data analysis and preprocessing in Pandas, series serves as a building block for complex data structures and analysis. Thank you for your attention. Happy data analyzing!

DataFrame Video• . Duration: 6 minutes

[Notes]

Welcome learners to the topic, Pandas DataFrames. By the end of this video, you will be able to understand what a Pandas DataFrame is. Create DataFrames from various data sources. Access and manipulate DataFrame data and perform basic operations on the DataFrame. A Pandas DataFrame is a two-dimensional label data structure that contains rows and columns similar to a spreadsheet or an SQL table. It is designed to hold multiple data types, making it versatile for various data analysis tasks. Each column in the DataFrame is essentially a series, which allows for efficient data manipulation and analysis across different dimensions of your data. Before we start working with DataFrames, we need to import the Pandas library into our Python environment. We typically import Pandas using the alias pd, which is a convention widely used in data science community. The import statement allows us to access all of Pandas functionality using the pd prefix,

making our code more concise and readable. One common way to create DataFrame is from a dictionary of lists. In this method, the dictionary keys become the column names and the list become the column values. This code creates a DataFrame. The resulting DataFrame will have three columns, name, age, and city, with corresponding values for each person. Another way to create a DataFrame is from a list of dictionaries. In this approach, each dictionary in the list represent a row in the DataFrame. This code creates a DataFrame from a list of dictionaries, where each dictionary contains the information of the person. The resulting DataFrame will have the same structure as the previous example, but the data is input in a row-wise manner. Sometimes we want to assign custom labels to the rows of our DataFrame. Pandas allows us to create a DataFrame with custom index, which can make our data more meaningful and easier to work with. In the code, we are creating a DataFrame with custom row labels, person1, person2, person3. Pandas provides two main ways to access columns in a DataFrame. We can either use the square bracket notation with the column name as a string, or the square bracket notation using the column name as an attribute. Both method returns a series object representing the column, allowing for further manipulation or analysis of the specific data. To access rows in a DataFrame, we can use the LOC or ILOC methods. The LOC method is label-based, allowing us to access rows by their index labels. The ILOC is integer-based, I standing for integer, allowing access by numerical position. Both methods return a series representing the row, providing flexibility in how we retrieve data from our DataFrame. Adding a new column to the DataFrame is straightforward process in Pandas. We can simply assign a new list or series to the new column name. And Pandas will add it to the right of the existing columns. This code adds a new country column to our DataFrame, demonstrating how easy it is to expand our data set with additional information. Pandas provides several methods for quick analysis of our DataFrame. The describe method gives a statistical summary of numerical columns, including count, mean, standard deviation, and quartiles. The info method provides a concise summary of the DataFrame, including column names, non-null counts, and data types, which is useful for getting a quick overview of our data. Pandas allows us to apply functions to columns in a DataFrame using the apply method. This method can be used with lambda functions or custom functions to transform our data in the column. The code shows how to apply a lambda function to increment each value in the age by one, demonstrating how we can easily perform operations across an entire column. Dealing with missing data is a common task in data analysis. Pandas offers robust tools for handling null values in a DataFrame. The isNull method checks for null values in the entire DataFrame, while dropNA removes rows with missing data. This example demonstrates how to add a row with a missing value and then how to identify and remove rows with null values, which is crucial for maintaining data integrity in our analysis. Let's summarize. A DataFrame is a powerful two-dimensional label data structure, central to data analysis and manipulation in Pandas. We have learned various methods to create DataFrames, including from dictionaries, lists, and other data sources, providing flexibility in data input and structure. DataFrames offer a wide range of methods for access and manipulation, allowing easy retrieval and modification and analysis of data. We have seen how DataFrames facilitate filtering, function application, and handling of missing data, making them essential for complex data analysis and preprocessing tasks. Thank you for your attention. Happy data analyzing.

Indexing a DataFrame Video• . Duration: 4 minutes

[Notes]

Welcome learners to the topic Indexing in Pandas DataFrames. By the end of this video, you will be able to understand different indexing methods in Pandas DataFrames. Use label-based and integer-based indexing. Apply various indexing techniques for data retrieval and manipulation. To begin our exploration of DataFrame indexing, let us create a sample DataFrame that we will use throughout the presentation. This code creates a DataFrame with information about four people, including their names, ages, and cities. We are also assigning custom index labels A, B, C, and D to each row, which will be useful for demonstrating label-based indexing later. Pandas provides straightforward methods for accessing columns in a DataFrame. To access a single column, we can use the square bracket notation with the column name as the string. This will return a series object representing that column. If we want to select multiple columns, we can pass a list of column names inside the square brackets. This approach allows us to create a new DataFrame containing only the specified columns. The LOC accessor in Pandas is used for label-based indexing, allowing us to access rows and columns using their labels. When we use `df.loc` with a single label, it returns a series representing the row with that label. We can also use LOC to select a range of rows and columns by providing slice notation with labels. This method is useful when we have meaningful index labels and want to select data based on these labels. For situations where we want to select data based on integer positions rather than labels, Pandas provides the iLOC accessor. Using `df.iLOC` with a single integer returns a series representation of the row at that position. We can also use iLOC to select ranges of rows and columns by providing integer slices. This method is useful when we need to select data based on its position in the DataFrame regardless of its labels. Boolean indexing is a powerful feature in Pandas that allows us to filter rows based on conditions. We can create a Boolean mask by applying a condition to a column and then use this mask to select rows that meet that condition. This technique can be combined with column selection to create more complex queries. For example, we can select the names of people who live in a specific city, demonstrating how Boolean indexing can be used for targeted data retrieval. To recap, we have discussed several key indexing methods, each serving different purposes in data manipulation. Basic column indexing allows straightforward access to columns using their labels or names. Label-based indexing with iLOC enables data retrieval using custom row and column labels, ideal for meaningful indices. Integer-based indexing with iLOC provides access based on numerical positions useful for positional data selection. Boolean indexing offers powerful filtering capabilities based on specified conditions. These diverse indexing techniques are crucial for efficient data retrieval and manipulation in Pandas. Thank you for your attention. Happy data analyzing.

Selection in a DataFrame Video• . Duration: 5 minutes

[Notes]

Welcome learners to the topic of Selection in Pandas DataFrames. By the end of this video, you'll be able to understand different selection methods in Pandas DataFrames. Apply various techniques to select columns, rows, and subsets of data. Use advanced selection methods for complex data retrieval. Let's begin by creating a sample DataFrame that we will use throughout this presentation. This code creates a DataFrame with information about five people, including their names, age, cities, and salaries. We are using a dictionary to define our data, which Pandas then converts it into a DataFrame structure. The result is shown there. Pandas offers two primary methods for selecting a single column from a DataFrame. We can use either the square bracket notation with the column

name, or dot notation using the column name as an attribute. Both methods return a Pandas series object representing the selected column. This is similar to what we have discussed in the previous video. When we need to select multiple columns, we have a couple of options. We can use square bracket notation with a list of column names, which returns a new DataFrame with only the specified columns. Alternatively, we can use the LOC accessor with a colon to select all rows, and a list of column names to specify which columns we want. To select rows by their label or index, we use the LOC accessor. When we provide a single label to LOC, it returns a series representing that row. We can also select a range of rows by providing a series of labels, which returns a DataFrame containing those rows. For selecting rows based on their position rather than their label, we use the ILOC accessor. Similar to LOC, providing a single integer to ILOC will return a series representing the row at that position. We can also select a range of rows by providing a slice of integer positions. How about selecting subsets? Pandas allows us to select subsets of data by combining row and column selection. Using LOC, we can specify a range of row labels and a list of column names to select. If it is with ILOC, we use integer positions for rows and columns to define a subset. Both will return a DataFrame. Boolean selection is a powerful feature that allows us to filter rows based on conditions. We can create a Boolean mask by applying a condition to a column, then use this mask to select rows that meet the condition. This can be combined with column selection for a specified data retrieval. An example is given in the code. The query method provides a more intuitive way to select data based on complex conditions. It allows us to write conditions as strings, which can be easier to read and write for complex queries. This method is useful when dealing with multiple conditions. The `isin` method is useful when we want to select rows based on whether a column's value is in a specified list. This is handy when we need to filter based on multiple specific values in a column. It provides a concise way to perform what otherwise would require multiple OR conditions. How about random selection? Pandas allows us to easily select random samples from our DataFrame. We can specify either the number of rows we want to sample or a fraction of the total data. This is useful for creating test datasets or performing operations on a subset of our data. The `nLargest` and `nSmallest` methods provide a quick way of selecting rows with the highest or the lowest value in a specified column. These methods are useful when we need to quickly identify top or bottom performers in our dataset based on a particular metric. To recap, we have explored single and multiple column selections, row selection by label and position using LOC and ILOC, Boolean indexing for conditional selections. We have also dwelt into advanced methods like `query` and `isin` methods for complex data retrieval. These selection techniques are crucial for efficient data manipulation and analysis, allowing you to extract precisely the data we need for our tasks. Thank you for your attention. Happy data analyzing!

Filtering a DataFrame Video• . Duration: 4 minutes

[Notes]

>> Welcome learners to the topic. Filtering in Pandas Data Frames by the end of this video, you will be able to understand different filtering methods in Pandas. Apply various techniques to filter data based on conditions Let us begin by creating a sample data frame that we will use. This code creates a data frame with information about five people, including their names, ages, cities and salaries. We are using a dictionary to define our data, which Pandas converts into a data frame structure. The most basic form of filtering in Pandas uses Boolean indexing. We create a Boolean

mask by applying a condition to a column, then use this mask to select rows that meet the condition. This returns a new data frame containing only rows where the condition is true. When we need to filter based on multiple conditions that all must be true, we use the Emerson OR and operator. It is important to enclose each condition in parenthesis to ensure correct evaluation. This code selects rows where age is greater than 35 and salary is over 60,000. For situations where we want to select rows that meet at least one condition of multiple conditions, we use the Pipe or OR operator. This allows us to combine conditions in an OR relationship, selecting rows that match either conditions. The ease in method is particularly useful when we want to filter based on multiple specific values in a column, it checks if the values in a column are present in the given list. This method provides a concise way to filter for multiple values without writing multiple OR conditions. Pandas provides string methods, that can be applied to text columns for filtering. These methods allow us to perform operations like, checking if a string starts with certain letter, contains a substring, or matches a pattern in the code. We are filtering for names that start with the letter A. The query method offers a more intuitive way to filter data using string expressions. It allows us to write conditions as strings, which can be easier to read and write, especially for complex queries. This code is particularly useful when we are dealing with multiple conditions. We can combine the lock sensor with a Boolean series for filtering. This approach allows us to separate the condition creation from the actual filtering, which can be useful for complex or reusable conditions, check out the code for details. Pandas provides methods to handle missing data, including filtering out rows with missing values. The drop NA method removes rows with nan or not a number values, and we can specify which columns to check using the subset parameter. This is crucial for dealing with incomplete data sets. Another method is to use between. It is a convenient way to filter for values within a specific range. By default, it is inclusive of both the endpoints. This method is useful when we need to select data within a certain interval. Finally, we can combine both row filtering and column selection to extract specific subsets of our data. The LOC accessor allows us to specify both row and column conditions in a single operation. This approach is useful when we need to filter rows and select specific columns simultaneously. To summarize, we have explored Boolean indexing for single and multiple conditions using the Easein method for membership tests and string methods for text-based filtering. We have also discussed more advanced techniques like query method for complex string-based conditions. These filtering techniques are essential for data cleaning, analysis, and extraction by specific subsets of the data. Thank you for your attention. Happy analyzing.

Operations on a DataFrame Video• . Duration: 3 minutes

[Notes]

Welcome learners to the topic Operations on Pandas Data Frames. By the end of this video, you will be able to perform arithmetic operations on data frames, apply functions to data frame columns and rows, use grouping and aggregation for data analysis, and merge and join data frames. Let us start by creating two sample data frames that we will use throughout this presentation. The code creates two data frames, df1 and df2, each with two columns, a and b. These simple data frames will help us demonstrate various operations clearly and concisely. Pandas allows us to perform element-wise arithmetic operations between data frames. When we add df1 and df2, Pandas aligns the data based on the column and index labels, performing the addition where the labels match. This operation demonstrates how Pandas handles arithmetic between data frames of the same shape.

We can also perform scalar operations on data frames, which apply the operation to every element in the data frame. In this code, we are multiplying every value in df1 by 2. This operation shows how we can easily scale all values in a data frame. Pandas provides a powerful way to apply functions to data frame columns. Here, we are using the apply method with a lambda function to square each value in the column a and create a new column c with the results. This demonstrates how we can derive new columns based on the existing data. When we want to apply a function to every element in a data frame, we can use the apply map method. In the code, we are multiplying every value in the data frame by 10. This method is useful when we need to apply a transformation on the entire data in the data frame uniformly. Grouping and aggregation are powerful tools for data analysis. In the code, we are grouping the data frame by category column and then calculating the mean, sum, and count, which are all aggregating operations for each group. It demonstrates how we can quickly generate summary statistics for different subsets of our data. How about merging? Merging allows us to combine two data frames based on a common key. In the code, we are performing an outer merge on the key column, which includes all keys from both the data frames. The operation shows how we can combine data from different sources while handling missing values. Joining is a similar operation to merging, but it is typically done based on the data frame indices. Here, we are performing an outer join of two data frames with different indices. It shows how we can combine data frames side by side, aligning them based on their index values. To conclude, we have discussed arithmetic operations between data frames and with scalars, demonstrating how to perform calculations on our data. We have also learned how to apply functions to columns, rows, and the entire data frame, allowing us to transform our data in complex ways. Grouping and aggregation techniques were introduced for generating summary statistics, which are crucial for data analysis. Finally, we explored merging and joining operations, which allow us to combine data from different sources. These operations form a powerful toolkit for manipulating and analyzing data with Pandas. Thank you for your attention. Happy data analyzing.

Lesson 1 Demo Video• . Duration: 16 minutes

[Notes]

Greetings, learners. We are about to embark on a practical journey through the pandas library, where we will bring to life all the code snippets we have explored in the lesson so far. Using Jupyter Notebook, we will execute these Python snippets and analyze their outputs together. Without further ado, let's dive in. First, we have to import the pandas library from the Python modules. Import pandas as pd helps us to do that. pd is the alias name that we provide. Then we create a series from either a list or a series with custom index or from a dictionary. Let's execute this and see how it works. So when you create a series from a list, the list is 10 to 55 elements. Python automatically provides an index which starts from zero and moves till four in this example. We can modify the index by providing a custom index list, A to E. If it is a dictionary, then automatically the keys will be converted as the index. And the values will be converted as the data or the value of the series. To access each element in the series, we use the indexing operation, right? Alternatively, we can use position also. So here it will give us the element at the index B, and this will give the element at the position one. We can also perform slice operation which will give you a set of elements from the position one to three, three is excluded. The attributes of a series include index values and D type. Index will return all the index values. Values will return all the values. And the data type will be

returned by D type. Note that the index and values are returning as a list. Basic operations on a series includes addition and multiplication. Addition will add both the series in element wise order, whereas multiplication with a scalar is a scalar multiplication, where it is broadcasted across all the elements in the series. So in the first one, we will get the result as 11, 22, 33, etc, for the addition operation. For multiplication, the value of S1 is multiplied by 2, resulting in all the even numbers. Filtering operation on a series will help us filter elements based on some criteria. So S greater than 30 will check whether it is returning true or false. And wherever it is true, that element alone will be returned. So we are expecting a result of 40 and 50 in this example. Corresponding indexes are also returned. When we apply functions using the apply method, we can either apply regular functions or lambda functions. This example shows applying a lambda function, which is essentially going to square each of the element, because we said $\lambda x, x^2$. So it will square each element and return a series. If we have to check for null values or remove null values, that is possible using `isnull` and `dropna`. `isnull`, which is for null value, `isnull`, it will return a Boolean list, where if it is Nan, it will return true, otherwise it will return false. `dropna` will remove this particular data as such, or the row as such. And as you can see, after applying that function, the element at index 2 is removed. Series can hold multiple data types also, which can be processed in the same way that we have discussed. Moving on, let's move to the lesson on data frame. A data frame can be created from a list. Here we have three lists, one for name, one for age, one for city, and the last one for salary. Now these four lists are associated as values for the dictionary elements name, age, city, and salary. This is created as a dictionary. So here data is a dictionary of lists. Then we create a data frame from the data using the pandas library and the data frame method. The result is shown as a table. We in effect get a table with all the values that we have mentioned. The index, again, it is numerical, it starts with zero. The index for the data frame can be added as a list and recreate the data frame using the data frame constructor. So the index can be customized. We can add a new column to the existing data frame just by providing the data frame in square brackets, the column name. This will go as a heading and the corresponding values as a list. In effect, we have added column country to the existing data frame. This is very useful when we are manipulating the data in data aggregation. There are two basic data frame operations, like `describe` and `info`. `describe` helps us to figure out the statistical data in the data frame. This is applicable only for the numerical data. So we have only age and salary as a numerical data. It will list out all the statistics for those two columns alone. Whereas `info` will give us more about the information regarding the data frame, like how many entries are there, how many non-null elements are there. This is one useful way to figure out if we have any null values. And what are the data types of each of the column in the data frame? Strings are by default given as objects, whereas int and float represents the numerical data. Similar to the other methods, we can apply any function over to the data frame. So here we are incrementing the age by one by applying a lambda function on each of the age value. So if you want to compare, the age for person one was 28. Now it has been modified to 29 after applying the lambda function. Let's go on to the next video where we are going to index each data frame. So we create a small data frame, a different one, which has three columns, name, age, and city. Then we will create an index appropriately for each of them, a very small data frame. Now, the indexing can be done on column, either as a single column or multiple columns. The first one will take out all the data regarding the column name, and second one will return the data regarding name and age. So it is two column index, a list of data. It will return a data frame. Each element can be indexed either using index-based or integer-based. So when we use `LOC` or the `lock` method, we are providing the index. It can be as a single row or it can be the entire rows and columns. When we say `LOC B`, it will return

the data of the row B. When we are saying LOC of A, C, and name, age, we are returning A, John 28, C, Alex 42, and all the things in between. So B is also returned. Instead of label-based indexing, we can go for integer-based indexing, where we will use ILOC or integer-based locator. ILOC of 1 will return the row at position 1. ILOC 0 to 2 will return all the data, all the columns and rows from 0 to 2. 2 is excluded. So A and B will be returned. We can also index based on a condition. So if you want to figure out who all are aged above 30, we create a criteria where age is above 30, create another data frame out of it, and then provide it as an index. So this will return whoever is having age above 30. That is a filtering that we do based on the Boolean index. Then we can select the column based on this Boolean index also. So it will list out the name where city is London. Next video talks about selection in data frame. Again, we create a sample data frame for us, which is going to have five data with four columns. We can select each column by providing the data frame and the column name. We can also use the dot notation. We can select multiple columns using a list of column names, either directly on the data frame or by using the LOC. If you are using LOC, note this usage of colon, comma, so that we can select the entire rows. If we need to select by rows, then we have to use the LOC operator. If we have to select by row positions, we have to use the ILOC or ILOCator. To create subset based on name and age, especially of the first two of the rows, one and two, we can use the location operator where we say one to three, four, getting one and two rows and their corresponding name and age alone. The same thing can be done with ILOC. We have to manipulate the numbers in such a way that we get all the data. For Boolean selection, we have to give a criteria. Say, for example, here we have given two criteria data. Age greater than 35, this will return all the rows where the age is more than 35. And this will return all the names and salary where the salary is greater than 60,000. Note the two different notations. This is from the direct selection and here we are using the location operator. Also note that in the first case, the entire row is selected. In the second case, only name and salary are returned. For advanced querying or filtering, we can use the query method, ESIN, and largest and smallest. So in query method, we give the criteria using the AND operator. So it will return all the rows which have age greater than 35 and city not equal to Tokyo. In the second case, it will return all the rows where cities are contained in this list. So if the city name is any, those will be returned. Random sampling can be done from the data frame by providing how many samples you need or as a fraction. This is completely random. Note that this is one and four, zero and three. If I execute it again, it will give a different set of data. The largest and smallest based on a particular criteria can also be selected. This will select the salary and the smallest age. The first criteria will give the first two largest values. Eval is also another method. This will evaluate this particular query. Another criteria for greater than 60,000. DF high salary is defining a new column appending to the existing data frame. And it's a Boolean data where only those data which satisfy this condition will have true. So we have row two and row four having true value in it. Moving on, in the video related to filtering a data frame, we are going to filter based on some conditions. Data frame age or DF age greater than 35 is a very simple condition. It will select the data frame. The result is again a data frame that you can see. It will create a data frame where these conditions are true. We can use and or operator. This is an example where we are using and operator. If the city is having a string which is in this list, then it will be filtered out. If you want to figure out if the string or if a name starts with the letter A, then that is also possible by using string commands. Str.startsWith is a string command that we use. We check it on the name column of the data frame. This will filter out all those data which starts with the letter A. In the first example, we had just said df square brackets and then the criteria. Instead of using df square brackets, alternative is used to use query. So df.query, we can do the same operation in a different

way. You can create a condition and use that condition with the loc operator. This is a different way of filtering data. The way we code is what is different. This is more readable when compared to the previous example. The between operator helps us to extract all the data where this particular range is satisfied. So between 30, 40 on age is applied and all the data, all the rows that satisfy this condition will be returned as a data frame. We can have complex queries being evaluated on the data frame using df.eval. Here the condition is age greater than 35, salary greater than 60,000 or city is New York. So any of these conditions, either this condition or the condition two has to be satisfied to return the data. Another way of doing this is to give a condition and filter out what are the columns that are required. We do not need all the columns that can be provided as a separate list to the loc operator, which will return the result. All these operations are providing a data frame as are. It is not a list or a series. What about the operations that can be performed on a data frame? We create a simple data frame, which is having elements, right? Both have two keys and three values each frame is two data frames are created. Now they have the same shape. So we can perform arithmetic operations like adding on that. Now when we do df1 plus df2, all the numerical data will get added. The element wise addition is what is happening. If we do a scalar operation like multiplication by two, each element in the data frame will get multiplied. This is for df1. So let me just take df1. Everything will be doubled in df1. We can apply a particular lambda function on a column itself and add that to different column in the data frame. So df1 of A will get squared and that result is stored in df itself by appending a new column. So new column is added. Can apply this on the entire data frame by providing the apply map function. Everything will get multiplied by 10 when we are using apply map. For grouping, we have to use the group by method. Group by will combine based on a category that we are providing or a criteria that we are providing. So in this example, we have a column by the name category, which is either A or B. So it will categorize. Let's execute this and see the results for ourselves. So when we do this, look at the category. It is A and the only two lists are there. Since we are saying group by dot group dot mean, it will compute the mean of each of the category and display that. We can also perform other aggregation operations like mean, sum and count. operation to be performed that can be created as a list and provided to the agg method of grouped data. Grouped is our data which is a data frame so we can apply on to that and it will perform all the three operations in one go. Two similar data frames can be merged by using the merge operation but you have to specify on which key it has to be merged. Now note that the data frame df2 does not have the key c similarly df1 does not have the key d so in those two places nan is being posted. Join can be done on two data frames by specifying on which key that you want to do. So here data frame one which has a and b with two different keys are appended with c and d again if the data is missing dan will replace that missing area. Nan stands for not a number. As we conclude I urge you to experiment these files independently. Enjoy coding and I appreciate your engagement during this session.

Descriptive Statistics for Numerical Data Video• . Duration: 7 minutes

[Notes]

Welcome learners to the topic Descriptive Statistics for Numerical Data. By the end of this video, you will be able to understand and calculate measures of central tendency, compute and interpret measures of dispersion, describe the shape of a distribution using skewness and kurtosis, interpret descriptive statistics in the context of data. Descriptive statistics are fundamental tools used to

describe the main features of a dataset. They encompass three primary measures, measures of central tendency, measures of dispersion and measures of shape. These statistics help us understand the typical values of our data, how spread out the data is and what is the overall distribution of the data. By using these measures, we can gain insights into our dataset's characteristics and make informed decisions based on its understanding. The mean is a measure of central tendency, calculated by summing all the values in the dataset and dividing by the total number of values as given in the equation. It is represented by the equation $\text{mean} = \frac{\text{sum of data points}}{\text{number of data points}}$. For example, given the data points, the mean would be the sum of all data points by 5, which is equal to 6. While mean is widely used, it is important to note that it can be sensitive to outliers and extreme values, which can skew the results. The median is another measure of central tendency, representing the middle value when the data is ordered. For odd number of values, it is simply the middle value. For even number of values, it is the average of two middle values. The median is useful because it is resistant to outliers, making it a more robust measure of central tendency when dealing with skewed data or datasets with extreme values. The mode is the most frequently occurring value in the dataset. For instance, in the data provided, mode would be 5. A dataset can have no mode, one mode or unimodal, or multiple modes, multimodal. The mode is useful for categorical data where concepts like mean or median may not apply. It provides insight into the most common category or value of the datasets. The code calculates the mean, median and mode of a sample dataset using NumPy and SciPy libraries. SciPy stands for Scientific Pi. NumPy's mean and median functions are used for the mean and median calculations, whereas SciPy's stats.mode function is to find the mode. These functions provide a quick and efficient way to compute the central tendency measures, which are essential for understanding the typical values in a dataset. The range is a simple measure of dispersion, calculated as the difference between the maximum and the minimum values in the dataset. For the given data, the range would be the difference between max 6 and min 2, which is 4. While easy to calculate and understand, the range is sensitive to outliers and only considers the two extreme values, potentially overlooking important information about the distribution of data between these points. Variance measures the average square deviation from the mean, providing insight into how spread out the data is. It is calculated using the equation, $\text{variance} = \frac{\text{sum of squares of differences of data points to its mean}}{\text{number of data points} - 1}$. For the dataset, the mean is 6 and the variance would be 10. The variance helps us understand the spread of data points around the mean. The standard deviation is the square root of variance, providing a measure of dispersion in the same units as the original data. Using our previous example, where the variance was 10, the standard deviation would be square root of 10, which is 3.16. Like variance, the standard deviation helps interpret the spread of data around the mean. It is widely used because it is in the same unit as the data, making it more intuitive to interpret. The interquartile range or IQR is the difference between 75th percentile and 25th percentile of the dataset. For example, in the dataset, Q1 or the 25th percentile is 3 and Q3 is 7, so the IQR is 4. The IQR is resistant to outliers and represents the spread of the middle 50% of the data. It is useful for identifying outliers and understanding the concentration of data around the median. The code snippet calculates the various measures of dispersion using NumPy functions. It uses the max and min value to compute the range, variance, and standard deviation using VAR and STD functions. The interquartile range are computed using the percentile methods. These functions provide an efficient way to calculate these important dispersion measures, which help us understand how spread out our data is and identify potential outliers. Skewness measures the asymmetry of a distribution. It is calculated using the equation. Skewness is the sum of cubes of

differences of data points to its mean by the number of data points times cube of variance. A positive skewness indicates a right-skewed distribution with a longer tail on the right side, while a negative skewness indicates left-skewed distribution. Kurtosis measures the tailness of a distribution, indicating whether data has heavy tails or light tails compared to a normal distribution. It is calculated by the equation given there. A kurtosis greater than 3 indicates heavy tails or more outliers, while a kurtosis less than 3 indicates light tails. Kurtosis helps in understanding the shape of the distribution and the likelihood of extreme values. A boxplot visually summarizes several key statistics, including the median, the quartiles, the range, and the potential outliers. The code creates a boxplot using Seaborn's boxplot function. Boxplots provide a quick and comprehensive view of the data's distribution, allowing for easy comparison between different datasets or groups within a dataset. They are useful for identifying outliers and understanding the spread and the skewness of the data. To recap, statistical summaries are essential for initial data exploration and understanding. It is important to choose appropriate statistics based on your data type and distribution. Thank you for your attention. Happy data analyzing!

Descriptive Statistics for Categorical Data Video• . Duration: 8 minutes

[Notes]

Welcome, learners, to the topic, Descriptive Statistics for Categorical Data. By the end of this video, you will be able to understand the nature of categorical data, learn how to summarize categorical data, explore frequency distributions and proportions, understand and interpret contingency tables, learn about measures of association for categorical data. Categorical data can be classified into three main types. Nominal data consists of categories with no inherent order, such as colors like red, blue and green. Ordinal data include categories with a meaningful order, exemplified by education levels such as high school, bachelor's, master's and PhD. Binary data represents information with only two categories, typically yes or no responses. Frequency distributions summarize the number of occurrences of each category in a dataset. To create a frequency distribution, we count the occurrences of each category and calculate relative frequencies by dividing each count by the total number of observations. These distributions can be visualized using a bar chart or a pie chart, providing a clear picture of the data's composition and allowing quick comparisons between categories. Let's consider an example of customer satisfaction ratings. In a survey of 100 customers, 10 rated their experience as poor, 30 as average, 40 as good and 20 as excellent. Note that these are the counts. To calculate relative frequencies, we divide each count by the total number of responses. This gives us 10% for poor, 30% for average, 40% for good and 20% for excellent. This distribution allows us to easily see that the majority of the customers had a positive experience. The mode is the category with the highest frequency in a dataset. For categorical data, the mode is useful as it describes the most common category. In our example, if good has 40 occurrences and is the highest, it would be the mode. The mode provides a measure of central tendency for categorical data where concepts like mean or median may not be applicable or meaningful. The code creates a sample dataset using Python and Pandas. We generate data for 100 products across 3 categories, electronics, clothing and books. Each product is assigned a satisfaction rating of poor, average, good or excellent based on the predefined probabilities. This dataset will be used to demonstrate the descriptive statistics and visualization for categorical data in the coming slides. The code snippet demonstrates how to create a frequency distribution using

Pandas. The `valueCounts` function is used to count the occurrences of each satisfaction rating and `sortIndex` method ensures the categories are in the logical order. The mode of the satisfaction ratings is also calculated representing the most common rating. This provides a quick summary of the distribution of satisfaction ratings in our dataset. Contingency tables, also known as cross-tabulations, show the relationship between two categorical variables. These tables are created by counting the occurrences of each combination of categories from both the variables. For example, we might count how many electronics products received each satisfaction rating and to do the same for clothing and books. This allows us to see how satisfaction ratings vary across the product categories. For instance, there are 20 electronics products rated as good. The row and column total provides marginal frequencies. This table allows us to quickly see patterns such as which product category has the highest number of excellent ratings. The code shows how to create a contingency table using Pandas. The `crossTab` function is used to generate the table with product category as rows and satisfaction as columns. The resulting table shows the frequency of satisfaction rating for each product category. This provides a comprehensive view of how satisfaction varies across different product types in our dataset. Marginal frequencies are the row and column totals in the contingency table. Row marginals sum up the frequencies of each row. Column marginals sum up the frequencies down each column. These provide summary information for each variable independently. Marginal frequencies are useful for understanding the overall distribution of each variable. Conditional proportions represent the proportion of cases in category of one variable given a specific category of another variable. Conditional proportions help in understanding the relationship between variables by showing how the distribution of one variable changes based on the categories of the other. The equation is probability of A given B is frequency of A and B by marginal frequency of B. For example, the proportion of electronics rated as good would be 20 by 50 which is 0.04 or 40 percentage. The code calculates conditional proportions using Pandas. It divides each cell in the contingency table by its row total resulting in proportions that sum to one for each product category. This transformation allows us to compare the distribution of satisfaction ratings across product categories even if the categories have different total number of products. Visualizations are crucial for understanding categorical data. Bar plots are effective for showing the frequency or proportion of each category. Pie charts can illustrate the composition of a whole. Heat maps are used for visualizing contingency tables using color intensity to represent frequency or proportion. These visualizations make it easier to identify patterns, trends, and relationships in categorical data. The code creates a heat map visualization of the contingency table using Seaborn library. The heat map represents the frequency of each combination of the product category and satisfaction ratings using color intensity. Darker colors indicate higher frequencies. This visualization allows for quick identification of patterns such as which product category has the highest concentration of excellent ratings. To recap, categorical data analysis involves understanding different types of categorical variables nominal, ordinal, and binary. The key techniques include creating frequency distributions, identifying the mode, and constructing contingency tables. Marginal frequencies and conditional proportions provide deeper insights into relationships between variables. Visualizations such as bar plots and heat maps are essential for effectively communicating patterns in categorical data. Thank you for your attention. Happy data analyzing.

Data Relationship: Correlation and Covariance Video• . Duration: 6 minutes

[Notes]

Welcome learners to the topic Data Relationship using Correlation and Covariance. By the end of this video, you will be able to understand the concept of correlation and covariance, learn how to calculate and interpret correlation coefficients, understand the differences between correlation and covariance. Covariance is a statistical measure that quantifies how two variables change together. It is calculated by the given equation. This measure helps us understand the relationship between two variables indicating whether they tend to increase or decrease together or if they move in opposite directions. The interpretation of covariance provides insights into relationship between variables. A positive covariance indicates variables tend to move in the same direction, meaning that as one variable increases, the other also tends to increase as well. Conversely, a negative covariance suggests that the variables tend to move in opposite directions with one increasing as the other decreases. When the covariance is zero or close to zero, it suggests that there is no linear relationship between the variables. Correlation is a standardized measure of the strength and direction of the linear relationship between two variables. The Pearson correlation coefficient is calculated using the equation as shown. Unlike covariance, correlation is always between negative one and positive one, making it easier to interpret and compare across different pairs of variables. The correlation coefficient given as r , r equals one, shows a perfect positive correlation. It indicates that as one variable increases, the other increases proportionally. Conversely, r is a negative value indicates a perfect negative correlation, which means that one variable increases, the other decreases proportionally. When it is zero, it suggests no linear correlation. The strength of the correlation can be categorized as weak for the coefficient is less than 0.3, moderate when the coefficient is in between 0.3 and 0.7, or strong if the coefficient is greater than or equal to 0.7. This provides a quick way to assess the significance of the relationship. The different correlation strengths and directions can be visually represented using scatterplots. It helps in understanding how the plot changes with different correlation coefficients from perfect positive correlation to perfect negative correlation, including examples of no correlation and varying strengths of positive and negative correlations. This slide demonstrates a step-by-step calculation of correlation using a small dataset. We start by calculating the means of X and Y , then compute the covariance by summing the products of deviations from the means and dividing by n minus one. We calculate the standard deviations of X and Y . Then we use these values along with the covariance to compute the correlation coefficient. The result RS 0.36 indicates a moderate positive correlation between the variables. This example illustrates how to interpret the results in the context of data. The code demonstrates how to compute covariance and correlation using NumPy. It generates a sample data, then uses `np.cov` and `np.corrcoef` functions to compute covariance and correlation respectively. The results show a strong positive correlation between X and Y , which is expected given how Y was generated as a function of X . How about using Pandas library? The code shows how to compute covariance and correlation using Pandas. It creates a data frame from the same data used in NumPy example, then uses `df.cov` and `df.corr` methods to compute covariance and correlation respectively. The results match those from NumPy. While both correlation and covariance measure relationships between variables, they have distinct characteristics. Correlation is a standardized measure ranging from negative 1 to positive 1, making it unit-less and easier to interpret. It allows for comparison between different pairs of variables. Covariance on the other hand

is not standardized and depends on the scale of variables, making it harder to interpret in isolation. However, covariance is useful in multivariate analysis such as principal component analysis or PCA. When using correlation and covariance, it is important to be aware of their limitations. Correlation does not imply causation, meaning that a strong correlation does not necessarily indicate that one variable causes changes to another. Pairs and correlation is sensitive to outliers and only measures linear relationships, which can be misleading for nonlinear relationships. Additionally, correlation can be affected by the range restriction in the data. Understanding these limitations is crucial for proper interpretation and application of these measures. To conclude, covariance measures how variables change together, while correlation standardizes this measure to a range between negative 1 to positive 1. Thank you for your attention. Happy data analyzing.

Univariate Analysis Video• . Duration: 4 minutes

[Notes]

Welcome, learners, to the topic Univariate Analysis. By the end of this video, you will be able to understand the concept of univariate analysis, learn various techniques for summarizing single variables, and explore different visualization methods for univariate data, interpret statistical measures in the context of data distribution. Univariate analysis is a fundamental statistical method that focuses on examining one variable at a time. It primarily concentrates on understanding the distribution, central tendency, and spread of the individual variables within a dataset. This type of analysis serves as a crucial first step in exploratory data analysis, helping researchers and analysts gain insights into the characteristics of each variable independently. By providing a comprehensive understanding of individual features, univariate analysis lays the groundwork for more complex multivariate analysis, enabling a thorough exploration of the dataset structure and properties. Univariate analysis employs several key techniques to summarize and understand single variables. Measures of central tendency, such as mean, median, and mode, provide insights into the typical or average values of a variable. Measures of dispersion, including range, variance, and standard deviation, and interquartile range help quantify the spread or variability of the data. Percentiles and quantiles offer a way to understand the distribution of values, while skewness and kurtosis provide information about the shape of the distribution. For categorical data, frequency distributions are used to summarize the occurrence of different categories. Visualization plays a crucial role in univariate analysis, making it easier to interpret and communicate findings. Continuous plots are useful for showing quartiles and potential outliers, providing a compact summary of the distribution of the data. For categorical data, bar charts effectively display the frequency or proportion of different categories. When dealing with continuous data, histograms and density plots are invaluable tools for visualizing the distribution shape, allowing analysts to quickly identify patterns, modes, and unusual features in the data. The z-score method is a common technique for detecting outliers in univariate analysis. The code calculates z-score for the age variable in a data frame. It subtracts the mean from each value and divides by the standard deviation. Outliers are identified as data points with absolute z-scores greater than 3, meaning that they are more than 3 standard deviations from the mean. The code then prints the number of outliers found, providing a quick way to assess the presence of extreme values in the dataset. To recap, univariate analysis is a fundamental technique in data analysis, focusing on examining one variable at a time. It provides insights into individual variables, forming the foundation of more complex analysis. By employing various statistical

measures and visualization techniques, univariate analysis helps in understanding data distributions, identifying outliers, and guide further statistical tests and modeling decisions. This initial exploration of individual variables is essential for building a comprehensive understanding of the dataset and informing subsequent analytical steps. Thank you for your attention. Happy data analyzing.

Bivariate Analysis Video• . Duration: 3 minutes

[Notes]

Welcome learners to the topic Bivariate Analysis. By the end of this video, you will be able to understand the concept of bivariate analysis, learn various techniques for analyzing relationships between two variables, explore different visualization methods for bivariate data, interpret statistical measures of association between variables. Bivariate analysis is a statistical method used to analyze the relationship between two variables. It explores one variable changes with respect to another, helping to identify patterns, trends, and correlations within the dataset. This type of analysis is crucial for understanding the interactions between different features of your data. Bivariate analysis forms the foundation for more advanced techniques, such as predictive modeling and hypothesis testing. By examining the relationships between pairs of variables, we can gain valuable insights that guide further analysis and decision-making processes. The key techniques used in bivariate analysis include correlation analysis, which is a fundamental method that measures the strength and direction of linear relationship between two variables. Correlation analysis provides a quantitative measure of how closely two variables are related. Covariance is another important technique that measures how two variables vary together. While similar to correlation, covariance is not standardized and depends on the scale of the variables. For categorical variables, contingency tables are used to analyze relationships. These tables display the frequency distribution of the variables in a matrix format, allowing us to see how different categories interact. Visualizations play a crucial role in bivariate analysis, making it easier to understand and interpret relationships between variables. Boxer plots with correlation coefficients are widely used to visualize the relationship between two continuous variables. Each point on the plot represents an observation, and the overall pattern of the points reveals the nature of the relationship. Box plots are useful for comparing the distribution of continuous variables across different categories. They show the median, the quartiles, and the potential outliers for each category. Heat maps are effective for visualizing relationships between categorical variables or for displaying correlation matrices, using color intensity to represent the strength of the relationships. To summarize, bivariate analysis is a powerful tool for examining relationships between pairs of variables in a dataset. It employs various techniques such as correlation analysis, covariance calculation, and contingency tables to quantify and describe these relationships. Visualizations like scatter plot, box plots, and heat maps help in understanding and communicating these relationships effectively. By providing insights into how variables interact, bivariate analysis guides feature selection for modeling, hypothesis formulation, and informed decision making in various fields of study and practice. Thank you for your attention. Happy data analyzing.

Lesson 2 Demo Video• . Duration: 12 minutes

Welcome everyone! We will be demonstrating the practical application of all the code snippets we have discussed in the lesson on EDA. We will be using Jupyter Notebook to execute these code snippets and examine their outputs in real-time. Let us begin our hands-on session. We'll start with a video on summary statistics. We are going to rely on a lot of libraries. One is NumPy, which we have already seen. Second is Pandas. Third one is SciPy, which stands for Scientific Pi, and in that we have a stats module. We are importing that to do the statistical computation. We will rely on two libraries, Matplotlib and Seaborn for our visualization. Matplotlib has a sub-part or sub-module called pyplot, which is responsible for the actual plotting. And we will be using plt as the alias for Matplotlib. And sns is the alias for the Seaborn library. Our sample data is just a simple list with some random numbers. Let's execute this so the libraries are loaded onto the memory. Now we can compute the measures of central tendency, mean, median, and mode by just calling the function from the NumPy. For computing mode, we are going to rely on the stats library, which we have used. To compute the measures of dispersion, we again go back to the np library, the NumPy library, compute the max, the min, do a simple arithmetic to get the range values. The variance and standard deviations are computed from the NumPy library. For IQR, we have to do a little bit more computation. We can use the percentile methods to get the Q1 and Q3 value, compute the upper bound and lower bound also to figure out the noise or the outliers in the data based on IQR. Let's execute this. Yes, we get the range, the variance, standard deviation, and IQR, which are the measures of the dispersion. For shape measures, we will have to use skewness and kurtosis. We are relying on the stats library to compute these two. Moving on, based on the video on correlation and covariance, we need to compute the correlation between two data and covariance between two data. That means we need sample data, which is of two dimensions. We create a random sample on X. We are going to populate 100 data or 100 values, and this will be stored as a list. Based on the X values, we are going to create, using some function, we are going to simulate or create 500 Y values, which is of normal distribution. So both X and Y are correlated in some way. Now, compute the covariance and correlation using NumPy library. We use cov and cor coefficient. We compute the respective measures. If you're going to rely on pandas, the first step that we have to do is convert the X and Y into a data frame. Then only we will be able to do it. We will not be able to directly work on the NumPy data. So convert it into a data frame, then use the correlation coefficient function and the covariance function to compute the data. On executing this, you can see that both return similar data. So we can rely on any one of them based on how you have your data with us, as a NumPy array or as a data frame. Now let's work with categorical data. So we'll create a data set which is categorical in nature. It's going to be a data frame, which has product category and satisfaction. Let's just print the data so that you can see. This is how the data frame looks like. It has 100 rows with two columns, product category and satisfaction. Product category will have electronics, clothing, and books. And the satisfaction ratings are poor, average, good, and excellent. By looking at the data, we do not understand much out of it. So we will create a frequency table out of it to figure out how many electronics, how many books, how many average, how many good, and all those things. So creating a frequency table requires a bit of computation. We are relying on value counts. Then from the results, we will sort it based on the index. This statement will generate a frequency table. If you want to convert it into a percentage, then again we have to figure out a way to do it, which is like divided by the length of the data frame multiplied by 100 so that it becomes a percentage. Now from the table or from the data frame, we can create a contingency table using the crosstab function. Let's execute this statement and see for ourselves. So this will give us a detailed

view of how many books are having excellent rating, how many clothing have good rating, how many electronics have excellent rating, like that. This can be converted into a proportion also by using the contingency table dot div. Contingency table is our table's object's name. Div is the division operation that we are going to do based on sum of each column divided by each row. So you get the proportional value for each of the data in the contingency table. Now the contingency table can be visualized also by using a heat map. Here we are going to rely on the Seaborn library. SNS is an alias name for Seaborn library. So we rely on the Seaborn library, create a heat map based on the contingency table. We are saying that annotation has to be done and we are going to give a color map. This stands for yellow, orange, red color map. We will provide a title and plt dot show will ensure that it is displayed in the notebook itself. The first line will set the figure size to 10, 6. This is how the contingency table looks like. So whatever data we had here 7, 11, 16, that is converted as a heat map where the color map will guide you on figuring out which is having the highest one. So if you look at the darkest color, it is for this squire which stands for electronics good 18. That means electronics and good combination is having the highest satisfaction level. For univariate analysis, we can combine all what we have learned till now, create a data frame, compute the measures of central tendency on each numerical data. So for that we use the for loop. We can also compute the measures of dispersion for each of the numerical data. So we have age, income, and education and for that we are computing the range variation and standard deviation by the same code that we have explained so far. Percentiles and quantiles also can be computed. Now this is possible only for the numerical data that we have age, income, education years. We have another column called satisfaction which is of categorical data. So to deal with categorical data, you have to rely on frequency tables. So based on the frequency tables, it will list out how much of medium satisfaction, how much of high satisfaction, how much of low satisfaction are there in our data. Skewness and kurtosis also can be computed for all the three numerical data that we have. How about outliers? Outliers are the extreme values that we have. So detecting outliers is done by using the z-score method. We compute the z-score for each column by relying on the stats library. Take the absolute value because it can rely, the outliers can be either on the right side of the mean or on the left side of the mean. So we can compute that and whichever satisfies this condition, if it is, if the z-score is more than 3, then we say that it is an outlier. So whichever satisfies that condition will be listed out as the outlier. So in this example, there are outliers on age, income and education group does not have any outliers. So that is left as it is. Let's visualize the histogram and the density plot for each of the data that is age, income and education. Remember histogram and density plot are possible for your numeric data. The histogram of age shows a nice normal curve, the bell-shaped curve or the normal curve, whereas the income is a skewed data. It is the education years is more or less uniform except for one spike at the end for more than 20 years. Till then, it is more of a uniform distribution. So looking at the visualizations, we can figure out what kind of distribution we are dealing with. Box plot helps us to figure out if there are any outliers. Bigger, let's just increase the plot size. So age has one outlier or small outliers. Income has got very large number of outliers, whereas educational years does not have any outliers. To look at the frequency table of satisfaction, we can rely on the bar plot. We had three satisfaction levels, medium, high and low. So this is just going to count how many medium levels we had, how many low levels have and plot that count as a bar plot. We can also have a correlation map based on all the data by creating a heat map. This heat map will create, as you can see, it will create a correlation matrix based on all the three numerical data. So this says that the first square says how is age correlated to age? It will obviously have a value of one because it is always correlated with itself. How about age correlated to income?

That is given as a negative value, negative 0.038. So it is negatively correlated. Whereas age and education years, it is not having much correlation. So this is how you read it. It is symmetric metric based on the diagonal data. We normally ignore the diagonal data. We look at only the upper triangle or the lower triangle. Any one needs to be looked at. And there is a color map which will guide us to figure out how much correlation is there in the data as such. Proceeding to bivariate analysis, similar to the previous case, we create a sample data set where we have five columns, age, income, education, year, satisfaction, and department. Let's review the sample data. `df.head` will help us look at the first five data in the data set. So we have three numerical data and two categorical data. We will create a correlation metrics based on the numerical data alone, age, income, and education years. So we have the correlation data with us. It comes out as a metric. Similarly, we can have a covariance metrics also. Remember, we are relying on to create a crosstab or the contingency table, the satisfaction and the department is the categorical data. So we pass those two columns to the crosstab function. So it will tell us how many people in HR gave a high satisfaction, how many people in sales gave a high satisfaction. This is how we have to read it. The correlation metrics and the frequency, the contingency table can be visualized. So let's first visualize the correlation metrics, similar to the previous one because the data is more or less same. We can also have a scatter plot to figure out how variation is. So here we are going to plot only age and income. So the relationship between age and income is more or less scattered. It is a weak correlation that we have. The contingency table also can be visualized by using a heat map. It needs to be separately listed out. We are going to rely on a color map of yellow, green, and blue. There are various options for color maps. Now the darkest shade is for this square, the bottom left square, where maximum number of people have given a rating for medium and they are all belonging to the HR department. This is how we read it and the minimum will go to this particular square, which is around 38, where it is from the sales department with low rating. To wrap up, I encourage you to run these files on your own. Enjoy your coding experience and I thank you for your focused attention.

Scatter Plots Video• . Duration: 8 minutes

[Notes]

Welcome, learners, to the topic Scatter Plots. By the end of this video, you will be able to understand the purpose and use cases of scatter plots, create basic scatter plots using Matplotlib, interpret scatter plots to derive insights from data. A scatter plot is a two-dimensional graph that uses Cartesian coordinates to display values for two variables. Each point on the plot represents an individual data point with its position determined by the value of the two variables. Scatter plots are primarily used to visualize the relationship between two continuous variables. They are invaluable tools for identifying patterns, correlations, and outliers in datasets, making them essential for exploratory data analysis. The scatter function plots each x and y in the space provided. The space is set by the figure size as given here as 10, 6. It also sets the marker style, the size to 50, and the color as blue, and transparency given by the alpha parameter as 0.7. The title and both x and y axis labels are added for clarity. This simple scatter plot provides a visual representation, the relationship between the x and y variables, allowing us to see patterns or trends in the data at a glance. Here we plot two datasets on the same scatter plot. We generate a second set of x and y values and use different colors to distinguish between the datasets. A legend is added to identify each dataset. This

technique is useful for comparing relationship between variables across different datasets or groups, allowing us to see similarities or differences in the patterns or trends. The code adds a linear trend line to the scatter plot. It uses numpy's polyfit function to calculate the line of best fit and poly1d to create the polynomial function. The trend line is then plotted on top of the scatter plot. Adding a trend line helps visualize the overall relationship between the variables and can make it easier to identify patterns or deviations from the general trend. Creating a scatter plot involves analyzing the visual representation of data points to understand relationships between two variables. The key aspects to consider include the overall relationship between variables, the presence of clusters or outliers, the linearity of the relationship and the strength of correlation. By examining these elements, we can get valuable insights into the data and formulate hypotheses for further investigation. The strength of the relationship in a scatter plot can be categorized as strong, weak or non-existent. A strong relationship is indicated by points forming a clear, narrow pattern with y increasing as x increases. A weak relationship shows a similar trend but with more spread of points. No relationship is evident when the points show no discernible pattern suggesting the variables are not related. When examining scatter plots, look for outliers which are points from the main cluster of data. Identify any clusters which are groups of points that cluster together. Note any gaps which are areas with no data points. Observe the range which is spread of data along both axes. Pay attention to density, noticing areas where points are more concentrated. These features can reveal important characteristics of the data distribution and relationship between variables. Scatter plots can reveal whether relationship between variables are linear or non-linear. In a linear relationship, points roughly follow a straight line, with y increasing linearly with x plus some noise. A linear trend line fits this pattern well. In a non-linear relationship, points follow a curve or pattern. For example, y might increase quadratically with x. In this case, a non-linear trend like a quadratic curve would better fit the data. Correlation in scatter plots can be positive, negative or non-existent. Positive correlation is shown by points trending from lower left to upper right, indicating y increases as x increases. Negative correlation is represented by points trending from upper left to lower right, showing y decreases as x increases. No correlation is evident when points are scattered randomly, suggesting y is unrelated to x. This slide illustrates the important distinction between correlation and causation. Using the example of ice cream sales and drowning incidents, we see that while these may be correlated both increase with temperature, ice cream sales does not cause drowning incidents. More study time causes better score results. It is crucial to understand that scatter plots can show correlation, they do not prove causation. This slide further explores correlation against causation with four plots. The ice cream sales against drowning incidents plot shows correlation without causation. The temperature against ice cream sales plot shown in plot 2 demonstrates causal relationships where temperature influences the sales. Plot 3 plots the temperature against drowning incidents to demonstrate causal relationships where temperature influences the incidents. The study time against test scores plot shows a case where correlation does imply causation. This is shown in plot 4. Scatter plots are particularly useful in three main scenarios. First, they excel at visualizing the relationship between two continuous variables, allowing us to see how one variable changes with respect to another. Second, they are invaluable for identifying patterns, trends, or correlations between the variables, which can guide further analysis or hypothesis formation. Lastly, the scatter plots are effective for detecting outliers or clusters in a data sets with two variables, helping to identify unusual data points or groupings that might warrant further investigation. To summarize, scatter plots are powerful tools for visualizing relationship between two continuous variables. They enable us to identify patterns, trends, correlations, outliers, and clusters in data.

Proper interpretation of scatter plots is crucial for deriving meaningful insights from the data. As fundamental components of exploratory data analysis, scatter plots provide a solid foundation for understanding complex relationships in data sets and guide further statistical analysis. Thank you for your attention. Happy data analyzing.

Line Plots Video• . Duration: 8 minutes

[Notes]

Welcome learners to the topic Line Plots. By the end of this video, you will be able to understand the purpose and use cases of line plots. Create basic line plots using Matplotlib. Customize line plots and interpret line plots to derive insights from data. A line plot is a type of graph that displays data points connected by a straight line. It is primarily used to show trends over time or any ordered sequence. Line plots are effective for visualizing continuous data and changes in values. They're commonly employed in time series analysis and trend visualization, making them an essential tool for understanding how data evolves over a given period of time or a sequence. This code generates a sample data set for our line plot demonstrations. It uses NumPy to create x values and two sets of y values with sine and cosine functions respectively. The x-axis typically represents the independent variable, while the y-axis represents the dependent variable. We use pyplot of Matplotlib library in Python for all the plotting. The PLT is an alias added by the programmers for easy reference to the library. A basic line plot is created using the code shown using Matplotlib's plot function. The plot function plots both x and y points provided by the data. It also sets the figure size, adds a title, and x and y-axis labels. The resulting plot shows a simple sine wave demonstrating how data points are connected by straight lines. This basic plot provides a clear visualization of how the y values change with respect to x, allowing us to see the overall trend and the pattern of the data. Here we plot two lines, the sine function and the cosine function on the same graph. Different colors and line styles can be used to distinguish between the lines, and a legend is added for clarity. This technique is useful for comparing multiple data sets or variables over the same range of x values, allowing us to see relationships, similarities, or differences between the sets of data. The code demonstrates how to customize the axis of a line plot. We set custom limits for x and y-axis using xlim and ylim methods, focusing on a specific range of values. A grid is added using the grid method and passing true as a parameter. This improves readability. This customization can help emphasize a particular aspect of the data or make the plot more informative for specific analysis needs. Annotations can be added to line plots to highlight specific points or features of interest. The code uses annotate method to add a text label with an arrow pointing to the maximum point of the sine wave. Similarly, for the minimum point of the sine wave. Annotations are valuable for drawing attention to key data points, explaining trends, or providing additional context to the visualization. The code creates multiple subplots within a single figure using subplots method. It is useful for comparing related data sets or showing different aspects of the same data side by side. In this example, we plot the sine and cosine functions in separate subplots, allowing for easy comparison of their shapes and trends. Interpreting a line plot involves analyzing the visual representation of the data points to understand relationship and trends over a time or a sequence. Key aspects to consider include the overall trend of the line, any recurring patterns, key important points, and the rate of change. By examining these elements, we can gain valuable insights into the data on how it behaves and changes over a given range. Line plot can reveal various trends in data. An increasing trend is shown by a line that

generally moves upward, while a decreasing trend is indicated by a downward movement. A stable trend is represented by a relatively flat line. A fluctuating trend shows the line moving up and down without a clear direction. For plots with multiple lines, crossover points can be significant, indicating where different data sets intersect or change their relative positions. Line plots can reveal different patterns in data. Cyclic patterns show regular repeating cycles, such as daily temperature and fluctuations. Seasonal patterns occur at specific intervals, often showing one major cycle with smaller cycles within it, like yearly temperature variations. Random patterns show no discernible repetition, with data points fluctuating unpredictably. Recognizing these patterns can provide insights into the underlying factors influencing the data. When interpreting line plots, it is important to look for discontinuities, which might indicate missing data or significant events. Outliers are points that deviate significantly from the general trend, and they should be noted. Always check the scale, particularly whether the y-axis starts at zero, as this can affect the visual interpretation. Consider the time frame covered by the plot and its relevance to the data being analyzed. Key points in the line plots include peaks, or the highest points, troughs, or the lowest points, and the turning points, where the trend changes direction. The rate of change is indicated by the slope of the line. Steep slopes show rapid change, general slopes show slower change, and flat sections represent no change. Understanding these features help in identifying important trends, events, and patterns in data over time. Line plots are useful in three main scenarios. First, they excel at showing trends or changes over time, making them ideal for time series data. Second, they are valuable for comparing multiple series of data over the same period, allowing for easy visualization of how different variables or data sets relate to each other over time. Lastly, line plots are effective for visualizing any continuous data where the order of data points matters, providing insights into sequential patterns or trends. To recap, line plots are versatile tools for visualizing trends and changes over time, making them especially useful for time series data. They effectively show how variables change over an ordered sequence, allowing for easy identification of patterns, trends, and anomalies. Proper interpretation of line plots is crucial for understanding the behavior of data over time and deriving meaningful insights. Thank you for your attention. Happy data analyzing.

Bar Plots Video• . Duration: 8 minutes

[Notes]

Welcome, learners, to the topic Bar Plots. By the end of this video, you will be able to understand the purpose and the use cases of bar plots, create basic bar plots using Matplotlib, interpret bar plots to derive insights from categorical data. A bar plot is a powerful visualization tool that displays categorical data using rectangular bars. The height or the length of each bar is proportional to the value it represents, making it easy to compare quantities across different categories. Bar plots are effective for showing the distribution and comparison of discrete data, allowing viewers to quickly grasp differences between categories. Let us create a sample dataset to demonstrate bar plots. The code defines a list of categories and generates random values for each category using NumPy. We use these categories and values throughout the presentation to illustrate different types of bar plots. This simple dataset allows us to focus on the mechanics and interpretation of the bar plots. We use PyPlot of Matplotlib library in Python for all the plotting. The PLT is an alias added by the analyst for easy reference to the library. The code creates a basic vertical bar plot using Matplotlib. The bar function is used to generate the plot which categorizes on the x-axis and values on the y-axis. We

set the figure size, add a title and the label for the axes for clarity. Vertical bar plots are the most common type and the x-axis typically represents categories or groups and y-axis showing the measured value or frequency. Next we create a horizontal bar plot using the `barH` function. This type of plot is useful when category labels are long or numerous as it provides more space for the label text. In a horizontal bar plot, the categories are on the y-axis and the values are on the x-axis. This orientation can sometimes make it easier to read and compare values across categories. A grouped bar plot is used to compare multiple sets of data across categories. The code creates two sets of data and plots them side by side for each category. We use NumPy to arrange the x-positions of the bars and set a width for proper spacing. The legend helps identify which set of bars correspond to which data sets. Stacked bar plots are excellent for comparing two or more related quantities across different categories. Stacked bar plots are useful for showing composition within categories. The code creates a stacked bar plot by plotting one set of bars and then adding another set on top using the `bottom` parameter to position the second set. Each bar represents the total value of the category with segments showing the contribution of each component. This type of plot is effective for displaying part-to-whole relationships within categories. A percentage stacked bar plot shows relative proportions within each category. The code calculates percentages for each component and creates a stacked bar plot where each bar totals 100%. This type of plot is useful when the focus is on the relative proportions rather than absolute values, allowing for easy comparison of composition across categories regardless of their total sizes. When interpreting bar plots, start by comparing the bar heights. Taller the bar represents larger the values, while shorter bars indicate smaller values. Look for patterns in the data, such as overall trends increasing, decreasing, or fluctuating, any outliers or exceptions. These patterns can reveal important insights about the relationships between categories and their associated values. Understanding the scale is crucial when interpreting bar plots. Always check if the y-axis starts at zero, as non-zero baselines can exaggerate differences. Note any breaks in the axes and be aware of the units of measurement. Pay attention to color coding, understanding what different colors represent. Carefully read the labels and legends to know what is being measured and what each color or pattern signifies. For grouped bar plots, focus on comparisons both within and between groups. Look for consistent patterns or differences across groups, as these can indicate important relationships or trends in your data. This type of analysis can reveal insights about how different categories of data are or trends in your data. This type of analysis can reveal insights about how different categories or groups compare across multiple measures or time periods. When examining stacked bar plots, consider the proportion each segment represents within each bar. For 100% stacked bar plots, focus on relative proportions rather than absolute values. This approach allows you to understand the composition of each category and how it compares to others, regardless of the total size of each category. Vertical bar plots are ideal for comparing a single quantitative value across different categories, displaying frequencies of categorical data or showing absolute values. Horizontal bar plots are preferable when dealing with long category names, visualize rankings or order data, or when you have many categories and limited vertical space. Grouped bar plots are excellent for comparing two or more quantitative values across different categories. They are particularly useful when dealing with data that has two categorical variables and one quantitative variable, allowing for side-by-side comparisons within categories. This type of plot can reveal complex relationships and patterns that might not be apparent in simpler bar plots. Stacked bar plots are best for showing the composition of each category and displaying part-to-whole relationships. They effectively show both the total and the breakdown of the components. Percentage stacked bar plots are ideal when you want to focus

on the relative proportion of the components within each category. Compare percentage composition across categories when the total value is less important than the relative proportions. To recap, bar plots are versatile tools for comparing categorical data, offering various types to suit different data structures and analytical needs. The choice between vertical, horizontal, grouped or stacked bar plots depends on your data characteristics and the message you want to convey. When designing bar plots, prioritize readability and clarity to ensure your audience can easily interpret the information presented. Proper use of bar plots can significantly enhance data visualization and communication in various fields. Thank you for your attention. Happy data analyzing!

Histograms Video• . Duration: 7 minutes

[Notes]

Welcome learners to the topic Histograms. By the end of this video, you will be able to understand the purpose and use cases of histograms. Create basic histograms using Matplotlib. Interpret histograms to derive insights about the data distribution. A histogram is a graphical representation of the distribution of the numerical data. It displays the frequency of data points within bins, which are intervals of variable being measured. Histograms are used to estimate the probability distribution of a continuous variable and help in understanding the central tendency, spread, and shape of the data. This visualization tool is essential for getting a quick overview of the data's characteristics. To demonstrate histograms, we will start by creating a sample data set. The code relies on NumPy and Matplotlib libraries in Python. It generates 1,000 data points from a normal distribution with a mean of 100 and a standard deviation of 20. We use random module to create this data set, which will serve as the basis for our histogram examples. The random seed is set to ensure reproducibility of the results. The code creates a basic histogram using Matplotlib. The hist function is used to generate the histogram, and Matplotlib automatically determines the number of bins. We set the figure size, add a title, and label the x and y-axis for clarity. This simple histogram provides a quick view of the data distribution, showing the frequency of values falling within each automatically determined bin. We can customize the histogram by specifying the number of bins and adding an edge color. By setting the number of bins as 30, we create a more detailed view of the data distribution. The edge color parameter adds a black outline to each bar, improving visibility, especially when the bars are adjacent. This level of customization allows for better control over the histogram's appearance and granularity of the data representation. A normalized histogram shows the probability density instead of frequency on the y-axis. This is achieved by setting the density as true in the hist function. We also added a vertical line representing the mean of the data using AXV line. This type of histogram is useful for comparing distributions of different sizes and for estimating the probability density function of the underlying distribution. We can plot multiple histograms on the same axis. We create a second data set and plot both histograms using different colors and set transparency for better visibility of the overlapping regions. This technique is useful for comparing distributions of different data sets or groups within a data set. The legend helps identify which histogram corresponds to which data. A 2D histogram, also known as a heat map, shows the relationship between two variables. This code generates two sets of normally distributed data and uses hist 2D to create a 2D histogram. The color intensity represents the frequency of data points in each 2D bin. The color bar is added to interpret the frequency values. This visualization is useful for understanding the joint distribution of two variables. When interpreting histograms, it is important to

understand their structure. The x-axis represents the range of values divided into bins. Y-axis shows the frequency or count of data points in each bin. The height of each bar represents the number of data points falling within that bin. Look for gaps in the histogram, which could indicate empty bins or large spaces between bars, potentially revealing interesting patterns or anomalies in the data distributions. Histograms reveal the shape of the data distributions. A symmetric distribution, like the normal distribution, has a balanced shape around the center. Skewed distributions have a longer tail on either side, either right skewed or left skewed. Bimodal distributions show two distinct peaks, indicating two subpopulations or clusters in the data. Uniform distributions have roughly equal frequencies across all bins, suggesting an even spread of values across the range. When analyzing a histogram, locate the center by finding the peak of the distribution, which can help estimate the median and mode. Observe the tails to see how quickly the frequency drops off at the end, indicating the presence of extreme values. Identify potential outliers by looking for isolated bars far from the main body of the distribution. These features provide insights into the data's central tendency and spread. Assess the spread of the data by looking at its range and observing how wide or how narrow the distribution is. In density histogram, the area of each bar represents the proportion of data in that bin, useful for probability estimations. When comparing multiple histograms, ensure that they use the same bin width and scales for accurate comparisons. These aspects help in understanding the variability and relative frequencies within the data. Histograms are ideal for visualizing the distribution of a single continuous variable. They help identify the shape of the data distribution, whether it is normal, skewed, or has multiple modes. Histograms are particularly useful for determining the frequency or probability of values falling within each range or bin. This makes them valuable tools for understanding the underlying patterns and characteristics of your data. To summarize, histograms are powerful tools for visualizing data distributions, offering insights into central tendency, spread, and shape. Matplotlib provides various customization options for creating informative histograms. Choose the appropriate bin sizes and types based on your data characteristics. Consider adding statistical information, such as mean or median lines, to enhance the visualization's informativeness. Proper interpretation of histograms can reveal valuable insights about the data's distribution and guide further analysis. Thank you for your attention. Happy data analyzing.

Lesson 3 Demo Video • Duration: 14 minutes

[Notes]

Hello, learners. It is time to put theory into practice. We are going to run through all the code examples we have discussed in our lesson on EDA visualization using Matplotlib. We will be using Jupyter Notebook as our platform to execute these Python snippets and observe their results firsthand. Let us get started with our coding demonstration. First, we will see how to plot scatter plots. We will rely on NumPy library basically for generating data and Matplotlib library for the plotting. We will be using PLT for the alias of matplotlib.pyplot. Let's execute it. The next statement or next three statements is going to generate some random data. Random.seed is for the reproducibility of the results that we see now. The first statement will create random 50 numbers using uniform distribution and that we will store it as a NumPy array x. Based on NumPy array x, we are going to create a function which will generate random 50 numbers and denote it as y, again a NumPy array. Now let's start the basic plotting. Look at the results. Scatter plot will generate a plot

of the data as points, as 2D points on the Cartesian coordinates. The first line will set the size of the entire plot figure. So the size that you see is a 10 by 6, 10 width and 6 height units. And that will be the size of the plot that we have. Scatter x, y is responsible for the actual plotting. So here the marker is O, so we will get it as circle. The size of each marker is 50 and the color of the marker is blue and it has a transparency. That is not very bright color. It has a transparency of 0.7. Let's slightly modify this instead of O, if I give asterisk and say here I'm going to give red, you can see that the plot changes. So you can experiment with these parameters as you go. plt.title will set the title of the plot. xlabel and ylabel will set the x-axis label and y-axis label respectively. plt.show is to ensure that the plot is displayed within the Jupyter notebook. Now what if we want to have multiple scatter plots? So we have to create two sets of data. So we have x and y in the previous case and now we are going to create two different sets of data also, x2 and y2. In the same plot, we can create two scatter plots. Let's just create it, see the results for yourself. Since we have not provided any parameters or any arguments for the marker or the color, it will take the default values that is being set by the library. scatter xy will generate the data, the blue dots for the x and y data set and scatter x2, y2 will generate for the data set too. Those are the orange dots, right? So if you want to change it, I can give it as marker. So you have asterisks for the second data set and the circles for the first data set. We set the title xlabel, ylabel. Now plt.legend will be displaying a legend at the default position and this position changes according to the data that you have. It shows that, like you will ask, how do you know which data is what? So looking at the legend, you can figure out that blue dots is from data set 1 and the orange asterisks is from data set 2. If you want to set a line, which we call a strand line, to know in which direction the data is moving, we can overlay this plot with a line and for that we rely on polyfit and poly1d. Let's see the results. So first it creates the scatter plot. Then we fit the line according to the data that we have. This is done by the NumPy library. Then the fitted line, which is given as p here, is going to be plotted. So again, for plot, we definitely need x and y coordinates for the line to be plotted. So we say x and p of x to note the line coordinates. This r dash dash shows the color and the form. So it is going to draw a line in red color across the data that we have. Next is how to plot line plots. We will create a sample data where we have x as a set of random data, 100 points within the range 0 to 10. Then we will rely on the trigonometric functions. We will create two functions, sine x and cos x and store it as y1 and y2 respectively. Now let's plot. Plot x comma y1 will plot the data given by x and y1 in the space that is provided. This is a very basic line plot without any extra items plotted on it. All we have done is set the title, the x-axis label and the y-axis label. Note that the coordinates or the units are automatically taken by the pyplot library and we have the absolute zero for both x-axis and y-axis drawn. If you have to modify this, maybe change the line style, change the color or have multiple lines in it, we can do that. For each line, you will have to have its own plot. So this is for a plot between x and y1, the sine line and plot for x and y2, which is the cos line, which will require another line. We can set the color parameter, we can set the line style, we can set the line width and also the label, which will be taken in by the legend. Note that we are doing plot.legend to show the legend also. So again, the legend position is based on the data that you're plotting. So this is a multi-line plot where you can see that the cos line is given by the dotted lines in blue color and the sine line is provided by the red curves in the line style of dash. Both the lines are plotted on the same axis. If you want to blow up a particular part of this, if you want to change the axis of the plot that can be done by providing xlim and ylim. So note the difference between the two one. For the sine plot, this is the original sine curve, the red one is the original sine curve that we have. We are going to zoom out a particular part of it and display only that particular section using the xlim ylim. So xlim ylim will set the limits of the x-axis and y-axis

respectively. And you can get to see the curve in detail. Additionally, you can plot the grid also by using `plt.grid true`. So it will set the grid lines for easy reference or for easy comparisons. Now what if you want to annotate? So in the sine plot, if you want to annotate the max point and the min point that is done by using `plt.annotate`, you have to specify what is the text that you want to see at what position. That is the arrowhead should be at this position is what it means. And the text should be at what position? That has to be mentioned. And what is the properties of the arrow? How do you want the arrow to be displayed? That is given by `arrow props`. The color, the size, all can be provided as a dictionary for the arrow alone. So both max and min can be annotated into the same plot. This makes it more readable. And how to draw two plots in the same graph? We have a graph of size 10, 5. If you want to split it into two plots or two spaces where we want to plot two different functions, then we rely on subplots. This will give me two axis, `ax1` and `ax2`. It is not the axis of `x` and `y`, it gives us two subspaces. In the first subspace, you plot one function and in the second subspace, you plot the second function. For individual spaces, you can set the title, you can set the `x-axis`, `y-axis` labels, you can add a legend. Let's see how to create bar plots. We will create a random sample where we have five categories and some random values are being generated. Around 100 values are being generated for these five categories. It is an `int` value that is getting generated. First let us plot and see the result. This is the bar plot that we get. Like five categories, the category label is given in the `x-axis`, the values are given in the `y-axis`. So `x label` and `y label` is what is used to set it. The title is given as basic vertical bar chart or bar plot. How to create the bar plot then? By using the `bar` function. There you specify the categories and values which we had created earlier. Pass it as `x` and `y` values, it will plot the bar plot on its own. The same thing can be flipped. Instead of `bar`, if you use `bar h`, it will become a horizontal bar plot. The same data, we are just shifting the orientation. How to plot two bar plots on the same diagram? You definitely need to have two functions. You have to specify what is the data for product A and what is the data for product B. You can specify the label. So we create two data, A and B. We create the categories also for that specific one. A width as 0.35 is set. This is going to set the width of each square or each rectangle in the bar plot. Now this will generate a group bar plot where we are grouping A and B's product together. A legend is automatically added and we will get to see the results. Always ensure that you are starting with 0 for the `y-axis` values. To stack it up, you have to specify the `bottoms` parameter. Whichever goes under `bottoms` parameter will be going below in the stack part. So first we plot A values and then we say that plot the B values, but it should be on top of the A values. So orange is for the B and blue is for the A values. So it will get plotted correctly. Stacked bar plot shows the actual count. If we want it as percentages, then we will have to compute the percentages manually and use the bar plot again to get the percentage stacked bar chart. So now this is a relative scale of 0 to 100 and it shows the proportion for each product A and B. Let's look at histograms. To generate histograms, we need numerical data. So we are going to generate random around 1000 points based on the normal distribution. Let's plot the histogram and see the results first before we explain it. This is how the histogram looks like. It's more like a bar plot, but without the vertical lines separating each of the rectangles. It shows the frequency count. So on the `X axis`, you have the value and on the `Y axis`, you have the frequency. It will automatically bin. That is the bin is defined here. How much value should go within 75 and 85 or 55 and 65. How much value should go in between each of them. That is your bin size. And then it is plotting the frequency. If you want to modify, we can provide with custom bins. How many bins do you want? How many bins are to be represented? And we are going to add an edge color also. That is a bit more customization on the histogram or `hist` function. So now we have a better histogram where each bin will represent 30 values. For those 30 values,

what is the frequency that is provided and plotted. And we can also demark it because of the edge colors. We can see where each rectangle is in the histogram. Can we plot the mean and median of this histogram? Yes, by using `np.mean` and `np.median`. Now this is going to be marked as a line. So `AXVLINE` will help us plot a vertical line at this position. We can customize that also provide the color, the line style, the width and the label. Now to get the density, we are adding `density equals true` also. So we will get the density plot out of it. And on top of that, we are marking the `AXLINE`. So you can see that there are two lines here to represent mean and median. It is this part, it is clearly visible. Rest of it, it is not that clearly visible, but two lines are plotted. We see the overlap because mean and median are one and the same in this particular example. This is also called as KDE plot or probability density plot. How about histograms of two different data? So we have `data1` which we have already used, we are going to create another data now. And in the same plot, we are going to use two different histogram function or `hist` function, one for `data1` and another for `data2`. We are not specifying any color, but we are just specifying the label so that legend will show correctly. The color will be taken by default. So `data1` is represented by the blue parts, `data2` is represented by the orange parts. How about heat map or a 2D histogram for the same data? We can plot the data as a 2D histogram, it's 2D X and Y. Specify the number of bins and you can specify a color map also. The color map used here is yellow, orange, red. The darker the color, more data it has, lighter the color, less data it has. Again note the access limits, it is automatically taken based on the data that we have. One more thing, if you right click on this image, you can save the image for later purposes. To finish up, please make sure to run these files yourself by making slight modifications for better understanding of the code and the corresponding plots that were generated. Enjoy the coding experience and thank you for your attentiveness throughout this practical demonstration.

Writing Data to Text Format Video• . Duration: 6 minutes

[Notes]

Welcome, learners, to the topic Writing to Text Format. By the end of this video, you will be able to understand methods for writing data to various text formats, learn to use Pandas for exporting data to text files, explore different options and parameters for data export. When it comes to exporting data, several text file formats are widely used. CSV or comma-separated values is one of the most popular formats using comma to separate data fields. Tab-separated values or TSV is similar but uses tabs as separator. JavaScript Object Notation or JSON is an excellent way for structured data and is widely used in web applications. Plain text files offer a simple, human-readable format for data storage. Each of these formats has its own advantages and is suitable for different use cases in data analysis and storage. Before we dive into writing data, let us set up a sample data frame to work with. The code creates a Pandas data frame with the sample data, including their names, ages and salaries. It then prints the data frame to show its contents. Writing data to CSV file is straightforward with Pandas. The `toCSV` method is used to export a data frame to a CSV file. By setting `index is false`, we prevent the row index from being written to the file, which is often desirable when exporting data. The code exports the data frame to a CSV file named `output.csv`. The `index is false` parameter ensures that the row index is not included in the output file. The code offers various options to customize CSV export. You can specify a custom separator, select specific columns to write and control whether to include the header row. These options allow you to tailor the output to match your specific requirements or match requirements of other systems that will be reading the

CSV file. The code exports selected columns, name and salary to a CSV file using semicolon as the separator and including a header row. TSV files use tabs as separators. Writing to TSV files in Pandas is nearly identical to writing CSV files, with the only difference being the separator specification. The format can be useful when your data contains commas, which would complicate using CSV format. The code exports the data frame to a TSV file by specifying the escape sequence slash T as the separator. The index is excluded from the output. JSON is a popular format for storing structured data, especially in web applications. Pandas uses toJSON method to export data to JSON format. The orient parameter allows you to specify the structure of the JSON output, with records being a common choice for list-like structure. The code exports the data frame to a JSON file using records orientation to create a list of objects. When writing JSON files, you can improve their readability by using the indent parameter. This creates a more human-readable format by adding line breaks and indentation. While this increases the file size slightly, it can be very helpful for debugging or when the files need to be read by humans. This code creates a formatted JSON file with indentation, making it readable for humans. For simple human-readable output, you can write data to plain text files. The toString method in Pandas formats the data frame as a string, which can then be returned to a file. This approach gives you a nicely formatted table in a text file. The code writes the data frame to a plain text file named output.txt, creating a formatted table without the index. When dealing with large datasets, memory management becomes crucial. Pandas allows you to write large datasets in chunks using the chunkSize parameter. This approach helps manage memory usage by writing the data in smaller portions rather than all at once. The code creates a large data frame and writes it to a CSV file in chunks of 10,000 rows, demonstrating how to handle datasets that may not fit in the memory. Sometimes you need to add new data to an existing file rather than overwrite it. Pandas allows you to append data to the existing files using the mode parameter. This is useful for scenarios where you are continuously adding new data to a file over time. In the code, the data frame is appending data to an existing CSV file without writing the header, and it is useful for adding new rows to an existing dataset. When working with data that contains special characters or non-ASCII text, it is important to specify the correct encoding. UTF-8 is a common encoding that supports a wide range of characters from various languages. Proper encoding ensures that special characters are correctly represented in the output file. This code creates a data frame with special characters and writes it to a CSV file using UTF-8 encoding to preserve the special characters. To summarize, Pandas provides versatile methods for writing data to various text formats, including CSV, TSV, JSON, and plain text. When exporting data, it is important to consider the file format, customizing options like separators, column selection, and how to handle large datasets efficiently. Pandas offers options for appending to existing files and managing special characters through proper encoding. Always verify the exported data for accuracy and completeness to ensure data integrity in your exported files. Thank you for your attention. Happy data analyzing!

Data Quality Issues Video • Duration: 6 minutes

[Notes]

Welcome, learners, to the topic Data Quality Issues. By the end of this video, you will be able to understand common data quality issues, learn to identify data quality problems in the dataset, explore techniques for assessing data quality, and recognize the impact of data quality on analysis

and decision making. Data quality issues are problems that affect the accuracy, completeness, and reliability of data. These issues can lead to incorrect analysis and flawed decision making, potentially causing significant problems in various fields such as business research and policymaking. It is crucial to identify and address these issues before proceeding any further on the analysis. By ensuring high quality data, we can increase the reliability of our insights and effectiveness in our decisions. There are several common data quality issues that analysts often encounter. Missing data can create gaps in our analysis. Duplicate records can skew results and waste storage. Inconsistent formats can make data difficult to process uniformly. Inaccurate data can lead to wrong conclusions. Outliers can distort statistical analysis if not properly handled. Data type mismatches can cause processing errors. Inconsistent naming conventions make it challenging to merge or compare data from different sources. Solving these seven issues is the first step in maintaining high quality data. To demonstrate data quality issues, we will create a sample dataset. The code creates a data frame with intentional quality issues such as duplicates, missing values, outliers, and format inconsistencies. The dataset includes ID column, names, age, and email addresses. By examining this data, we can practice identifying and addressing the data quality problems. Identifying missing data is crucial for accurate analysis. The code counts the number of missing values in each column and calculates the percentage of missing data. By using the `EaseNull` method and summing the results, we can quickly see which columns have missing values and to what extent. This information helps us decide how to handle the missing data, whether through imputation, removal, or other methods. Duplicate records can skew data analysis. This code counts the number of duplicates in the dataset and displays all the duplicate rows. The `duplicated` method identifies rows that are complete duplicates of previous rows. By setting `keep` as `false`, we can see all the instances of duplicates, which is useful for deciding how to handle them. Understanding the data types of each column is important for proper data handling. The code displays the data types of all the columns and identifies columns with objects, mainly string data type. Knowing the data types help in choosing the appropriate analysis methods and identifying potential type mismatches that could cause errors in data processing. Outliers can affect statistical analysis. This code uses interquartile range or IQR method to detect outliers in the age column. It calculates the IQR and identifies the lower bound and the upper bound. This method helps in identifying unusual values that may require further investigation or special handling. Inconsistent formats can cause problems in data processing and analysis. The code checks for basic email format by verifying the presence of the `@` symbol in the email addresses. It then identifies entries that do not match this expected format. This simple check can help spot potential data error or format inconsistency that need to be addressed. Another issue is inconsistent naming, which can lead to problems when aggregating or comparing data. The code shows the frequency of each name, including null values, and demonstrates how inconsistent capitalization can affect counts. By converting all names to lowercase before counting, we can see the true frequency of each name, regardless of capitalization differences. To address data quality issues, several steps can be taken. Handling missing values through imputation or removal is crucial. Removing or merging duplicates ensures data integrity. Standardizing formats and naming conventions improve consistency. Assigning outliers appropriately prevents skewed analysis. Implementing data validation rules prevents future issues. Using data quality tools and frameworks can automate many of these processes. Establishing data governance practices ensures ongoing data quality maintenance. To conclude, data quality issues can significantly impact analysis outcomes, leading to flawed decisions. Common issues include missing data, duplicates, inconsistencies, and outliers. To identify these

issues, analysts can apply statistical methods, pattern matching, or use data profiling tools. Addressing these issues involves cleaning data, implementing validation rules, and establishing robust data governance practices. Better data quality assessments can be crucial for maintaining data integrity and ensuring reliable data analysis results. Thank you for your attention. Happy data analyzing.

Lesson 1 Demo Video• . Duration: 7 minutes

[Notes]

Greetings, data quality champions. We will demonstrate all the code snippets we have covered in our lessons on identifying and addressing data quality issues. We will run these examples in Jupyter Notebook and examine their outputs together. Let us dive in. We will see how to read from a text file using the pandas library. So we'll import the pandas library and numpy library as required. First, we will see how to read a CSV file. I'm expecting that you already have a CSV file by the name input.csv in the existing folder or in the current working directory. We will use the read CSV method to access the file, open it up and start reading it. If we have to import with specific header names, we can provide it as part of the input for the read CSV file, specifying the column names, A and B in this case. We are also specifying the separator. So other separators like semicolon can be provided using the separate parameter. Next is how to read from JSON files using the pandas library. We'll rely on read underscore JSON to execute the statement and print the contents of the data frame. In CSV file, if you want to skip rows and comments that are provided in the very beginning, we can use in the read CSV function, the parameter skip rows and comment. The comment can be provided by using other characters also. And you can specify how many rows to be skipped. This will skip row one and row two. To read specific columns, you can provide columns as a list to read only those columns from the given file. This is based on the input.csv file. You may have to change this according to the file you are reading. Let's continue with writing data to the text format. We are creating a sample data using a data frame, which has three columns and several data associated with the three columns. To convert this to a CSV file and write to a CSV file, we will rely on the toCSV method. We will also include index equals false because we don't want the index numbers to be returned to the CSV file. Only the data has to be returned. Let's execute these two statements. It will write to the current working directory by the name output.csv. If you want to export or if you want to write only specific columns, say name and salary in this case, we can do that. And we can also provide whether we want to keep the header or not as provided in the data frame. The separator can be any as you want to specify because it's a CSV file. To write to a TSV file, that is tab-separated files, you have to use the same method toCSV, but the separator will be considered as slash T, which stands for the tab. The index is maintained as false because we do not want to write the index to the file. How to write to a JSON file? Using the toJSON method of the pandas library. So we call it as df, which is our data frame, toJSON and provide the file name. And we will say how to load in the records, right? How the records are to be given one after the other. If you want to provide a specific intent, which is the spacing in the JSON file, you can provide that as intent. This generates beautiful JSON files for easy references. In all the cases, if you want to update existing files, instead of creating a new file or overwriting the existing file, we have to use the mode format. Mode equals A will ensure that we are appending to the existing files that are available in the current working directory. Definitely the header and the index should be maintained

as false so that the contents in the file are pure. We will look at the samples provided in data quality issues lesson. We'll create a sample data frame, which has some data quality issues, like not available, referred to as none or not a number. You can see that there is a none coming in under name of the sample data. Also, there is a invalid email provided in the sample data. Now, how do we process these two? Are we going to remove the entire row or can we do something else? First is to check whether we have some missing data, right? None is considered as a missing data. For that, we will rely on isNull method and isNull.sum will just tell us how much of data is missing. So it says that in ID, there are no missing data. In name, there is one. In age and email, there are no missing data. Converted into percentage, it shows the percentage of how much of missing data is available in your dataset. In the same data, we can check whether the data is duplicated. Let's just look at it. The data gets duplicated. The row index one and row index two are duplicates of each other. So let's look at it and treat it in a way that we want it to be. So we'll check for the number of duplicates using df.duplicated. And when we use the sum, it will show in which one it is getting duplicated. So there are total number of duplicates is zero, right? And the duplicated rows are in these columns. To check the data types of each data in the data frame, we will use df.dtypes. And if you want to specify only the string data, you want to select only the string data, you can use df.selecttypes and specify it as object. String is by default converted as object and give the column names. So as you can see, the column names name and email are returned because only those two are of string data type. Rest are int. If you want to check inconsistent formats, for example, in our data, we have email. Email by default should have the at the rate symbol and it should contain the dot symbol. If we want to check such inconsistency issues, we can write the string command for that and use it to apply it on the data frame. So the statement not df email str contains at the rate, that is considered as an invalid email that is extracted and given as the output. And this shows the count and how many of them are consistent and how many of them are inconsistent. Similarly, we can go for inconsistent naming issues if the names do not have some proper convention that has to be followed, especially if we have a non-data. In that case, we can use that. So here, when we execute this statement, df name value counts drop NAs false, it will count the number of names that exist in the data and say that these data are available. And it shows that non-count is one. Now we are converting into a lower case to check whether it is the same. So this will remove any of the partial matches or partial inconsistencies because of the case is available. So we can rely on the lower function to convert data into the lower case and do the counting. To wrap up, I encourage you to run these files yourself. Enjoy coding and exploring your data. Thank you for your focus throughout this session.

Filtering out Missing Data Video• . Duration: 6 minutes

[Notes]

Welcome, learners, to the topic Filtering Out Missing Data. By the end of this video, you will be able to understand the concept of missing data and its impact, learn techniques to identify missing data in datasets, explore methods for filtering out missing data using Pandas, recognize the implication of removing missing data. Missing data refers to data points that are not present in the datasets. In Pandas, these are typically represented as not a number or none. Missing data can occur due to various reasons such as issues in data collections, non-responses in surveys, or data corruption during storage or transmission. The presence of missing data can impact data analysis and model performance, potentially leading to biased results or reduced statistical power. Understanding and

properly handling missing data is crucial for ensuring the validity and reliability of your analysis. To demonstrate the techniques for handling missing data, we will create a sample dataset with different patterns of missingness. This code creates a Pandas data frame with four columns, each exhibiting different patterns of missing values. The sample data will allow us to explore various methods for identifying and filtering missing data. The code creates a data frame with four columns A, B, C, and D containing different patterns of missing values represented by NAN. This sample data helps demonstrate various missing data scenarios. Before we handle missing data, we need to identify where it exists in our dataset. Pandas provides several methods for this. The `isNull` function creates a Boolean mask showing which values are missing. By summing this mask, we can count the number of missing values in each column. We can also calculate the percentage of missing data per column, which helps us understand the extent of missingness in our dataset. The code demonstrates three ways to identify missing data, creating a Boolean mask using `isNull`, counting the missing values per column using `isNull.sum`, and finally calculating the percentage of missing data in each column. One approach to handle missing data is to remove any row that contains at least one missing value. This can be done using `dropNA` function in Pandas. While this method ensures we are only working with complete data, it can significantly reduce the size of our dataset, potentially leading to loss of important information. The code uses `df.dropNA` to remove all rows containing any missing values. It is simple, but potentially aggressive approach to handling missing data. A less aggressive approach is to only remove rows where all the values are missing. This can be achieved by using `dropNA` function with the argument `howisall`. This method preserves more data compared to the removing rows with any missing value, but it still may retain rows with substantial missing data. The code demonstrates how to do this using the parameter `howall` in the `dropNA` method. It is less aggressive than removing rows with any missing values. Sometimes certain columns in our dataset are more critical than the others. In such cases, we can choose to remove rows based on missing values in specific columns. This approach allows us to preserve more data while ensuring completeness in the most important variables. This code removes rows with missing values only in columns A and B. It is useful when certain columns are more important for our analysis. Instead of removing rows, we might choose to remove entire column containing missing values. This can be done by specifying `axis1` in the `dropNA` function. While this approach can eliminate variables with missing data, it may result in the loss of important variables from our analysis. The code removes all columns with any missing values. It can result in significant data loss if many columns have missing values. When filtering out missing data, it is important to quantify how much data we are losing in the process. This helps us assess the impact of our data cleaning on the overall dataset. By comparing the size of our original dataset to the clean dataset, we can calculate the percentage of data loss. The code demonstrates the same. It helps assess the impact of data cleaning on the dataset size. While filtering out missing data can seem a straightforward solution, it comes with several considerations. Removing data can potentially introduce bias into our analysis, especially if the missingness is not completely at random. It can also lead to a loss of statistical power due to reduced sample size. For small datasets, removing missing data may be not as suitable as it could significantly reduce the amount of data available for the analysis. It is important to consider the mechanism of missingness and balance the trade-off between data quality and quantity. To summarize, filtering out missing data is one approach to handle incomplete dataset, but it requires careful consideration. Landus provides various methods for removing missing data, allowing flexibility in how we approach this task. When filtering data, it is crucial to consider the impact of dataset size and representation as well as the pattern of missingness. In many cases,

alternate approach like imputation might be more appropriate. Please assess the implications of data removal on your analysis to ensure the validity of your results. Thank you for your attention. Happy data analyzing.

Filling in Missing Data Video• . Duration: 6 minutes

[Notes]

Welcome learners to the topic, filling in missing data. By the end of this video, you will be able to understand the concept of data imputation. Learn various techniques for filling in missing data, explore Pandas methods for data imputation, and recognize the implications of different imputation strategies. Data imputation is the process of replacing missing data with substituted values. This technique aims to preserve data integrity and maintain a sample size, which is crucial for many statistical analysis. There are various methods for imputation, each based on different data characteristics and analysis goals. Imputation is an important step in data preprocessing, as it allows us to work with complete datasets and potentially improve the accuracy of our analysis. To demonstrate various imputation techniques, we will create a sample dataset with missing values. The code creates a Pandas data frame with both numeric and categorical data, including intentionally placed NaN values to represent missing data. This sample allows us to explore different imputation strategies and their effects. One of the simplest imputation method is fill all missing values with a constant. This approach replaces all NaN values with a specified value such as 0. While this method is straightforward to implement, it may not preserve the relationship between the variables in the data. It is important to consider the nature of your data before using this method, as it can introduce bias if the constant does not accurately represent the missing values. This code uses the fillNa method to replace all NaN values with 0. It is a simple approach, but may not be suitable for all types of data. A more nuanced approach is to fill missing values with the mean of each column. This method only applies to numeric columns and can help maintain the overall distribution of the data. By using the mean, we ensure that the imputed values are representative of the central tendency of each variable. However, this method can be sensitive to outliers and may not be suitable for skewed distributions. This code fills NaN values in each column with the mean of that column. It is only applicable for numeric columns and maintains the column's average. Similar to using mean, we can fill missing values using the median of each column. The advantage of using median is that it is less affected by outliers compared to the mean. This can be useful when dealing with skewed data or when there are extreme values in your dataset that might disproportionately influence the mean. The code fills NaN values with the median of each column. It is less affected by the outliers compared to using the mean. Sometimes, different columns in your dataset may require different imputation strategies. This approach allows you to apply tailored imputation methods based on the characteristics of each column. For example, you might use mean imputation for a numeric column and a constant value for a categorical column. The code demonstrates filling specific columns with different strategies, mean for numeric column A and a constant unknown for categorical column D. Interpolation is a method that estimates missing values based on the surrounding data. This technique is useful for data with a logical ordering, such as time series data. Interpolation can provide more accurate estimates than simple filling methods, especially when there is a trend or a pattern in the data. The code uses the interpolate method to estimate missing values based on the surrounding data points. It is useful for data with a logical ordering. For more

granular control over data imputation, you can use a dictionary to specify different fill values for each column. This method allows you to apply different imputation strategies to different columns in a single operation, providing flexibility in how you handle missing data across the dataset. The code uses a dictionary to specify different values for each column, allowing for more precise control over imputation. For more advanced imputation techniques, we can use Skykit Learn's Simple Imputer. This method can be integrated into machine learning pipelines and offers various strategies for imputation. Using Skykit Learn can provide more sophisticated imputation methods and ensure consistency between your data preprocessing and modeling steps. The code uses SKLearn's Simple Imputer for mean imputation. It can be integrated into machine learning pipelines. When performing imputation, it is crucial to consider its impact on data distribution and relationship. Imputation can potentially introduce bias, so it is important to choose a method suitable for your data type and patterns. Always document your imputation strategy for transparency. For statistical analysis, consider using multiple imputations to account for uncertainty. Finally, evaluate the impact of your imputation on your downstream analysis to ensure it does not lead to invalid conclusions. To wrap up, data imputation is a crucial technique for handling missing values in datasets, helping to preserve sample size and maintain data integrity. Pandas offers imputation methods, each with its own strengths and limitations. The choice of the imputation method should be based on the data characteristics, the missing data patterns, and the analysis objectives. It is essential to consider the implications of imputation on subsequent analysis and to document the chosen strategy for transparency and reproducibility. Thank you for your attention. Happy data analyzing!

Lesson 2 Demo Video• . Duration: 4 minutes

[Notes]

Welcome Missing Data Detectives. We are about to bring to life the code examples from our lesson on handling missing data. Using Jupyter Notebook, we will execute these snippets and analyze their results. Shall we begin? We will check how to find missing data and then remove the missing data from them. Let's first import the files that are necessary. There is a new library that we are going to rely on, which is called sklearn. It stands for Skykit Learn. And in that, we have a module called Impute, which helps us to impute missing data or impute new values to fill the missing data. We will be relying on a simple imputer in the Impute module to do the same. Let us set up a sample data. Now, the sample data has stored a lot of NPNANs. They're not a number. The column A has one, column B has two, and columns D has around four NAN values. Let's just print it out for our reference. To identify whether we have missing data or not, we will rely on the isNull method. It will generate a mask for each of the row. So wherever there is NAN, it will return true for us. The count will return how many of them are missing and whether we have any missing data or not. So we can see that in the column C, there are no missing data, whereas in all the other columns, there are some missing data. The last line, `df.isNull.sum` divided by length of `df` multiplied by 100, is going to compute the percentage of the missing data per column. So the column D has got around 80% missing data. How to remove? There are multiple ways. The first one and the simplest one is to remove as it is. So let's use `dropNA` and remove the row. So now only the fourth row will remain as it is because that is the only data that is existing. Now if you want to find out whether you have missing data in specific columns, for the `dropNA` method, you can use the `subset` function and provide the column names into the `subset`. So this will generate only those data which are missing in

column A and B. If you are removing rows using dropNA, we can see that there is an 80% loss, which is not ideal in most of the data analysis cases. We will try to retain as much as possible. That means we need to look at how to fill in missing data. Now in this section, we will look at how to fill in missing data where there are a lot of npNans. So we'll create the sample data as previously. Now let us fill the NaN with a constant, in this case 0. So wherever we had NaN, we are having 0 replacing it. Or a better way is to fill it with mean. Now remember mean gets affected by outliers, so some cases it is ideal, some cases it is not ideal. Instead of mean, if we think that we have a lot of outliers and we want to modify it with median, you can use that. So for all the three methods, the command that we used was df.fillna in the brackets, df.mean or median or a constant. If you want to fill only for specific columns, we can use that in the square brackets. So we are going to create a copy of the, it is called as df.filled, and take the column A out of it and fill only in the column A. On running this, you will see that only column A NaNs are getting filled by the mean. If instead of NaN, you may prefer it to have unknown or some other data where you have a third or a fifth category to do so. So in that case, you can use a dictionary to fill values. So wherever A is present, you will get it as 0, wherever B is present, you will get it as 1, like that. In case we do not have any value, you will be filling it with unknown. How about using the sklearn library? We will be using the simple imputer function and specify the strategy as mean. And we will have to train the imputer with the data that we are providing. And what is the data? We will specify the columns, whichever columns you want. You can provide it as a list also instead of this. It will train the imputer based on the data that we are providing and it will impute wherever missing data is available. For each one, it will take in a different way. The strategy can be mean or median or some other interpolation technique. Enjoy filling in the gaps in your data and may your analysis always be complete. Thank you for your focused participation in this demonstration.

Removing Duplicates Video• . Duration: 5 minutes

[Notes]

Welcome learners to the topic, Removing Duplicates. By the end of this video, you will be able to understand the concept of duplicate data and its impact, learn techniques to identify duplicate records in datasets, explore methods for removing duplicates using pandas, and recognize the implications of duplicate removal on data analysis. Duplicate records are identical or very similar entries in a dataset. They can occur due to various reasons such as data entry errors, system glitches, or when merging data from different sources. The presence of duplicates can significantly impact data quality, reduce the accuracy of our analysis, and waste storage space. It is crucial to identify and handle these duplicates appropriately to ensure the integrity of our data. Let us start by setting up a sample dataset to work with. We will use pandas to create a data frame that contains some duplicate records. This data frame will include both exact duplicates and partial duplicates, which will help us demonstrate various duplicate handling techniques. The code we are using creates a data frame with columns for id, name, age, and city, intentionally including some duplicate entries. Now that we have our sample data, let's look at how to identify duplicate records. Pandas provides a handy method called duplicated, which returns a Boolean mask indicating which rows are duplicates. We can also use this method to count the total number of duplicates in our dataset. The code shows us which rows are duplicates and which gives us a count of how many duplicates we have in total. Before we start removing duplicates, it's often useful to inspect them. We can use

the Boolean mask we created earlier to display all the duplicate rows, including the first occurrence. This step is crucial as it allows us to understand the nature of the duplicates and make informed decisions about how to handle them. Once we have identified and inspected our duplicates, let us remove the exact duplicates. Pandas offers a straightforward method called `drop_duplicates` for this purpose. By default, this method keeps the first occurrence of each duplicate and removes the subsequent ones. This approach is often suitable when we are dealing with exact duplicates caused by data entry errors or system glitches. Sometimes we might want to consider only certain columns when determining what constitutes a duplicate. This is useful for handling partial duplicates. For example, we might consider two records duplicates if they have the same name and age even if the other fields differ. Pandas allows us to specify which columns to consider using the `subset` parameter in the `drop_duplicates` method. After removing duplicates, we often want to reset the index of our data frame to ensure we are having consecutive indexing. This step can make our data easier to work with and understand. We can combine the `drop_duplicates` method with the `reset_index` method to achieve this in one step. In some cases, we might need to deal with near-duplicate records that are very similar but not exactly the same. For instance, we might want to treat names as duplicates regardless of their case. We can achieve this by applying transformations to our data before removing duplicates. When removing duplicates, it's important to consider the broader implications. Duplicate removal can significantly impact your data set's size and statistical properties. You might lose important information such as frequency data. It is crucial to understand where the duplicates come from and whether they represent actual errors or meaningful data. Consider how you will handle partial and near duplicates. Always document your duplicate removal process for transparency and reproducibility. In summary, we have explored the crucial process of duplicate removing in data cleaning. We have learned to identify, inspect, and remove duplicates using Pandas. Considering various scenarios like exact, partial, and near duplicates, remember, understanding your data and the impact of duplicate removal is key to maintain data integrity and ensure accurate analysis. Thank you for your attention. Happy data analyzing.

Transforming Data Using Mapping Video• . Duration: 5 minutes

[Notes]

Welcome, learners, to the topic Transforming Data Using Mapping. By the end of this video, you will be able to understand the concept of data mapping in Pandas, learn different methods to apply mapping transformations, explore various use cases for data mapping, and recognize the importance of mapping in data preprocessing. Data mapping is a crucial process in data analysis that involves transforming data from one format or structure to another. It is essentially about applying a function or rule to each element in a series or data frame. This technique is widely used for data cleaning, normalization, and feature engineering. Data mapping is an essential step in preparing your data for analysis or machine learning tasks, as it helps ensure your data is in the right format and structure for your specific needs. To demonstrate data mapping techniques, let us start by setting up a sample dataset. We will create a data frame that includes both categorical and numerical data, which will allow us to showcase various mapping methods. The code creates a data frame with columns for ID, product, category, and price, giving us a good mix of data types to work with. One of the simplest ways to perform data mapping is by using a dictionary. This method is useful when you want to replace specific values with others. In the code, we are creating a new

column named product name by mapping the values in the product column to more descriptive names. This technique is often used for labeling or categorizing data in a more meaningful way. Sometimes, we need more complex logic for our mapping. In such cases, we can use a custom function. The code shows how to create a new categorical column called price category based on the values in the price column. This function demonstrates how we can use conditional logic to create new features from existing data, which is a common task in feature engineering. The apply method in pandas is a powerful tool for performing row-wise operations. It allows us to apply a function to each row of a data frame, which is particularly useful when we need to consider multiple columns in our transformation. The code calculates a discounted price based on the product category, showcasing how we can use complex logic in our transformations. The series.map method is an efficient way to replace values in a series. It is useful for encoding categorical variables, which is a common preprocessing step. The code creates a new category code column by mapping each category to a single-letter code, which could be useful for algorithms that require numerical input. Lambda functions provide a way to define simple functions in line without the need of a formal function definition. They are useful for quick one-off transformations. In the code to double the price for each product using a lambda function. This demonstrates how we can perform simple mathematical operations across a series. Sometimes we need to apply different transformation to multiple columns simultaneously. Pandas provides transform method, which allows us to do this efficiently. In this code, we are applying different functions to each column, lowercase for products, uppercase for categories, and a custom formatting for prices. This tells us how we can handle complex multi-column transformations in a single operation. Data mapping isn't just for feature creation. It is a tool for data cleaning. The replace method in Pandas is useful for correcting misspellings or standardizing values. The code corrects potential misspellings in the category column, demonstrate how mapping can be useful to clean and standardize data. Mapping techniques are versatile and can be applied in various scenarios. They are useful for value replacement, standardization, ensuring consistency across your datasets. Mapping is also a key tool in feature engineering, allowing you to create new features based on existing data. It is often used for data type conversions, transforming data into the format required for specific analysis or algorithms. Finally, mapping is crucial for categorical encoding, converting categorical variables into a format that can be used by machine learning algorithms. To recap, data mapping is a fundamental technique in data preprocessing, offering various methods like map, apply, transform, and replace for data transformations. It is essential for tasks ranging from simple value replacement to complex feature engineering. When mapping, consider efficiency, data types, and handling of missing values. Please document your mapping processes for transparency and reproducibility in your data analysis workflow. Thank you for your attention. Happy data analyzing.

Replacing Values Video• . Duration: 5 minutes

[Notes]

Welcome, learners, to the topic Replacing Values. By the end of this video, you will be able to understand the importance of value replacement in data cleaning. Learn various methods to replace values in Pandas. Explore different scenarios where value replacement is necessary. Recognize the implications of value replacement on data analysis. Replacing values in your dataset is a crucial step in data cleaning and preparation process. It allows you to correct errors and inconsistencies

that may have crept into your data during the collection or processing. By replacing values, you can standardize data representation across your dataset, ensuring that all information is in a consistent format. This process is also essential for handling missing or invalid values, which can skew your analysis if left untreated. Ultimately, value replacement helps prepare your data for analysis or machine learning tasks by improving overall data quality and consistency. To demonstrate various value replacement techniques, let us start by setting up a sample dataset. We will create a data frame that includes different data types, missing values, and a sentinel value, say, negative 999, commonly used to represent missing data in some systems. This code creates a data frame with four columns, each representing different scenarios which might encounter in real-world data. One of the most basic replacement operations is replacing a single specific value. In this example, we are replacing the sentinel value, negative 999, with NAN or not a number. This is a common operation when working with datasets where missing values are represented by a specific number. The code uses the `replace` method to perform this operation on column C. Handling missing values is a critical part of data cleaning. The code is demonstrating how to replace NAN values itself, in this case, with `unknown`. This operation is performed on column D using `fillna` method, which is specifically designed for handling NAN values in pandas. Sometimes, we need to replace multiple values at once. In this example, we are replacing the lowercase letters A, B, and C with their uppercase counterparts in column B. The code shows how we can perform multiple replacements in a single operation, which can be more efficient than doing them one by one. When we have many values to be replaced, using a dictionary can be an efficient approach. The code is using a dictionary to map X, Y, and Z to high, medium, and low, respectively, in column D. This method is useful when you have a clear mapping between old and new values. Other expressions provide another tool for pattern-based replacements. In this example, we are using a regex pattern to replace any single lowercase letter from A to C with the word `lower` in column B. The code demonstrates how we can use regex for more complex string manipulations and replacements. Sometimes, we need to replace values based on a condition. In the code, we are replacing the values in column A with 999 if they are greater than 3. This showcases how we can use Boolean indexing in pandas to perform conditional replacements. We can also replace values with the results of calculations. The code doubles the values 40 and 50 in column C. This demonstrates how we can perform data transformations as part of replacement process. By default, many pandas methods return a new object. However, we can often modify the original object directly using the `inPlace` parameter. The code is replacing lowercase d with the uppercase d in the column B, modifying the original data frame instead of creating a new one. In more complex scenarios, we might want to replace values in one column based on conditions in another column. In this code snippet, we are replacing the values in column B with the corresponding values from column D, but only when column A equals 999. This demonstrates how we can perform inter-column operations using replacement. To conclude, value replacement is a critical step in data cleaning and preparation. Pandas offers various methods for replacing values, including regular expressions and conditional replacements. When performing replacements, consider their impact on data integrity. Always document your replacement operations for transparency and verify the results after making changes to ensure accuracy. Thank you for your attention. Happy data analyzing.

Discretisation and Binning Video• . Duration: 5 minutes

[Notes]

Welcome learners to the topic discretization and binning. By the end of this video, you will be able to understand the concepts of discretization and binning. Learn various methods to discretize continuous data using pandas. Recognize the impact of discretization on data analysis.

Discretization is the process of converting continuous variables into discrete categories. This technique is also known as binning or grouping. The main purpose of discretization is to group continuous data into discrete intervals or bins. There are two primary types of discretization, equal width binning and equal frequency binning. These methods allow us to transform continuous data into more manageable and interpretable categories. Discretization helps reduce noise in the data by grouping similar values together. This technique is also effective in handling outliers, as extreme values can be grouped into broader categories. Additionally, discretization simplifies data representation, making it easier to understand and analyze complex datasets. The applications of discretization include data analysis, machine learning, and data visualization. By converting continuous data into discrete categories, we can often uncover patterns and trends that might not be apparent in the original continuous data.

Equal width binning is a discretization method that divides the range of data into bins of equal size. The code snippet demonstrates equal width binning using `cut` function of pandas and specifying the number of bins. This creates age groups and displays the count of records in each bin. The process of equal width binning involves three main steps. First, we determine the range of data by calculating the difference between the maximum and minimum values. Next, we choose the number of bins we want to create. Finally, we calculate the bin width by dividing the range by the number of bins. Equal width binning is straightforward and preserves the relative scale of the original data, making it easy to interpret. However, it can be sensitive to outliers and may result in an uneven distribution of data points across bins, especially if the data is not uniformly distributed.

Equal frequency binning, also known as quantile binning, divides the data into bins with approximately equal number of data points. The code snippet demonstrates equal frequency binning using `qcut` function and specifying the number of quantiles. The process of equal frequency binning starts by sorting the data in the ascending order. Then, we choose the number of bins we want to create. Finally, we divide the sorted data into equal sized groups. Equal frequency binning handles outliers well and ensures a more balanced distribution of data points across bins. However, it may group dissimilar values together and can sometimes obscure the underlying distribution of the data.

When choosing between equal width and equal frequency binning, consider the nature of your data and your analysis goals. Equal width binning is often suitable for uniformly distributed data when you want to preserve the scale of your data and when there are few outliers. On the other hand, equal frequency binning may be more appropriate for skewed distributions when you want to balance bin population or when you are dealing with many outliers. Custom binning allows us to define our own bin edges and labels, which is useful for domain-specific categorization. The code creates age categories with custom bin edges and labels. The resulting age category column will contain labels like young, adult, middle-aged, senior, and elderly based on the custom bins we defined. We can assign custom labels to bins, which is useful for creating meaningful categories. In the code example, we are converting the score column into letter grades. We define the bin edges to correspond the common grade cut-offs 60, 70, 80, and 90 and assign labels F, D, C, B, and A to each bin respectively. Discretization can be useful for data visualization. The code creates a bar plot using matplotlib with the x-axis representing the age groups and y-axis showing the count of individuals in each group. This type of visualization can quickly reveal patterns in age distribution of our dataset, which might be less apparent in continuous data. To summarize, discretization is a powerful technique for converting continuous data into categorical bins, offering

benefits like noise reduction and simplified data representation. Pandas uses cut and queue-cut methods for binning. When possible, use domain knowledge to guide your binning decisions for more meaningful results. Thank you for your attention. Happy data analyzing.

Encoding Categorical Data Video• . Duration: 4 minutes

[Notes]

Welcome, learners, to the topic, Encoding Categorical Data. By the end of this video, you will be able to understand the need for encoding categorical variables, learn various methods of categorical to binary encoding, explore different encoding techniques using Pandas and Skykit Learn, recognize the impact of encoding choices on data analysis and modeling. Encoding categorical variables is a crucial step in data preprocessing for machine learning. Many algorithms require numerical input, yet categorical data often contains valuable information that we cannot or we do not want to discard. By encoding, we can preserve this categorical information in a numerical format that algorithms can work with. It is important to note that different encoding methods are suited to different types of categorical variables. A wrong choice of encoding method can significantly impact the model's performance. To demonstrate various encoding techniques, we will start by creating a sample dataset. This code generates a data frame with three categorical variables, color, size, and category. These variables will allow us to showcase different encoding methods and their applications. One-hot encoding is a popular method for handling nominal categorical data. It creates binary columns for each category in the variable. The code uses `getDummies()` function of Pandas to perform one-hot encoding on the color column. The resulting encoded columns are then concatenated with the original data frame. This method is useful when there is no inherent order in the categories. Often we need to encode multiple categorical columns at once. Pandas makes this process straightforward with the same `getDummies()` function. By specifying the column names in a list as `columns` parameter, we can encode all of them in a single operation. This code demonstrates how to use one-hot encoding for both color and size columns simultaneously and automatically generating appropriate column names for each category. Label encoding is another common technique useful for ordinal data. It assigns a unique integer to each category. The code uses Skykit Learn's label encoder to encode the category column. It assigns 1, 2, and 3 for the categories A, B, and C. Each unique category in this column will be assigned a unique integer. While simple and memory efficient, it is important to note that label encoding can imply an ordinal relationship that may not exist in the original data. Ordinal encoding is specifically designed for categorical variables with clear order. The code encodes sized columns using a predefined order dictionary. This method preserves the inherent order in the categories. Small less than medium, less than large will be assigned the numbers 1, 2, 3 respectively. Use this method only when there is a genuine ordinal relationship among the categories. Category encoding replaces categories with their frequency of occurrence in the dataset. This method is useful for high cardinality category variables. The code calculates the frequency of each color and maps these frequencies back to the color column. This encoding can provide valuable information about the distribution of the categorical variables. One-hot encoding is excellent for nominal data as it does not assume any ordinal relationship between categories. However, it can lead to high dimensionalities with many unique categories as the number of columns increases. Label encoding is simple and memory efficient, but implies an ordinal relationship that may not exist. Ordinal encoding preserves the order of the categories and is

memory efficient, but it requires a known ordering of the categories. When choosing an encoding method, consider the nature of your categorical variables. Use one-hot encoding for nominal categories where there is no inherent order. Apply label or ordinal encoding for ordinal categories where the order matters. Consider frequency encoding for high cardinality variables to capture the relative importance of each category based on its frequency. To summarize, categorical to binary encoding is essential for many machine learning algorithms. Pandas and Skykit Learn offer various encoding methods. The choice of encoding can significantly impact model performance, so consider your data characteristics and model requirements carefully. Always validate your encoding approach using cross-validation to ensure it enhances the model's performance. Thank you for your attention. Happy data encoding!

Detecting Outliers Video• . Duration: 5 minutes

[Notes]

Welcome, learners, to the topic, Detecting Outliers. By the end of this video, you will be able to understand the concept of outliers and their impact on data analysis, learn various statistical methods for detecting outliers, explore visualization techniques for identifying outliers, recognize the importance of domain knowledge in outlier detection. Outliers are data points that significantly differ from the observations in dataset. These unusual values can arise from various sources, including measurement errors, data entry mistakes, or genuine anomalies in the data. It is crucial to understand that outliers have a substantial impact on the statistical analysis and the model performance. Depending on the context and nature of your data, outliers may need to be handled differently. In some cases, they might represent valuable insights, while in others, they could be errors that need correction. Identifying and appropriately handling outliers is an essential step in ensuring the quality and reliability of your data analysis. To demonstrate outlier detection techniques, we will start by creating a sample dataset. This code generates a data frame with 1,000 normally distributed values and introduces five outlier points. By using a seed for the random number generator, we ensure reproducibility of our results. The describe function provides a quick statistical summary of our data, giving us an initial overview of its distribution. Box plots are an excellent tool for visualizing the distribution of data and identifying potential outliers. In the visualization, the box represents the interquartile range, or IQR, with the median shown as a line inside the box. The whiskers extend to 1.5 times the IQR, and any points beyond the whiskers are plotted individually, representing potential outliers. A histogram is another useful visualization tool for detecting outliers. It shows the distribution of values in your dataset, with the height of each bar representing the frequency of values falling within that range. The outliers often appear as isolated bars far from the main distribution. In this example, we are including a Kernel Density Estimate, KDE plot, which provides a smooth estimate of the probability density function of the data, further highlighting any unusual patterns or outliers. The Z-score method is a statistical approach to identify outliers. It calculates how many standard deviations each data point is from the mean. Typically, points with an absolute Z-score greater than 3 are considered potential outliers. Unless they fall more than 3 standard deviations away from the mean. This code calculates Z-score for each data point and identifies outliers based on this criterion. Interquartile range, or IQR method, is another common statistical approach for detecting outliers. It uses the concept of quartiles to define boundaries on what constitutes an outlier. This method is less affected by extreme outliers

compared to the Z-score method, making it more robust in certain situations. The code calculates the IQR, defines the lower and the upper bounds, considering any points outside these bounds as potential outliers. After detecting outliers using statistical methods, it is often helpful to visualize them in the context of the entire dataset. The scatter plot shows all data points with detected outliers highlighted in red. This visualization helps us understand the position of outliers relative to the rest of the data, providing insights in their potential impact and whether they truly stand out from the overall distribution. When detecting outliers, it is crucial to consider several factors. First, think about how outliers may impact your specific analysis or model. It is important to distinguish between true outliers and measurement errors, as they may require different handling approaches. Domain knowledge plays a vital role in interpreting outliers. What seem unusual statistically might be perfectly normal in the context of your field. Detection methods can be sensitive to extreme values in different ways, so use multiple methods that provide a more robust analysis. Remember that outliers can sometimes provide valuable insights, so be careful considering it is necessary before deciding to remove them from your dataset. Outlier detection is crucial for maintaining data quality and ensuring accurate analysis. Various methods, including statistical approaches like Z-score and IQR, as well as visualization techniques such as box plots and histograms, can be used to identify outliers. It is important to consider multiple methods for robust detection and always interpret outliers within the context of your domain. Remember, detecting outliers is often just the first step, followed by appropriate handling of further investigation. Thank you for your attention. Happy data analyzing!

Filtering Outliers Video• . Duration: 5 minutes

[Notes]

Welcome, learners, to the topic Filtering Outliers. By the end of this video, you will be able to understand different approaches for handling outliers, learn techniques for filtering outliers using Pandas, explore methods for treating outliers without removing them, and recognize the impact of outlier filtering on data analysis and modeling. Filtering outliers is an important step in data preprocessing and can improve the accuracy of statistical analysis. By removing the extreme values, you can enhance model performance and generalization, reducing the impact of data errors or anomalies. This process is crucial when preparing data for algorithms that are sensitive to outliers. However, it is important to approach outlier filtering with caution, as indiscriminate removal of data points can lead to the loss of important information. The decision to filter outliers should always be made with the consideration of the specific context and goals of your analysis. To demonstrate outlier filtering techniques, we will start by creating a sample dataset. This code generates a data frame with 1,000 normally distributed values and introduces 10 outlier points. The describe function provides a quick statistical survey of our data, giving us an initial overview of the distribution and the presence of outliers. Before applying any filtering techniques, it is better to visualize our data to understand its distribution and the presence of outliers. A box plot is an effective tool for this purpose. It displays the median and the quartiles and the potential outliers in a single visualization. This graph will serve as a baseline for comparison after we apply our filtering methods, allowing us to see the impact of the outlier removal techniques. The graph shows several outliers beyond the whiskers of the plot. The z-score method is a common approach for identifying and filtering outliers. It calculates how many standard deviations each data point is from the mean.

Typically, points with an absolute z-score greater than 3 are considered outliers and are removed. This code calculates the z-score for each data point, removes those with absolute z-scores greater than 3, and prints the number of remaining data points. The interquartile range method is another popular technique for outlier filtering. It uses the concepts of quartiles to define boundaries of what constitutes an outlier. This method is less affected by the extreme outliers compared to the z-score method, making it more robust in certain situations. The code calculates the IQR, defines lower and upper bounds as 1.5 times IQR, below the first quartile and above the third quartile, respectively, and removes any points outside these bounds. After applying our filtering methods, it is important to visualize the results to understand the impact of our actions. This plot shows the impact of applying z-score method. It removes most of the outliers. The plot displays the effect of performing filtering outliers using IQR method. It is more stricter than the z-score method and it is not affected by the extreme values in the data. When filtering outliers, several important factors need to be considered. Consider the impact of your sample size and statistical power. Removing too many data points can weaken your analysis. There is also a risk of losing important information if outliers represent genuine but rare phenomena in your data. The appropriateness of filtering should be evaluated in the context of your analysis goals. Domain expertise is crucial in interpreting whether outliers are errors or valuable extreme cases. It is important to document all the decisions made in handling outliers for transparency and reproducibility. Lastly, consider investigating the root cause of outliers as this can provide valuable insights into your data generation process or reveal important patterns. To recap, outlier filtering is a critical step in data pre-processing, with methods like z-score and IQR available for implementation. It is important to consider the impact of filtering on your analysis and use visualization to understand its effects. Always interpret results in the context of your domain and document your outlier handling approach for transparency. Thank you for your attention. Happy data analyzing!

Lesson 3 Demo Video• . Duration: 16 minutes

[Notes]

Hello, data transformers. It is time to put your knowledge on data transformation techniques into practice. We will run through all the code samples from our lesson using Jupyter Notebook and preview the transformed outputs. Let us get started. Let's look at how to remove duplicates from our data. We will rely on pandas, NumPy, matplotlib, and sklearn, which stands for Skykit Learn. In sklearn, we have a module called preprocessing, which in turn has a function called label encoder, which we will be relying on for doing the encoding of categorical data. Now we will create a sample data where there are some duplicates provided. Our aim is to remove duplicates. Note that the duplicates are coming in index 1 and 2. Duplicates are available in 4 and 5. The duplicates in index 4 and 5 are called as partial duplicates because the case only when it is case insensitive, it becomes duplicate. Otherwise, it's a totally different data. How to identify duplicate records? We use `df.duplicated`. This will list out all the duplicates and `df.duplicated.sum` will return the total count. So by default, it is case sensitive. So you will see that it is 4 and 5 are written as false, saying that it is not duplicated. So the number of duplicates is 1. How to view the duplicated records alone? We use `df.duplicated.keep equals false`. So it will list out all the duplicates that we have in the data. One simplest way to remove duplicates is to drop it out completely. So `drop duplicates` is the method that we will rely on. So as you can see, the data, the index 1 is retained. Index 2 data is completely

removed. So index 2 is missing. If you want to remove duplicates based on specific columns alone, that can be done by specifying the column names as part of subset in drop duplicates. So it will check only the name and age for duplicating. And if there are duplicates present, that alone will be removed. Now when you remove duplicates, as you can see, the index is retained as such. So we will have to reset the index by providing drop duplicates.resetindex and drop equals true. Drop the duplicates, then reset the index from the very beginning. So 2 is maintained or there is a reshuffling done and the indexing is done properly. Now what about this data? How about 3 and 4? So here we have to have it as a partial duplicate or a near duplicate. So what we will do is we will convert everything into lower string, especially the name. And if it is duplicated, we will remove it. So `df.name.str.lower` will convert the name into lowercase. Then based on the lowercase, we will check whether there is any duplicates and remove if there are any. As you can see, this works well, right? It is converted to lowercase and the duplicated value is removed. You can see that gap in the index based on the original. How much of data is lost because of duplication? Let's do a computation, original size minus clean size, and then convert it into a percentage. So around 15% is lost because of removal of duplication. The next segment is on transforming data using various mapping. So we'll create a data frame with id, product, category, and price. Id is unique, product is discrete data, which has three categories, A, B, and C. Category is again discrete, which has three categories, electronics, clothing, and books. And price is the price of each of the product that we have. Let's create the data. Now we want to add simple mapping to say that wherever it is A, it should be a laptop, B should be marked as t-shirt, and C should be marked as novel. So we will use the map function to do it. So product map is a dictionary. We will use the dictionary in map function to rename the map. The product and product name, if you compare, it is doing it correctly. Now another way to do it is to use mapping with a numerical value. So this is one way of discretization. If the price is less than 50, we are going to say it as low. If the price is less than 100, we are going to say it is as medium. And if it is more than 100, we are going to make it as high. And we want to create a new column by the name price category and map, use this kind of rules to give values for price category. So the same map function is used. We will pass on this function, price category, and ask it to convert. So as you can see, a different column is added based on the price the category is assigned correctly. Another way to do this is by using apply function. We'll create a discount column where we are going to provide discount to those which are electronics. The rest will not have a 10% risk. So the rule is, if the category is electronics, the price is computed, otherwise it will remain the same. We'll use the apply function and it is on the axis equals one that is on each row. So if you look at price, the discounted price, if you compare, you can see that the values are changing. Another way to do such mapping is by using a series method. We'll create a series and then use that category code to map it. So again, map function is used to map the category code to each category, but the differences are input is in the series format. Instead of regular functions, we can use lambda functions also using the same map function. So here the price is doubled based on the original price. If you want to do multiple transformations, a convert into lower, upper, or apply some lambda function, that is also possible, provided it is given as a dictionary. This part is the dictionary and you specify onto which column you want to apply what function. And then you use the `df.transform` method to apply these transformation on our data frame. Finally, we can clean up. If there are any spelling errors and all, we can replace. You can use the replace method to clean up the spelling errors that can happen in the data. `df.replace` will help us. The data will have to be given as a dictionary. How about discretization and binning? Let's create a sample data where we have a lot of numerical data with us. The age, salary, and score. Now we will discretize. We will say that we will

bin them and put them into different categories based on their age or salary or score or all of them. The first type of binning that we are looking at is equal width, where the width of each bin is made as equal. Now for this, we will rely on the pandas library, cut method in the library is used. We have to specify how many bins. So based on the age, we are going to have five groups. So five bins. So you can see that it will return the bin interval and how many, what is the frequency of data in each bin? So the first bin starts with 17.9 and ends with 29.8. The next bin starts with 29.8 and ends with 41.6, one value less than that, right? And since we said five bins, you have five bins with the frequency mentioned. It also adds a new column where it will specify to which bin it belongs to. So this one belongs to the bin 53.4 to 65.2. If you're going for equal frequency binning for the same five bins, we will have to rely on the Q cut method of pandas library. It will create the bin interval in such a way that the frequency in each bin is the same. So everywhere you will see that it's 20. So each bin will have only 20 data points. It's not the same here. It has various numbers of data points. It will specify the lower and the upper bound and for each data frame that is also getting added. How about custom bins? You do not want to rely on the bins. You do not want to give the number of bins, but you want to specify the bin width. Like say for example, for the age, this is a commonly used technique. So all these people who are between the age zero and 25 should be marked as young, between 25 and 35 should be marked as adult, 35 and 50 should be marked as middle aged, 50 to 65 should be marked as senior and 65 to 100 should be marked as elderly. So the width changes, the upper and lower limit also are changing. So if this is required, then we have to create two lists, age bins and age labels, then use pd cut to specify on which column we are binning, what is the limits and what are the labels. On executing it, you can see that each of the age gets appropriately categorized based on the bin labels that we have. The same can be done for the score data using pd cut, where you specify the bins and the labels associated with it. Reviewing the result shows that the score grade is changing according to the bins that we have provided. Now let's visualize the effect of discretization that we have done on age. So you can see that the bin width is taken and for each bin width, the frequency is given as a bar plot. How about encoding categorical data? So we will create a data frame which has a lot of categorical data in it. So in this data frame, we have color, size, and category. Color has red, blue, and green in various combinations. Size has small, medium, large, and category has A, B, and C. Five different rows. Now one way of doing encoding is using one-hot encoding. For that, we will use pd.getdummies. That is, we will rely on the pandas library. We have to specify on which column we are going to create the dummy values. And we will specify what is the prefix that has to be given in the column name. Let's execute this. Then you will understand better. Color, size, category is the original data. Now after dummies, you will see that there are three different columns that is getting added with the prefix that is mentioned. Color blue, color green, and color red. Color blue will have one if in the original data it has blue in it. Color should, in dot, what we are doing with getdummies is create three columns which is going to mark the presence and absence of that particular discrete value. This is effective for doing decision making or making a decision tree which is going to rely on a lot of discrete data. We can bin on multiple columns in one go. So you have to specify the columns as list. In this case, color and size. Same method, getdummies is used. So in effect, you will get to see that there are three columns for color and three columns for the size. Another way to do it is by using label encoder. This will use the labels or numbers to convert the category to, by default, it goes alphabetically. If you want ordinal encoding, then you have to use the map method. So we do not have any specific method for ordinal encoding. We have to give our own rules where small will take a value of one, medium will take a value of two and large will take a value of three and then we use the map function to apply on the

data frame. If you're going to use frequency encoding, again, we will rely on the map function. First, you have to create the frequency by using the value counts and then normalize it so that it will scale up to 100 or we will scale up to zero to one. Then use the map function with that frequency encoding on the data. How about outliers and how to detect them? We will rely on matplotlib, library, seaborn and skyp other than the pandas and numpy. First, we will create random data. This will create `np.random.normal` will create a thousand random data in the normal distribution and `np.random.normal` will create five data, which is again from the normal distribution. This is what we are going to consider as outliers. So first we will use describe. In describe, you will see that there are totally 1,005 data. You can see that the mean is 0.006 and all the other statistics, but it does not specify whether you have outliers or not. But if you visualize it with box plot, you can see that there are some outliers. The outliers are marked by these kite symbols or the rectangular symbols that you see here. There are outliers below the data also. Histogram will also show the same thing. If I go for a histplot and use the distribution to plot it, you can see that there are some outliers which are towards the tail of the data. So this much is where your tail comes in, where you have three, plus or minus three is where you have the entire actual data. Whatever goes beyond plus or minus three will be considered as outliers. We do have some outliers, which can be visualized. Another method to figure out outliers is by using z-score. So we rely on stats library, compute the z-score based on the value. If it goes beyond three, then we are saying that it is having an outlier and it will detect those outliers. So in this case, we have five data, which is outlier and it is detecting which one are this. Another method is to use IQR. So you compute the Q1 and Q2 based on the quantiles, compute the IQR, then compute the lower bound and upper bound based on the IQR. Then use simple logical statement. That is, if the value is less than lower bound, or if the value is greater than upper bound, then it will has to be converted as an outlier. It will detect all the outliers. So this is a little more trickier than the z-score method. It detects around 13 outliers based on the IQR boundaries. Let's visualize these using the scatterplot. So we generate a scatterplot of X and Y. We generate the scatterplot of the outliers that we have detected using the z-score method and those are colored in red. So those outliers are seen separately. How about removing the outliers? So let's set up the same sample data. First we will figure out if there are any outliers. Yes, we do have some outliers beyond the quantiles that are mentioned and there are below the quantile also. We will use z-score method to first identify the outliers. This statement will identify the outliers. Then we are going to say that whichever is having less than 3, that is our filtered data. So to detect outliers, we use the condition that it should be greater than 3, but we want the original data back. We do not want the data with outliers. So we are saying, select those data which is having z-scores less than 3. So it is the filtered data that we have. Instead of z-score, if we use quantiles or IQR, the same procedure is followed. We will have a different equation now. Whatever is above the lower bound and whatever is below the lower bound, that is extracted from the data frame and that becomes our filtered data. As we said, IQR is a bit stricter than the z-score. It removes around 16 data points. Let's visualize after filtering. Compared with the original month, there are more points at the end. The number of outliers are reduced. This is based on the condition of z-score. What if we are using IQR? You can see that only one point is outside the quantiles. Depending on your data requirements, you can decide which method is better and use it. To finish up, please execute these files on your own data sets. Happy coding and may your data transformation always yield meaningful insights. Thank you for your attentiveness throughout this practical session.

Split - Apply - Combine Video• . Duration: 4 minutes

[Notes]

Welcome, learners, to the topic Split-Apply-Combine. By the end of this video, you will be able to understand the core concept of Split-Apply-Combine strategy in data analysis, identify the main three steps of Split-Apply-Combine process, and recognize the benefits and applications of this approach in data manipulation. The Split-Apply-Combine is a powerful data analysis strategy designed for processing large data sets. Its primary purpose is to simplify complex data manipulation and make them more intuitive for analysts and data scientists. This approach breaks down big data problems into more manageable pieces, allowing for more efficient and organized data processing. The core idea behind Split-Apply-Combine is to break down a big problem into manageable pieces, operate on each piece individually, and put them back together. This strategy consists of three main steps, Split, Apply, and Combine. Each of these steps plays a crucial role in data analysis process, allowing for a structured approach on handling complex data sets. The objective of the Split step is to divide the data into smaller, more manageable groups. In this process, data is partitioned based on one or more variables. For example, you might split a sales data set by product category or region. This step is typically implemented using the group-by function in Pandas, which creates separate groups of data based on the specified criteria. In the Apply step, the objective is to perform operations on each group independently. This process involves applying a function or set of operations to each group created in the Split step. For instance, you might calculate the average sales for each product category. The Apply step allows for parallel processing of data, as each group can be operated on independently of the others. The Combine step aims to reassemble the results from each group into a single output. In this process, the results from the Apply step are merged back into a cohesive structure. For example, we might want to combine the average sales from all product categories into a single table. The output of this step can be a new data set, a summary statistics, or visualization, depending on the analysis needs. The Split-Apply-Combine strategy offers several advantages. It simplifies the complex data manipulation, making them easier to conceptualize and implement. This approach enhances code readability and maintainability as each step is clearly defined. It also facilitates parallel processing for large data sets, improving computational efficiency. Furthermore, it promotes a consistent approach to data analysis tasks, making it easier for teams to collaborate and understand each other's work. This strategy has numerous applications in data analysis. It is useful for data aggregation and summarization tasks. In time series analysis, it can be used for creating monthly or yearly summaries. For cross-sectional analysis, it allows for easy comparison across categories. Additionally, this approach is valuable in feature engineering for machine learning pipelines, helping to create new features based on group-level statistics. To recap, the Split-Apply-Combine strategy breaks down complex data manipulations into manageable steps. Split, Apply, and Combine. This approach simplifies handling of large data sets, enhances code readability, and facilitates parallel processing. It is just not a technique, but a mindset for efficient and effective data manipulation. Thank you for your attention. Happy data analyzing!

Split Step Video• . Duration: 3 minutes

[Notes]

Welcome, learners, to the topic, Split Step. By the end of this video, you will be able to understand the concept and purpose of the Group By operation in Pandas, recognize the basic syntax and structure of Group By operations, apply Group By operations to perform data analysis tasks, and appreciate the flexibility and power of Group By in data manipulation. The Group By operation in Pandas is a powerful tool for splitting data into groups. Its primary purpose is to enable analysts to perform operations on these groups and combine the results efficiently. This functionality is used in various data analysis tasks, allowing for aggregation and manipulation of data based on specific criteria. Group By follows the Split-Apply-Combine paradigm, which is a fundamental concept in data analysis. When you use Group By, the result is a new data frame with one row per group and aggregated values. This structure allows for easy comparison and analysis across different groups within your dataset. In Pandas, you can perform Group By operations on a single column or multiple columns. For a single column, you would use the syntax `GroupedDataFrame.groupby(column)`, which creates a Group By object based on the category column. For multiple columns, you can use `GroupedDataFrame.groupby([column1, column2])` as a list you will provide the column names, for example category and group. This groups the data based on a combination of values in both category and group columns. In both cases, the result is a Group By object that can be further manipulated for various analytical purposes. Group By operations have numerous practical applications in data analysis. Sales Analysis. You can use Group By to calculate total sales by a product category, giving insights into which category are performing best. For Customer Segmentation, Group By allows you to compute average purchase values by a particular customer group, helping to identify high-value customer segments. In Time Series Analysis, Group By is valuable for creating monthly or yearly aggregations, enabling the identification of trends over time. For Performance Metrics, you can use Group By to calculate employee productivity by department, facilitating comparisons across different parts of an organization. In Quality Checks, Group By helps in identifying inconsistencies in group data, which is crucial for maintaining data integrity. Lastly, in the field of Machine Learning, Group By is a powerful tool for feature engineering, allowing you to create new features based on group-level statistics that can enhance model performance. To summarize, Group By is a fundamental operation in Pandas for data aggregation and analysis, implementing the Split-Apply-Combine paradigm. It allows grouping of single or multiple columns, enabling complex analysis with concise code. This tool enhances data manipulation capabilities, making it essential for various analytical tasks in data science and business intelligence. Thank you for your attention. Happy data analyzing!

Apply Step Video• . Duration: 4 minutes

[Notes]

Welcome, learners, to the topic, Apply Step. By the end of this video, you will be able to understand the concept of aggregation in the context of Group By operations. Identify common aggregate functions used with Pandas Group By. Apply multiple aggregations on grouped data. Use custom aggregation functions with Group By. Aggregation in the context of Group By operation is a technique that computes a single result for each group after a Group By operation has been performed. The primary purpose of aggregation is to summarize data within each group, allowing for concise and meaningful insights to be drawn from large datasets. This process is essential in data analysis as it enables us to distill complex information into more manageable and interpretable

forms. Aggregation follows the split-apply-combine paradigm, which is a fundamental concept in data analysis. When you perform an aggregation operation after a Group By, the result is typically a new data frame with one row per each group and aggregated values. The structure allows for easy comparison and analysis across different groups within your datasets, providing a clear overview of group-level statistics. Pandas provides several built-in aggregate functions that can be easily applied to group data. The SUM function calculates the total of values in each group, which is meaningful for the metrics like total sales per category. The MEAN function computes the average of values in each group, providing insights such as average sales per category. These functions can be applied using very simple syntax as given by the code. Other useful built-in functions include COUNT, which gives the number of non-null values in each group, and it is helpful for understanding the group sizes. The MIN and MAX functions find the minimum and maximum values in each group, useful for identifying ranges or extremes. Lastly, the FIRST and LAST function return the first and last values in each group. This can be useful when working with time series data. Pandas allows for applying multiple aggregations simultaneously. You can apply multiple functions to a single column, for instance, `DF.GROUP BY category sales aggregation` and as a list `SUM, MEAN, and MAX`. This would give you the SUM, MEAN, and MAX sales for each category. You can also apply different functions to different columns in one operation, as given in the code. This flexibility allows for comprehensive summaries of grouped data. Sometimes built-in functions may not suffice for specific analytical needs. In such cases, Pandas allows you to use custom aggregation functions. You can define your own function, such as a range calculator, and then apply it using GROUP BY operation. For simpler custom functions, you can use lambda functions, like `lambda X: X.MAX-X.MIN`, for the range operation. This flexibility enables you to perform virtually any calculation on your grouped data. After performing aggregation, you often need to further manipulate the results for analysis or presentation. You can rename columns after aggregation for clarity, reset the index to turn group labels into a column, or sort the aggregation results. The example code provided for each of these operations, like naming, resetting index, and sorting, helps us to figure out how Pandas helps in doing the operations. To conclude, aggregations with GROUP BY is a powerful tool for computing summary statistics on grouped data. It offers built-in and custom functions for various analytical needs. The results can be further manipulated for in-depth analysis, making it an efficient method for deriving insights from complex datasets. Thank you for your attention. Happy data aggregations!

Combine Step Video• . Duration: 3 minutes

[Notes]

Welcome, learners, to the topic Combine Step. By the end of this video, you will be able to understand the concept of combining results after group by operation, recognize different methods for combining group data, apply various techniques to merge or join grouped results, utilize transformation functions to align grouped results with original data. Combining after group by operations is a process that involves merging or aligning grouped results with the original or other data. The primary purpose of this step is to create comprehensive datasets that incorporate both individual-level and group-level information. This process allows analysts to enrich their original data with insights gained from group-level aggregations, providing a more complete picture for analysis. The Combine Step follows the Split-Apply-Combine paradigm. The result of combining after group

by operation is an enhanced dataset that contains both individual and group-level information. The combined dataset allows for more nuanced analysis, as it provides context for individual data points within their respective groups. One method for combining grouped results is using the Merge function. This code demonstrates how to use Merge to combine grouped results with the original data. Note the usage of ResetIndex, the operation Merge. The approach preserves the original data while adding group-level information, allowing for comparisons between individual values and the grouped aggregates. Another method for combining results is using the Join function. We are grouping the data based on category sales and aggregation operation of sum. To compute the results, we use the Join function, which demonstrates how the grouped results can be merged together with the original data. Like merging, this method also preserves the original data while adding group-level information. The choice between Merge and Join functions often depends on the specific structure of your data and the desired output format. Transformation functions offer a powerful way to apply group-level calculations directly to the original data frame. The code uses the TransformMean function, which shows how to use the transformation function to calculate and add group averages for each row. For custom transformation, you can use expressions like `df.sales by df.groupbyCategorySales.transformSum`. This gives an advantage of using transform in such a way that the results align perfectly with the original data frame, eliminating the need for additional merging steps. To wrap up, combining completes the split-apply-combine paradigm in Pandas's data analysis. The key methods include merge, join, and transform for integrating grouped results. The transform function is useful for aligning group-level calculations with the original data structure. This process enables the creation of rich, insightful datasets that blend individual and group-level information, enhancing analytical capabilities. Thank you for your attention. Happy data analyzing!

Lesson 4 Demo Video• . Duration: 5 minutes

[Notes]

Greetings, learners. Today, we will demonstrate the code snippets from our lesson on data aggregation techniques. We will use Jupyter Notebook to execute these examples and examine their aggregated outputs. Are you ready to begin? In this video, we will look at the steps split, apply, and combine. We will rely on Pandas, NumPy, and Matplotlib libraries. Let's create a sample data frame where we have four columns, category, region, sales, and profit. Category and region are categorical, whereas sales and profit are numerical. Let's see a sample data. Now, let's look at the info of the data. What kind of data are we dealing with? As we said, the first two are string, and the last two are numerical data. We have around 1,000 entries. Let's look at the data carefully. Based on the category, we have three categories, A, B, and C, which will help us to perform aggregation. And similarly, for region, we have four categories, north, south, east, and west, again, scope for aggregation. So assume that we are working with a data where you are selling a product based on three categories and in four regions. Now, we want to aggregate maybe based on the region or based on the category. How much profit did you gain or how much sales did you do? How to do that? The first step is to provide the aggregation. On which column do you want to aggregate? So we are going to say that we will aggregate by the category column, that is the product names, A, B, and C. So we're saying `data frame.groupbyCategory`. This will create a new data frame which will have the data in group. Let's just look at how the group data looks like, print the contents. It is an object. Now, look at the contents of the data. We are going to use the head method. By default,

head will list out the first 15 values in the data frame. So let's look at the first 15 values of the group data based on the category column. Now, let's aggregate based on these categories, right? So we have the group data. Now, based on the group data on category, we will compute the mean of all the sales. Very simple aggregation. Now, look at the mean. It gives us the mean sales of each of the category. So it is grouped based on the category column, and it is giving out the mean of each category. Now, if you want to combine multiple aggregations, like you want to compute the sum and the mean of sales and mean of the profit, that is also possible. That means we are doing multiple aggregations. On sales itself, we are doing sum and mean, and on profit, just the mean. If you want to figure out what is the profit margin based on the sum computation, you can give the equations that you want to provide and create a custom function based on it. Apply this custom function on the group data. It will tell you how much is the margin profit. This equation is based on your business logic. If you want to filter based on these groups, or based on these group by definition that we can have, that is also possible. So here we are creating a lambda function where we are saying that if the sales mean is more than 500, we are going to filter it out and say that it is a high sale category. Now, look at the sales data. All the three are sales more than 500. So all of them will be listed out. Now, if we change this to 550, we can see that it corresponds only to the category A. The rest two categories do not have that much sales. In the combine step, we are going to merge back the results with the original data. So how to combine the sales mean with this original data? Let's create a new data frame where you are going to say that the category average sales, a new column is added. How to do it? You can use the merge method. So in the merge method, we give the original data frame. What do you want to add on to the totals, which we have computed separately? On which category you want to merge on, right? And what should be the suffix of the column name that you get added? So based on this, it will add the column average based on the category and add the sales data. Multiple operations can be merged together in group by. So we create a group by on category and specify that you have to perform lot of aggregation. On sales, we want sum and that should be saved as total sales. On profit, we want mean and it should be maintained as average profit. And we also have to have a profit margin, which is going to execute a lambda function based on the profit data. And we want to reset the index after the group by operation, apply operation, and the merging operation. And finally, we'll show the result. On executing this statement, you can see that only three rows exist. First row is for category A with all the details. Second row is category B with, again, the sales profit and profit margin. And finally, the category C. As we conclude, I encourage you to dive into these files with your own data sets. Enjoy summarizing and analyzing your data and may your aggregations always reveal powerful patterns. Thank you for the engagement and enthusiasm during this hands-on demonstration. Thank you.

Dimensionality Reduction Video• . Duration: 7 minutes

[Notes]

Hello everyone, in this lesson we will talk about dimensionality reduction. By the end of this video, you would have understood different kinds of data types which are available in the real-world applications of data analytics and you should be able to extract features from the raw data to create dimensions and distributions about the underlying dataset. We will also see an introduction to dimensionality reduction. There are several data sources in the real-world such as images, text, time series and high throughput data from bioinformatics experiments. We would like to use these

datasets as raw data for the data analytics modeling and analyze the techniques involved for collecting the data, storing it, cleaning it to get good quality data and then evaluating it for different models using simple machine learning algorithms and advanced analytics concepts based on deep learning. The feature types which are there in the real world include fundamental data types such as float, integer and boolean but also in data analytics, they can be extended into causal, structural and synthetic data types where structural data is about combination of the underlying data types to create more meaningful data attributes for the modeling and causal relation of cause and effect between the different parts of the data. Synthetic data is generated data from the given dataset which is treated as the original data distribution. For all these data types, we have several feature selection techniques which can be categorized into supervised, unsupervised and semi-supervised techniques. For each of these techniques, we will have to provide the training dataset or the input data from the available raw data. The raw data can be categorized into quantitative data useful for statistical modeling which includes numeric data types available in CSV files and qualitative data for interpretation of the raw dataset such as images and text using non-numeric data attributes in real-time. On the right side, we can see an example of an RGB color image which has been subject to several mathematical transforms to find the infrared and thermal versions of the image useful as data attributes for the modeling. On the top right, we can see products such as a phone which is treated as the dimension of the data and a particular type of phone like iPhone which is treated as an attribute for the data. We would like to be able to store, retrieve and model such data types. In this slide, we can see examples of such data types for the web pages which are available on the internet. These include statistical, syntactical or language-based, temporal or varying in time. For example, we can see that average, standard deviation, entropy, median are statistical measures associated with the text and words on a web page. The grammatical rules such as verb-object pairs and topics of the web page are the syntactical or language-based features in the web page. The time interval between two packets on the network which is rendering that web page is our example of a temporal feature. When we create the training dataset, we have to decide the most important features for the modeling purpose based on the advantages and disadvantages offered by the features which are listed in this slide. Once we have the features defined from the raw data for the training purpose of the modeling, we treat them as data distributions where on the top figure, we can see there is a normal distribution with a mean which is very high compared to another normal distribution whose mean is not so high. We can see that for the first distribution, the value is 0.8, around 0.9. For the second distribution, it is around 0.5. And correspondingly, the variance of the distribution is comparatively less for the first distribution and more for the second distribution. Based on this, we decide the most important components of the distribution for further modeling. The figure on the bottom shows the corresponding probability distribution function for the probability density function shown on the figure on the top. There are several distribution functions occurring in nature, some of which are listed here such as the uniform distribution, Poisson distribution, Gaussian distribution, and exponential distribution. We treat the underlying distribution for any given dataset as belonging to one of these categories for statistical modeling in data analytics. However, in practical scenarios, we only have the discretized version of the data represented using histograms and the overall trend which is followed in the histograms is taken to be a distribution. In this case, the wine dataset which is publicly available is visualized and follows normal distributions across different attributes such as taste and aroma. Once we have the features for the modeling, we would like to find the most compact set of features which are most useful and have the most information for the modeling purpose and also reduce the computational effort involved in the modeling. So

there are several techniques in dimensionality reduction to create the condensed compressed representations are called dimensions. These dimensions reduce the number of input features and remove irrelevant features to find informative features to model the target concept. In the examples on the right side, we can see that for the sentence, a quick brown fox jumps over the lazy dog and another sentence, the dog jumped. There is an encoding in terms of the frequencies of the words in the text, which is being mapped to floating point numbers representing the dimensions of the sentences in the text across different documents. In the diagram below, we can see that original image of different colors has been mapped to its components and with these components, we can reconstruct the original image and these components will be useful for modeling the target concept of detecting an object in the image. Thus, dimensionality reduction increases computational efficiency of the algorithms used in data analytics. It minimizes information loss to find the most important patterns in the data. In the subsequent videos, we will see several techniques for dimensionality reduction, such as feature projection, selection, and clustering. Thus, in this video, we have seen how to create a mathematical representation of the raw data available for data analytics and identify most relevant features for data analytics using dimensionality reduction techniques. Thank you.

Numerosity Reduction Video• . Duration: 8 minutes

[Notes]

In this video, we will look at numerosity reduction which is complementary of dimensionality reduction to reduce the size of data sets. We will look at several data transformations to extract informative features from the original data to create the training data set. The size of data has a big impact on the modeling results of data analytics. Consequently, the quality of modeling is based on the quality of data. So data must be preprocessed so as to improve the efficiency and ease of building the analytics model. The factors comprising data quality include accuracy, currentness, completeness, consistency, and so on. On the right side of the figure we can see how a workflow is created for obtaining a reduced data set from the input raw data. The algorithm takes the reduced data set as input and produces several outputs which must be evaluated for consistency. The data is influenced by several factors such as noise, missing values, outliers and so on. And without dealing with each factor separately we cannot obtain the correct data for building a consistent analytics model. The process of obtaining good quality data is called data cleaning. It removes noise and inconsistencies in the data. Several mathematical transforms such as normalization and smoothing are applied to obtain the cleaned data. The clean data is further reduced in number of records and number of attributes to obtain a good sample for building the analytics model. The numerosity reduction workflow is given on the right side of this figure where the steps followed are normalization, binning, smoothing in that order, or discretization or combination of binning, discretization or skip one of the steps in this workflow such as go directly to smoothing from normalization. The purpose of normalization is to get the ranges of different attributes into the same range of values such as 0 and 1. And the purpose of binning and discretization is to create a discrete version of the original data set useful for algorithmic processing. Purpose of smoothing is to obtain good data that is easily learned by the analytics model. There are several parametric and non parametric techniques to affect these steps in the data and they have a significant impact on the quality of the results produced by data analytics. The non-parametric techniques include production

of histograms as described in the previous video sampling, data aggregation, so on, and the correctness of data analytics improves with the usage of such techniques. One can also generate a concept hierarchy of several discrete categories of data by using these techniques. The workflow for doing data transformation using mathematical operations on the raw dataset is given in this figure. The data transformation can be thought of as consisting of two steps, the cleaning and integration of multiple data sources and extraction of meaningful features useful for the data analytics purpose. The corresponding modeling can be thought of as having two or multiple steps where we produce the data analytics output across these steps. There is feedback loops from the modeling to the feature extraction and the modeling to the data collection. If the data is insufficient for modeling then such feedback loops come into effect and if the quality of the process data is insufficient for modeling, such feedback loops come into effect. Some of the techniques for non-parametric numerosity reduction of the data are given in the subsequent slides starting with binning. There are two kinds of binning, equal width and equal frequency binning which replace the value given in the list of enumerated in the feature in terms of different discrete categories called bins. For example, there is value of price of a item given in dollars such as \$4, \$8, \$28, \$34. This is mapped into three bins 4, 8, and 15. The values of these bins are based on calculation of either equal frequency or equal width formula which is used to create these binned values. For example, using equal frequency binning we wish to put the first three values 4, 8, 15 into the first bin and the next three values 21, 24 where 21 is repeated two times into the next bin which is bin 2 and the next three values 25, 28, 34 into the third bin so that each bin has three values each satisfying the condition of equal frequency. Similarly, there are other ways to partition the original data into bins which are described in this slide. The partitioning rules provide a histogram representation of the original data and they discretize a numeric attribute into categories which can be further combined into higher level categories using a top down or bottom up merging strategy. The idea is that the class the frequencies of the values which are assigned to each bin should be consistent according to a selected formula. Discretization is an extension of binning where there are several intervals such as 0 to 10, 11 to 20 assigned to the bin values. And these intervals in turn can be assigned labels such as the age of a person, whether it is youth, adult, or senior. And this can be done recursively to organize higher level concepts into a concept hierarchy of the numeric attributes. Smoothing is a further step from discretization and there are several techniques such as regression which will be seen in the subsequent modules used for smoothing. Linear regression fits a curve to the data like in the figure where the difference between the scatter of points and the curve should be minimized. And for any given scatter of points, if we cannot minimize the distances between the points and the fit which is a straight line. We can use a curved line to do that, in which case the problem changes to that of nonlinear regression. Multiple linear regression or linear regression are extensions of linear regression where the data is fit on multiple attributes. And with recent ability to collect data on different dimensions such as collection of data on demand, collection of data of different sizes such as gigabytes and terabytes, and collection of different types of data such as photos, audios, video, social media, so on. There is need for building such smoothing techniques on complex multidimensional surface representing the data. Hence there are several issues found in such data sets such as missing values, outlier values, inconsistent values, invalid values, duplicate values, so on. The purpose of numerosity reduction is to deal with each of these issues using algorithms so that we are able to extract patterns on a reduced data set which is compressed and representative of the original problem at hand. We wish to achieve a small approximation error in the data analytics model which are built on the data set, and there are several optimization algorithms and objectives

that can be formulated around this requirement. Thus, we have reviewed the need for numerosity reduction, the mathematical transforms to do numerosity reduction and the how to set up a workflow for numerosity reduction including binning, discretization, and smoothing. These techniques will be useful for creating a representation of the raw data that is useful for data analytics. Thank you.

Sampling Methods Video• . Duration: 5 minutes

[Notes]

In this video, we will look at sampling methods to create the training dataset for analytics models. Sampling is useful for setting up a statistical experiment in the data analytics process, starting with cross-validation procedures. The most common kind of cross-validation is the k-fold cross-validation, where there is a holdout or test dataset separated from the training dataset and such a separation is done k times and that is illustrated in the figure on the right side. We wish to build a model on the training dataset and report its performance on the test dataset. The test dataset should be separate from the training dataset to get a statistically valid model. Here is an example of training dataset ABC given to us. This is split into the various subsets of ABC, AB, AC, BC. The remaining values of C, B and A are separated out from the model and the training of the analytics model is done on AB, AC, BC respectively while the testing is done on C, B, A respectively. The average performance across different runs is reported as the final performance of the model. There are several sampling procedures which can be used to create the cross-validation experiments in data analytics. In this figure, we see how several data examples can be randomly generated using underlying statistical properties of the data, such as this is pseudo-random generation and this is based on a statistical formula. Depending on the characteristics of the data, such as we can see red, violet, light blue and dark blue points here, we wish to select the sampling procedure. In random sampling, the chance of selecting red, violet, etc. points is equal. So we expect to find colors of all points in almost the same proportion in the training and test data. If this condition does not hold, then we do stratified sampling, where depending on the proportions of the points belonging to different colors, the chance of selecting those points is determined, such that there is a lesser chance of selecting red points compared to blue points, so that we get a dataset which is balanced across different colors of points for both training and testing purpose. This is the idea used in the SMOTE algorithm used for imbalanced data, where there is less light blue points compared to the dark blue points, so that after applying the technique, there is equal weightage given to both light and dark blue points. SMOTE algorithm searches through the neighborhoods of points and then interpolates from the given points which are marked blue in the figure here into points which become red after applying SMOTE so that the random sampling and stratified sampling can be further done on the recreated data. There are several other sampling methods which can be used depending on the data characteristics, such as rare classes can be either over sampled to give more weightage to rare classes compared to commonly occurring classes. There can be different weights assigned to the points based on their performance in the analytics model and several models can also be integrated into the sampling procedures. The probability of sampling is influenced by the weights assigned to the points. Sampled data should be representative of the underlying population and should be useful for building analytics models in an efficient manner. Any errors which creep in into this process can be assigned as weights also called misclassification costs to the points which are hard to classify or predict or model. Also not all examples in the data may be informative, so we

want to get the examples which are most useful for the analytics procedure and the acquisition of such examples can be viewed as a cost centric process that assigns labels to the points which are hard or easy to classify. This is the procedure of active sampling. The key question in active sampling is how to select examples to label the most accurate model at a given cost. This leads to several modeling assumptions about querying the most informative examples in the sample and in algorithms it leads to several black box oracle algorithms to create the queries on the examples. Thus we have reviewed stratified sampling, imbalance sampling, adaptive resampling and active sampling to reduce the size and complexity of the original dataset to be able to prepare a training dataset for the analytics modeling. We can further optimize these sampling procedures based on insights on the performance of the analytics models with respect to hard and easy to build and model points. Thank you.

Filter Methods Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video, we will look at filter methods to do feature selection. In the previous video, we were understanding the ways to prepare training dataset for the analytics problem. Once the dataset is prepared, we will be in a position to build a good model. The original dataset can be used as the prepared dataset for the analytics modeling or new attributes can be derived from the original dataset. These attributes are called features and there are several techniques to derive these features starting with the filter methods in this video. We wish to identify and select the most relevant features for any given data analytics task. And these features are independent of the analytics task, they are able to handle large and complex data and they are reusable across different algorithms. The resulting patterns from these features must be easily understood by the end user, validate statistical hypothesis in an application domain and produce some unknown information from the analytics process. The algorithms to produce such patterns include recursive and iterative algorithms, serial and parallel algorithms, linear and convex algorithms, exact and approximate algorithms. These are all different algorithms available in the literature. I encourage you to refer to online resources on the definitions of these terms. We are normally assuming a problem consisting of millions of low-level features and we want to select the most relevant features to build better, faster, easier to understand analytics models and in general machine learning, which is represented in the graph here where Q is much much greater than Q dash, which is the set of features which we want to model from the original dataset X . The features can be modeled one feature at a time, multiple features at a time using the filter methods. The filters produce ranks to each feature giving us the importance or relevance of the feature to the analytics task at hand. And to remove an irrelevant feature, a search procedure is developed in the filtering algorithms that evaluates all possible subsets of the features to find the best subsets which are most relevant for the analytics task. Thus, this is an exponentially complex problem to solve using algorithms. So the optimization problem in filtering methods can be formulated as find a procedure to remove redundant data and find relevant data in a tractable manner on the given computer and use this procedure as a pre-processing step independent of any chosen data analytics task at hand. Sort the different features according to an importance index which could be interpreted as a ranking measure, a quality index or a weighting criteria. Select several subsets of features to process and identify as many relevant features as possible as distinguished from irrelevant and redundant

features. This same algorithm is represented in these slides where we initialize a set F to the whole set of P features and for every subset of features in combination with features which have been already selected, there is a coefficient calculated such that it maximizes the performance gain from the data analytics process and the set S is incrementally updated until the set S reaches expected size of the features which we find to be relevant for the data analytics problem represented as a predictor in the workflow here. Suppose we can think of this diagram which is summarizing the various components of the filter methods such as how to generate the feature subset, what are the search algorithms to do that and how to evaluate the feature subset, what are the performance criteria to do that and what are the search and optimization procedures which are created to do this and finally how to distinguish relevant features from irrelevant features. Relevance may be based on contribution of single feature to the analytics model or multiple features or it could be dependent on the context of the feature subset which is used for analytics modeling or it could be based on a mathematical definition. Multiple statistical hypothesis tests can also be devised to evaluate feature subsets. Thus we have seen that filter methods order features whether individual or multiple according to a ranking measure and the ranking is based on a notion of relevance of the features to the data analytics task. This notion is evaluated using several mathematical criteria. The results of filter methods create models which generalize to new test data and are useful in the real-world application and are computationally tractable on a given computing machine and capture our notion of relevance, irrelevance, redundancy and interestingness in the data. Thank you.

Correlation Based Filters Video• . Duration: 14 minutes

[Notes]

Hello everyone, in this video, we will look at correlation based filter methods to analyze linear correlations in the data and set up hypothesis tests for statistical significance of linear and non-linear correlations in the data. One desirable property of data is there is a dependency between multiple variables in the data. Suppose there are two variables X and Y , which are the random variables associated with features x and y respectively. The probability distribution that Y is dependent on X is represented by the conditional probability. If there is no dependence, then this conditional probability will be the same as the probability of Y independent of the probability of X . And this is the independence condition of probability theory used in defining a correlation. And if this is not the case, then there is a dependence between X and Y that can be measured using several probability and statistical measures, such as the entropy measure or mutual information measure here, giving the dependence of X and Y as a joint probability conditioned on the independent probabilities of X and Y using the KL divergence from information theory. If two features X and Y are correlated, they may be present in the training dataset given to the analytics model and they may be useful for learning the target concept Y , which is represented as a function of the input data X . Filters provide a way to score the different features that are correlated to each other and the predicted concept. The interactions between different features are quantified by using certain coefficients such as the Pearson coefficient. The Pearson coefficient is a measure of linear dependence between two features and it is defined as the subtraction of the mean of a feature from every value of the feature for both features divided by their standard deviation. And this can be expressed mathematically as expectation of the random variable underlying the feature and the mean of the random variable underlying the feature. And it is simplified as shown in the figure. The

ranges of different values produced from the Pearson coefficient are also given in the figure, where we can say a coefficient value of 0 indicates independence between features X and Y, whereas value of minus 1 or plus 1 indicates strong direct or inverse dependence. Any value between 0 and plus 1 can be thought of in the range of 0 to 0.3, 0.3 to 0.7 and 0.8 to 1.0 and probably 0.6 to 0.7 or 0.6 to 0.8. These values can be labeled as representative of weak, moderate, strong and very strong correlation in the data. And the same is represented using a correlation matrix here for a standard data set, where the color and the size of the ellipses which are shown in the figure is indicative of the strength of the correlation between multiple variables in the data. So for example, displacement is related to weight using a green color, which is indicative of strong correlation according to our categorization of coefficients and the strength of the correlation is moderate. The figure shown on the right side of the slide is a correlation matrix, which shows the dependence coefficients between every pair of the features present in the data. Correlation matrices help us visualize the mean and variance of all pairs of features in the data. Thus, we can find highly correlated feature pairs. We can also find portions of the correlation matrix, which show multivariate correlations. Note that we are discussing linear dependency between the variables using Pearson coefficient. Using other coefficients, the correlation matrix can represent nonlinear correlations. Spearman rank correlation is an example of a nonlinear correlation coefficient, which replaces the values of the observations themselves with the rank of the observations in the sorted list of values for that particular feature, such as in a feature containing n points, d is the rank of the ith record in the feature and this summation is on the numerator of the formula is being done for all the data points in the training dataset. The denominator acts as a normalization term so that the value of the coefficient is in the range of minus 1 to 1. Normalization of such nonlinear correlation coefficients can be interpreted as a filtering method where the value of the coefficient is the importance of the feature in the data analytics task and it has a direct bearing on the errors and residuals produced by the data analytics error is the difference between the observed value and the predicted value in the data analytics residual is same as error but it is calculated on the given training dataset using errors and residuals and their distribution across different data records. We can set up hypothesis testing in the filter methods to test for correlation relationships not only in the sample which is given as input to the analytics model but also in the underlying population or distribution of which the sample is representative. If there are bidirectional relationships between the features x and y that can be termed as a correlation. If there is a unidirectional relationship between features x and y that can be termed as a regression. Normally we start with a scatter of points where the x and y axis of each plot are the features x and y to be analyzed for correlations. Using the scatter of points we can see several trends in the data such as a decreasing trend from right to left, an increasing trend from right to left, an increasing trend from left to right, a decreasing trend from left to right so on and to capture such trends in the data we use correlation coefficients where correlation is to be computed not only on the sample on the underlying distribution of the data and it has to be statistically significant in both cases to create a reliable estimate of the correlation. This is done by hypothesis testing that checks for statistical significance of the correlations represented by their p-values which to draw conclusions on the underlying populations from observed data samples. Statistical significance tests are described on the expectations of either the random variables or functions of random variables on the samples and populations. Then statistical inferences are drawn on such expectations to determine the coefficient relating multiple variables in a correlation function. The hypothesis test lets us decide whether the correlation is significant or not in the underlying population or distribution given the training set on which it is computed. A value of correlation coefficient close to zero implies

that there is no correlation in the population even if there is a correlation found in the sample. So probably such a correlation is a chance result found in the sample and we do not have sufficient statistical evidence to generalize it to the underlying population and in such a case the correlation coefficient is significantly different from zero. At the same time the p-value supporting a null hypothesis which must be rejected to establish an alternate hypothesis is also very low. In a hypothesis test involving null and alternate hypotheses we estimate the factors to generalize a conclusion from a sample to the underlying population. Both null and alternate hypotheses are mutually exclusive theories which we derive from the data. Null hypothesis is the hypothesis which we are trying to disprove. Alternate hypothesis is the hypothesis which we are trying to prove and we wish to find correlation coefficient that is significant in the underlying population to establish a alternate hypothesis. So we can summarize the alternate hypothesis as the correlation coefficient ρ is not equal to zero whereas the null hypothesis says ρ is equal to zero. Then several inferential statistics can be used to generalize our conclusions from the sample to the population such as the t-test. The t-test is calculated from the correlation coefficient and the number of examples used in the data sample to calculate such a coefficient. We can see that even if there is a weak correlation in the data for a large number of samples the t-test will give a significant result. The t-test is a summary of the underlying data distribution and a corresponding probability value for the t-test is the p-value. It is a probability which ranges between 0 and 1. In our case we wish to disprove the null hypothesis and we express that saying that there is a 5% chance that the null hypothesis is true in the underlying population which means that there is a 95% chance that it is false and the alternate hypothesis is true. The p-value can be obtained from several tables which are available for the inferential statistics and these tables are also available in modern software such as in Python. The p-value is compared against a threshold value of alpha and the alpha in our case is taken to be 0.05. Alpha is normally taken to be a very small number such as 0.01 or 0.05. If p-value is larger than alpha this implies that we fail to reject the null hypothesis. So we conclude that the correlation is not statistically significant. On the contrary if p-value is less than alpha then we conclude that the correlation is significant. Thus a hypothesis test helps us to determine if there is an effect in the data sample which generalizes to the population and typically null hypothesis states there is no effect whereas alternate hypothesis states that there is an effect which is a non-zero effect and p-values are the probability values supporting the null hypothesis. So we wish to provide evidence against supporting the null hypothesis. The relationship between alpha and p-values helps us to set up such a null and alternate hypothesis for our significance testing. Lower alpha indicates that more evidence is required to reject the null hypothesis. There is an example given in this slide where an all-you-can-eat pizza joint thinks that its customers eat four slices on an average and based on that it is designing a marketing campaign whereas we think that the average is more than four slices it is probably 5.6 slices. We want to establish this hypothesis in the data. So to set up our statistical test we collect data from different customers on how many slices of pizza are eaten in each visit of the customer and we associate a probability of consuming so many slices across different customers in the pizza chain. Then we plot all these probabilities to produce a normal distribution. We analyze the mean and variance of the normal distribution and set the statistical significance of our test as 5% that is alpha is 5%. Since the data distribution follows a normal curve we can take a t-test and decide what is it that we are missing as shown in the violet shaded region here that is how many customers are eating more than 5.6 slices probably 6 7 8 10 slices compared to the average we want to establish to be 5.6 where the whereas the pizza chain thinks it's four slices this is tested as a statistical significance test involving the t-test inferential statistic where the

p-value is found to be less than 0.05. So we can conclude that in 95% of the customers we can generalize that the number of pizza slices which are consumed are less than 5.6 thus in this video we have understood an introduction to correlation analysis and we have learned how to find linear correlations in the data then generalize such correlation calculations to nonlinear correlations using statistical hypothesis testing. Thank you.

Entropy-Based Filters Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video, in continuation to the previous video, we look at nonlinear correlation methods based on the concept of entropy. By the end of this video, we would have understood the reason for analyzing nonlinear correlations in data using entropy-based filters to do this and use such filters in Python. Correlation coefficients such as the Pearson coefficient can detect only linear dependencies between pairs of features in the data and corresponding dependencies between the features and the target concept to be learnt by the analytics model. If we wish to learn nonlinear dependency structures in the data, then we have to create a model that fits to the data to make the prediction required from the analytics model, such as a regression fit. Then we have to evaluate the goodness of the fit to the given dataset such that it satisfies some statistical criteria. The filter-based ranking methods which do this are suitable for nonlinear correlation analysis. They sort the given set of features according to an evaluation metric that is directly calculated on the training dataset. A feature or subset of features is evaluated according to a discrimination criteria where the discrimination could be based on classification techniques which we will see in later modules or the features can be evaluated based on dependence and independence between each other with respect to the target concept being learnt by the analytics model. In such cases, entropy is measured in terms of the fraction of data points belonging to a class i.e. the target concept to be predicted for a particular value of the attribute. This is calculated for all attribute values and classes and entropy is a statistic encoding the information present in the dataset to find the dependency structure in it. Higher values of entropy means there is a mixing of target concepts or class labels in the data. Higher mixing is not desirable because it does not discriminate between several target concepts and a value of 0 to entropy implies that there is perfect separation between the different target concepts being learnt by the analytics model giving greater discriminative power to the filter method and the subsequent analytics model. An alternative to entropy is the chi-square statistic which is also a statistical test based on data that has been discretized into different class labels and it is not making any further assumptions about the underlying data distribution whereas entropy is an information theoretic statistic that quantifies the mutual information between multiple dependencies in the data and it is useful to quantify joint data distributions in the underlying population. A regression equation is normally fit to the data to build non-linear correlations. Regression will be covered in subsequent modules. The geometric interpretation of the regression fit is used to interpret the importance of the dependency structures in the data such as whether the regression line is on an increasing or decreasing trend and where does it cut the y-axis and where does it cut the x-axis. The corresponding correlation coefficient is a measure of the proportion of variation of the various dependent features in the regression equation. These coefficients are estimated using an algorithm in regression. Regression equations can be extended to cause-effect relationships also relating dependent and independent features and when the assumptions of simple regression

models are violated more complex assumptions are needed to get a good fit to the data. There are several feature ranking techniques available in Python and other programming languages such as R and Java and these techniques are listed here. The techniques include mutual information and its variations such as information gain and gain ratio, principal components and chi-square. We encourage you to use Python libraries to visualize such linear and non-linear correlations in the data using correlation matrices and different filter methods. We will explore feature ranking techniques to create new features in the subsequent videos. Thank you.

Wrapper Methods Video• . Duration: 7 minutes

[Notes]

In this video, we will look at wrapper methods which can be contrasted with the filter methods to produce features for data analytics. Several search and optimization criteria are combined in wrapper methods and we will look at their components. The purpose of wrapper methods is to improve the data analytics performance and produce faster and cost-effective analytics models while producing an understanding of the underlying statistical process generating the data for analytics. The wrapper methods can be interpreted as belonging to a block where a subset of the features is generated and included into the evaluation of the analytics model. Assume the set of all features present in the data. Wrappers hence assess the usefulness of subsets of features in the data and search through all possible feature subsets and to various degrees of importance, learn relevant and spurious features in the training data. Only a few of the features present in the original data may be relevant for the target concept and these few features are sought after by the wrapper methods such that they are correlated to the target concept and include most of the necessary information in the given data for modeling the target concept. Thus, the purpose of wrapper methods is also to remove redundant and irrelevant features to reduce noise in the data and estimate errors such that target concepts are predicted correctly in the analytics model. Out of all possibilities of the wrapper methods, we wish to select the smallest and the easiest and the simplest model for considering to be the best model. This is the minimum description length principle that selects the best set of features in the data that explain the data in the simplest possible way. Wrappers reduce the computational complexity of the algorithms involved in building the models and remove irrelevant features in the data called predictors in this slide. The workflow in wrapper methods is further detailed in this diagram where the various subsets of the wrappers are generated according to a search algorithm or strategy. They are evaluated in a prediction algorithm that is based on statistical hypothesis testing and it returns a performance estimate for each feature subset that has been generated in the search algorithm. Combination of the search algorithm and the prediction algorithm is the wrapper method and it produces feature sets which are finally used for building the analytics model and reporting the performance of the model on a cross-validated test dataset. The prediction algorithm partitions data into training and test or validation datasets and we wish to report the best evaluation of the feature subset on the test dataset whereas the algorithm builds the analytics model on the training dataset. The resulting model after going through the wrapper method is evaluated according to a cross-validation procedure on the test dataset. The simplest strategy for generating feature subsets is a brute force search of generating all possible subsets which is the for a given set we can generate the power set of the set but for a large number of items in the set this is a computationally prohibitive procedure. So we wish to devise efficient procedures according to the

order and number of feature subset evaluations that must be specified in the search algorithm. An optimal feature subset algorithm is well devised to be integrated with the prediction algorithm without carrying forward any biases present in the data with respect to the target concept learned by the analytics model. The algorithm for wrapper methods can thus be formulated as create an augmented set of features by adding feature subsets to the current feature set. Use a prediction algorithm to evaluate the performance of the features and decide to keep the augmentation or remove the augmentation for the next iteration repeat until there is no improvement in the feature set selected or a maximum number of iterations is reached. The augmentation to the feature set can be done in several ways which creates several different wrapper methods. Features may be added by random sampling, a greedy strategy can be used to find the features with the greatest discriminative power such as in the entropy concept and features may be sensitive to the choice of the prediction algorithm. Thus the components of the wrapper methods are similar to those of filter methods except the focus of wrapper methods is on finding relevant features and their subsets and it is combined with all the previous filtering methods such as evaluating the feature subsets, finding the best analytics model, enumerating the feature subsets and statistical significance testing. Forward feature selection and backward feature elimination are two commonly used wrappers that optimize the search strategy in the prediction algorithm. The difference between filters and wrappers is illustrated in this diagram where filters are independent of the prediction or analytics model whereas wrappers are closely integrated with it. Wrappers are useful for feature construction, feature subset generation and evaluation with respect to the analytics task at hand and they differ from filters in all these respects. Thus wrappers also measure the usefulness of the feature set for the analytics task at hand and they are useful for extending the cross validation procedures such that the best version of the data is used for analytics modeling. But they are prone to overfitting on the data that is they work very well on the training data but do not work so well on the test data. They are not scalable to high dimensional data which leads us to the next video on Embedded methods. Thank you.

Forward Selection Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video, we will understand the forward feature selection procedure, which is a wrapper method. In forward feature selection or forward selection, the search space for generating the feature subsets is organized in a forward manner. That is, we start with single features and go to multiple features. This leads to several optimization problems and corresponding search algorithms. Forward selection is an iterative search algorithm that starts with an empty set of features as its initial state. In the first iteration, each feature is evaluated individually for its contribution to the analytics model and its performance. In subsequent iterations, a single feature is added incrementally to the previous feature subsets to create new feature subsets that are evaluated again and again. After some number of iterations, the algorithm selects k best features that are retained for subsequent iterations. The algorithm terminates if the accuracy of the target concept and analytics model at hand does not improve after consecutive iterations or if the desired number of features has been learnt by the analytics procedure. The computation time of feature selection can be reduced by reducing the number of evaluations of the feature subsets in the algorithm while not compromising on the quality of the analytics results. k is typically chosen to be a low value to reduce

the running time of the algorithm and it can be fine-tuned to the specific model in question. The disadvantage of this search algorithm is it may slowly loop through all the feature subsets and take a long time to find the best subsets. This slide is an illustration of the feature selection procedure in forward feature selection where the items a, b, c are being combined with the items x, y, z to create items such as item sets such as a, b, c, x, a, b, c, y, a, b, c, z and so on. Each item set is evaluated independently to decide whether to retain the new items x, y, z which are being appended to the previous item set a, b, c. Out of the evaluations, suppose we find that x is to be retained and y is to be retained but z is not to be retained then a, b, c are extended into a, b, c, x and a, b, c, y for the subsequent iterations. The wrapper method thus conducts a search on all possible feature subsets and the search space is organized as an algorithm consisting of an initial state, a termination condition and a search algorithm. For n features, we can represent the search space using a bit vector of n bits where absence of a feature leads to a value of 0 and presents to a value of 1. We have to then choose operators to connect between search states by adding or deleting features from the current search state. The search terminates if no new search states are found to improve the accuracy of the analytics model. And the search algorithm seeks to find the search state with the highest evaluation criteria which is normally the accuracy. There are several search algorithms to go through the search states starting from the brute force search and extending into hill climbing search, greedy search, simulated annealing so on. This can be understood in terms of the fitness or objective function landscape that finds the best feature subset for the wrapper. Several fitness landscapes are illustrated in this slide where we could have only one solution or we could have multiple solutions and we could have multiple solutions that are hard to find and we could have many multiple solutions that are ill formulated in terms of our objective function. In all these landscapes, we must find the best solution. That is done by a search procedure which goes stepwise according to the direction of search and the step to take in that direction. Some of the search algorithms are able to converge directly depending on the fitness landscape. Some take a long time to converge. Some never converge. All the search algorithms can be thought of as consisting of these steps in the algorithm where there is an open list that is to be updated using the best feature subset for the fitness landscape of the accuracy metric of the analytics model which is being evaluated. Open is updated and subsequently there is a closed which is a separate set which is being updated. Both open and closed are evaluated on the current best subset of the features. So long as there is an improvement in the performance, the search continues across different search states otherwise it terminates. After the end of the search, we find the best set of features from set operations on the open and closed sets. Thus, we have seen the forward selection procedure in the wrapper methods and we have seen the algorithm various search algorithms in such a forward selection procedure and how to optimize them for the best feature set in the wrapper method. Thank you.

Backward Elimination Video• . Duration: 7 minutes

[Notes]

Hello everyone, in this video, we look at the backward elimination procedure for wrapper methods. By the end of the video, we understand how the search space is organized in backward elimination compared to forward selection and we look at the recursive feature elimination algorithm. In backward elimination, all the features in the data are considered to be important for the analytics

task at hand and the interactions between more important feature subsets in the data are found by the algorithm. For example, if we are able to imagine that every feature is following a polynomial distribution, then higher order of the polynomials will have more significance to lower order of the polynomials in the features to test the final analytics model and hypothesis to be fit to the data. We want our model interpretation in terms of the feature sets selected by the wrapper method to include such higher order interactions between the features. Backward elimination is a simple wrapper method which deals with such situations. It starts with all the features, defined as the initial feature subset to start the search algorithm. Then similar to forward selection iterations, it removes one feature at a time. It terminates when none of the generated feature subsets improve upon the prediction accuracy or performance of the analytics model. By the end of the termination of the search algorithm, we can find the best feature subset over all possible feature subsets and we wish to do this in a manner which is not exhaustive and computationally expensive. Recursive feature elimination is an algorithm that allows that, where for each subset of size from the subset containing all the features, we calculate the model performance by training the model on a cross-validation dataset and reporting its performance on a test dataset. The performance is indicative of the importance of the feature subset in the search algorithm. We keep recalculating the performance for each feature subset until the search algorithm converges across iterations. After the algorithm converges, we have performance evaluation measures for every feature subset and the best fitting model for the selected feature subsets. The search proceeds backward through the search space by removing feature subsets one feature at a time or multiple features at a time. Such a search organization of the search space is needed to design computationally efficient strategies which are not exhaustively searching through the feature sets. However, there is no guarantee that the optimal feature subset will be found. To search through the various feature subsets effectively and efficiently, we can design compound operators in the backward elimination procedure that lead to new feature subsets from the current feature subsets in an efficient manner, where instead of removing every feature and evaluating the model on the resultant feature subsets, we design operators that capture higher-order interactions between the features in the search algorithm. This can be understood with this diagram, where starting with all features, we are searching through the relevant features to find strongly relevant features quickly, using compound operators that are able to skip through regions of the search space instead of searching one feature at a time. Similar to the forward selection procedure, irrelevant features are not considered in the search algorithm. The relevant features are either weakly relevant or strongly relevant, and the dotted arrows show a compound operator compared to solid arrows which show the search organization by removing one feature at a time. By the end of the search, there will be no more features left to be searched. Compound operators are applied on the current set of features such that the features that are farther away in the search state space are found quickly in the search algorithm. The evaluation of the search algorithm continues so long as compounded states increase the accuracy of the analytics model. We can plot a search path through the search space through the discovery of search states in the algorithm. However, the algorithm is sensitive to context-sensitive features that may require probabilistic reasoning in the compound operators, and it is able to remove features that cause least change between the performance of the analytics model with respect to the underlying data distribution of the selected feature subsets. There are several variations of the wrapper methods that combine both forward selection and backward elimination such that additional features searched in the algorithm can be optimized for the target concepts to be learnt beyond what are already found. High-performance ML models would be required to evaluate such

hybridized wrapper methods. Therefore, we have seen an introduction to backward elimination and contrasted it with forward selection in terms of the search strategies used in the algorithm and understood the use of compound operators to organize the search space and seen a variation of backward elimination which combines with forward selection in high-performance models. Thank you.

Embedded Methods Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video, we look at the embedded methods that are an advancement on filter and wrapper methods. We will understand the feature selection procedure in embedded methods and contrast them with those in filter and wrapper methods. In embedded methods, the usefulness of feature subsets is included into the algorithm building the analytics model, and each embedded method is specific to the particular algorithm which is building the analytics model for the task at hand. And the solution to the model, in terms of the target concepts being learned, often has relevant information about the embedded method to be chosen. In this slide, we see a diagram of embedded method, where a feature subset is included into the algorithm used for building the analytics model, and the feature subset is subsequently selected in the algorithm if there is no performance degradation in the model. A variety of data distributions, feature subsets, optimization objectives, target concepts, and learning machines can be included into embedded method. Embedded methods are an efficient extension to the wrapper methods such as those used for non-linear correlation analysis, and they are useful as feature selection methods in data analytics. They find a function that represents prior knowledge about the statistically good model in complex data. They are useful for performing analysis of the analytics model with respect to changes of the output of the model on the input, called sensitivity analysis. Sensitivity analysis studies the effect on the dependent variable or the output from the model based on changes to the independent variables. Statistical methods use all the elements shown in this diagram, where feature subsets are generated according to the analytics model being built for the target concept, their relevance is evaluated on different criteria, according to the performance of the model in a statistical environment. Some of the common embedded methods are listed in these slides. Previously studied techniques such as mutual information and correlation analysis can also be extended into analytics methods. High performance black box models for data analytics such as neural networks and support vector machines can also be included into the embedded methods. Statistical testing and rule-based methods are also another possibility. In embedded methods, an iterative algorithm is built such that a small number of features are removed in each iteration. Compared to the wrapper methods, a set of features is removed after each iteration rather than one feature and any features which are useful for the data analytics task are given more weight to create a pruned set of features that are suitable for the modeling. The weights are estimated again and again in the iteration until all relevant features are found. The relevant features are responsible for generating a model that works not only on the original dataset given to us as training data sample but also generalizes to datasets which were not available during training time but will be available during test time. The relationship between the performance of these embedded methods on various datasets such as the training set, test set and validation set are evaluated in embedded methods to find the best method that has the least complexity. Thus, in this video, we have seen that embedded methods are useful for

evaluating the relevance of feature sets in the data analytics process and they can be used to search through the feature sets which give the best analytics model and they are tightly coupled with the model in cross-validation process. Embedded methods are less computationally intensive than wrapper methods and they can be integrated into the model selection procedure of the target concept. They can be optimized according to different interaction effects required in the dependency structure of the features. Thank you.

Mutual Information Video• . Duration: 10 minutes

[Notes]

Hello everyone, in this video we look at mutual information as an example of embedded method. By the end of this video, we will have seen an illustration of embedded methods and have introduced to the computing mechanisms involved in such an illustration. We would have understood the concept of entropy. Entropy is an information theoretic measure to quantify dependencies between multiple variables across features in the embedded method. The ranking criteria for dependency structures between features is the Shannon definition of entropy that represents the uncertainty or information content in a target concept with respect to the input present in the analytics model. For observed input x , the entropy conditioned on x for y is defined as the joint probability of x and y occurring together in the data and the estimated probability of y given x has been observed in the data. By observing a variable, we reduce the uncertainty in the output from the analytics model in the target concept. The decrease in uncertainty is the mutual information, which is defined in terms of H_y and H_y given x , whose equations are given in the previous slides. If x and y are independent, then mutual information will be greater than 0 if x and y are dependent. Dependency implies that information about one feature affects the information about the other. We replace the summations in the previous definition with integrals for continuous data and this leads to the Kullback-Leibler divergence between two densities. To estimate mutual information, we require notion of dependence and independence in the data with respect to ability to discriminate with the target concepts. This is represented using probability density functions on the output and input given to the analytics model based on the available data samples to do the training of the model. The mutual information can be interpreted as a feature ranking measure and such algorithms can be devised over a fitness landscape of mutual information, similar to what was done in the wrapper methods. Maximizing mutual information between features finds the most relevant features for the target concept and minimizing mutual information between features finds the best features for the target concept. This can be formulated as a search algorithm. The search algorithm will use an objective function such as this, which is the mutual information between y and x , to design an embedded method that finds the interfeature dependency. We will illustrate the search algorithm with an example. Suppose we are analyzing a data set consisting of houses, where we want to know whether the house is owned by somebody or rented by somebody, relationship between this fact and their annual income and marital status. We can imagine a scenario where a bank would analyze such data set to decide whether to give a loan or to not give a loan to such a homeowner. We can see that all components of the data set are categorical values and based on this we wish to calculate the mutual information. Out of 10 records which are shown in the previous slide, 7 records correspond to a defaulted borrower, that is defaulted borrower is equal to yes corresponds to 3 records and 7 records correspond to defaulted borrower is equal to no. For such a data set, the

marital status over 10 records can take single, married or divorced. Now in such a case, we wish to calculate the probability that the defaulted borrower is equal to yes, that is given as 3 by 10, because 3 records correspond to defaulted borrower is equal to yes out of total of 10 records. Similarly probability of defaulted borrower is equal to no is 7 by 10. The entropy for defaulted borrower is $-\frac{3}{10} \log_2 \frac{3}{10} - \frac{7}{10} \log_2 \frac{7}{10}$ which computes to minus 0.265. Now to partition the data set further based on entropy criteria, we check for different conditions of marital status. That is, do we check for the combination of marital status is equal to married as opposed to the combination of marital status is equal to married, divorced or do we check for combination of marital status is equal to single as opposed to marital status is equal to single and married. In this way, we create several subsets to be searched by the embedded method. For each subset, we then calculate the probability of defaulted borrower is equal to yes. This gives us mutual information compared to including marital status is equal to married with respect to marital status is equal to single, married, divorced. We can see that the mutual information computes to minus 0.021. Similarly in the other case of marital status is equal to married, divorced, the mutual information computes to minus 0.541. I encourage you to verify these calculations. Now to compute mutual information and make a decision on whether there is a relationship between x and y, we check for another attribute homeowner. Earlier we were checking for the relationship between defaulted borrower marital status. Now we want to check for the relationship between homeowner and defaulted borrower. Using similar computations, we find that the mutual information is 0.021, which is more than the computation in the previous case where the mutual information was minus 0.244 and 0.276. Using these two split conditions on the marital status, we select the condition of 0.276 where there is an increase in the mutual information. Now comparatively, that value is compared with mutual information found from dependence between homeowner and defaulted borrower. The optimization selects the feature marital status is equal to married given features homeowner is equal to yes and homeowner is equal to no are selected with optimized mutual information which is greater than the entropy value minus 0.265 associated with marital status equal to single, married and divorced. Because entropy models are driven by the data and estimate a likelihood and statistics based on the likelihood from the data that are representative of the uncertainty or surprise in terms of the expectations of the underlying data distributions. They give minimum information needed to encode the data in terms of its probability distribution among various features. They assume what is known in the data and do not assume what is unknown in the system generating the data. Uniform probability of selecting the features leads to a condition of high entropy and adding constraints to such a situation raises the likelihood of the data by bringing the joint probability estimation between the X and Y closer to the uniform distribution which is considered to be the ideal distribution. Several entropy based information theoretic measures are available in the research literature and they quantify the multivariate dependencies between multiple features involved in the analytics modeling in terms of their underlying joint data distributions. Comparative statistics include the chi-square statistic which is an inferential statistic on the goodness of fit for data that has been discretized. Thus we have seen the mutual information technique to build embedded methods for feature selection and we have understood the concept of entropy and mutual information in a given data set. Thank you.

Covariance Analysis Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video we look at the feature extraction procedure in feature engineering. By the end of the video, you would have been introduced to the feature extraction techniques in feature engineering, understood covariance analysis as an example of feature extraction, understood the difference between covariance analysis and correlation analysis and regression in general. Feature extraction objective is to find the most compact and informative feature set in the dataset to improve the processing time and computational efficiency of the data analytics. The data is assumed to be stored as feature vectors suitable for a pattern recognition system. Features which are extracted then encode the problem-specific knowledge in structured, unstructured, polystructured, multistructured big data. Applications of feature extraction are in internet, marketing, supply chains, financial markets, biotechnology, etc. We can decompose the steps in feature extraction into feature construction and feature selection. Feature construction is the transformation of the given input features to generate a new set of features useful in the improvement of model performance. Feature construction leads to concise and accurate models. It reduces the data dimensionality and provides an interpretable understanding of the underlying model and the target concept. Some of the feature construction techniques are given in this slide. These include the feature subset selection procedures seen in filter wrapper and embedded methods. We wish to create such procedures with respect to a utility criteria for the end user with a minimum of user intervention required in the algorithm constructing the features. The algorithm must be computationally efficient and should lead to models which generalize beyond the training data available for the analytics. Some of the motivations for feature extraction and its extension into feature selection are to find relevant and informative features, to reduce the dataset, to increase understanding about the data and reverse engineering of the pattern recognition system underlying the data, reduction of mislabeled examples in the data, and improvement of filter wrapper methods. To perform feature extraction, we require a feature subset generation algorithm, an evaluation criteria, and an estimation of the evaluation criteria in the given dataset. We illustrate these concepts using covariance analysis, where while correlation tells us the dependence between one feature and another, covariance tells us the variation of one feature and another. Sample covariance between two features x and y having standard deviation sdx and sdy is defined as follows, where the mean value of a feature x is subtracted from each value of the feature x , correspondingly similar calculation is done for a feature y and it is divided by the size of the data for every combination of values in x and y , leading to a matrix of covariances. The sign of the values in the matrix indicates whether the variation in one feature is in the same direction as another feature, whereas the magnitude of the covariances indicates the proportionality of such variation. Covariance is then defined as the expected value of the joint change in the variance of two features, averaged over the entire training set. A zero value for covariance says that both features x and y are independent, and correlation can be interpreted as a normalized covariance. The covariance matrix equation is given in this slide. The definition from probability theory is also given. For every i and j corresponding to features x_i and x_j , the covariance is computed, resulting in a square symmetric matrix between all features in the training set. Suppose the training set is the human faces set here, which is showing various human emotions in the images. The covariance matrix for this set can be written in terms of the data matrix, representing every image as a vector in the matrix. The dot product of the data matrix with itself gives the covariance matrix. The off-diagonal elements in the covariance matrix correspond to most-variant, least-variant or redundant features in the data. These features are found using several techniques on the covariance matrix, such as the most popular principal component analysis, extending into discriminant analysis and regression modeling. Thus, we have seen the difference

between feature extraction and feature selection. as a stepping stone for feature extraction and we have been introduced to the eigenfaces dataset for pattern recognition. Thank you.

Principal Component Analysis Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video, in continuation to the lesson on feature extraction, we study the principal component analysis and see an application on the human faces dataset. We have already seen an introduction to covariance analysis. Now we will perform a matrix decomposition of this covariance matrix to form the eigenvectors and visualize them for the human faces dataset. We have seen that the feature extraction method reduces the dimensionality of the data and describes the most important feature vectors in the data. In principal component analysis, these feature vectors are orthogonal vectors that best represent the data. The input data can be expressed in terms of these orthogonal vectors as a linear combination of the vectors. Such vectors are called principal components. The principal components are sorted in decreasing order of their significance to obtain a reduced representation of the original data. Thus, principal component analysis PCA can be thought of as a data projection method that compresses the original data to a much smaller feature subspace. For example, we can see on the left-hand side, there is an image that has been projected to a new feature space. The reconstructed image is having more clarity. Thus, principal component analysis can be used for different stages of the data analysis. The feature subspace in principal component analysis is determined by the matrix decomposition technique called eigen decomposition. The decomposition is done over the covariance matrix. Matrix and in general tensor decompositions are a way to reduce the covariance matrix into its constituent parts. The eigenvectors are the constituent parts in PCA and they are projections of the data along axes which maximize the variance of the projected data. They can be used to produce a sorted ranking of the most important components in the original data according to the eigenvalues corresponding to the eigenvectors. Once eigenvectors are found, we can project the original data into a feature subspace via matrix multiplication. For a square covariance matrix σ , the eigenvectors w are found by solving the equation given in this slide. This equation is called the eigenvalue equation, where λ are the eigenvalues and w are the eigenvectors and σ is the covariance matrix derived from the data matrix. Eigenvalue equation can be solved as a system of equations. It can also be solved as a regression problem. The eigenvalues and their corresponding vectors for the human faces dataset are visualized in this slide. The images which are shown in this slide form the basis vectors for the original dataset, so that a linear combination of these images is useful to reconstruct every image in the original dataset. The original dataset and its covariance matrix is reconstructed from the eigenvectors according to the equation given in this slide, where W is the matrix consisting of all the eigenvectors and W^{-1} is its inverse. Original matrix in the middle consists of their corresponding eigenvalues. The importance of the eigenvectors in reconstructing the original data is given by their eigenvalues. The eigen decomposition is carried out with an optimization algorithm solving for the eigenvalue equation and this can be solved as a system of equations or as a regression problem. I encourage you to refer to python packages to see how to arrive at a solution for the eigenvectors. The PCA then learns data projections from the original dataset and its covariance matrix to the eigenvectors to model higher-order statistical dependencies in the eigenvectors. Eigenvalues give the magnitude of each eigenvector by scaling them in the direction

of the eigenvectors. The combination of eigenvectors and eigenvalues can help us understand important, redundant and noisy features in the training data. The resulting data points from projecting the data on the eigenvector matrix resulting in the matrix V can be said to be a good representation of the original data that is able to separate the most variant features in the data apart from the noisy features in the data. The set of eigenvectors in terms of linear algebra forms an orthonormal linear basis for the dataset and each element of the projected matrix is obtained by taking a dot product between the training data points and the eigenvectors matrix. The resultant projected data matrix gives us a reduced dimensional representation of the data. PCA can be used to visualize high-dimensional data, find hidden relationships in the data, and compress large datasets at reduced computational complexity. There are many variations of PCA that account for noise, latent variables, and predicted target concepts in the covariance calculation. Ideally, we want to minimize the error resulting from subtracting the original matrix and the projected matrix leading to dimensionality reduction, outlier detection, denoising, image reconstruction, and other applications of PCA. Thus, we have seen an introduction to PCA and the application of PCA on human faces dataset where the human faces were described in the previous video and their eigenvectors were described in this video. The solving of the eigenequation has been introduced and software packages such as in Python will be used to find the eigenvectors. Thank you.

Applications Video• . Duration: 5 minutes

[Notes]

Hello, everyone. In this video, we will look at applications of linear regression. By the end of the video, learners will be introduced to the uses of regression in real world and be able to formulate data analytics problems based on regression. Regression models are useful for hypothesis testing in statistics where a presupposition about the data can be tested to be true or false. The strength and consistency of correlation's target concept is quantified by regression models. Most popular uses of regression are; data fitting, functional approximation, forecasting, and predictive modeling. Regression fits into the general data mining functionalities used for analysis of the given dataset, such as optimization, classification, and correlation listed in this slide. The patterns found by regression must satisfy the conditions listed in this slide, such as they must validate a hypothesis must be easily understood, should be useful in an application domain, and must be novel or previously unknown. The quality of regression can be measured with pattern interestingness measures which can be subjective or objective. These act as additional constraints in the optimization of the regression model. The constraints for optimizing regression can be categorized into the taxonomy shown in this slide, such as data constraints, dimensionality constraints, rule constraints, and knowledge constraints. Regression has several applications, both in science and business. In scientific research, populations underlying data distributions can be quantified by regression in domains such as biology, ecology, physics, and chemistry. In industrial process control and business process automation, regression can be used to identify patterns in time series to forecast future behavior. In commercial applications of regression, we can predict the response to an event quantified by causal relationships, such as in the drug discovery process to predict whether the drug is effective on the population from clinical trials. Regression has several other applications, such as in agriculture, where agriculture field can be discretized into components such as shown in this figure to interpolate quantities of interest in agriculture from one set of components to another.

The data sources for such an application include satellite imagery, both hyperspectral and multispectral data, geographic information systems, and sensors planted in the field to collect data. The regression models can be built for farm level or plant level analysis of the crop yield to improve efficiency of crop production using accurate plant monitoring, monitoring plant diseases, and using sustainable farming practices. For such an application scenario, the questions for modeling in a regression problem are; what are the correlations between data attributes that affect the yield, whether there is an influence of fertilization on the yield, and what is the effect of photosynthesis creating chlorophyll in plant with respect to yield prediction. Regression has application in financial markets where stock market indices such as BSE, NIFTY, S&P, SENSEX, NYSE can be used to obtain time series data on which auto regression can be used to make forecasts about prices, volumes, and other information about the trades conducted on the stock market using forecasting methods based on regression. Some of the application areas for regression in this context include price prediction, risk management, hedging strategy, portfolio management, etc. The modeling question in this context would be how to predict stock exchange prices in a business investment plan using regression. What is the effect of hedging premiums on firm value? The data is prepared for regression using formulations such as in this figure, where we can select suitable representation of the data, which represents the price volume information used for forecasting. Also finds complex structures in the data with respect to correlations across multiple attributes. Corresponding machine learning models would involve more and more sophistication, going from the left to right in the figure. Thus, we have surveyed the applications of regression in agriculture and finance in this video where the uses of regression, motivating the need for data mining models has been discussed. One has to identify a dataset for a given application domain to define a regression problem. Thank you.

Simple Linear Regression Video• . Duration: 2 minutes

[Notes]

Hello everyone, in this video, we will look at simple linear regression to obtain an intuition about the working of linear regression and use it to find overall trend in the data. We will look at model fitting criteria to formulate a regression equation. Linear regression is useful for quantifying the mathematical relationship between attributes and distributions in the data. It is useful for making predictions on the dependent variables using a model that fits to the data attributes. Most popular attributes used in regression are numeric attributes. They define the relationship between independent and dependent variables in the data such that linear regression fits a straight line in multiple dimensions over the given data set. The slope-intercept format of basic algebra is used to create the straight line, such as $y_i = \sum \beta x_i + \epsilon$, where β are the coefficients of regression to be estimated and they act as intercepts and slope coefficients in a geometric representation of the straight line representing the linear regression equation. Using test data, we can predict the future value of output y given information about the observations of x . Thus a straight line is formed by regression to fit a scatter of points. This inevitably leads to error where the fitted point is only an approximation of the real point. We seek to minimize several such errors across all the data points. Using linear regression, non-linear data can be mapped to a linear format, however, it is at the expense of obtaining more data and linear regression is not sensitive to duplicate and correlated features in the data. Linear regression does not identify irrelevant features in the data to be removed for further analysis. It is sensitive to the type of the data, it works best for

numeric data and not for categorical data. It cannot identify non-linear structures underlying the data. Therefore, we have seen an introduction to linear regression in this video, where the underlying structure of data can be meaningfully interpreted in terms of the regression coefficients. Thank you.

Ordinary Least Squares Regression Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video, we will look at the ordinary least squares regression model. By the end of this video, we would have understood the error function to be optimized in the regression model, adjusted the parameter estimates in the regression model to minimize the error function using an optimization algorithm that is iterative. The error function is the difference between the predicted and the actual value available for regression. We would like to minimize the error function such that for the given data set, the regression model is the best fitting model. The coefficients for regression are estimated for each feature in the ordinary least squares regression model. Each feature has a separate effect on the predicted value of the regression output. For a given feature set x_0 to x_3 , OLS regression predicts a value y by estimating values β_0 to β_3 corresponding to x_0 to x_3 , such that the error resulting from the prediction is minimum. This can be expressed using the equation $y = x\beta + e$, where x and y are matrix and vector representations of the data, and e is the error. Regression coefficients β are solved by minimizing the error across all the records in the data, called the sum of squared error. This term acts as an objective function for finding the optimal solution to the regression problem. There are several packages available in Python to solve for this objective function using matrix algebra. Now we will discuss the pros and cons of using OLS regression. OLS regression makes strong assumptions about the data and does not work with missing data. It has a linear relationship between the dependent and independent variables and works best for numeric data. It requires us to provide independent features that add new information to the dependent variable being predicted, and they are expected to represent the underlying population being modeled. The squared errors used in OLS regression can be thought of as a measure of goodness or lack of fit of the regression model to the given data set. Thus, in this video, we have looked at the OLS regression and ways to estimate the coefficients of the regression such that the target concept or dependent variable is predicted accurately without being influenced by unusual observations in the data to find overall trend in it. Thank you.

Linear Models Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video we will look at linear models for regression as contrasted with non-linear models for regression. By the end of this video, we should be able to formulate a regression problem given a dataset and fit a linear regression model to the dataset. Use several kinds of regularization in the regression to optimize the regression model. In regression, the input and output variables are numeric. The goal of regression is to minimize the squared error of the prediction of these variable values. The input variables are referred to as explanatory variables or predictor variables or independent variables or regressors, whereas the output variable is referred to as response variable, dependent variable or regress. The prediction process in regression is to build

a model relating the output with the input. We commonly pre-process the data to ensure that the scale of the different attributes in the input are the same to get weight estimates from the regression that are comparable. Thus, we construct a data matrix D of size n by d , where n is the number of instances in the dataset and d is the dimensionality of the dataset. In such a dataset, let x_i correspond to a feature vector representing one of the independent variables. Let y_i be the response variable. Then the regression equation is expressed as $y_i = w^T x_i$, where matrix multiplication is being performed. w is a d -dimensional row vector and weights w must be learnt to fit the regression model to the data. In linear regression, the fit is a straight line across a scatter of points where error is inevitable. To address such errors, we can look into nonlinear regression, where there is a curve being fit to the scatter of points rather than a straight line. In both cases, the objective is to minimize the error of regression, which acts as an objective function given as O in this slide, such that gradient descent algorithm can be applied on the objective function to find the weights w that give the best fitting regression model. The gradient of the regression equation or the inverse of the data matrix multiplied with itself are both useful to find the weights w that fit best to the data. Note that DTD^T is a symmetric matrix and its inverse must be found using the Mohr-Penrose generalized inverse formulation of finding the inverse of any given matrix. Such a formulation is solved using several data analytics techniques such as principal component analysis, singular value decomposition, and novelty detection filters, where the matrix DTD^T is factorized into component matrices such as Q and Λ . The columns of Q are orthonormal eigenvectors if the factorization is based on singular value decomposition, whereas Λ contains the eigenvalues. Reducing the rank of the matrix DTD^T can be achieved by reducing the dimensions of Q . For problems where there is a wide variation in the weight values represented in the matrix w , we add regularization terms such as λw^2 to O , the objective function for regression, such that the solution based on the inverse of the matrix is expressed as $(DTD^T + \lambda I)^{-1} D^T Y$, where I is the identity matrix. $DTD^T + \lambda I$ is a positive definite matrix whose inverse always exists. Regularization is useful for preventing overfitting in the data, where the model works very well on training set but not on test set. At the same time, it helps in reducing the regression error for different data instances while providing a score for each feature contributing to the prediction in the regression model. In this video, we have looked at linear regression models in general and ways to obtain best fitting regression model for the given dataset by formulating the objective function for the models and further optimizing them using regularization. Thank you.

Gradient Descent Video • Duration: 7 minutes

[Notes]

Hello everyone, in this video we will look at the gradient descent optimization algorithm to find the best fitting regression model to the given dataset. By the end of the video, you would have understood the gradient descent algorithm, visualized the convergence properties of the algorithm and understood the importance of the various hyperparameters used in the algorithm, such as the learning rate. Gradient descent is an optimization algorithm. It requires the definition of an objective function to measure the model performance. In the case of regression, the objective is the error function matching the predicted output with the actual output. It is given by the equation $\sum (y - \hat{y})^2$ and the squared error is given by the norm of this equation. The goal of gradient descent in this case is then to find w that minimizes the function O . This is achieved by calculating the gradient of function O .

over the w space rather than the d space. This is a figure representing the surface of the error that is to be optimized by gradient descent in the regression model. Let us say a random point on the x -axis is selected to start the search for the minimum point at the lowest position in the figure. Then the gradient of the function at the random starting point gives the direction in which the minimum is present from the current point. This direction is found repeatedly across several iterations and we would have to decide not only the direction but also the magnitude of the step in the direction to be taken to obtain the minimum point. If the random starting point is closest to the actual minimum point, then the search time to find the minimum point is reduced. Otherwise, several iterations are required to arrive at the minimum point. Thus, in the case of regression, for each data point, the value of weights to be estimated in the regression model are updated by using the derivative represented with delta in this equation. Delta is based on the partial derivative of the error with respect to every weight which is being optimized in the regression model, where η is the learning rate and it is a constant. Thus, the learning algorithm in our case consists of adjusting w 's for every input record until the local optimum is reached. The equation to update each w is called the update rule and the algorithm to update each w is the gradient descent algorithm because it is descending towards the minimum point in the data set. The gradient descent algorithm can be described in terms of these steps where the change to the weight for every weight corresponding to each feature in the data set is proportional to the partial derivative of the error function with respect to that weight. The magnitude of the step in the direction opposite to the gradient direction is determined by a line search procedure which is either exact line search or backtracking line search. Once the magnitude and the direction are decided, we know the update to each w until a stopping condition is reached where the regression error is minimized. The same can be visualized using contour plots that are a projection of three-dimensional error surface on two-dimensional figure where the search starts from a point randomly selected to minimize the regression error and it proceeds in a zigzag manner to the optimum point where the zigzag is happening across the contour plots having the same value for the error on all parts of the dashed curve moving from one curve to another. The constant η also has an impact on the rate of convergence of gradient descent to the minimum value. For a small η , small steps are taken to the final minimum value. For a large η , large steps are taken. Depending on how the error landscape is shaped, whether there is a single optimum point or multiple optimum points connected by a flat surface, the value of η has an impact on the runtime of the gradient descent algorithm where in the figure d for example, if we start with a position which is on the top left, we would take a long time to converge to the final solution that is actually present here but then the algorithm terminates here because immediately after the local optimum is reached, there is a flat surface which is making it difficult for the algorithm to find the next starting position for the search. Gradient descent is an iterative algorithm. It requires the objective function and its derivative such that the gradient which is the set of all partial derivatives of the objective function with respect to the weights leads to the direction of steepest decrease in the regression error. Gradient descent across several iterations converges to an approximation of the target function sought by regression. However, it may overstep the minimum regression value and keep searching for additional iterations until convergence. Thus, the key idea to use gradient descent in regression is to make the hypothesis space of possible weight vectors that produce the best fitting regression model to the data. Gradient descent is the basis for several other learning algorithms that search through parameterized hypothesis spaces building on the ideas of regression. In this video, we have looked at the concept of gradient of a function as a partial derivative of a function. We have used the gradient descent algorithm to minimize the regression error and update the weights of the

regression using the minimization criteria produced on the error surface. Experimental evaluation of the optimization solutions produced is necessary to find the best hyperparameter values such as the learning rate for gradient descent. Thank you.

Evaluation Metrics Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video, we will look at performance metrics to evaluate a regression built for the dataset. By the end of this video, we would have reviewed the model performance evaluation metrics useful in regression modeling, found the best fitting regression model among several models and reported the performance of the regression which is best fitting. Regression can be used in several kinds of predictive models for different input datasets, be it image, text, graph, sequence or data stream. For time series input, regression is used for forecasting and regression is a major tool in data mining and data analytics. It is based in statistical analysis of the data and it can produce prediction as well as explanation associated with the prediction. The typical assumption in regression is that the distribution of the residuals or errors from the predicted values of regression follows a normal distribution such that no error is more important than any other error. This is shown with the frequency versus residual plot where residual is the difference between the predicted value and the actual value found for regression. The same plot can be analyzed across other dimensions such as percentage versus residual plot, residual versus fitted value plot, residual versus observed order of the input records plot. In all these plots, we can see the variation in the regression errors and we expect that the variation follows a normal distribution. Under such a condition, we can define several statistical measures for evaluating the performance of regression to check its ability to predict future observations based on current modeling of the weights that fit best to the data. The performance or the accuracy of regression is modeled using the out-of-sample error such that the performance on the new dataset which comes in the future is to match with the performance of the model found for the given dataset presently available with us. Here, residual is defined as the difference between the predicted value and the observed value of regression for each input record. The residual can be expressed using this equation where minimizing the residuals leads to a good model for the regression. There are several error measures to define the difference between the predicted value and the observed value in regression, some of which are listed in this slide such as the sum of squared estimate, root mean squared error and mean absolute deviation. Sum of squared estimate SSE is evaluated on a separate dataset different from the one used to build the regression model and it can be used to discriminate between several regression models. It is defined in terms of the error or residual produced from subtracting the predicted value xw from the observed value y_i . A squared term is introduced to not allow negative values for error to affect the summation term. Thus, SSE is a measure of the goodness of fit of the regression model to the data. Lower SSE means there is lesser error in the regression and it indicates a better fit of the regression model to the dataset. Any unexplained variance in the errors and residuals of SSE leads us to formulate the need for a more sophisticated regression model having more complex RMSE is derived from SSE where mean squared error is SSE divided by n where n is the number of points in the dataset. RMSE is then the square root of MSE and it measures the variability in the regression errors. A large non-zero RMSE indicates bias in the predictions of the regression model because the underlying data distributions input to the model may have changed, however, the data attributes or

feature representations of these data distributions have not been able to track such a change leading to biased predictions in the regression values being generated. Mean absolute deviation MAD is also defined in terms of the norm of the error or the absolute value of the error divided by number of points in the dataset. Similar to RMSE, MAD is a measure of the standard deviation in the regression errors where the regression errors or their residuals are assumed to follow a normal probability distribution. If there is any deviation from this assumption, this is indicative of the fact that our regression model must be refined using more sophisticated parameters. Here we see a % vs error plot for the fitted data represented by a straight line. Any deviation from this is the residual in the regression found for each record where residual is calculated by subtracting the observed value from the predicted value. The distribution of these residuals across given data and future data will form the error distribution in the regression. In this video, we have seen the performance evaluation criteria of regression models and defined three different metrics to be reported in the experimental evaluation of regression. We have also visualized the regression residuals and errors to formulate the need for more sophisticated regression models. Thank you.

Model Selection in Regression Video• . Duration: 5 minutes

[Notes]

Hello everyone. In this video, we will look into model selection in regression. We will be looking at the concept of overfitting and how the best fitting model must be selected to find regression that is able to generalize to out of sample data. And we will look at some of the recommendations provided by information theory to measure the best fitting model. The model will be evaluated using the R squared statistic. Concentrating too much on model that produces best historical fit leads to overfitting. The best approach is to select the results having the smallest standard deviation or mean squared error in out of sample predictions, which are about the data coming in the future based on predictions found in the data currently available with us. We measure the out of sample performance for model fitting by several performance metrics and include this into the cross-validation procedures used to improve the regression. The R squared statistic is defined in this slide as the division of the residual sum of squares in the numerator with the total sum of squares in the denominator. Maximizing the R squared statistic is equivalent to minimizing the sum of squared errors. Larger values of R squared thus suggest a good fit to the data. As R squared always decreases when new features are added, R squared tends to encourage overfitting, leading to more than necessary features and parameters in the model, resulting in a very complex model. Thus a large value of R squared does not ensure a best value for the out of sample regression error. There are variations of R squared, such as the adjusted R squared and the Akaike information criteria and Bayesian information criteria. Their definitions are given in the slides here, where AIC and BIC penalize the sum of squared residuals that lead to addition of unnecessary parameters. Thus models that have small values of AIC and BIC are good fitting models. An asymptotically efficient model selection procedure for regression selects the sequence of regression models such that they approach the likelihood function estimated in the data, which is taken to be the true model in data in information theory. Thus based on information theoretic criteria, we should ensure that the regression model is not very complex. It is the smallest model that best fits the data. It is a model that is consistent with observations in the data and it is able to remove redundant and irrelevant features in the selection procedure. Redundant features may be due to noise or hard to predict

records, which lead to an increase in the error. At the same time, we seek to reduce the computational complexity of the optimization algorithm, understanding the parameters for regression and the cross-validation procedures seeking to reduce the regression error. This can be represented by this figure, where each person can be thought of as a model and the whole shape of the elephant is obtained by combining the predictions of the model, where the elephant represents some unknown data distribution represented in the data sample input to the model. These ideas from information theory are the basis for ensemble modeling. About the regression is more accurate than random guessing and correct decisions are reinforced in the algorithm shown on the right side, where training data set is partitioned into several different data sets input to multiple regression models and their predictions are combined using a combination model, which is taken to be the final model for prediction. These ensemble techniques are able to address the various information theoretic ideas discussed in the previous slide. The theoretical basis for these ideas is the no-free-lunch theorem that says that there is no one regression algorithm that is the most accurate, no matter how sophisticated it is or how much data is input to it. This implies that we can expect some kind of a variance in the performance of the regression and it is a good idea to combine the results of multiple regression models to get the best model, such that the prediction and the variance in the prediction is addressed by the best model, taken to be an ensemble model in this diagram. In this video, we have seen an introduction to model selection and the need for it in finding the best fitting regression model. These ideas must be combined for performance evaluation metrics for regression to select the best model for a given data set. We have also looked into information theoretic criteria for model selection. Thank you.

Smoothing Methods Video• . Duration: 5 minutes

[Notes]

Hello everyone. In this video, we will look at smoothing methods to produce nonlinear regression models. Smoothing separates the noise from the signal in the data. And it is useful in regression, at the same time in other data analytics tasks such as feature selection. Smoothing is achieved by binning the data. Binning is a process to binarize the data by discretizing a continuous values associated with the independent variables in regression, such that each bin is able to assign a unique class label to the different components of the data. Binning trade-offs the precision available in a continuous measurement of the independent variable with the discrete measurement available in the trend underlying the variable. Discretization generates a concept hierarchy of several bins. It is useful for qualitative data analysis, where the regression equation expresses the trend in the data using discretized input. In linear regression, the slope and intercept of the regression line are able to express the trend in the data. In nonlinear regression, the trend is expressed in terms of the weights or parameters or coefficients in the regression model. Regression curve, in general, is able to explain the signal or trend in the data. And the variation of the curve is explained by the regression errors. Random variation is seen for unexplained differences between regression prediction and observed value, which is where smoothing the random variation for nonlinear regression results in new features acting as dependency structures used as independent variables in the regression. Many real-world phenomena are nonlinear rather than linear, thus smoothing is an integral part of building regression models. The goal of smoothing is to uncover and highlight patterns present in the data with a minimum of preconceptions and assumptions about the patterns to find the smooth

or signal in the data as opposed to the noise components in the data. This is achieved by reducing the effects of noise in the data by considering several alternative hypotheses and models for regression, such that the rough component or the noise component contains as little dependency structure as possible useful for the regression modeling. The remaining dependency structures useful for regression modeling are obtained by the smoothened dataset using techniques such as binning, correlation analysis, frequent pattern mining, copula, etc. used for the analysis of multivariate probability distributions underlying the regression model. Basic smoothing is called binning and it replaces each observation by a mean of its surrounding observations so as to reduce the noise while smoothing based on the nearby data. The smoothed value is insulated from observations that are far away, leading to a discretization of the dataset. There are several other smoothers available research literature such that the pattern or signal in the data is quantified by the output of such smoothers. They include sampling procedures, grandeur causality, sequential pattern mining, time frequency methods, splines, and so on. They are useful in several statistical models such as parametric models, wavelets, quantiles, analyzing the underlying multivariate probability distributions in the regression model. In all these cases, we assume that the predicted values and their associated residuals and errors follow a normal distribution such that the predicted value is composed of a combination of several basis vectors used in the prediction. If the equation is on the left-hand side plot is smoothened using one of the smoothing techniques, we can see that rough edges in the regression curve are smoothened out to form a smoothened line which is not overfitting to the data. In this video, we have motivated the use of capturing complex nonlinear relationships in data for regression modeling and introduced smoothing as a technique to do this. Thank you.

Regularised Models Video• . Duration: 7 minutes

[Notes]

Hello everyone, in this video, we will look at the use of regularization in nonlinear regression models. By the end of this video, we would have seen the L1 and L2 regularization in regression modeling and understood the role of regularization in robust regression modeling. Regularization methods introduce a penalty function that express prior belief about the smooth behavior of the signal present for regression modeling. Adding such penalty functions to the objective function, which is the error function in regression, prevents overfitting in the data. Regularization parameters can be understood as constraints to model the model complexity and model fit for a given training dataset of limited size. They not only control overfitting, but are also able to filter out the features which are not relevant for the regression modeling. Ideally, we want to produce penalty terms that are able to induce sparsity in the parameter updates. That is, few updates should be able to produce a simpler model and not compromise on the performance of the regression model. Estimating such weights smoothly as a function of features requires us to prepare the dataset using several preprocessing techniques. And it results in improved error performance of the regression model, at the same time reducing the complexity of the model without information loss about the dependent and independent variables in the regression model. The total error function to be optimized can be expressed as given in this slide with the additional term for regularization that governs the complexity of the model in terms of the underlying basis functions and features used in the regression. We would like to produce strategies for the additional term that reduce the test error of the regression on the test dataset without compromising the training performance on the training dataset. This is the basis for

the bias-variance trade-off to study robust regression models. Here robustness is the ability of the regression model to deal with noise in the data, such that uncertainty about the dependency structures used in regression is considered in the regression model. Presently, that is done by adding the λw^2 term, where w^2 is the dot product of the weight vector with itself and it is expressed by this summation. Here λ is a constant that governs the relative importance of the error term as compared to the penalty term in regularization. Thus, we have the total error function to be minimized using gradient descent and the penalty term here is based on the squared of the norm of the weight vector. This is called the L2 regularization penalty, where the remaining aspects of the regression modeling are the same as seen in the previous videos, where non-linear function f of x is being multiplied with the weights w to produce the prediction, which is being subtracted from the actual value to produce the error function to be optimized using the non-linear function f expressed using parameters w . The use of L2 regularization in regression is also called ridge regression and this is one of the non-linear regression models available in the research literature. Normalization of the data to make the various independent attributes or features fit into the same range is useful for getting a good ridge regression model. Several other ideas discussed earlier about feature selection, cross-validation can be included into the ridge regression modeling. If the penalty term is only a norm of the weight vector, then that is the condition of L1 regularization, such that the objective is to remove irrelevant features from the regression modeling that are noisy and dirty. In the sense that they are misspecified, they cannot be separated easily from the signal. The use of L1 regularization in regression is also called lasso regression. Feature regression model tends to produce sparse weights, that is, there are few weight values that are non-zero and most of the values are zero. Only relevant variables having non-zero weight values will be able to produce the regression output. L1 regularization thus produces a sparse solution. It acts as a feature selection procedure and it is further studied in signal processing under the compressed sensing field. Beyond L1 and L2 regularization, we should be able to deal with robustness of the regression model in terms of its ability to deal with noise in the data input. There are several solutions to do this, such as the use of synthetic data to augment the training data to understand the noise structure and separate it from the signal, or injecting noise by design into the regression model so that the weights it learns are able to classify such a noise. Equivalently, regularization terms of different categories can be used to deal with noise. Uncertainty modeling of the variation in the modeling weights using probability theory is also one solution. The end goal of these techniques is to produce a target concept that is useful for regression modeling in spite of quirks in the input dataset. Thus, we would like to encourage regression parameters that are not very influenced by small perturbations of the weights estimated in the regression model, and they are hence insensitive to the variations in the data. Thus, in this video, we have seen the need and role of regularization in regression modeling in particular and data analytics in general. We have looked at the L1 and L2 regularization terms, where nonlinear regression models are built using them, looked at the need for noise robustness in robust regression models, which are nonlinear regression models. Thank you.

Nonlinear Models Video • Duration: 4 minutes

[Notes]

Hello everyone, in this video we will look at the framework of non-linear models for regression and as contrasted with the linear models for regression. By the end of this video, we would have been introduced to the non-linear regression techniques and understood the basis vectors used for non-linear regression, with an example in the k-nearest neighbors algorithm. The basis vectors are functions of the independent variable that are used for regression modeling, such as, instead of multiplying w with x , we multiply it with a function of x , which is non-linear and the resulting $w f(x)$ is the predicted value from the regression, such that it is matching up with the actual value t . There are several possibilities for f and each new f which is introduced into the error function for regression must add additional information to reduce the regression error. Changes in one region of the feature space representing the basis functions would have an effect on the other regions of the feature space. In polynomial regression, where polynomial splines are used as the basis functions, piecewise polynomials shown in the figure are combined to create the regression output. There are several other basis functions possible for the non-linear regression models, such as Gaussian basis functions, where the mean and variance of the Gaussian curve decides the structure of the independent variables in the regression modeling. Sigmoidal functions have a format of exponentials used in the denominator and they vary between 0 and 1. Including the mean and variance of the Gaussian curve, we can pass it through a sigmoid function to form the basis functions. In k-nearest neighbors, the closest samples to a particular value are used for regression. Thus, the neighborhood is considered to be the basis function for every data point used in regression. The distance between the points to decide the neighborhood of a data point is found using the Minkowski distance in general, which is a generalization of the Euclidean distance used in clustering. For two data points consisting of components x_{aj} and x_{bj} , the Minkowski distance is defined in terms of the norm of the Q th power of the difference between the components of the data points. This is done for along every dimension of the records used in finding the neighborhood structures and a Q th root is taken where the Q of the root matches with the Q of the norm. Q is any real number greater than 0. All features x_a and x_b must be in the same range or scale for Minkowski distance to give meaningful results. The k in k-nearest neighbors is a hyperparameter value which is set by the user of the algorithm and we must consider the disadvantages of k-nearest neighbors while building regression model based on that, such as finding the neighborhood of every data point is a time consuming exercise and it requires high computation capacity. The local structure which may be present in the data is not considered in the k-nearest neighbors to remove irrelevant features for finding the neighborhood structure. Noise also has a disproportionate effect on k-nearest neighbors. With respect to these caveats, we have seen how k-nearest neighbors is useful for finding the neighborhood structures that are taken to be the basis vectors for nonlinear regression. There were other basis vectors used for nonlinear regression such as polynomials, piecewise polynomials and sigmoid functions. In all these cases, the error is to be optimized in nonlinear regression using gradient descent on basis vectors that are differentiable. Thank you.

Applications Video • Duration: 5 minutes

[Notes]

Hello everyone. In this video, we will look at several applications of classification. By the end of this video, you would have understood the role of classification in data mining for analyzing several complex data types, and we would have seen an introduction to the frameworks for building

classification models. In customer target marketing, to tag groups of customers which are interested in a particular product according to their previous buying behavior and features from their demographic profiles, we can use classification. For example, the products can be mobile phones and customers may be browsing an e-commerce platform to buy a high-end phone or low-end phone or popular phone, based on different locations in different cities in India. In medical disease management, historical data is available about the patient's medical tests and treatments prescribed by doctors previously. Based on this, we can predict the treatment outcomes using classification models into the future. For document categorization and filtering on the internet, we can identify specific topics of interest in the news portals using classification, where features for classification correspond to words in the document and labels for classification correspond to various topics to be identified. A real-time classification system can be deployed on any news portals using classification models. In web linkage mining, the logical structure of the internet can be mined using classification, where the social and information networks on the internet are the features and the links between the various networks are the classes to be predicted. There are several data types which can be used in classification models. We normally think about the multidimensional data, such as shown in this slide, having several attributes, perhaps obtained from a SQL table, of different data types, such as categorical, numeric, and binary. However, the data types can also be complex graphs, such as shown in this figure, coming from scientific applications, such as chemical compounds and co-expression networks of genes representing graphs. The techniques of classification used for such scientific applications can also be used on the internet datasets, such as social network datasets and program flow datasets found in software engineering. Classification is the basis for predictive analytics, and it maps input variables to output variables, such that the observed input is put through a workflow to come up with the predicted output. This predicted output is to be matched with the actual values available in the data to come up with an error which is determining the merit of the predicted output. If the error is not according to our expectation, we can use further machine learning to optimize the classification model, by using additional objective functions and constraints which are parameterized in the optimization procedure. To come up with a prescriptive output from the predictive output, that can be presented as a recommendation on the decisions produced by the classifier. The overall framework for classification can be understood in this figure, where the data for classification is prepared to build the model for classification. The patterns used for classification are then built using several optimization criteria, resulting in the notion of discrimination as opposed to classification, producing decision support. Any feedback obtained from the end user of classification can be given back to the classification model to make further improvements to it, and new data characteristics can be used to come up with the discrimination models. There are several classifiers available in the research literature, such as logic-based classifiers, for example decision tree, neural network-based classifiers, such as multilayer perceptron, statistical techniques, such as Naive Bayes, instance-based techniques, such as k-nearest neighbors. We will be covering a sample of the various classifiers available in the literature. For every classifier, there is a deep and machine learning counterpart which can be built to make the classifier better, such as rule-based techniques in frequent patterns can be improved with generative modeling in deep learning. In this video, we have surveyed the applications of classification in medicine, marketing, and internet, the use of classification in various data mining models, and formulation of classification problem in a given application domain.

Binary Classification Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video we will look at binary classification to build a classifier. By the end of this video, we would have understood the binary classification framework, evaluated it using confusion matrix and used the k-nearest neighbors algorithm to build the classification model. Binary classification is useful in many different setups where we have to classify each instance of the data to predict whether a particular class label is present or not, such as whether a sad face is present or not in the eigenfaces dataset. We transform the raw data into a training dataset typically consisting of numerical features and then apply parametric methods to build such a model. The parameters in the parametric methods are based on analysis of the underlying probability distribution, whereas by contrast, non-parametric methods analyze representatives of the data points to make the prediction. We would ideally want to separate out the various class labels by using a straight line like in the figure on the left. However, in real-world application, there is a significant overlap across different class labels to be predicted, which leads to several algorithms for classification. In the k-nearest neighbors algorithm applied to the classification setting, we have a test record and we compute the distance between the test record and other training records. Depending on the nearest points to the test record, we take a majority vote of the class label to be assigned to the test record. For example, a duck looks like a swan, a duck does not look like a parrot, a duck does not look like a peacock, so on, so depending on the nearest neighbors to a duck which is a swan and another duck, we classify the test record to be a duck rather than say a peacock. One nearest neighbors where k is 1 is the simplest kind of classifier. It is used to classify two class labels. It is based on distance computation between the data point to be classified and the dataset available in the training examples. For any unknown test data point, the distance between the data point and its neighbors is found to find the closest matches in the neighborhood of the data point and predict the class label to be the same as the majority label in the closest matches. In one class classification, the objective is to find only one class label as opposed to the remaining class labels which are not known. In binary classification, the objective is to find the positive class label as opposed to the negative class label which is separate from it. In positive unlabeled classification, positive data is available but negative data is not available which is taken to be unlabeled data. In positive confidence classification, the probability of predicting the positive label is calculated in the setting of positive unlabeled classification. Thus, k nearest neighbors and its variations can be built for several variations of the binary classification model. For each of these variations, we would like to define what is a true positive. When the predicted class label matches the actual class label, that is taken to be the true positive case, where we would like to get more number of true positives or true negatives where the model is able to predict correctly the positive as well as the negative data in the training dataset, whereas the off-diagonal elements of false positives and false negatives have to be minimized. This leads to several performance metrics related to classification and they can be analyzed by building the confusion matrix for the binary classification case where we are either predicting a positive class or a negative class. Various metrics on true positives, true negatives and so on can be derived from the confusion matrix. Thus, in this video, we have seen the fundamental dogma of classification in binary classification to predict whether a class label is present or not in the dataset and we have seen how the one nearest neighbor and k nearest neighbor algorithms solve this problem. We have also seen an introduction to the confusion matrix used in the evaluation of classification models. Thank you.

Multiclass Classification Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video we will look at multiclass classification, which is a generalization of binary classification. We will also look at the Naive Bayes classifier as an example algorithm to build multiclass classification model. Multiclass classification requires a labeled training set of the form $XICl$, where every record has a corresponding class label. There are k such class labels which are assigned to the dataset, and the model has to predict one of the class labels Cl , where i varies from 1 to k , for a test record XI . In binary classification, by contrast, the class labels are either plus 1 or minus 1. The k -nearest-neighbors algorithm used for binary classification can be extended to multiclass classification. Several other ensemble techniques and black-box models such as support vector machine can also be used for multiclass classification. In multiclass classification, there is no clear boundary between two particular class labels, and there is a smooth change from one class label to another class label. This leads to scenario of multiple class labels. This can be expressed using the probability of a class label being assigned to a record in comparison to the probability of another class label, where X is denoting a single feature of a record. Naive Bayes is an example of a probabilistic classifier that constructs the relationship between the features and the target class as a probability. Naive Bayes models the class prediction probability conditioned on the information of the selected feature set used to make the prediction. This is based on the Bayes theorem, where the predict class conditional probability is on the left-hand side, and it is dependent on the likelihood estimated on the right-hand side. The denominator of the term on the right-hand side is a normalization term, and the multiplication with the class probability is based on prior information which is available in the dataset. Naive Bayes makes the assumption that each feature is independent of the other features in the dataset used to build the classification model. So the conditional probability to be estimated on the right-hand side of the Bayesian theorem is simplified into a product of the probabilities of each feature existing in the dataset given that the corresponding class label exists in the dataset. Because of modeling the probability of the features and their relationship with the classes, Naive Bayes is also referred to as a generative classifier. It is based on the assumption that features are independent of each other and class labels are generated according to a probability distribution following this assumption. There are several other probability distributions which can be included into the Naive Bayes classifier. In all those cases, we wish to estimate the class conditional probability also called the maximum a posteriori probability. This is based on the conditional probabilities of associating the features with the classes in the dataset. The easiest way to associate the probabilities of the features with the classes is to find their relative frequency under the Naive Bayes assumption. Then we choose the prediction of the class label that maximizes the probability. k nearest neighbors can be used for multiclass classification and there are two different scenarios in which it generalizes from binary classification to multiclass classification. We can think of the prediction problem across k classes as decomposing into k binary classification problems where each problem discriminates whether a given class is present in the prediction or not. To be able to achieve this classification model, we require to build k binary classifiers and one test instance will go through all these classifiers to find the best class label predicted by all the classifiers. In the all versus all scenario of k nearest neighbors applied to multiclass classification, each class is compared to every other class. To build k combinatorial two binary classifiers and a test instance with the maximum number of votes across all the classifiers is

assigned the corresponding class label. Thus, in this video, we have seen the generalization of binary classification to multiclass classification. Thank you.

Building Decision Trees - Part 1 Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video, we will look at decision trees, which is a popular rule based model to do classification. By the end of this video, we would have applied a decision tree on a test instance and we would have motivated the training algorithm for building decision trees. This slide shows an example of a decision tree where there are sub trees associated with every node in the tree, on the left side and right side, to form a recursive hierarchical decision structure across the various feature variables arranged across the tree from the root to the leaves of the tree. Following the path from the root to the leaves gives us a rule to test on a given instance across different ranges of the feature values as given in the conditions, splitting the sub trees in the decision tree. The decision tree at a particular node is based on a split condition on one or more feature variables in the training dataset and the goal is to identify those branches or leaves in the tree that reduce the difference between the various records according to the class labels which are assigned to them. So we are trying to produce a tree that involves minimum mixing between different class variables in the tree. Each node in the decision tree represents a subset of the feature space and it is defined by a combination of the split criteria with reference to the nodes above it, leading to the root node. The decision tree induction algorithm starts with the full training dataset represented at the root node, then it recursively partitions the data into the left and right sub trees until the lower level nodes are reached in the tree according to the split criterion. The decision tree algorithm stops according to a stopping criterion, otherwise it stops when all the records found at a particular node belong to the same class and we prefer shallow decision trees to deep decision trees to reconstruct simpler models on the training dataset which induce the same error as more complicated models. For a given categorical dataset such as a homeowner mapped to their annual income and their marital status, to make the prediction whether they will default on a loan or not, the corresponding decision tree has to split on various attributes of the dataset and it has to decide the best attribute such as homeowner based on the split criterion and recursively go down the tree to order other attributes which will split the data into multiple partitions where at the leaf the prediction is either yes or no that is the homeowner will either default or not. Traversing the tree from the root to the leaf leads to a prediction which is available for a test instance. Given a test record such as homeowner is no, marital status is married, annual income is 80,000, we wish to traverse the decision tree along the path formed by the root and the edges leading up to the leaf. Every edge has a label associated with it which is the value of that particular attribute defining the left or right subtree. In our case where homeowner is no, we have to traverse to the right subtree and then check for marital status. Since marital status is married, we traverse further and find that the prediction for whether it is a default borrower or not is no. The path taken from the root to the leaf is affected by the values of the test instance. It is also affected by the attributes on the path. There could be more than one tree that fits the same data. So the classification task in decision tree is to induce or learn the model for the decision tree and to apply the model for deducing the prediction for test records. In this video we have seen an illustrative example of decision tree and we have seen the traversal in decision tree across different subtrees from the root to the leaf. Thank you.

Building Decision Trees - Part 2 Video• . Duration: 2 minutes

[Notes]

Hello everyone, in this video, we continue with the training algorithm for building decision trees. We will look at several recursive algorithms for building the tree. Hunt's algorithm, which is one of the most popular algorithms, will be described. There are several other algorithms available in the literature, such as ID3, C4.5, Cart and Cubist. The general structure of Hunt's algorithm is based on a recursion. Let DT be the training records that reach a node T . Then the algorithm has to partition the records such that they belong to the same class YT , leading to a leaf node. And if there are multiple classes after partitioning, then the recursion has to continue. We take the same example as before, where we have to partition the dataset containing information about whether the borrower has defaulted or not across 10 different records into 3 and 4 records, based on the attribute HomeOwner and its values yes or no. So that if HomeOwner is yes and defaulted is no, then there are 3 records which are present in the dataset satisfying this condition. Otherwise, the remaining records satisfy the condition of HomeOwner is no and defaulted is no. Similarly, once we split on HomeOwner is yes and defaulted is no, then we have reached a leaf node and no further splitting is required. However, when HomeOwner is no and defaulted is no, then we have to split further based on the marital status attribute, such that HomeOwner is no, marital status is married, defaulted is no leads to a leaf node and further on HomeOwner is no, marital status is single or divorced, annual income is less than 80k leaf node and annual income greater than equal to 80k leads to another leaf node. Thus, we have seen the recursion procedure followed by decision tree algorithms and we have looked at the Hunt's algorithm for building the decision tree. Thank you.

Design Issues Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video we will look at some of the issues in the design of the induction algorithms for building decision trees. The main criteria used to conduct the recursion in the decision tree is the split criteria and in this video we will see how a gain calculation is used to decide the splits. The main question is how should training records be split in the recursion and we require testing conditions to express this split condition. Then after the splitting the data we require a way to evaluate the goodness of the resulting partition of the data. We stop splitting if all the records in the partition belong to the same class label or if the sufficient depth in the tree is achieved. Depending on the data type of the attribute used to split the partitions we have several conditions as the split criteria. Some of the most common attributes are binary, numeric, nominal and ordinal. The test condition for nominal attributes such as marital status is single, divorced or married is based on different categorizations of the values which the nominal attribute takes such as married and single, divorced, single, married, divorced, single, divorced and married. Now in these different partitions the question is how to decide the best split condition. We seek to find those splits and partitions that find the same class label assigned to the different records. We can split on one or multiple values of the nominal attribute. In binary split we divide attribute into two different subsets, in multi-way splits into more than two different subsets. In ordinal attributes we preserve the order of the values assigned to the attribute while splitting. In nominal attributes we preserve the order of the attribute

values on which we are splitting. In continuous attributes we require to find a threshold such as 80k to split the attribute on. In multi-way splits we can have multiple thresholds. In binary splits we require one threshold. Before splitting we count the number of records belonging to different class labels then we select an attribute to split on and then we count the number of records belonging to different class labels after splitting. We seek to find those records which belong to one class label and not multiple class labels after splitting. This is called the pure class distribution that is preferred and there are different criteria on the splitting based on a calculation about impurity of a node. For example, records proportionately distributed across different class labels leads to high degree of impurity whereas records belonging to a single class label leads to low degree of impurity. Thus, we compute an impurity measure before and after splitting the data into partitions and this impurity measure is calculated recursively. The gain then from splitting on a particular attribute and its value is the difference between the impurity measure after splitting and before splitting. We are looking for lowest impurity measure after splitting such as illustrated in this diagram where n_{00} n_{01} number of records assigned to classes c_0 and c_1 before splitting n_{10} n_{11} n_{20} n_{21} assigned to the class labels c_0 and c_1 after splitting. The gain is calculated based on these number of records and it is given as $b-a_1$ when the split node 1 is used and $b-a_2$ when the split node 2 is used. Now the question is which of these gains is the best gain? Calculation of the split criteria will be shown in the subsequent videos. In this video, we have seen the recursive procedure used for splitting a decision tree based on a greedy selection of the attribute values. Thank you.

Measures of Impurity - Part 1 Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video we will look at various measures of impurity in splitting the partitions across nodes in a decision tree. We will look at the definitions of the impurity measures and computing procedures. This is a list of the commonly used impurity measures. Any index is based on a probability of a set of records belonging to a class label and it is calculated as the relative frequency of the records belonging to the class label, represented by the value P_{IB} in the equation. The same probability when it is used in the formula for entropy is a different impurity measure. When it is used in the calculation of a misclassification error, it leads to a different impurity measure. This probability is calculated for every set of records associated with every class. Gini index is the summation of the probabilities of splitting an attribute across k different classes and the maximum value it can take is $1 - \frac{1}{k}$, where records are equally distributed among all classes. This is the least beneficial situation for classification modeling. The minimum value which Gini index takes is 0 when all records belong to a single class and this is the most beneficial situation for classification. Such a computation is used in the CART algorithm. We will see an example for binary classification where there are two classes plus and minus. The corresponding probability of minus can be expressed in terms of the probability of plus as $P_{\text{minus}} = 1 - P_{\text{plus}}$. Hence substituting the value of P_{minus} in the calculation for Gini index, we get the formula as $2 \times P_{\text{plus}} \times (1 - P_{\text{plus}})$. This is the simplification of Gini index for binary classification. For multiclass classification, we must find the number of records belonging to a particular class label and then find the relative frequency of the records and then substitute in the formula for Gini index to find whether the Gini index is close to 0 or away from 0. This is done for every split condition on every attribute. For example, if we have to compare three different class 2 record proportions where 6 records

belonging to class C2 compared to 0 records belonging to class C1 versus 5 records belonging to class C2 and 1 record belonging to class C1 versus 4 records belonging to class C2 versus 2 records belonging to class C1, the corresponding Gini index computation is based on the relative frequencies of the records belonging to the class that is 0 by 6, 6 by 6, 1 by 6, 5 by 6, 2 by 6, 4 by 6 respectively. These relative frequencies are taken to be the probabilities substituted into the Gini index formula to find the gain obtained from using the Gini index. When a node B is split into m partitions, the Gini index is calculated as Gini of i for the i th partition weighted by the number of records in the i th partition after the splitting compared to the number of records before the splitting. We wish to find larger and purer partitions. Computing the gain after splitting in comparison with the Gini index before splitting, we find that the gain is non-zero. So the selected attribute and its value can be used for splitting. In this video, we have looked at the Gini index and its computation in finding the split in a decision tree. Thank you.

Measures of Impurity - Part 2 Video• . Duration: 3 minutes

[Notes]

Hello everyone. In this video, we continue with computation of impurity measures. We look at the entropy impurity measure and its variants from information theory. Entropy takes a maximum value of $\log_2 c$, where c is the number of classes. That is, records are equally distributed among all classes, which is the least beneficial situation for classification. It takes a minimum value of zero, that is, all records belong to a single class, which is the most beneficial situation for classification. We wish to find those split values which lead to a zero or close to zero value for entropy. Entropy is used in the C4.5 algorithm to build decision trees. For example, if we have to decide between splitting across zero records belonging to one class and four records belonging to another class, one record belonging to one class and three records belonging to another class, two records belonging to both classes, the entropy computation is based on the relative frequency of the records belonging to class shown in the example, substituted into the definition of entropy, giving the values of 0.8 and 1 for entropy. We can clearly see that the situation of zero records belonging to one class and all four records belonging to another class is the most beneficial situation for classification. The entropy calculation for various class-to-record proportions is given in this slide. The gain obtained from splitting on entropy is then given according to the entropy obtained by splitting on every class, weighted by the number of records after splitting compared to the number of records before splitting, subtracted from the entropy of the data before splitting. This gain is also called information gain and we could include several variations of entropy in its calculation. In case there is noise in the data, the decision tree tends to overfit to random characteristics of the noise present in the training instances and most of the effect of noise is seen in the lower levels of the nodes in the decision tree, which contain a small number of data points easily influenced by noise. Information theoretic measures can also be used to address this situation where we can explicitly penalize the noise and model complexity resulting from the noise using ideas such as regularization or we can try to find a simpler model among multiple competing models of the decision tree which satisfy the minimum description length principle that is defined based on a cost associated with different subtrees in the tree. This cost is a weighted sum of the error resulting from making misclassifications in the tree and it is a measure of the tree complexity. Thus we can build decision trees to not only optimize the error but also the complexity or cost of building the decision tree. Thus we have seen how information

theoretic principles such as entropy are used to measure the error and cost in a decision tree to split the decision tree into partitions. Thank you.

Cross-Validation Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video, we look at model selection in decision trees to evaluate them using cross-validation procedures. By the end of this video, we would have understood the importance of separating the train, test and validation datasets for modeling and sampling procedures to achieve this. In cross-validation, we shuffle the available records randomly and then split them into k different partitions. For each partition, we hold a small percentage of data as the test dataset and use the remaining data as training dataset. The classification model is built on the training dataset. However, its performance is reported on the test dataset. Across different folds, the test performance is reported. And an average of these performances is used to select the best classifier model, giving the smallest test or evaluation score on an average. This is the k -fold cross-validation procedure, which is commonly used in classification models. It is illustrated in the diagram here, where a particular partition of the data is used for further training the model and it is split into training and test data across different folds to report the performance on the test data and take its average as the performance of the model. Thus, the dataset is typically split into training and test dataset to create a general classification model. A separate validation dataset can also be used for further fine-tuning the model. Some of the user-defined parameters, such as the hyperparameters, can be fine-tuned on the validation dataset. Any model which works best for training dataset but does not work for test dataset is said to overfit and we wish to find such a model that is working best on both training and test dataset. This can be achieved by techniques such as regularization. It can be evaluated with a confusion matrix. A training dataset is sampled from an underlying data distribution to build a model for classification and a separate test dataset from the same underlying distribution is used to evaluate the model and report its performance. A validation dataset is used to further fine-tune the model after training it to monitor the model performance and do steps such as hyperparameter optimization. This is in contrast to steps such as feature selection used in training the model. Combination of feature selection and model selection creates the best-fitting model and we have to report the performance of the model on such a combination of training and validation datasets in classification. The corresponding sampling procedures have to create the training, test, and validation data samples from the given dataset. Some of the common sampling procedures are random sampling where the probability of selecting a record is equal across all the records, holdout sampling where a predefined fraction of data points is selected and retained for training and the remaining data is used for testing. Sampling with replacement where out of n records a fraction f records is selected randomly and no duplicates are included in this selection. Sampling with replacement where records are sampled sequentially and independently allowing duplicates over the sequential selections. This leads to duplicate data in the training dataset, biased sampling where certain parts of the data have greater importance for the modeling than other parts such as recent records have greater chance of inclusion in the training set than older records, erroneous records are more important than easy to classify records, records are sampled according to a probability distribution which is non-uniform. All of these techniques can be contrasted with the random sampling technique which is typically used in cross-validation. In stratified cross-validation important

parts of the data are based on rarity of the classes and their corresponding records such that across different sets of classes we get a desired representation of the data according to predefined proportions of the records belonging to the classes. This leads to greater robustness in the analysis of the classifier. Thus in this video we have looked at several cross-validation procedures including k-fold cross-validation, holdout cross-validation, stratified random sampling for cross-validation and then some other variations of the sampling procedures used in cross-validation. These procedures are useful for designing the experiments in training the classification models. Thank you.

Overfitting Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video we will look at the concept of overfitting for model selection in decision trees. All data analytics procedures can be understood as a combination of model representation, model evaluation and search, where model representation is the language used to describe patterns and limited representation cannot produce accurate models. High dimensional representation increases the complexity of the models. Thus, a suitable model representation is required to generalize the model from training dataset to unseen test dataset. Model evaluation criteria are quantitative statements about how well the model fits to the training dataset. This is dependent on the goals and patterns found from the data mining and the evaluation of the patterns is important and it is done on test dataset according to different dimensions of interest such as novelty, utility of the model. The search methods used for data analytics include those based on rules, those based on parameters, those based on models and those based on optimization. In all these search procedures, we can improve them by getting a high quality representation of the data that is able to generalize from the training set to the test set and remove irrelevant and redundant information in the data to find the patterns of interest in a computationally cost effective way and discover the underlying data distribution using various feature selection procedures. These procedures can be combined further with smoothing and regularization procedures that aid the search process. In this context, overfitting leads to models that are very complicated and work very well on the training dataset but do not generalize to test dataset. That is, the performance of the model is high on training dataset but not on the test dataset. We can interpret this situation by saying that the model is fitting to random characteristics of the training dataset that are not generalizing to the structures found in test dataset. Overfitting can be assessed with learning curves that plot the number of instances of the training set on the x-axis and the accuracy of the model on the y-axis. With increase in the number of training instances, we expect the accuracy to increase and eventually lead to very high values such as more than 80%. The corresponding error which is plotted on the y-axis leads to a reduction of the values found for training sets of different sizes to obtain errors which are less than 10% for increasing sizes of the training set. We can also plot the loss function which is the objective function for the model leading to the probability of prediction associated with the class label producing the error and accuracy and we can have several values of hyperparameters on the x-axis not only the size of the training set. However, in case of decision trees, there is no loss function used and the loss function is the same as the error function resulting from a deep decision tree having impure nodes near the leaves. We wish to find the situation in the model where the model is neither underfitting nor overfitting to the data that is the model is neither too complex resulting in a reduction in training error but an increase in the test and validation error nor is the model too simple

where there is a high error associated with both training and test data sets. The corresponding loss or error function is plotted on the y-axis and the size of the data number of iterations of the algorithm or hyperparameter values are plotted on the x-axis. We wish to find the model falling in between the colored regions such that the performance of the model is high and error is low on both train and test data set. Thus, we have seen the use of learning curves in building classification models in this video. Thank you.

Random Forests Video• . Duration: 5 minutes

[Notes]

Hello everyone. In this video, we will look at random forest, which is a generalization of decision trees using multiple decision trees in an ensemble to create the output. By the end of the video, we would have looked at the random forest algorithm and a sampling method to create the random forest. And we would have seen an introduction to ensemble methods. Ensemble methods combine several models to form a metamodel, such that each model brings its own unique bias and combining predictions from multiple models results in a better output and performance of the metamodel. Several iterations can be conducted over the ways and means to combine the various predictions coming from the model, leading to improvements in the design of ensemble methods. Ensemble methods are self-organizing and adapt to the task at hand. They can work with multiple classification models, which are treated as a team of experts. Ensemble models are able to create strong classifiers by combining several weak classifiers and the weak classifiers can be a decision tree in the case of a random forest ensemble. The algorithmic question is how to combine the predictions to make the final prediction. The training dataset can be partitioned into multiple datasets and the corresponding decision tree can be built for each dataset. So a majority vote of the predictions coming from the decision tree can be taken as the final prediction. Another way to do this is, same algorithm is used under different parameter settings to create multiple decision trees on the same dataset and the predictions from multiple decision trees are combined to give the final prediction. These techniques can be categorized into bagging, boosting and stacking, which are different sampling procedures used in ensemble models. Several base classifiers are built and combined into a meta-classifier in the learning algorithm. Bagging is short for bootstrap aggregating. It generates the training dataset by bootstrap sampling of the original dataset. Bootstrap sampling is a variation of stratified random sampling. The set of models resulting from building decision trees on each bootstrap results in the ensemble model, which is combined using majority voting scheme. In bagging, the bootstrap is created by sampling with replacement and we train each new decision tree on a new bootstrap to use a voting scheme in the ensemble method. The voting scheme is illustrated in this slide, where there is a weight associated with every prediction y_j created by the classification tree. This weight is a positive number and all the weights across all trees sum up to 1. This is represented in the diagram, where y_1 to y_n number of trees is being input to several weights, which are being combined using a summation operation to give the final prediction. The voting scheme used in this slide is similar to the voting used in a democratic setup. The classification produced is the final prediction by combining the predictions and their weights in the voting scheme. Producing such final predictions for every test instance and building several ensembles of decision trees is the algorithm for random forest. It uses bagging to create the ensemble and bootstrapping to create a sample in the ensemble. The advantage of random forest compared to decision trees is it

can handle extremely large datasets. It has performance comparable to more complicated black box models like deep neural networks and support vector machines. It can handle noisy and missing data. At the same time, it can also handle numeric and categorical data similar to a decision tree. However, it is not easily interpretable compared to a decision tree. The algorithm for creating an ensemble method is given in this slide, where different partitions of the data are mapped to different tree construction algorithms and a combination of their predictions is made to arrive at the final prediction. Thus, in this video, we have used multiple decision trees to create a random forest and we have seen how the final prediction of the random forest is arrived at with the voting scheme. In general, the ensemble techniques used for building a random forest can be used to generalize other classification models.

Decision Boundaries Video• . Duration: 8 minutes

[Notes]

Hello everyone, in this video, we will look at the use of decision boundaries in model selection for decision trees. By the end of the video, we would have visualized the decision boundaries with Voronoi diagrams. A connection with bias-variance trade-off involved in the modeling of classification using decision trees is also given. A decision tree is breaking the attribute into range of values based on split positions such that the resulting records belong majorly to a single class and not other classes. In this case, the classes are Cheat, No and Yes subject to annual income which is split on positions 55, 65, 72, 80 etc. determined by the impurity matrix resulting in distribution of records across class labels Yes, No and the improvement seen in the gain corresponding to Gini index. These split criteria will decide the learning curves and decision boundaries coming from the decision tree. Since decision tree is splitting on thresholds of attributes, we can visualize the resulting splits from decision tree as following straight lines parallel to either y-axis or x-axis depending on the threshold values. For multiple decision trees such as decision tree A and B, this figure shows the resulting decision boundaries. Now, we can see that blue points which are closer to maroon points are not classified correctly by both of these trees and such a classification is possible by the straight line separating the blue points of class A from the maroon points of class B. This is another example where the light blue points are overlapping with the violet points across a decision boundary such as in the illustration compared to a smoother decision boundary which is including the light blue points in the dark blue portion of the figure and the violet points in the light blue portion of the figure in a smoother fashion compared to the one shown in the figure on the left hand side. In both these cases, we can see that decision tree is producing non-smooth separation between the points belonging to two different classes. The diagrams which we have seen just now are called Voronoi diagrams and they produce different partitions of the training data points across class labels separated by curves called decision boundaries that decide the regions between classes based on thresholded attribute values found by the decision tree. Each region expresses a set of rules characterizing a hypothesis underlying the dataset. In case of decision tree, the hypothesis is based on rules formed by traversing the tree from the root to the leaf and the corresponding attribute values and their thresholds forming the non-smooth curves in the decision boundary. As we can see in the example, the decision boundary formed by decision tree is not suitable for data points that are not easily separable across two different class labels. Such a decision boundary can be regularized or smoothened to get a better result of separating the inseparable data points near the decision

boundary that are hardest to classify or easiest to misclassify. We wish to arrive at straight lines like shown in the figure to isolate hardest to classify points placed between easy to classify points. Such a straight line is arrived at by the ensemble models which combine multiple decision such as the decision tree shown by the pink line and the blue line to come up with the dashed line which is a combination of the decision boundary formed by both the trees and hard to classify points which are closest to this line are represented between the two parallel lines here and these points are specifically addressed by the ensemble model resulting in a boundary like shown on the figure on the right where the boundary is more fine grained than the one formed by decision tree and it is a better approximation of the straight line separating the blue points from the pink points. The estimation of the decision boundary is based on the rules learnt by the decision tree and corresponding rules learnt by the random forest. The impact of the decision boundary is quantified by several data examples and expectations over them over all possible predictions which are formed over the training set. The decision boundary is affected by the variance in the predictions formed by the model as well as the bias in the model subject to assumptions about the underlying data represented in the training set whether they are easy to classify or hard to classify examples. An expected value of the error resulting from the decision tree is defined in terms of several error terms such as the expectation over the bias resulting due to the underlying data used in building the decision tree, expectation on the variance of the decision boundaries in the tree and any intrinsic error which has not been accounted for. These equations lead to the bias-variance trade-off in building ensemble models where decision tree is a base model, several samples are created on the underlying data to get the training dataset and ensemble models are built on the samples so as to look at hard to classify features or labels in the data with respect to their bias and variance components. Model centered ensembles create different models for different regions of the data so that a combination of models is useful for making prediction whereas data centered ensembles are driven by the sampling procedures. Model centered ensembles assume that majority of their ensemble components are not affected by particular test instances so that we can generalize the combination of multiple models. Therefore in this video we have seen a decision boundary analysis of decision trees and corresponding random forests, we have quantified the error and decision made by the decision trees in terms of the bias-variance trade-off and the effect of smoothing, regularization and further optimization of the decision tree on the decision boundaries. Thank you.

Logistic Regression Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video we will look at logistic regression as an example of parameter based classification model. By the end of the video we would have looked at the training algorithm for logistic regression, the objective function and optimization procedure used in logistic regression. The target variable in logistic regression is assumed to be drawn from a Bernoulli distribution, similar to the one associated with tossing a coin. The distribution mean is parameterized by the logistic function, which is applied on the features input to the classification model. This type of a classifier is called a discriminative classifier because it is making a prediction on the class conditional probability associated with the class label. It can also be understood as a probabilistic classifier, similar to Naive Bayes. It can be interpreted as a linear classifier too. Logistic regression directly models the class prediction probabilities for different features input to the model. The logistic function is shown

in this slide for both positive and negative data. The parameters underlying this function are estimated in a data-driven manner using the training algorithm. The maximum likelihood approach is used to estimate the best parameters θ . For the likelihood function given in this slide, which is a concave function, the gradient ascent method is used to find the optimal parameters of the vector θ , maximizing the log likelihood. And wherever there is a mistake produced from the prediction of the classifier, that is, an actually positive class label point is classified as a negative class label point or vice versa, those mistakes are accounted for making adjustments to the weights θ computed by the gradient ascent method. Thus, the update rule for the weights is driven by the mistakes made by the predictions across positive and negative class labels. The magnitude of the step to be taken in the direction of the steepest ascent can be found using a line search procedure with or without backtracking and using variations such as in binary search. The equation shown in this slide is the final update rule applied by the logistic regression to every parameter value θ_i . Adding additional regularization terms to the log likelihood function reduces the danger of overfitting in logistic regression. Several optimization procedures from convex and concave models that use mistakes made on predicting the positive class data point to be the negative point and vice versa can also be used to make improvement to the algorithm. Logistic regression is a special case of generalized linear models used in statistics. Thus, we have seen an introduction to logistic regression, objective function, optimization algorithm and probabilistic classification in this video. Thus, in this video, we have seen an introduction to logistic regression and its associated objective function optimization algorithm in probabilistic classification models. We have seen how log likelihood estimation in logistic regression is done using mistakes made in the prediction of the model. How the gradient ascent algorithm acts as the optimization method used in the procedure. Thank you.

Discriminant Analysis Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video we will look at discriminant analysis as an example of parameter based models for classification. We will look at the linear discriminant analysis based on the Fisher score and its use in binary classification problems. Fisher score is designed for numeric attributes to quantify class separation between them. It measures the ratio of average interclass separation to average intraclass separation. The distance between the multiple classes and the distance between the multiple points in the same class. These distances are found using the mean and variance of the points in the class where p_i is the fraction of data points belonging to our class i and μ_j σ_j are mean and standard deviation of the points for a particular feature. We wish to filter features with the largest Fisher score to find the most discriminative features available for classification. That is, these features are able to separate out the data points into two or more classes. This same idea is generalized into linear discriminant analysis LDA to combine various features in the original set that are most discriminative in forming the classifier. The direction of the new features is designed to have high discriminatory power with respect to the class labels and it is a supervised dimensionality reduction method where information about the class labels as well as the feature values is used for discrimination and classification. LDA is also able to preserve the variance in the data for class specific discrimination in the feature space. LDA is defined for binary classification as between class scatter along vector divided by within class scatter along a vector. This in turn is expressed in terms

of the covariance matrices along the direction of the vector giving the maximum discrimination and the corresponding means and fractions of records belonging to different classes. μ_1 , μ_2 , w are d -dimensional row vectors, σ_1 , σ_2 are d by d covariance matrices. The matrix multiplying μ_1 minus μ_2 with itself is referred to as the between class scatter matrix and the matrix in the denominator multiplying the covariances with the fractions of the records in the class is called the within class scatter matrix. The goal of the classifier is to determine a direction w that maximizes the score. This direction is proportional to the between scatter matrix and the inverse of the within class scatter matrix. The directions of w are determined by an iterative algorithm that projects data into orthogonal subspace such that the discriminant computed in the previous equation is reduced in subspace. The final data representation found from LDA is more discriminative in the reduced subspace than in the original space and it produces a classification model driven by the parametric representation of data in w . For each w , the perpendicular hyperplane wx plus b is taken to be the most discriminative direction where b is a biased term selected based on accuracy requirements in the training set. w is able to geometrically project the data points along the most discriminating direction and b is able to find the best separation line between the two classes generalizes into higher dimensional space. Thus, in this video, we have seen an introduction to LDA. It can be contrasted with PCA on unlabeled data. LDA is useful for both dimensionality reduction and classification using parameterized representation of the original dataset. Thank you.

Classifier's Performance Evaluation - Part 1 Video• . Duration: 8 minutes

[Notes]

Hello everyone, in this video, we will look at performance evaluation of classification models, whether they are rule-based or parameter-based. End of this video, we would have looked at several evaluation metrics used in cross-validation procedures to analyze the confusion matrix constructed for binary and multiclass classification. We will arrive at several metrics to assess the accuracy and performance of the model and understood the importance of analyzing the error in the model. Model performance is required to discriminate between competing models for classification. Depending on the application, the performance is assessed according to the error distributions found in the application and their associated prediction probabilities calculated by the classification model. The improvements in the error are achieved by not only analyzing training and test instances leading to the error, but also by analyzing the runtime and memory requirements of the classifier, as well as the optimization objectives leading to the error. The modeling process, which includes model validation in it, can be described according to this slide, where problem definition, data analysis, and data collection are done before fitting the best model to the data, and then model evaluation, model validation, and model deployment are done after fitting the best model to the data. Validation experiments have to consider all the steps after fitting the best model to the data. We have already seen the holdout, stratified, and bootstrap cross-validation procedures, such that they are neither overfitting nor underfitting the model to the data, where the class label information provided to match up the fit of the model is called the ground truth information. There are several techniques to evaluate the performance of the classification models, some of which are listed in this slide. We will look at the confusion matrix and ROC curve techniques. The evaluation measures should not only match up the predicted class label to the actual class label, but also look at the prediction probability

associated with the class label, to ensure that there is an informed prediction being made and it is not a chance prediction. The commonly used evaluation measures in binary and multiclass classification are shown in this slide, such as accuracy, F1 score, recall, specificity, area under curve, so on. Confusion matrix is a contingency table between observed and predicted classes available in the classification model. The co-occurrence frequencies of the prediction categories with respect to frequency analysis of the predictions, as they match up with the actual values, creates the various evaluation measures. Thus, a frequency analysis of the confusion matrix can be conducted using statistical techniques such as frequency tables, tables of proportions, cross tabulations, so on. In general, confusion matrix can be interpreted as a contingency table, where interdependence between known and unknown classes is represented in terms of frequencies of records belonging to the classes, either predicted or actual. This is an example of a contingency table, where the predictions are compared with the actuals in terms of frequencies of matching and corresponding marginal probability distributions are derived. Here is an example, where different application areas of human-computer interaction, such as concurrent engineering, virtual reality, human-robot interaction, are being mapped up with the responses provided by participants in a survey on whether they are involved with those areas of human-computer interaction. The number of responses is represented as a frequency in the table. These frequencies can in turn be used to derive marginal probabilities. Let O_{ij} be the observed absolute frequencies from such a contingency matrix for n number of observations. Then the probability P_{ij} that the i -th category of domain X matches up with the j -th category of domain Y is given by the hypothesis testing involving P_i and P_j , where the null hypothesis says that there is no difference in the probability estimated from the categories of domain X compared to domain Y , whereas the alternate hypothesis says that there is a significant difference. In this way, we can set up several hypothesis tests on the contingency tables. Some of the common hypothesis tests are chi-square hypothesis test, which is based on subtraction of the frequencies which are observed and estimated, and the expected frequency, which is based on the product of the observed frequencies. There are several other algorithms to estimate the hypotheses, such as the expectation maximization algorithm. Interpreting confusion matrix as a contingency table allows us to analyze the frequencies and associated probabilities to form hypothesis tests on those probabilities. This leads to several evaluation metrics derived from the confusion matrix, such as sensitivity, specificity, precision, recall, accuracy, error. There are two kinds of errors which are apparent on the non-diagonal elements of the confusion matrix called the type 1 and type 2 errors associated with the false positives and false negatives. One has to define what is a true positive to arrive at what is a true negative, what is a false positive and what is a false negative. When the predicted class matches with the actual class, we say that we have arrived at a true positive. Several statistical measures and hypothesis tests can be devised over the confusion matrix by treating it as a contingency table to quantify the advantage offered by one classifier over another in a given dataset. Accuracy is the most common evaluation metric used in classification. It is defined as the fraction of points which match on the predicted and actual observed values for both positive and negative classes. There are many variations of accuracy, such as the cost-sensitive accuracy, when all classes are not equally important in making the prediction and improving the performance of the classifier model. In imbalanced classification problems, where there are few points of one class compared to many points of another class, the misclassification of the minority class containing few points is less desirable than misclassification of the majority class. The differential cost associated with such misclassification is used as a weighting term in the accuracy which is computed in such scenarios. Therefore, in this video, we have seen the need for

performance metrics in classification evaluation and we have looked at several variations of commonly used metrics such as accuracy. We have also interpreted the confusion matrix as a contingency table and taken advantage of the resultant hypothesis testing. Thank you.

Classifier's Performance Evaluation - Part 2 Video• . Duration: 4 minutes

[Notes]

Hello everyone, in continuation of the previous video, we continue with ROC curves for performance evaluation of classification models. By the end of this model, you will appreciate the need for associating prediction probabilities with class labels in classification and understand the use of ROC curve which is the receiver operating characteristic curve in evaluating classification models. There is a prediction probability associated with every training and test instance in the formation of the confusion matrix. The prediction is associated with the maximum probability across several prediction probabilities associated with each class for every test instance. Thus, we can compare different test instances in terms of their relative propensity to belong to a specific class in terms of their prediction probabilities. ROC curves evaluate the overall trade-off between labeling test instances as positive as opposed to labeling them as negative. They threshold the prediction probability for positive class to define binary classification performance. If the threshold across set of probabilities associated with prediction across different class labels for a test instance is selected too aggressively, then it minimizes the number of positive class instances and classifier will miss true positive class instances increasing false negatives. If the threshold is too relaxed, it will lead to too many false positives. Thus, there is a trade-off between false positives and false negatives which is quantified in the ROC curve. The correct threshold is never known in a real-world scenario because of reasons such as we never know the underlying data distribution for classification model and we never know the true error function for classification. In such a situation, ROC comes to our aid in evaluating the trade-off between the false positives and the false negatives. There are several other alternatives to ROC curve such as the precision recall curve and cost curve which will be not covered in this course. ROC curve is based on the false positive rate which is computed from the confusion matrix. It is defined as the percentage of falsely reported positives out of ground truth negatives. It is also based on the true positive rate which is defined as the percentage of ground truth positives that have been predicted as positives. The corresponding equations for true positive rate and false positive rate are shown in this slide. The false positive rate is plotted on the x-axis for different runs of the classifier so that one point the true positive rate is plotted on the y-axis so that one point on the ROC curve is representing one run of the classifier and one configuration of the classification model. The ROC curve changes from 0,0 which is the origin to the maximum value of 1,1 which is there on the top right of the plot. The area under the curve which shows the shaded region in the curve is expected to be more than 0.5 which is the area of this triangle. A random method exhibits the performance along the diagonal forming the triangle connecting the point 0,0 and 1,1 and any lift obtained above the random performance method is the performance gain obtained by the classification model. Thus, area under the ROC curve provides a quantitative evaluation of the effectiveness of the ROC curve. It also provides a way to compare relative trends between different algorithms and different models for classification. We always increase the area under the curve to find classification models that strictly dominate the other ones in terms of the

area under the curve. There are several possibilities for area under the curve which are shown in this diagram. In the perfect situation model is able to clearly distinguish between different classes with area under the curve equal to 1 otherwise it is a good enough model that is able to predict correctly most of the time. Any model which falls below the diagonal line such as this model is able to distinguish classes but in a reverse fashion that is confusing between the positive and negative classes perfectly. In this video, we have looked at the ROC and AUC evaluation metrics for comparing several classifiers. Thank you.

Applications Video• . Duration: 2 minutes

[Notes]

Hello everyone. In this video, we will look at several applications of classification. By the end of this video, you would have understood the role of classification in data mining for analyzing several complex data types, and we would have seen an introduction to the frameworks for building classification models. In customer target marketing, to tag groups of customers which are interested in a particular product according to their previous buying behavior and features from their demographic profiles, we can use classification. For example, the products can be mobile phones and customers may be browsing an e-commerce platform to buy a high-end phone or low-end phone or popular phone, based on different locations in different cities in India. In medical disease management, historical data is available about the patient's medical tests and treatments prescribed by doctors previously. Based on this, we can predict the treatment outcomes using classification models into the future. For document categorization and filtering on the internet, we can identify specific topics of interest in the news portals using classification, where features for classification correspond to words in the document and labels for classification correspond to various topics to be identified. A real-time classification system can be deployed on any news portals using classification models. In web linkage mining, the logical structure of the internet can be mined using classification, where the social and information networks on the internet are the features and the links between the various networks are the classes to be predicted. There are several data types which can be used in classification models. We normally think about the multidimensional data, such as shown in this slide, having several attributes, perhaps obtained from a SQL table, of different data types, such as categorical, numeric, and binary. However, the data types can also be complex graphs, such as shown in this figure, coming from scientific applications, such as chemical compounds and co-expression networks of genes representing graphs. The techniques of classification used for such scientific applications can also be used on the internet datasets, such as social network datasets and program flow datasets found in software engineering. Classification is the basis for predictive analytics, and it maps input variables to output variables, such that the observed input is put through a workflow to come up with the predicted output. This predicted output is to be matched with the actual values available in the data to come up with an error which is determining the merit of the predicted output. If the error is not according to our expectation, we can use further machine learning to optimize the classification model, by using additional objective functions and constraints which are parameterized in the optimization procedure. To come up with a prescriptive output from the predictive output, that can be presented as a recommendation on the decisions produced by the classifier. The overall framework for classification can be understood in this figure, where the data for classification is prepared to build the model for classification. The patterns used for classification

are then built using several optimization criteria, resulting in the notion of discrimination as opposed to classification, producing decision support. Any feedback obtained from the end user of classification can be given back to the classification model to make further improvements to it, and new data characteristics can be used to come up with the discrimination models. There are several classifiers available in the research literature, such as logic-based classifiers, for example decision tree, neural network-based classifiers, such as multilayer perceptron, statistical techniques, such as Naive Bayes, instance-based techniques, such as k-nearest neighbors. We will be covering a sample of the various classifiers available in the literature. For every classifier, there is a deep and machine learning counterpart which can be built to make the classifier better, such as rule-based techniques in frequent patterns can be improved with generative modeling in deep learning. In this video, we have surveyed the applications of classification in medicine, marketing, and internet, the use of classification in various data mining models, and formulation of classification problem in a given application domain.

Types of Clusters Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video we will look at different types of clusters. By the end of the video you would be able to formulate a suitable clustering model based on the type of cluster. The definition of the clustering depends on the desired result from the data analytics since this is an unsupervised learning problem and the nature and characteristics of data must be considered to define the number of clusters. In the red points on the left hand side of the figure we can see that all the points have been assigned to the same cluster and on the right hand side of the figure six different clusters have been formed for the same set of points whereas on the bottom of the figure four different set of clusters have been formed whereas on the bottom left of the figure there are two clusters assigned to the same set of points. So the question is which clustering is the best clustering? That depends on the desired result from the clustering algorithm. Here is another illustration of four different clusters which have been assigned to a set of points. Typically clustering is defined based on a partitioning criteria in the data set and we can define multiple clusters based on borders formed between different partitions of the data set. This is the concept of exclusive clustering. If a data point can belong to multiple clusters at the same time with a non-zero chance the scenario for clustering is called fuzzy clustering where each point is assigned a weight of belonging to more than one cluster. Sometimes it is possible to cluster the data fully and sometimes it is not possible. Sometimes it is possible to cluster the data such that several points of the same label belong to the same cluster and sometimes it is not possible. Typically we are thinking about a circular boundary or an elliptical boundary around clusters to form notion of well-separated clusters. Here are examples of six well-separated clusters based on the center point of the cluster belonging to the middle point in the circles. However in this case where the data points cannot be separated through circles and ellipses other notions of clustering must be included to separate the blue cluster from the red cluster the yellow cluster from the blue cluster and so on. Here is another example where the red cluster within the larger cluster may belong to it because the points in this cluster are closer to the larger cluster than the smaller cluster from which they are far away. This is an example of two overlapping clusters which cannot be separated using a circular or elliptical boundary. Thus we have seen examples of different types of clusters and we have understood the notions of well-separated,

center-based and other clusters. Thank you.

Types of Clustering Algorithms Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video, we will look at different types of clustering algorithms. By the end of this video, you would have gained insights on the algorithm to be selected for building the patterns in a cluster based on the data characteristics discussed in the previous video. The clustering problem can be formulated in different ways, depending on the similarity between points in a cluster, number of clusters to be formed, and the optimization criteria for clustering. The clustering elements in such a case would include external supervision information to validate the results of clustering, manual human supervision to validate the results, and meta-algorithms which combine the results of multiple clustering algorithms. The data characteristics to be considered include complex data types, different kinds of labels to be assigned to the clusters, different representations of the data, and noise in the data. Based on these various different scenarios about the underlying data distribution, the resulting cluster's size, dimensionality, and scalability is affected in the results. This slide shows a taxonomy of various clustering algorithms, most popular being partitioning-based clustering. However, problems which cannot be separated using ellipses and circles as boundaries between partitions of the clusters can be solved by other kinds of clustering algorithms, such as hierarchical clustering, density-based clustering, and model-based clustering. We will look at some of these algorithms in the subsequent videos. This is a different taxonomy of clustering, where hierarchical clustering has been further classified into agglomerative clustering and divisive clustering. Similarly, partitional clustering has been classified into hard clustering and soft or fuzzy clustering. The leaps of this taxonomy show the various clustering algorithms available for implementation. Clustering can be also classified based on heuristics or models used in the clustering. This is an extension of density-based clustering, which are able to detect clusters that are otherwise not processed by partitional clustering. Heuristic-based techniques include prototype-based techniques, and model-based techniques include mixture-based techniques. We have to decide the pros and cons of each technique to decide the best clustering algorithm for the data analytics task at hand. Partitional clustering is useful for spherical-shaped clusters, and it is based on a mean or medoid point that represents the center of the cluster. Hierarchical clustering is suitable for creating multiple levels of partitions in the data, and it is based on several distance or similarity metrics between the partitions, forming a hierarchy. Density-based clustering can detect arbitrary-shaped clusters, unlike partition-based clustering. However, it requires the end-user to set some hyperparameter values for suitable clustering result. Thus, we have surveyed the taxonomy of clustering algorithms and correlated them with several application scenarios. Thank you.

Similarity Measures Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video, we will look at several similarity measures for partitional clustering. By the end of this video, you would have surveyed similarity measures used in clustering algorithms to model various different data types of the attributes used in clustering. A good clustering method that is partitioning the data should produce high intra-class similarity and low inter-class similarity. That

is, points belonging to the same cluster should be close to each other and far away from the points belonging to another cluster. Distance measures are defined based on this notion for different pairs of points present in the dataset. The Minkowski distance measure is given in this slide and special cases of this distance measure include Manhattan and Euclidean distance measure where the power term given in the equation takes the values of 1 and 2 respectively. p and q are vector magnitudes of the data point in each direction where each data point is represented as a feature vector. r is a real number which can go from minus infinity to plus infinity when it takes the value of 2. This distance measure is called the Euclidean distance measure. A positive value is taken for the difference between the vector components such that the summation is proportional to their magnitude in all dimensions. For attribute types that are non-numeric such as nominal, ordinal, and interval, the distance between pairs of points is redefined as shown in this slide. For nominal attributes, the distance is 1 if the values match, otherwise 0. For ordinal attributes, which are ranked nominal attributes, the distance is based on not only the magnitude but also the rank of the categorical value. For interval attributes, the distance is based on the lower and upper limits of the interval. A common situation is that the data points consist of only binary attributes and computing the distance for such binary attributes is shown in this slide where the matches between the 0s and 1s forming the bit vector representing the data point leads to four different variables where if there is a match between the 1s of both the data points the variable is called b_{11} , if there is a match between the 0s of both data points the variable is called b_{00} and so on. Based on this, the Jaccard coefficient for measuring distance between binary attributes is defined as the ratio of number of matches by number of attributes. That is, the numerator is telling in how many bits the bit vector representing the data point matches with the bit vector of the other data point where the bit values are 1 and 0 and the denominator tells the number of mismatches between the bit vectors. Simple matching coefficient and Jaccard coefficient are the most popular distance measures between binary records. Cosine distance is also used to compute the similarity between binary and numeric data types. It is based on the dot product between the data points. It is normalized by the magnitude vector representation of the data points. Here is an example where for two data points represented by their integer and binary values the cosine distance between them corresponds to the multiplication of the integer and binary values across each position of the bit vector. And the L1 norm of the bit vector gives the magnitude of the vector such that the cosine distance is evaluated to 0.315. In high dimensional data, there are several features which are found to be irrelevant and uncorrelated to the distance measure and data cannot be separated into meaningful and cohesive clusters using the computations we have seen until now. Several feature selection methods can be employed to reduce the dimensionality of the data before applying the techniques discussed in this video. However, it is not possible to remove features that have an impact across various search spaces in the feature space. Thus, any notion of feature relevance based on feature selection is local to the feature space and not global. The optimum feature set depends on the underlying data distribution on which the clustering is defined. Different views of the data distribution can lead to different solutions of clustering. Several clustering solutions are equally valid in arbitrarily oriented subspaces. Thus, there is a need for creating a robust clustering solution that combines the results of various clustering algorithms in an ensemble. Therefore, in this video, we have seen the clustering techniques used for partitioning numeric categorical binary data and their extensions into high-dimensional spaces. Thank you.

Distance Matrix Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video, we will discuss the distance matrix, which is a key computational element in partitional clustering algorithms. We will introduce the distance matrix and see examples for computing it. Distance matrix has several other uses in data analytics. Distance matrix is formed from the data matrix. It is a square matrix containing the distance between all pairs of points in the dataset. It is of the size n by n , where n is the number of points in the dataset. It can also be interpreted as a weighted adjacency matrix, where the weights are the distances between the points. It is useful in the computation of clusters in partitional clustering. It is also useful in the analysis of the data distribution using graph theoretic, linear algebra and other mathematical ideas, leading to notions such as symmetric matrix and non-negative matrix. For a set of points p_1 to p_4 having x and y coordinates as given in the slide, the distance matrix is calculated by mapping every pair of points p_1 to p_4 and computing the distance between each pair. This is done across every iteration of the clustering algorithm, until the clusters which we require are found. For each kind of clustering distance measure or similarity measure, a separate distance matrix can be derived. Mathematically, distances such as Euclidean, Manhattan and Minkowski distances follow the triangle property, which is an extension of the Pythagoras theorem. Distances must be positive or strictly speaking non-negative and they must be symmetric, i.e. distance between pair of points p, q can be computed by pairing them up either as p, q or q, p and distances must satisfy the triangle inequality, i.e. if there is a distance computed between p and r and then there is a point between p and r called q , then the distance between p and q summed up with the distance between q and r should be the upper bound for the distance between p and r . A distance that satisfies these properties is called a metric in mathematics. Weighted notions of distance can also be defined, where the weights are based on the similarity or dissimilarity of data points of belonging to one or more clusters. The weighted distance and similarity then measures the possibility of assigning a data point to one or more clusters. However, the weight to each distance measure is given according to whether a k th attribute has the same value across both data points. The weight attribute to the distance can also be measured based on several other data characteristics such as the sparseness of the data consisting of a data set having many zero values compared to non-zero values, the dimensionality of the data in high dimensional spaces, underlying data distribution and its statistical properties, the effect of noise on the data and so on. Thus, using a distance matrix, we can not only find the partitions in the data, but we can build several algorithms for data analytics, such as in distance metric learning and instance-based learning. There are several applications for such clustering algorithms, such as in bioinformatics, using the dynamic programming from algorithm design for the construction of clustering algorithms. Therefore, we have seen the use of distance matrix in clustering and machine learning algorithms used in data analytics. Thank you.

k-Means Algorithm Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video we will discuss the k-means algorithm for partitional clustering. k-means is the canonical example given for partitional clustering and it is able to find representative mean points to separate data set into spherical or elliptical partitions. k-means partitions or segments n data samples into k subsets or buckets called clusters. The clusters are

non-overlapping subsets and each cluster is defined by the central cluster representative point. Every other data point belongs to exactly one cluster and not more than one cluster. The challenges in partitional clustering include the computation of the distance matrix and we have to select the suitable clustering algorithm based on data characteristics. Such characteristics may involve partial information present with us about the clusters or labels in the data set and any user feedback. The working of the k-means clustering algorithm is illustrated in this slide, where the data points have been grouped into two clusters and one cluster respectively. Partition starts with an initialization of the representative points, which are randomly selected from the set of points available to us for partitioning and clustering the data. Then the distance between the remaining points and the representatives is computed. The points are reorganized such that the distances are minimized. After each iteration of clustering, we are able to partition the data into spherical clusters centered around the mean points. The same illustration is given as a pseudocode in this slide, where for a given value of k set by the user, such as k is equal to 2 clusters, the algorithm determines the center representative point for each cluster and computes the distance between that point and the remaining points in the data set until convergence. In this slide, we can see the various cluster configurations discovered by k-means. In the first iteration, the clusters are illustrated in terms of the blue, green and orange points, which change shape in the second iteration and this process keeps repeating until the representative points around which the remaining points cluster are found and there is no change in the clustering configuration, which is the convergence condition. Thus, the key steps in the k-means algorithm are the calculation of the new centroids and the calculation of the distance between the centroids and the remaining points in the cluster and the data set. This is kept track by using distance matrices. For data types that are non-numeric, clustering can be used by representing such data types, for example, categorical data in terms of a smoothened version such as in a histogram, where we can define a value p_{ij} in the histogram as the fraction of number of objects which belong to a particular cluster, where each attribute of each object takes on values which are discrete. Depending on the size of the number of categories belonging to each attribute, we can estimate the time complexity of the clustering algorithm. Once such histograms are formed, matching histograms can be used as the distance matrix to calculate the distance matrix. Several probabilistic measures can also be assigned to the similarity computation between histograms. In case the histograms are highly skewed due to attribute values representing data skew in the underlying data distribution, further preprocessing of the attribute values can be arranged. Based on frequency of discrete categories of the categorical attribute, the resulting weights can be used as abilities to compare histograms and compute the distance or similarity between them. Thus, we have gained insight on the k-means algorithm in this video and have applied it to not only numeric data but also categorical data.

Fuzzy C-Means Algorithm Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video we will look at the fuzzy c-means algorithm as a form of soft partitional clustering. We will see an introduction to fuzzy logic that overcomes the limitations of k-means algorithm and we will look at the fuzzy k-means algorithm also called the fuzzy c-means algorithm. Soft clustering algorithms assign a probability of a data point belonging to more than one cluster such that there is a non-zero assignment probability associated with every data point. This is based

on fuzzy logic which is a mathematical reasoning system to deal with uncertainty in the data set. A soft solution can be converted to a hard solution by selecting the largest assignment probability. Here is an example to describe the weather on a given day as sunny, cloudy or rainy where a crisp difference between the different categories of the weather corresponds to a hard assignment of data points to clusters whereas a soft transition from sunny to cloudy and cloudy to rainy corresponds to a soft assignment of points to clusters. The probability assigned to the weather being sunny or cloudy or rainy and its variation from 1 to around 0.5 and then increase to 1 is called the membership function where on the x-axis we have the attribute of interest whose probability of belonging to a cluster must be computed and on the y-axis we have the value of the probability assigned to the data point containing the attribute height in this case. Depending on prior knowledge available for clustering algorithms can be categorized into soft or hard clustering algorithms and extended into learning systems such as expert systems, fuzzy systems, data systems and neural systems mapping prior knowledge about the application to the processing logic used in the clustering algorithm which may be driven by rules or numbers. We have to make a distinction between possibility and probability to understand fuzzy logic. Probability theory is based on learning a probability density function for an underlying random process representing the data distribution where the sample space cannot change between various statistical experiments and the frequency associated with the event to compute the probability density from random variable cannot change and repeatable experiments are assumed such that the outcome of one experiment does not affect the outcome of a previous or future experiment. In possibility theory, real-world data is analyzed using fuzzy sets which assign membership function values to various elements of the sets. They allow us to reason with imprecise information, partial truth and uncertainty to assign the membership function values over fuzzy variables based in multivalued logic systems extending into adaptive systems theory. The familiar k-means algorithm can be adapted to the fuzzy context by redefining the distance metric used in the computation of the distance matrix. Recall that the distance metric is based on pairwise distance between points belonging to several clusters. Any point that is wrongly assigned to a cluster leads to error and a corresponding error term which can be summed up over all the data points and clusters to form a sum of squared errors which can act as an optimization objective for the clustering algorithm. Depending on the cluster and data characteristics such as size, density and shape of the clusters, the distance metric can be defined. Further pre-processing such as removal of outliers is also recommended. In this slide, we see examples of clusters differing in size, density and functional properties. The objective function for fuzzy k-means algorithm includes a weight term in the distance computation such that the power of the weight decides the extent to which the algorithm is assigning data points to clusters in a fuzzy manner. To minimize the objective function representing error terms in the clustering, we fix it and determine the fuzzification properties and recompute it until the clusters do not change. The initialization of the algorithm assigns weights randomly such that they sum up to one. Such weights can be interpreted as probabilities associating points to clusters. The centroids of the clusters are based on the weighted sum shown in this slide such that the weight deciding the fuzzification in the distance metric is based on a ratio determined by the distance metric. An example of how fuzzy k-means algorithm works is shown in this slide where the color gradient shows how different points are assigned to different clusters such as the red points whereas the clusters themselves belong to triangle, circle or square shapes. Across iterations of the fuzzy k-means algorithm, we wish to converge on to red points such that they belong to more than one cluster, similarly pink and blue points. Because we have gained insights on partially labeling data using the fuzzy k-means or fuzzy c-means algorithm, we have

looked at its illustration and contrasted it with the classical k-means algorithm. Thank you.

Bottom-Up Agglomerative Methods Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video, we will understand the hierarchical clustering algorithm that can be contrasted with the partitional clustering algorithm. We will look at the bottom-up agglomerative methods to do hierarchical clustering. Such methods are visualized using the dendrogram, which will be introduced in the video. Hierarchical clustering algorithms produce a nested structure of clusters organized as a hierarchical tree, and this tree is called the dendrogram. The dendrogram splits the data points into a sequence of merges in a bottom-up fashion to arrive at the superset cluster consisting of all the data points in the dataset. No particular assumption is made on the number of clusters in advance. Any desired number of clusters can be obtained by cutting the dendrogram vertically parallel to the x-axis, such that the clusters correspond to meaningful taxonomies such as found in biological sciences about the animal kingdom phylogeny reconstruction in DNA sequences. Bottom-up agglomerative clustering method is one of the most popular hierarchical clustering techniques and the algorithm is straightforward. It computes a proximity matrix between different sets of clusters, such that a decision is made iteratively on whether to merge the cluster or update the proximity matrix until a single cluster remains. The key operation is the computation of the proximity between two clusters and it is used to define the distance between the clusters. This computation starts with clusters of individual points such as shown in this figure. We wish to group individual points into sets of clusters and then approximate the distances between the clusters. For every grouping which happens across the levels of the dendrogram, we get new sets of clusters that can be used to form a distance matrix showing the proximity between the clusters. The key question in the computation is how do we update the proximity matrix? The updates can be performed based on distance metric calculation. The various popular metrics are listed in this slide. The min metric finds the distance between the clusters to be the minimum distance between the points which are closest to each other but belonging to the separate clusters. The maximum distance, on the other hand, finds the distance between the pair of points that are maximally separated to each other and belonging to the two separate clusters. Similarly, there are many other distance metrics that can be used to update the values of the proximity matrix. The group average finds the distance between all pairs of points across both clusters and takes their average value. The distance between centroids finds the center point of each cluster and finds the distance between them and calls it as the distance between the clusters. Thus, we have reviewed the hierarchical clustering methods at different levels of granularity to discover patterns and groups in the data by considering various distance measures. Thank you.

Top-Down Divisive Methods Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video, we will review the top-down divisive methods for hierarchical clustering that can be contrasted with the bottom-up methods. These are also called the divisive methods for clustering. We will also contrast the hierarchical clustering methods with the partitional clustering methods. While agglomerative or bottom-up methods start with the data points to form the clusters,

the divisive or top-down methods start with the all-inclusive cluster and divide it into sub-clusters iteratively. At each step of the iteration, the current cluster is split into sub-clusters such that the iteration repeats until each cluster contains a point. The same distance matrix used for bottom-up clustering can be used for top-down clustering. The proximity measures reviewed in the previous video can also be reused. Here is an example where the points shown according to the X and Y axes are mapped to a distance matrix such that each element of the matrix gives the distance between pairs of points such as P3 and P5. The corresponding dendrogram starts with the cluster containing all the points and it is recursively able to partition the data into clusters containing single points. The corresponding mapping of the points for the distance matrix shown in the previous slide is illustrated for the nested clusters formed in this slide. As top-down divisive methods employ a top-down strategy to form the clusters, they divide a root cluster into sub-clusters recursively until the recursion is exited by reaching lowest level clusters containing similar points or single points. The challenge is to partition a very large cluster into small clusters and there are exponentially large number of ways to do such partitioning. Backtracking of the clusters is not allowed for efficiency reasons since the algorithm would become computationally intractable. In the real application of clustering, agglomerative methods or bottom-up methods are more popular than divisive methods because of this reason of having to partition a cluster into sub-clusters in an exponentially complex computation. Hierarchical clustering also cannot handle data points which are close to each other but belong to different clusters. Hierarchical clustering cannot handle data points that do not belong in a circular or elliptical shape and belong to different clusters even while being close to each other such as in the yellow and green points in the figure in this slide. However, hierarchical clustering algorithms are able to handle such shapes. Thus, we have reviewed the top-down and bottom-up algorithms called Diana and Agnes to create the hierarchical clusters and analyze the data at different levels of granularity to gain insights on it in an unsupervised manner. Thank you.

Distance Measures in Hierarchical Methods Video• . Duration: 2 minutes

[Notes]

Hello everyone, in this video we will look at the distance measures used in hierarchical clustering and see several illustrations of the same. We would like to understand how data distributions are handled by hierarchical clustering and any computational issues thereof. Recall that we were computing the proximity between every pair of clusters using several distance metrics in hierarchical clustering. This proximity was represented by a matrix between all pairs of clusters. Depending on various clustering similarity metrics, we were able to arrive at different configurations of the clustering for the same set of points. And we saw that the clustering similarity computation was determined by the size of the proximity matrix and it was it can be said to be squared in the size of the matrix where n is number of points and cubed in the size of the matrix which has to be updated and searched. This computational complexity can be reduced further by using improvised algorithm design techniques in the cluster proximity calculation. Once a decision is made in hierarchical clustering to combine clusters, it cannot be undone. This is a disadvantage for minimizing various kinds of optimization objective functions in clustering schemes. Hierarchical clustering is also sensitive to noise and outliers. It cannot handle different shapes of clusters and tends to break large clusters into smaller clusters. For example, if points mixed up and noisy as illustrated in this

diagram, hierarchical clustering cannot find the correct clusters. If the points belong to one large cluster, it tends to break the points into smaller clusters rather than detecting the large cluster. Thus, we have reviewed the advantages and disadvantages of hierarchical clustering and looked at algorithmic methods for producing such clusters. Thank you.

Aspects of Cluster Validation Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video, having reviewed the algorithms for producing clusters, we will look at techniques for evaluating the quality of clusters, we will see the need for cluster validation and look at several measures to evaluate the quality of clustering. Given any random dataset, such as in this slide, we can partition it into various clusters. The question is, which partitioning is the best partitioning? If we had prior information about the class labels associated with the clustering, then we can derive several performance measures such as accuracy, precision, recall to evaluate the cluster quality. Then the question of evaluating the quality is similar to the question of evaluating the goodness of fit of the clusters to the dataset. However, when such supervision information is not available, then we have to compare clustering algorithms by comparing each set of the partitions found in the clustering. And this is based on the patterns underlying the clustering. The major issues in cluster validation are cluster evaluation, clustering stability and clustering tendency. Evaluation is a measure of the goodness of the clustering fit to the given dataset. Stability is a measure of the sensitivity of the clustering result to various algorithmic parameters in the model. Tendency is the suitability of clustering to any inherent grouping structure in the dataset. Thus, to determine the quality of clustering, we can compare the results of clustering to externally given class labels, or we can look at any structure that actually exists in the data to distinguish between different clusters and determine the correct number of clusters to compare various clustering configurations. We can also decide to further evaluate the entire clustering scheme or individual clusters. Depending on all these criteria, several clustering evaluation measures have been proposed in the literature. They are designed to evaluate the goodness of fit of the clustering to the dataset. And no one best measure exists because clustering is an unsupervised problem. The most common categorization of the measures is given in this slide. And the measures are called external, internal and relative indices. External indices use externally supplied class labels, which are provided by end-user to evaluate the quality of clustering. Thus, this is a supervised technique. Internal indices measure the goodness of fit of the clustering structure present in the data without any external information. Thus, these are unsupervised techniques. Relative indices compare different clustering algorithms or clusters using statistical measures such as entropy. Thus, we have seen the need for assessing the quality of clustering and the indices used in the evaluation of clustering. Thank you.

External Indices Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video we will discuss the external indices for cluster validation. The most popular indices for cluster validation are based on matching criteria. Given a ground truth which is a user supplied external information about the clustering called T , the problem is to match T with the clustering configuration C output by the clustering algorithm as given by a quality measure Q . Q is a

good measure if it is able to analyze the following characteristics of clustering such as homogeneity of clustering, points which are similar to each other belong to the same cluster, completeness of clustering, points that belong to one cluster do not belong to other cluster, rag bag property where points which are wrongly put in different clusters should be penalized in the clustering algorithm, rag bag property where points put in the wrong cluster should lead to penalties in the clustering algorithm, cluster preservation property where splitting small clusters is more harmful than splitting large clusters. Based on these criteria, there are several matching based quality measures used as external indices in clustering algorithms such as purity, maximum matching and F measure. Information theory based measures derived from entropy include normalized mutual information, conditional entropy and variation of information. The correlation structure underlying the clustering can also be analyzed using external indices such as Jaccard coefficient and Huber statistic based on a notion of true positives matching output clusters with the prior information about class labels available in the dataset. Once a true positive is defined, we can similarly define false positives, true negatives and false negatives. In the subsequent slides of the video, we will look at several definitions of these clustering quality measures. Purity is a measure which quantifies the extent to which a cluster contains data points which are similar to each other. It defines how pure each cluster is based on the number of points that are assigned to the cluster which are common to both the cluster and the ground truth partition C and T respectively. The purity of a cluster is then defined as the weighted sum of the data points belonging to the cluster. For example, if there are 20 points belonging to first cluster C1 and belonging to the second partition T2 and there are 30 points belonging to the third partition T3, then the total number of points belonging to the first cluster C1 are 50 and they are distributed among three different partitions. Using such a matrix, purity is computed based on number of points that belong to both clusters and partitions in the ground truth. Maximum matching measure is another external index that is based on matching clusters with partitions in the ground truth. It constructs a bipartite graph such that a weight is associated with the edges in the graph where on one side of the graph clusters are present and on another side the partitions are present. The graph is said to be matching graph if the edges in the graph are pairwise non-adjacent that is they do not have any common vertex mapping the same cluster to multiple partitions. The maximum matching measure is then defined on the graph as the optimal weight matching the clusters with the partitions. The sum of all edge weights matching multiple clusters to the partitions is the final matching measure. Precision, recall and F1 score are other external indices used to assess clustering quality. Precision of a cluster is same as the purity of a cluster discussed in a previous slide. It measures the fraction of points in cluster that belong to partitions. By contrast, recall measures the fraction of partitions that are shared by a cluster. F measure is the harmonic mean between precision and recall values for each cluster. It tries to balance the precision and values across clusters so that clusters are mapped to partitions in a one-to-one manner. Thus we have looked at the external indices for cluster validation including purity, maximum matching and F measure. Thank you.

Internal Indices Video• . Duration: 3 minutes

[Notes]

Hello everyone, in this video we will discuss the internal indices for cluster quality validation. We will look at an introduction to the most popular internal indices and gain an intuition about their use in

assessing the separation between clusters. Cluster cohesion and separation are two measures which are intrinsic to the clustering configuration. Cluster cohesion measures how closely the clusters are related to the data points, whereas cluster separation measures how distinct or well-separated clusters are from other clusters. This can be expressed as an optimization problem, where the within-cluster sum of squares expressed in terms of the difference between the representative points in the cluster and the remaining points in the cluster and the between-clusters sum of squares is based on the distance between the representative points across clusters. The total sum of squares is then the summation of both terms and it is typically a constant as derived in this slide. The sum of squares is expanded into its constituent terms across all clusters. In this slide, an illustration is shown for different clustering configurations, where the total error adds up to a constant value of 10. A graph-based view of cohesion and separation between clusters can also be derived using a proximity graph, where the data points are the vertices or nodes of the graph and edges between the graphs are formed between every pair of data points, such that the weight of the edges gives the proximity between the data points. The cohesion of the cluster can be defined as the sum of the weights of the links in the proximity graph that connect points within the cluster, whereas the separation between clusters can be given as the sum of weights of the links from points in one cluster to points in another cluster, such as illustrated in the figure. The silhouette score can then be derived based on the intra-cluster and inter-cluster distances. Corresponding to the silhouette score is an index which varies between minus 1 and 1, where a value of 1 means that clusters are well separated from each other and clearly distinguished, whereas a value of minus 1 means the clusters are wrongly assigned. A value of 0 implies that the distance between the clusters is not significant. The Davies-Bolden index is another ratio which is based on the within-cluster and between-cluster distances. It is defined as given in this equation, where d_{ij} is the within to between-cluster distance ratio for i th and j th cluster respectively. Optimal clustering configuration has the smallest db index value. Thus, we have seen internal quality validation indices for clustering, such as the silhouette coefficient and db index. Thank you.

Data Privacy Video• . Duration: 7 minutes

[Notes]

Hello everyone. In this module, we will look at the ethical implications of data analytics, starting with data privacy. By the end of this video, you would have received an introduction to data privacy and scoped out the privacy risks on the data available freely on the internet and understood the need for including defense mechanisms in data analytics methods. With the proliferation of emerging technologies such as Internet of Things, cyber-physical systems, data is everywhere, computing is ubiquitous. We have the capacity to collect both structured and unstructured data, leading to several complex data types and data structures in the data analytics pipelines. There is an intersection of hardware, software, and networking technologies with data analytics. The corresponding technology architectures must be included into the data analytics methodology that is suitable for privacy aspects. We require reference architectures for data analytics that include not only processing of data, but also the system design and architecture associated with the underlying hardware, network, downstream applications, and the presentation seen by the end user. To accommodate these requirements, there is need for new data analytics technology that is both robust and reliable, allied areas such as smart industry, smart agriculture, smart healthcare, smart homes, etc. Privacy risks

are seen with personal data of customers stored by a company, transferred over the internet, that affect the confidentiality and integrity of the data. Any faulty provisioning of data privacy in such data analytics models and methodologies will result in a malfunctioning of the developed technology. Ideally, we would like to include privacy considerations by design into the data analytics modeling such that decision-making on data is restricted according to access controls put on the data and policies in an organization. Transparency of the privacy associated with a data analytics solution is critical. From a mathematical modeling standpoint, we would define data privacy in terms of the information content in the data, such that after encoding the information, there is minimum loss of information while preserving the utility of the data in the data mining procedure. To achieve this, there are several solutions available on the environment from which data is sourced, such as data provenance and data management. Data provenance traces the changes occurring in data from the source until the model consumes the data, including anonymizing the data, users to whom the data is visible, owners of the data, and user access available on the data. All such privacy and security policies must be implemented in the technology solution as a data management aspect of it. In cyber-physical systems, there are several other layers associated with the data analytics modeling, such as the physical layer consisting of the infrastructure from which data is sourced and the human layer consisting of the human users who are consuming the data. All such considerations must be included into the design and development of algorithms and models for data analytics. Unlawful entry of personal information should be regulated such that personal data is managed according to privacy considerations at the database level and at the algorithm level of data analytics pipeline. There are several definitions of privacy which are relevant for data analytics, some of which are listed in this slide, such as bodily privacy from invasive measures of technology on human beings, communication privacy on how information is shared on channels such as the internet, territorial privacy on enterprise privacy within different buildings of a company, and misuse of personal information leading to recognition of individuals by public authorities. The data available for privacy requirements may be leaked inadvertently or maliciously. This leads to different scenarios under which data privacy must be assessed and the corresponding metrics for assessing the performance of the underlying algorithm. Risk mitigation of data leakage is expressed in terms of several risk factors such as frequency of accessing data points, size of data sets available, data available for public use, and data targeted by malicious adversaries. In all such scenarios, personal data and sensitive data must be protected to preserve data privacy. The easiest way to preserve sensitive attributes is to delete them. But this violates the data integrity of a database and adds to a missing pattern in the model that leads to increase in uncertainty and risk in the analytics model. Other alternatives to preserve data privacy are applying data augmentation, that is, inferring attack data that is able to mislead the data analytics model and include that dataset along with the original training dataset, train and tune the parameters of the data analytics model. Separate data samples such as training set, test set, and validation set can also be created to evaluate the performance of the data analytics according to different privacy requirements. A risk analysis, a risk assessment, and a threat modeling can also be included into the privacy preserving data analytics. In this context, data obfuscation perturbs sensitive attributes and records leading to information that is masked for algorithmic processing. The obfuscation is done by the designer of the data analytics model who considers potential privacy attacks on the model. The adaptive mechanisms that anticipate such attacks lead to several inference algorithms. Data sanitization is an alternate strategy which disguises sensitive information by rewriting it with statistical information that is derived from the original dataset. This is possible by learning synthetic data distributions from the

original dataset and also flipping the class labels in a classification problem according to certain specifications to preserve data privacy. Recent developments in generative models are very useful in such a context. Thus, we have seen the need and mechanisms for including data privacy in data analytics methodology. We have also seen examples of the intersection of data analytics with other areas of technology in practical applications such as smart cities and possible ways and means to develop solutions using such technologies in the data analytics, modeling, and algorithm development. Thank you.

Model Privacy Video• . Duration: 9 minutes

[Notes]

Hello everyone, in this video, we will extend the study of data privacy into model privacy. By the end of this video, we would have understood privacy preserving data mining techniques such as randomization and k-anonymity models. With the proliferation of internet, we have seen several cyber attacks occurring on internet such as phishing calls leading to security and economic implications in the usage of technology. In such cases, there is need to preserve individual's information using data mining algorithms that account for data privacy. Such algorithms must not only preserve the data privacy, but also retain the utility of data at the same time. Such techniques are referred to as Privacy Preserving Data Mining Techniques or PPDM for short. In the architectures for PPDM, there are several models, datasets, and tasks. The design is around the categorization and utilization of various attributes related to data privacy in the modeling of the data analytics. This includes models around data and feature leakage and specialized models such as differential privacy. Adversarial attacks, as listed in this slide, are also a concern where a malicious adversary targets the data analytics model with several attack scenarios such as the reconstruction attack. The adversary can be defined as a malevolent user in this context, where potential vulnerabilities in trained model are exploited by the adversary. Vulnerabilities include a model that overfits to the training dataset, a model which is not able to hide the sensitive data attributes, and a model that is not accounting for privacy attacks in its design. The data mining algorithms that include privacy preservation into their design reconstruct learned representations of the original training dataset such that they are suitable for modeling the data analytics. These include black box, white box, and gray box scenarios. In black box scenario, there is only a query access of the adversary to the data analytics model. By contrast, in white box scenario, all details of the model such as its input, output, architecture, and parameters are available to the adversary. It is realistic to assume that in real-world settings, gray box scenario, which is between black box and white box scenarios, describes the real-world situation. In such a scenario, the PPDM algorithms must include data analytics procedures and model validation procedures for the exchange of confidential information. Such models and algorithms are listed in this slide. Examples include clustering, classification, and association rule mining applied to the privacy problem. These algorithms consider concepts defining privacy in terms of the data distribution's input to them, and their capacity to distinguish between original and modified users, to estimate the information loss thereof, expressed as trade-offs between privacy and accuracy in the data analytics model. Typically, these techniques reduce the granularity of the underlying data in order to preserve privacy without compromising on the utility of the data. Several mathematical transforms are performed to achieve such representations for the original data. Some of the popular techniques for privacy preservation are listed in this slide. These

include mathematical transforms such as the geometric transformation, online and offline techniques suitable for black box scenarios, and cryptography techniques suitable for granular scenarios. We will see some examples of these techniques. In randomization method, noise is added to the data by the designer of the PPDM algorithm such that noise masks the attribute values of records, and individual values cannot be recovered from the processed data. Data mining techniques using randomization create an aggregate distribution from the perturbed records which can be expressed mathematically as, for a set of data records X , a probability distribution f of y is added to create set of distorted records x_1 plus y_1 , x_2 plus y_2 , so on x_n plus y_n , such that the variance of f of y is large enough such that records in x cannot be guessed easily. Thus, we can think of the original data as x , onto which y is superimposed as noise to create a randomized z which is used for data mining. For datasets having large enough number of examples, the distribution of noise z can be approximated with kernel density estimation, and the distribution of x can be inferred with likelihood estimation methods such as the expectation maximization EM algorithm. However, randomization has challenges dealing with high dimensional data, and it is not computationally feasible for estimation of data distributions in multivariate scenarios. At the same time, it has advantages such as it can be implemented at the time of collecting data and it does not require a trusted server to perform the anonymization process. Adding more noise susceptible to outliers decreases the utility of the randomization method, and such considerations are included in the design of data mining algorithms such as classifiers, association rules, collaborative filtering, etc. Several kinds of noise are described in this slide such that additive noise preserves statistical properties of the original distribution after reconstructing it by using masks, however it is sensitive to extreme data points such as outliers. Multiplicative noise is more effective than additive noise since reconstruction of original individual values is more difficult because data is randomized by multiplying noise with known statistical distribution. However, it is also vulnerable to data points such as outliers which severely degrade the utility of this technique. This leads us on to k -anonymity which is an alternative to the randomization method. It reduces the granularity in data by mapping one record in the processed data to at least k records in the original data. This is called the k -anonymity condition and this condition is further extended into several other properties that the data analytics model must satisfy such as the l -diversity across different class labels where k -anonymity holds for different groups of datasets. k -anonymity model is able to aggregate the results from original data partitioned into several entities such that aggregate results maintain their privacy over the entire dataset. It satisfies several mathematical properties such as generalization, suppression and perturbation described in this slide. For example, generalizes the range of the attribute values in such a way that the original data cannot be guessed easily. Perturbation replaces original data with synthetic data following a statistical distribution that closely matches the original data. k -anonymity model is able to reduce the risk of identification with the use of public records and any identifiers of a particular record will be matched against k different records in the original dataset. However, solving for the optimal k -anonymization algorithms is a NP-hard problem. The several properties extending k -anonymization are given in this slide such as l -diversity, t -closeness, epsilon differential privacy. Thus, we have seen the threats and risks to model privacy, extending data privacy. We have also seen some of the popular PPDM techniques and compared and contrasted several privacy models. Thank you.

Privacy Enhancing Strategies Video• . Duration: 7 minutes

Hello everyone, in this video, we will look at several privacy enhancing strategies useful in both data privacy and model privacy. By the end of this video, you would have understood privacy preserving mechanisms such as the use of noise processes, several types of adversaries and privacy preserving metrics used in the evaluation of data analytics. Privacy preserving mechanism can be defined as an algorithm that randomizes data or a function derived from the data in order to thwart unwanted statistical inferences around the privacy of the data. It differentiates between privacy and security requirements in data. Privacy preservation is then included into the different stages of data analytics process such as in the KDD process. Information privacy on processed data is the privilege to control the information obtained from raw data and the security mechanism to facilitate this is studied in subsequent topics. The difference between privacy and security of data is given in this slide, where privacy is based on user's information and control and access of that user or other users on that information, whereas security is defined in terms of the popular confidentiality, integrity and availability requirements satisfied by data analytics. Privacy is relevant in terms of individual users who decide about the utilization of their data, whereas security is about institutional arrangements that are providing a security system in an organization such as a company. It is possible to have poor privacy but good security, however, it is difficult to have good privacy without good security. Two examples are given in this slide. Such considerations of privacy are to be included into the data pipelines used to develop the analytics model such that productive enforcement of policies is made possible in the data analytics process. It includes methodologies such as ETL extract transform load to create processed data sets and reports from the database. The data generation process and storage virtualization are two other strategies used to preserve privacy in the data. In data generation, false data is added onto the real data to make it hard to distinguish between private and public data. In storage virtualization, several virtualization technologies are exploited in data storage systems such as databases and data warehouses. Differential privacy is an algorithmic strategy used to obtain useful information from databases. It finds the minimum distraction in the information provided by the database system such that the distraction is large enough to protect privacy, while at the same time distraction is small enough so that the provided information from the database is useful for the downstream analytics task. This is an illustration of differential privacy where the user provides a query to a database and the results of the query are differentially processed with noisy data overlaid on the retrieved data to provide results to the user which make it hard to guess the original data in the protected database. Differential privacy thus controls the shift in the output due to perturbations made on the input and it minimizes information leakage at the same time to satisfy several privacy-preserving performance metrics required by the data analytics. This is done by a study of several noise mechanisms included into the privacy-preserving mechanism. These mechanisms are obtained from information theory, some of which are listed in this slide, such as the privacy tunnel from the rate distortion theory, information bottleneck from statistical modeling, information obfuscation from sampling methods. All of these techniques are used in a data-driven fashion in modern machine learning systems. From the viewpoint of an adversary who is observing the designed solution, the adversary makes inferences on the designed solution and receives utility from such inferences. This is included as a threat model on the computational limits and assumptions of the adversary. This lead several assumptions about the underlying computation leads to several types of adversaries. The main goal of privacy-preserving mechanisms is not to prevent information leakage and make it zero, but to arrive at an optimal solution where there is some information leakage, but privacy is achieved

to the desired level and the desired utility with respect to several types of adversaries who estimate sensitive information about the model. In such an optimization problem, utility of the model is application dependent and several aspects of privacy must be included into the model and algorithm development. These include privacy metrics on the data sources used and probability distributions derived from them, subsequent computational assumptions around the adversary making inferences on such distributions. The adversary's goal can be stated as breaking the privacy of the model and the probability of doing this, the success rate of the adversary. In information theory, such probability distributions are summarized using metrics such as mutual information and F-divergence. In such privacy-preserving scenarios, there is minimal assumption made by the adversary about the targeted black box model and the model makes minimum assumption about the computational requirements from the adversary's side. The resulting mechanisms are privacy-preserving mechanisms which include adversarial capabilities. Differential privacy is an example of such a mechanism, where the result of a statistical query put by the end-user should be the same no matter whether the underlying dataset contains original data or adversarial data. The corresponding noise mechanisms in differential privacy include several perturbation schemes on the process data, adding noises such as Gaussian noise, Laplacian noise, exponential noise. The change in output probability distribution induced by such noise is quantified in adversarial models. Therefore, we have seen several algorithms to quantify privacy, especially with respect to use of information theory and adversarial capabilities. Thank you.

Data Fairness Video• . Duration: 4 minutes

[Notes]

Hello everyone, in this video, we will move on to fairness considerations in data analytics models, starting with data fairness. By the end of this video, you would have understood several notions of fairness in data analytics in particular and AI in general. You would have also seen the techniques relating data fairness with model fairness. We must observe that AI-based decision-making techniques are not objective. They are data-driven and this leads to several requirements on the AI models from real-world applications, ethical and legal motivations, political and security motivations. Resultant AI-driven decisions must consider machine learning models on data that is fair. Bias in algorithms can be defined in a fashion similar to bias in humans, where data labeled by humans input to the algorithms leads to discrimination by the algorithms. Such a discrimination is represented in terms of data characteristics such as skewed samples and tainted samples, where skewed samples are misrepresentation of the training data that has evolved over time, leading to biased predictions in the analytics model, whereas tainted samples are reflecting human bias in the AI system. Such sensitive features must be included into the data analytics modeling to create fair solutions using limited set of features including minority partitions in the data and reliable sample sizes that are representative of the underlying data distributions, while separating the correlation between sensitive and normal attributes in the modeling of the prediction from data analytics. This leads to several definitions of data bias in terms of the fairness requirements imposed on data analytics, such as individual fairness, grouped fairness, subgroup fairness so on. Each such definition is listed in this slide. The learned model for data analytics according to such definitions must satisfy certain performance metrics associated with each notion of fairness, such as statistical parity in grouped fairness. The data biases associated with such algorithmic biases can be

described in terms of several sampling methods such as selection bias and labeling bias. These methods lead to training data which is incomplete and has impact on the generalization ability of the subsequent AI model that is learning inaccurate predictions on limited data that may be flawed. Several performance metrics such as listed in this slide are presented in the literature as a solution to evaluate and fit such data analytics and AI models better to the given data set. Statistical parity difference that calculates difference in probability of favorable results with respect to hypothesis set on the unprivileged and privileged groups in the data set. Most of these metrics can be derived from the confusion matrix of a classifier. Some of these metrics are based on information theoretic ideas such as entropy. Associating a measure of fairness comparing individuals in the data with respect to outcomes in the algorithm. Thus, we have seen an introduction to data analytics fairness and AI fairness in general and we have seen several mathematical formulations of fairness accounting for data bias. Thank you.

Model Fairness Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this lesson, we will extend the topic of data fairness into model fairness. By the end of this video, you would have understood several notions of model fairness extending into AI fairness, especially those concerning pre-processing, in-processing and post-processing of data analytics modeling. A workflow for data analytics is shown in this slide, where data collection, data labeling, and setting up of the training dataset account for the data bias present in the AI model, whereas algorithm bias concerns the algorithm used to train on the given dataset, which is further optimized under several training strategies. A post-processing evaluation of the results of the data analytics includes several metrics for data analytics that are further fine-tuned according to user feedback and data characteristics. Model fairness and AI fairness address several complex interactions of such a workflow. It is multifaceted in nature to achieve equitable outcomes and account for the risk in the data and algorithms used for the modeling. Such accountabilities include storage, use, sharing, ownership, collection, partitioning, etc. of the data. The human factors contributing to the bias and fairness in the algorithms include the features input to the algorithm, the training algorithms used in the model, optimization objectives and model fine-tuning, which is done while training the model, parameter updates corresponding to user feedback provided on the model. All these techniques fall under the area of robust AI. Algorithm accountability must include such robustness considerations to mitigate bias and ensure fairness in the data analytics and underlying machine learning process. Potential influence of sensitive attributes must be separated from the non-sensitive attributes in the evaluation metrics used, such that any inherent bias in the existing machine learning system and potential incompleteness in the data analytics model is addressed. This can be done easily by using feedback loops from the output side of the data analytics and machine learning model, such that the feedback does not perpetuate and reinforce existing biases in the data. This leads to the notion of complex adaptive systems and associated algorithms included into the design of data analytics and machine learning systems. A simple way to assess such designs is to check the effect of noise in the predicted labels of the data analytics model as a performance criteria, thus pre-, post- and in-processing algorithms to reduce bias in data, trade-off between bias and performance of the model. There are several representations of the data which can be used as an underlying modeling framework to arrive at fair models. These include

adversarial learning frameworks. Pre-processing creates new representation of the input by removing information correlated to sensitive attributes, while preserving the information relevant for the modeling, such that downstream analytics are able to perform as expected while preserving privacy and fairness of the model. The corresponding data transformations which are applied to create the new representations of the input control discrimination, limit distortion, and preserve utility in the analytics model. As part of in-processing, we can use cost-sensitive classifiers and randomize the model to make it highly specific to the task at hand. But this requires a modification of the algorithm underlying the classification model. Another alternative is to use data processing at training time to increase the performance metric of the model that considers fairness notions. These performance metrics can also be included into the optimization objective of the model as regularization terms constraining the optimal solution. After the model is built, post-processing metrics can be defined to include fairness definitions into the model. These metrics are thresholds on scoring functions, for example. These assumptions can be thought of as data characteristics and computational assumptions on how to go about protecting the data, and subsequently the model. We wish to achieve the best trade-off between accuracy and fairness, such that minimum changes are required on the trained model. The preceding ideas of pre-, post- and in-processing of fairness in the data analytics model have been summarized in this slide. They can be compared with more mathematical models, such as those based on regularization and counterfactual examples. Thus, we have seen notions of model fairness in data analytics in particular and AI in general. We have seen techniques to include accountability of the algorithm into the data analytics process. Thank you.

Algorithmic Fairness Video• . Duration: 7 minutes

[Notes]

Hello everyone, in this video, we will look at extension of data and model fairness into algorithmic fairness. By the end of this video, you would have understood notions of fairness in a data-driven manner and the need for incorporating them in the design of algorithms for data analytics. Data-driven algorithms are being used nowadays to support decision-making in mission-critical applications such as in healthcare and defense. Equitable algorithm design is required to achieve fair data analytics and machine learning in such scenarios. The ideas of equity in this context include anonymization of data, inclusivity, labeling, transparency, etc. of the sensitive attributes in the data. We wish to address biases such as human-prejudiced bias, sampling bias due to data not representing the population, feedback loop bias in which loops reinforce the biased predictions made by the AI model, lack of diversity, and over-reliance on AI as other biases. Such biases lead to biased prediction and perpetuate discrimination in the model such that poor generalization is seen in the model across different data samples. As we have seen earlier, preprocessing techniques can be used to address such bias. These include assigning weights to different records in the dataset, sampling the different parts of the underlying population to augment the original dataset and optimize the analytics model using such a dataset. Further preprocessing techniques such as clustering and sampling can also be used as a first step to create the training dataset used in subsequent downstream analytics models. However, such pipelines are not mathematically guaranteed to provide the solution which we are looking for. In-processing techniques include adversarial techniques such as adversarial training and adversarial debiasing. These techniques

model the effect of the sensitive attributes on prediction probabilities made by the analytics model. They can be further enhanced with techniques based on causality and ensemble modeling. Post-processing techniques have several fairness measures incorporated into the analytics model and further included into the objective function for the analytics model. They improve the likelihood of the predicted outputs for particular inputs which are hard to predict. However, such a calibration is not guaranteed mathematically to remove all biases in the model. This leads to further mathematical properties being studied around the algorithm design suitable for fairness in data analytics. Such designs include adversarial regularization and optimization extending adversarial training, covariance shift regularization on the underlying statistical properties of the data, use of uncertainty inference models in the regularization of the data analytics model such as with mixture density neural networks. Such techniques based on counterfactual examples include assigning different weights to instances based on counterfactual scenarios such that trade-offs between different partitions in the data are accounted for in the evaluation of the analytics model. Using the techniques in the previous slides, we can come up with several variants of the classification clustering regression algorithm studied earlier which are listed in the subsequent slides in this video. In fair classification, there are fairness measures that are alternatives to standard measures such as precision and recall used across different populations in the data. These measures assume low distance in the covariate space, that is, similar records in the input should lead to similar predictions in the output. In regression models that predict a real-valued target, the probability associated with the prediction is balanced across different data groups such that fairness is achieved. In recommendation models, the ordering of the candidate items which are relevant for user and the subsequent rankings must be fair. In matching problems such as in economics, the matched items on both sides of the demand supply curve must be fair. In risk assessment algorithms, notions of risk must include notions of fairness such as based on counterfactual interventions. Several learned representations of the input data can also be derived to separate the sensitive attributes from the insensitive attributes in the analytics modeling. Fair clustering looks at different partitions of the data and ensures that subpopulations of such partitions are represented in at least one cluster. Fair anomaly detection looks at anomalous or surprising points in a fair manner to equalize salient features or quantities of interest across the underlying data populations so that members of minority groups are also present in the model. In fair spatiotemporal learning and fair districting, outcome-based fairness notions are used that evolve in time and space. In fair resource allocation, classification problems are discussed in terms of feature subset selection that allocates resources and goods in a fair manner. Fair data summarization presents equitable summaries about subpopulations of interest so that most important features are selected with dimensionality reduction techniques. Fair data generation creates synthetic data that is an equitable representation of the different class labels in the underlying data distribution. This synthetic data is able to evaluate biases of the existing models. It is also able to recreate sensitive attributes. Fair graph mining looks at representations of the underlying graph that are fair. Thus, we have seen several notions of fairness and bias in the algorithms used for data analytics, specifically with respect to the pre-, in- and post-processing stages of such algorithms design. We have also looked at particular techniques and examples of such fairness, such as regularization technique and fair classifiers. Thank you.

Model Security - Part 1 Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video, we will move on to notions of security in data analytics starting with model security. By the end of this video, you would have appreciated the data-driven insights obtained on data and model security and understood the several frameworks to model security in data analytics derived from threat and risk modeling. While privacy is the protection of people's information from unauthorized disclosure, security is the protection of the model and it is correlated with the requirement of data privacy. Data privacy is focused more on the end-use of the individual data, whereas data security is focused more on protecting data in an institutional scale from malicious attacks and misuse for profitable purposes. While security is fundamental for protecting data, it is not sufficient for addressing privacy that must be investigated further. In a previous video, we have seen the comparison between privacy and security that is reproduced here. Model security is concerned with stealing information and intellectual property about models using mechanisms such as phishing, malware, botnet, Trojan, which compromise the availability of available data analytics, AI, and machine learning models and cannot be addressed without corresponding countermeasures actively included into the training of the analytics model. Such measures target the CIA triad from the information security area, standing for confidentiality, integrity, and availability attributes in the data. This is an illustration of the CIA attributes for the data analytics models, where increasing false negative rate with causative attack scenarios leads to the poisoning of the existing analytics models, whereas targeting particular class labels by using more training data creates data sets that are able to mislead analytics models. The resultant security attacks can be indiscriminate or targeted, exploratory or causative. Based on such mathematical properties studied in the data analytics model, we can evaluate the performance of the model in terms of the CIA triad overlapping with causative, exploratory, targeted, and indiscriminate attributes of the security attack. For example, the goal of integrity attack is to increase false negatives in the prediction of the model, whereas the goal of causative attack is to influence the training data set used to build the model. By contrast, the goal of exploratory attack is to evaluate test examples that are able to mislead the model. These attack settings can be studied under the game theoretic framework of causative game and exploratory game, where two players, attacker and defender, are interacting to create the final classifier. The attacker poisons the data, while the defender retrain the classifier on the original and poisoned data set to detect such poisoned examples. In this case, the utility of the classifier is severely degraded for legitimate users when the availability goal of the CIA triad is targeted by the security attack. Similarly, there are several other taxonomies on the attacker's goal. Game theory is able to account for the trade-offs made by the attacker and defender or adversary and learner in our case, to adapt to several training strategies that are able to arrive at the optimal solution for the analytics model. The trade-off made by the defender balances between benefit of correct attack detection and cost of false alarm, whereas the attacker wants to have maximum impact on degrading the performance of the defender with minimum cost on their side. The optimum solutions of the analytics model then determine the suitable strategy for both attacker and defender in adversarial security attacks. The strategic interaction between both attacker and defender players can be used to arrive at the final outcome of the optimization based on best responses found in game theoretical formulations that optimize the attack solutions for adversary and defense solutions for the learner. These can be represented as payoff functions in game theory. The resultant algorithms are able to incorporate the notions of adversarial risk into the training of the analytics model. For further detail, games are defined in terms of players, interactions between players, actions taken by each player in their search space, the timing information associated with such actions, order of play of these players, resulting in several payoff or profit functions for each player,

the potential choices available for each player to come up with a beneficial solution for themselves, the resultant budgets and incentives available for the entire game in general, and the reasoning framework which arrives at the optimal solution. A utility theory is developed in game theory to model several players' interests and preferences in the analytics. In these models, several AI agent architectures are used to deal with uncertainty in the learning environment and such architectures define the utility functions and their expected values in the game. These utility functions are estimates of the underlying probability distribution of the dataset on which the model is being built and the optimal solution can be solved using game theoretical solution concepts such as Nash equilibrium and Perito optimality. The purpose of data analytics in this case is to arrive at a reasonable case of the adversaries that quantify the security evaluation of the analytics models. What is a reasonable case of the optimum solution can be further formalized in game theory using ideas of rationality and learnability that are mathematical properties to be satisfied by the data analytics model. Proposed AI agents lead to an economics equilibrium simulating market of interactions. The knowledge representation of such agents can be further expressed in terms of their beliefs, actions, reasoning, consequences, timing information, and different notions of payoff functions, rationality, and computational capabilities of such agents. Therefore, in this video, we have seen several model security frameworks for data analytics especially those based on game theory. Thank you.

Model Security - Part 2 Video• . Duration: 6 minutes

[Notes]

Hello everyone. In this topic, we will continue the study of model security. We will understand the relationship between data analytics and machine learning using a pattern recognition framework and the subsequent workflows that must be included into the model security. Here is a workflow for model security that looks at different stages of the data analytics, including data preparation, using that data for training a model, evaluating the model using statistical performance metrics, deploying the model in a real production system and making inferences about the model, accounting for user feedback. In all of these stages, there is vulnerability associated with model security that is detailed in the lower portions of the figure, where, for example, an adversary can have access to the training dataset or the predictions made by the model. Similarly, access can be available to the inferences made by the model and the user feedback in such inferences. Based on that, several application scenarios for data analytics that accounts for model security can be developed. All these application scenarios can be summarized in a pattern recognition system that extends the classification model using ideas of discrimination that include feedback loops coming back to the classifier on its performance, which may further percolate down into the dataset used for classification. Pattern recognition can be understood in terms of feature selection and model selection, where the feature engineering associated with preparing the data for the modeling is the focus of practical application of pattern recognition, whereas the model selection associated with the mathematical properties of the model that lead to the several patterns used for prediction and validation of the output is the domain of research. Active feedback between both domains of practical application and research must be encouraged to come up with the best solution. Sensory data is obtained from real-world measurements. It is used to construct representations of the measurements, which are useful as training data in the data analytics and subsequent machine learning modeling. The training data is

represented in terms of feature vectors, which are used as inputs to several analytics models, such as classification models. Several versions of a classification model are possible, out of which the best model is selected according to a post-processing evaluation criteria. Further, a validation data sample can be used to fine-tune such model, before producing its output used for decision-making and interpretation in the application domain from which the measurements were sourced. This slide is another example of a pattern recognition framework, where feature extraction is followed by model training. The model training estimates a probability distribution associated with the features found in the input data, such that the prediction scores obtained from such distributions are useful for decision-making. The corresponding machine learning framework consists of the training or learning algorithm and the inference or testing algorithm. Combination of both of these algorithms with the pattern recognition framework and the several sampling methods to create the training, test, and validation datasets produces the complete machine learning solution. In all these phases and steps, we must account for model security in terms of not only the CIA triad, but also other risks and threats associated with the data analytics model, such as those due to adversarial thinking and systems thinking. This leads to models which are hard to optimize in terms of the directive autonomy and discriminative autonomy of the models, whose special cases are clustering, classification, and regression algorithms that we have studied in the previous lessons, so that models which have both discriminative and directive autonomy are those which are most vulnerable to adversarial attacks. In cyber security, such attacks are further analyzed in terms of threat modeling, which associates additional metadata with the training data used to build the analytics model. The additional metadata includes information such as threat sources, capability, intent, target, relevance, to build probability distributions on the data on the impact of the threat and the risk level associated with it. The risk levels can be further analyzed in terms of risk management and assessment frameworks, such as identification, analysis, and evaluation of risk, treatment, monitoring, and review of risk. This leads to several adversarial and cyber attack scenarios in model security, such as the evasion attack and poisoning attack, that must satisfy existing legal norms. Most of the existing law pertaining to technology does not consider such scenarios. However, the technology has far outpaced the legal developments, including weaponization of the analytics and AI models. The several stages of such a weaponization, followed by a botnet, for example, are listed here. These include weaponization, delivery, exploitation, installation, command and control, actions of intent. They are further illustrated in this diagram, where several feedback loops can be obtained from these stages into the data analytics workflow. Thus, we have seen several risk and threat analysis models to be included into model security. Thank you.

Cost-Sensitive Classification Video• . Duration: 5 minutes

[Notes]

Hello everyone, in this video, we will look at the mapping of model security to cost sensitive classification. By the end of this video, we would have appreciated the usage of cost in building a classifier in addition to the confusion matrix and their impact on the prediction probabilities in the classification. A direct motivating example for cost sensitive classification is, is given in this slide. Several examples are given where the cost of lending to a defaulter for a bank is greater than the cost of refusing a loan to a non-defaulter. Thus, the class labels of defaulter and non-defaulter cannot be treated in the same way. Similarly, the cost of failing to detect an environmental threat

such as an oil slick is far greater than the cost of a false alarm from a model. The cost of gearing up electricity generators for a storm that doesn't hit is far less than the cost of being caught completely unprepared. The cost of misidentifying problems with a machine that turns out to be free of faults is less than the cost of overlooking problems with one that is about to fail. The cost of sending junk mail to a household that doesn't respond is far less than the lost business cost of not sending it to a household that would have responded. In all these scenarios, we can see that class labels should not be treated in the same manner for binary, multiclass and multilabel classification settings. We must take into account the cost of making wrong decisions or wrong classifications along with the right classifications. And the optimization of the classification predictions should be for not only errors in the performance metrics, but also the costs associated with the wrongly classified data points. If the costs were known in advance, they can be included into the decision-making process of the classifier to have an impact on the false positives and false negatives that would have different costs according to a cost matrix mapping to the confusion matrix. Similarly, the two types of classes in binary classification may have different benefits associated with them. Thus, we would like to replace the performance metric of success rate measured by accuracy in terms of average cost or profit per decision made by the classifier. This is shown here in terms of 3x3 confusion matrix, where the cost of misclassifying A into B is 2, which is less than the cost of misclassifying B into A, which is 5. Similarly, the cost of misclassifying C into B is 7, which is more than the cost of misclassifying A into B, which is 2. The same idea has been formalized in terms of false positives and false negatives in a cost matrix mapping to each element in the confusion matrix. The decision-making process of the classifier can thus take costs into account to assign weights to the training data and segregate hard and easy to classify examples in it. This will create a decision structure around the cost matrix that is used at the prediction time in the classifier. The subsequent predictions can be assessed using several curves such as the ROC curve, precision recall curve, and cost curves. These curves characterize the trade-off between hit rate and false alarm rate in false positives and false negatives, such that the performance of the classifier is assessed with respect to several data distributions input to it. The classifier is trying to select samples that have a high proportion of positives in the prediction of true class labels. The resultant true positive rate and true negative rate are shown in this slide and they are plotted in this slide where the false positive rate is on the x-axis and the true positive rate is on the y-axis to create the ROC curve. The ROC curve is created in a stepped fashion for different ranges of false positive and true positive values used in the confusion matrix. It can be made smooth by taking average of several such ROC curves across different experiments. The averaging is done using cross-validation procedures that are extension of sampling methods seen earlier. The resulting predictions have an associated probability with them and create a probability distribution on the matching which happens between the predicted class labels and true class labels. These probability distributions can be further optimized using data-driven learning techniques. If we count the number of true positives and false positives in the test set, we arrive at an ROC curve. Calculations of such counts as weights on the instances used for training the model define several cost curves. These weights can be included into the training of the model to connect cost-sensitive classification with cost-sensitive learning. Thus, we have seen how to build classifiers that are cost-sensitive and evaluate them using not only the confusion matrix but other performance metrics such as the ROC curve. Thank you.

Cost-Sensitive Learning Video• . Duration: 9 minutes

Hello everyone, in this video we will look at the extension of cost sensitive classification into cost sensitive learning. We will compare cost sensitive classification with learning such that prediction probabilities on the cost estimation process of the learning algorithm used in the data analytics model. We note that the model outputs not only a prediction but also a probability associated with the prediction. This probability can be adjusted to minimize the expected cost over the several instances in the training set. The prediction probabilities are normally estimated to find the best model for the given data set such that the maximum probability is associated with the prediction. Instead, the probability in the model can be associated with the expected cost and its misclassifications on the training set. Expected misclassification cost measure can be the performance metric that must be optimized by the analytics model. Such a measure is especially pertinent for data points that are very close to the decision boundary found by the classification model. For two classes represented by the pink and blue points in the figure, the boundary between the two classes that has a mixture of data points which are hard to separate using traditional classification models have to be studied using such a misclassification cost notion. The change in the prediction probabilities from the pink to the blue to the pink is shown in the adjacent figure. Such changes have to reflect the underlying changes which are happening in the data. If they do not reflect that, then the model is not able to adapt to the changes which are happening in the data and it will start malfunctioning. Thus, we must consider points which are close to the decision boundary and have not been accounted for in the classification problem. The expected cost of dealing with such points in a cost matrix must be included into the training of the model in cost sensitive learning. This can be assessed using cost curves in a binary classification setting and it can be extended to multiclass classification setting using a one-to-many mapping, simplifying n-dimensional confusion matrix, simplifying a confusion matrix having multiple class predictions into two class predictions via one-to-many mapping and many-to-many mapping used in ensemble techniques. Cost curves plot expected error against probability of predicting a class such as the positive class. Its intercepts are false positives and false negatives on the sample used to create the predictions. Let us assume a binary classification scenario. In this case, the diagonals of the cost curve are performance of two extreme classifiers which always pick one or other class, whereas the line in the curve parallel to the x-axis having expected error of 100% is a classifier that is always wrong, whereas the x-axis itself is a classifier that is always right. The classifier which we build such as the A classifier in the figure is what we would expect to find on a given dataset and cost characteristics in it between the horizontal and the diagonal lines. Several such classifiers can be compared across different regions of the probability associated with a prediction such that a classifier that has an expected error rate of around 25% is more performing in some regions and less performing in other regions compared to a classifier whose error rate is increasing from a low value to 25% and beyond 25% to 50%. In this way, several classifiers and their false positive false negative rates can be compared using the cost curve. Associated variants of the same classifier can also be plotted on such a curve to include such an analysis into the training of the classification model. The x and y-axis of the cost curve are defined in these equations where c_{+} plus given minus is the cost of predicting plus when the instance is actually minus, thus there is a focus on misclassifications which happen in the dataset and associated probabilities of the misclassification. With different class distributions and class-to-record distributions in different batches of data, cost curves are able to visualize the best performing classifier. They can be combined with ROC curves to extend cost-sensitive classification into cost-sensitive learning. The prediction probability in the cost curves is minimizing an empirical risk

associated with each classifier. The risk is based on the loss function and the target function in the classifier, which is mapping the output class labels to the input records. An expected value of the loss function, which is predicting the probabilities, is to be optimized using the cost curves. This procedure is called the empirical risk minimization procedure. It is useful for problems which are having multiple solutions and multiple optima, such as in the adversarial learning scenario, where the adversary can provide the cost matrix using sampling methods to the classifier and cross-validation of the classifier's predictions is to be done using probability estimates on the misclassification rates of the classifier. We can obtain optimal solutions in such a scenario correctly and efficiently using problem-specific decisions on the prediction probabilities. The problem can be specified in terms of an optimization algorithm whose search space, candidate solutions, and cost function to be optimized must be designed in the classifier. Thus, our goal for training the model is not only to reduce the error of the model, but also to reduce the cost of the model, where the error is reflecting the performance of the model on test dataset, whereas the cost is reflecting the risk associated with making a wrong prediction in the model. Several ensemble models can also be included into this training procedure, such as resampling and reweighting procedures on the prediction probabilities associated with the input records or the output confidence limits from the model. This is done in decision trees, for example, where the splitting and pruning criteria in the tree are cost-sensitive. This is also done in game-theoretical models for model security, where surrogate loss functions are defined based on game theory. In decision trees, the split criteria is penalized using the information-theoretic idea of minimum description length to obtain trees that are not very complex and deep. The cost of tree is defined in terms of not only the error produced by the prediction from the tree, but also the complexity associated with coming up with such a prediction, such that simpler trees are preferred to complex trees in terms of the depth of the tree. Thus, we have seen the idea of cost-sensitive learning and its use in model security extension into fairness and privacy, where the prediction, probabilities, misclassification costs are further analyzed in the classifier design, incorporating cost curves and theoretical ideas such as empirical risk minimization to create answerable learning, meta-learning frameworks and surrogate loss functions in game-theoretical models of model security.

Adversarial Data Mining - Part 1 Video• . Duration: 8 minutes

[Notes]

Hello everyone, in this topic, we will study adversarial data mining in further detail and as an extension of cost-sensitive classification. Adversarial data mining is an emerging area of data mining where adversarial examples are created for the classification models based on several risk and threat models on the classifiers. Outstanding results of machine learning, deep learning, data analytics, and artificial intelligence algorithms are only possible in laboratory conditions. In real-world applications, malicious users can influence the inputs to AI to change the predictions from AI. Thus, there is a need for including awareness of the security threats and risks associated with such inputs given to AI. Several adversarial attacks and examples can be created for the data analytics and machine learning-based systems used for pattern recognition, extending into machine learning, AI, and autonomous systems such as drones. Thus, state-of-the-art classifiers published in the literature work very well on standard data under laboratory conditions, but they are vulnerable to adversarial examples targeted at fooling the algorithms, leading to enormous security implications in

model security. Malicious adversaries can actively try to manipulate data to fool the accurately built classifier to pursue a benefit for themselves. Classification algorithms in mission-critical AI systems must be able to deal with such adversaries to come up with robust AI solutions, account for possible modifications in input data in the algorithm design of the classification models. The behavior of such classification models to adversarial perturbations or examples is one of the areas of study in robust AI. In this figure, we see an example of a spam filter that is being misled to call ham as spam and spam as ham by an adversary who is making up a manipulation or perturbation to the training dataset in a poisoning attack scenario. In such classifiers or learners, the threat models formally specify the types of adversaries and the assumptions made in the prediction of the model. This includes the information, the interaction between the adversary and the model, and the final optimum solution found by the model in the presence of an adversary. Such threat modeling can be incorporated into game theory frameworks. Game theoretical AI agents share beliefs and preferences about each other such that in real-world applications, they have the ability to hide and conceal information to arrive at the optimum solution. In such real-world applications, the threat models for adversarial learning include black-box, white-box, and gray-box models, where black-box models assume restricted access to the classifier. That is, adversary can provide a query input to the classifier and see the output produced by the classifier and evaluate a performance metric based on that to come up with some targets for misclassification. This is a good scenario to come up with surrogate models of the classifier whose internal details are not known and analyze the surrogate models to achieve the target of misleading the classifier. By contrast, white-box attacks attempt to generate adversarial examples that are optimal for a specific target, assuming full information about the classification model. Several such adversarial attacks at training time and test time are listed in this figure, where the adversary could target either the training set or the test set or both. Adversary could target the architecture of the model or particular parameters of the model to achieve a particular prediction or misclassification in the output. Or, in the worst case, adversary could have only an inkling of the pipeline in which the model is used, but not the internal details of the model. Based on all these taxonomies, we can come up with strong and weak adversarial attack settings. Formally, the adversary learns a conditional distribution of adversarial examples with respect to the training dataset, where the training dataset is expressed in terms of the loss functions and learned representations of the original data in the loss functions, such that the distance between the adversarial examples and original examples is negligibly small. An adversarial attack will be imperceptible to a human, but it can fool the classifier. The classifier must be optimized further for such adversarial examples. These examples can be generated using a min-max game or maxi-mini game, which is also called a Stackelberg game in the game theory literature, such that the distribution of the adversarial examples achieved at the optimum point is able to completely mislead the classifier. And hence, if the classifier is trained on such examples, it comes up with a robust hypothesis which is not going to be misled by the adversarial examples. Such a hypothesis can be incorporated into the loss function design for classification. Entire hypothesis classes can be found on known and unknown targets in the classification setting. The most common of such settings is the Stackelberg game or two-player zero-sum game leading to a Nash equilibrium, which has been expressed mathematically here. There are two functions f and g , which are interacting with each other. g is the classification function, which is relating x with y , but in the presence of adversarial examples z , which are provided by the adversary by solving for the probability distribution P_z that is able to increase the values of loss function L . This increase in values is checked for every record x and y . Several L s are possible, most common of which are generative and discriminative

classification models. In generative models, the joint probability distribution between x and y subject to z is learnt by the model, whereas in discriminative classifiers, the posterior probability distribution i.e. the prediction probability associated with the class label given the input data is learnt, rather than the joint probability distribution, which is a harder distribution to learn. In both these models, several parameterizations are possible to study the misclassification errors increases thereof due to adversarial examples. These increases in errors could be due to inaccurate assessment of the learning environment by the classifier and likelihood of incorrect predictions increasing in the classifier, classifier fitting more to false positives than true positives, and this leads to unnecessary reactions and time-consuming operations in retraining the classifier subsequently. All these situations pertaining to uncertainty associated with the prediction probability can be parametrically studied using game theory, where hard-to-classify models that have increasing directive autonomy and discriminative autonomy can be studied further in the game theoretical setting. Thus, we have seen an introduction to adversarial classification in contrast to cost-sensitive classification. We have looked at several taxonomies of adversarial examples and how they are generated using game theoretical models with application in binary classification specifically for statistical spam detection. Thank you.

Adversarial Data Mining - Part 2 Video• . Duration: 12 minutes

[Notes]

Hello everyone, in this topic, we will continue the study of adversarial classification. By the end of this video, you would have investigated several sub-problems in adversarial classification related to binary classification, multiclass classification and multilabel classification. This would have been understood from the perspective of game theory. As we have seen along with the CIA triad, adversarial learning has other causative, exploratory, targeted, indiscriminate goals specified for the adversary, which affects different parts of the confusion matrix to increase misclassification in the classifier. It is better to have an estimation of an adversarial environment and simulate that into the training of the classifier to evaluate the impact of classification, rather than react to adversarial attacks. This is shown as four stages of the classification process, where modeling adversary is required to simulate an adversarial attack and include that into the countermeasures of training the classifier in an adversarial setting. In ubiquitous computing systems, such as in the Internet of Things, there are several interdependent, interconnected, and interactive components in the system. And when data analytics, pattern recognition, machine learning, artificial intelligence models are deployed on such systems, this leads to new attack surfaces for the adversary to exploit, vulnerabilities in the model, and optimization landscapes for the model to control mechanisms to defend against such vulnerabilities, leading to new developments in data analytics. Such increase in developments is both vertical and horizontal, vertical in numbers and volumes of the devices which are used in such AI systems, horizontal in terms of the functionality in which they are used. Common examples of systems include social networks, smartphones, and IoT devices. In all these cases, the adversarial attack can manifest at different stages of the classification process. This depends on the adversary's knowledge about the classifier, put into the black box, gray box, and white box scenarios, affecting the classification outcomes, and any pre-processing, post-processing steps associated with the classification. In a game theoretical framework, to create attacks which affect not only the CIA triad, but also targeted indiscriminate poisoning evasion kind of attack

scenarios in the classifier. All these different scenarios can be formally specified in terms of bias-variance trade-offs of the classifier, which is overfitting to noisy data produced by the adversary as adversarial examples. The goodness of fit of such a model can be expressed in terms of its errors for bias-variance and noise components in the analytics evaluation. The corresponding data samples would include training, validation, and test samples that estimate the model performance. In this case, adversarial samples correspond to validation samples. The bias-variance trade-off would lead to the best performing model in the given hypothesis classes for the target class labels. Therefore, such classifiers trade-off between different notions of robustness in the data mining, where learnability can be defined as the ability of the classifier to predict the correct labels without regard to noise, whereas robustness of the classifier is the ability to come up with prediction that is the same with or without presence of noise. Here the noise is presented by the adversary to the classifier and more learnability comes at the cost of less robustness. These are mathematical properties which must be included into the data analytics procedures to deal with misclassification errors and costs due to adversarial training and learning. ROC curves and cost curves can be used to discriminate between several possible solutions for the adversarial learning process, leading to increasingly complex solutions with decreasing information that the adversary has about the classifier in black box settings. In such scenarios, the risk and threat modeling is not only at a particular misclassification error, but a targeted misclassification and a target from a particular source class label to a destination class label, such that the prediction probability associated with it leads to misclassification in the performance of the classifier. This is achieved by information on the architecture of the classifier or only on the input data available to it or on any sample data available from the same data distribution on which the classifier was built. With decreasing knowledge the adversary has, we are tending towards a black box scenario where achieving a particular source to target misclassification is a harder task to do. Such scenarios must be included into the analysis of cost-sensitive classification and learning to come up with adversarial training and learning. The adversarial attracker trade-offs between the cost of adapting that attack to the retrained classifier that is looking to balance between having a correct detection in its prediction versus having a wrong detection. This interaction can be modeled using game theory where the optimum points found by the adversary affects the optimum points found by the classifier for the classification problem such that the best model is found at the optimum point and the various strategies of interaction between the adversary and the classifier lead to several algorithms for adversarial classification. The strategic interactions between both the player's adversary and classifier or attacker and defender can be modeled using game theory specifically idea of payoff functions in game theory which give a relative ranking of each player in terms of the final optimum outcome such that several trade-offs are made by each player. The attacker trade-offs between increasing attack strength and chances of getting caught by the security system. The defender randomizes between different prediction probabilities to balance between classifications and misclassifications. The Nash equilibrium is the optimum point of the game which can be found in polynomial time. Such a game can have multiple stages and players who update their beliefs and distributions iteratively until the optimum point is reached. All these trade-offs can be included as game theoretical requirements of the security on top of the confidentiality, integrity and availability requirements discussed earlier. This leads to several hypothesis classes for adversarial learning extending adversarial classification of data analytics into adversarial learning of machine learning. The associated concept, complexity and hypothesis classes are related to several mathematical such as those found in optimization theory and probability theory where the worst case empirical risk is to be optimized under uncertainty of the

estimations using multi-level optimization theories where multiple players could be involved in the game. The computing resources available for such a game would trade-off between privacy and utility and security, cost-sensitive adversary on adversary side and classifier side, defense mechanisms for the classifier in an adversarial setting and this leads to new attack surfaces for the adversary. The resultant cost-benefit trade-offs for both adversary and classifier would lead to several privacy violations and security breaches as given in a risk analysis based on adversarial learning and further extension into game theory based on payoff functions or utility functions and mechanism design. This leads to highly specialized adversaries who have lot of computing capacity and capability associated with them and they are able to create cyber attacks for critical infrastructure such as in cyber physical systems such as in power grids. Such algorithms must be built for very specialized tasks and they cannot be built on existing libraries which use the products available in the market. Thus the optimization problem can be summarized into attacker and defender trade-offs where attacker has a cost associated with adapting to the classifier and a benefit from executing the adversarial attack. The defender has a benefit of detecting the correct class label and detecting the misclassified wrong prediction. Several such trade-offs lead to interaction between attacker and defender which must be expressed in game theory using a search space of strategies possible for the interaction, payoff functions resulting from such an interaction, cost of participating in the game for both players, relative ranking of each player after each iteration of the game such that the decision boundaries of the classifier are set to classify not only the original data but also the hard to classify adversarial examples. The associated risk and threat models and attack scenarios define the various types of adversaries who choose their strategy for interacting with the classifier based on the current state of the classifier. This leads to several optimum solutions for several objective functions and several cost functions relating cost-sensitive learning with adversarial learning. The defense mechanisms used by the defender should be superior to the attack mechanisms used by the adversary to arrive at a best-case solution. Similarly, the adversary is able to make new adversarial examples based on recent advancements in AI. From the perspective of the classifier, more learnability comes at less robustness. That is, classifier has to expend more energy in dealing with misclassified data points at the cost of reduction in its performance for correctly classified data points. And this has impact on security and privacy of the classifier. The amount of training data available for the classifier, the learning environment in which the classification is happening, perhaps in a resource-efficient manner for error-resilient system design, are all to be formally specified as mathematical properties to be satisfied by the classifier to achieve robust AI. Thus, we have seen a taxonomy of adversarial attacks in classification problems and an extension of adversarial classification into adversarial learning using game-theoretical models. This leads to different assumptions about the types and trade-offs of adversaries and their interactions with the classifiers. Thank you.

Robust Data Mining - Part 1 Video• . Duration: 7 minutes

[Notes]

Hello everyone, in this video we will look at robust data mining, which is an extension of adversarial learning. By the end of this video, we will look at robust classification in contrast to adversarial classification and motivate the defense mechanisms in robust classification with respect to mathematical properties that are to be satisfied by the classifier. A game theoretical framework will

also be provided, integrating the adversarial data mining with the robust data mining. Robust AI studies the assumptions about the error, noise, uncertainty in the model structure for classification, regression, and other data analytics models. The sensitivity of the algorithms in these models to missing values, outliers, distributional shifts, noise processes, discontinuities, adversarial examples, etc. found in the data are studied in a mathematical model for robust data mining. It generalizes existing models to higher dimensional search spaces, provided relevant data is made available to them. This is a practical view of applying data mining theories in real-world applications. The uncertainty in the model weights is represented via a probability distribution, and an optimum probability distribution is sought after using algorithms that learn a target function mapping the output with the input in the classification. Such functions should alleviate not only overfitting in the model, but also overconfidence in the model associated with the prediction probabilities. This is done by new loss function designs and feature engineering specific to the dataset and task at hand. The general idea of robustness is to create a parameter space where small perturbations in the parameters have small influence on the output of the classification model, rather than a large influence. This idea is pursued using several theories from robust statistics over complex error, cost, and optimization surfaces to arrive at one or more solutions for the optimization by reducing the complexity of the underlying algorithms, which are robust to different properties of the dataset used for training the analytics model. One simple way to come up with robust design is to use ensemble techniques, which vote on a prediction using a meta model. The optimal design will be robust to several worst-case settings found in the dataset, and this is very important in safety-critical, mission-critical systems of using AI, such as in self-driving cars. Thus, the robustness properties of data analytics models can be summarized in terms of effect of noise on the weights or parameters learned by the model at the pre-processing, in-processing, and post-processing stages, where noise is included into the training of the classifier to make it effective for misclassified data points. Such noise can be generated from fake or synthetic data that has prediction probabilities close to the target function and data examples close to the original data which is available, such that the resultant classification model is relatively insensitive to small variations in the weights. One simplification of such a noise process into the classification design is label smoothing. Label smoothing replaces hard 0-1 classification targets with fuzzy values between 0 and 1, such that any noise in the fuzziness can be included into the prediction made by the classifier. Thus, we can define robustness as the ability of the classifier to make correct predictions given noisy data or data with missing values, where robustness is typically assessed with a series of synthetic datasets of increasing noise and complexity. This definition fits very well with the cost-sensitive classification model, where a cost matrix can be created to account for the effect of noise on the classifier. Fewer errors will be made with inclusion of such a cost matrix because false positives will be weighted more heavily than false negatives. Such robust AI models work very well on a wide variety of problems, but they have to be created using efficient algorithms that are able to arrive at an accurate solution without being influenced by errors due to measurement or noise. This leads to several optimization trade-offs that must be made in the classification and prediction model. One of the theoretical frameworks to study such optimizations is game theory. In systems engineering or complex adaptive systems, robustness is inspired from biological definitions and it is defined as the ability to perform well in the face of a mutation or perturbation coming from the learning environment modeled as noise in the classifier. Such a noise perturbation can occur due to inaccuracies in the training of the model or incomplete information about the environment from which the data is sourced to train the model or complexities associated with the parameters and weights estimated in

the model for optimization purposes. Efficient algorithms are required to achieve robust AI in case of such corrupted costly and adversarial examples such that both error and cost are optimized by the robust AI. The corresponding statistical methods and defense mechanisms are studied in data mining. These methods create a robust classifier that is able to deal with imprecision in the class distributions and misclassifications in the data points. The frameworks under which such robust performance is analyzed include game theoretical and decision theoretical frameworks focused on error rate minimization and its associated costs. Such frameworks are summarized in this slide where adversarial training has been discussed at some length but it can be combined with ensemble and generative models for example to create classifiers that are robust and interpretable. This is not only possible for data analytics models but also for machine learning and deep learning models. Thus we have seen robustness properties of analytics models and the use of robust statistics in achieving these properties and the subsequent optimization algorithms which are used on data analytics models specifically classification models. Thank you.

Robust Data Mining - Part 2 Video• . Duration: 6 minutes

[Notes]

Hello everyone, in this video we will continue study of robust data mining. By the end of this video, we would have segregated the attack and defense mechanisms in robust data mining with respect to adversarial learning. From the perspective of adversarial learning, robust data mining has to deal with uncertainty in the predictions with respect to the attack scenarios produced by the adversary. These scenarios will provide adversarial examples that will model the misclassified data points for the classifier. The error rate of such a classifier can be optimized further using adversarial training procedures, where several perturbations on the training set can be obtained using the procedures. Adversarial examples also provide a means to accomplish variations of classification such as in semi-supervised machine learning, where the conditional probability of predicting the class label given the input record, joint probability of estimating both the input record and the predicted class label. This combination of unlabeled and labeled information is called semi-supervised learning. It can be used to create a search space suitable for classification. This search space can be created using adversarial examples combined with original training set. The trade-offs between several classification models such as the generative and discriminative models can be explored further with respect to parameters shared between the conditional and joint probability distribution models and prior beliefs associated with the solution of the classification problem. For example, take the logistic classification scenario, where the classification probability of predicting the positive class plus 1 is given in terms of the logit function and similarly there is a probability given for the negative class. Aggregate probabilities for all the records, this loss function can be differentiated further to arrive at the gradient, which can be rearranged in terms of the probability of making a mistake of calling a positive data point as negative data point and vice versa. These mistakes can be further optimized as misclassification points in logistic regression using the gradient descent optimization procedure obtained from the derivative of the loss function. Such procedures are in contrast to the normal optimization procedures, which are trying to reduce the error of the prediction from the model. These procedures can be extended into mistake-driven learning frameworks in machine learning. The mistake-driven learning framework considers sequence of training examples and predicts target value for each example such that minimum number of mistakes are made by the classifier for the

target concept. This is mathematically expressed in the optimization equation here, where $\hat{p}$ denotes a posterior probability of predicting a class label for a given input. However, that input has been corrupted by noise, which is due to adversarial examples and the adversary is also making an estimation of this posterior probability such that the classifier is fooled. Given a loss function for binary classification that extends into multi-class and multi-label classification, the mistake-bounds learning similar to the logistic regression case can be applied in this case for any multi-label or binary classifier in a black box setting such that the interaction between the losses of the classifier and adversary is expressed in a game theoretical formulation using parameters α and w , where the cost of misclassifying a point for the classifier is dependent on the cost of generating a point that is hard to classify for the classifier from the adversarial side. Such interactions between adversary and classifier formalized in terms of the parameters w and α here can be used to create a max-min game or a max-max game here in terms of the payoff functions of the game which are also parameterized. The interaction between the adversary and the classifier is expressed in terms of the interaction of their costs such that the max-max game is solved first for w by the classifier learning its parameters and then for α which is the adversarial perturbation created by the adversary to generate adversarial examples. The solution of this objective function is found by an optimization algorithm driven by the mistake-bounds framework. Since this optimization function is a non-convex function in general, we cannot use gradient descent directly in solving for this problem and several techniques from optimization theory must be investigated. The aggregate value of the loss functions found at the optimum point gives the empirical risk. The objective of the optimization is not only to reduce the error of the classification model or its variants, but also the cost associated due to the risk term. Thus, we have seen differences between adversarial training and adversarial learning. From a game-theoretical perspective, analyzed risk and cost associated with such learning procedures. These learning procedures will further be helpful to integrate robust AI or data mining with adversarial AI or data mining. Thank you.

Adversarial and Robust Data Mining Video• . Duration: 7 minutes

[Notes]

Hello everyone, in this video, we will continue to explore the connections between adversarial data mining and robust data mining. By the end of this video, you would have appreciated the defense mechanisms used in robust data mining from the perspective of adversarial data mining, especially with respect to game theories. Several adversarial perturbations can be included into such frameworks for classification. This figure shows the tic-tac-toe game for the max-min formulation of it, where depending on the dots and crosses, several strategies are explored from the root of the tree to the leaf of the tree. At the leaf, several payoffs or utilities are achieved for multiple configurations of the game. The path from the root to the leaf gives the optimal solution for the game. This is the classical adversarial search procedure used in AI. Such procedures can be extended into several game theoretical formulations, such as in the Stackelberg game for the prisoner's dilemma problem, where we assume there are two prisoners and there are several possibilities on how they interact with each other, leading to a result of whether they will get jail time or not and for how many years. These outcomes are summarized using a matrix, where if both the prisoners choose to remain silent in the face of interrogation, then they will get 5 years jail sentence term each. If both of them confess, then they get 10 years each. If one of them confesses, then the

other goes free. These possibilities are analyzed using a Venn diagram of the optimum solutions of whether they should cooperate with each other or whether they should defect based on what the other player has chosen or whether both of them should not cooperate with each other. Based on these several possibilities of the solution space are explored in the Venn diagram to come up with a robust solution. Now such game theories can be included into the defense mechanism of the classifier across multiple stages of interaction between one or more adversaries and the classifier. Such stages lead to an optimum solution that updates the final decision and the prediction made by the classifier with respect to the interactions made with the adversaries. This leads to robust classifiers that directly minimize the cost of making mistakes as an optimization problem and the interaction between the classifier and the adversary leads to several evaluation metrics. The adversary can construct several loss functions that are cost sensitive acting as surrogate loss functions in the adversarial learning procedure. The underlying empirical risk can be solved for in a tractable manner using optimization algorithms. Several mistake bound frameworks and no regret learning frameworks are available as the optimization algorithms which will reweight the training data consisting of the original data adversarial data and any misclassified costly data having different sets of weights associated with them. All of these weights can be estimated as parameters in the classification problem. The estimations can be done using gradient or Hessian based methods provided the objective function is convex. But as discussed in the previous video there are several objective functions that are non-convex in which case alternatives to gradient and Hessian based methods must be investigated such as the generative methods and the game theoretical methods. However if the gradient and Hessian of the objective functions in adversarial learning can be solved using optimization algorithms then gradients can be computed for the objective functions using algorithms listed in this slide. However if the gradient and Hessian of the optimization objectives are known then we can use algorithms listed in this slide. If the objective function is non-convex and it is complex where the gradient cannot be found then game theoretical models are one way to solve for the optimization problem. Some of the optimization algorithms are listed in this slide. Note that several deep learning techniques are also listed here. Now all these interactions and classification models can be abstracted as a defense mechanism for the classifier in presence of one or more adversaries interacting with one or more classifiers. The risk and threat modeling associated with such a case is given in this slide where an agent paradigm is given for designing the classifier. In such an agent paradigm depending on the knowledge of the adversary several taxonomies can be produced on the optimum point found by the classifier. For example the adversary can have knowledge about the loss function used by the classifier or the feature set used by the classifier. In these scenarios the optimum solution found by the classifier will be different. Using the knowledge about the classifier adversary can come up with attack strategies that will guide the classifier towards the optimum point. These attack strategies will solve for the perturbations on the original data set which will create the adversarial examples. Some of these strategies can be based on game theoretical models. Some of them can be based on gradient descent models or they can be based on simply a search algorithm. This leads to several defense mechanisms on the classification side extending adversarial training. The defense mechanisms can consider different stages of the data mining process in terms of the interaction between the attack and defense models at training and test phases of the data mining. This slide summarizes all the available defense mechanisms in the literature to come up with robust AI solutions based on adversarial learning. Such robust AI solutions will become harder and harder as we move towards a black box attack from a white box attack in the adversarial learning without information about the

underlying data architecture and hardware used to implement the classifier. As we have seen several game theoretical formulations integrating robust data mining and adversarial data mining using both differentiable and non-differentiable optimization algorithms leading to defense mechanisms in adversarial classification. Several further game theoretical formulations such as normal form games, zero form games, multi-level games have also been illustrated. Thank you.